

The ALF (Algorithms for Lattice Fermions) project release dev.

Documentation for the auxiliary-field quantum Monte Carlo code

The ALF Collaboration*: F. F. Assaad^{1,4}, M. Bercx¹, F. Goth¹, A. Götz¹, J. S. Hofmann²,
E. Huffman³, Z. Liu¹, F. Parisen Toldin¹, J. S. E. Portela¹, J. Schwab¹

¹ Institut für Theoretische Physik und Astrophysik, Universität Würzburg,
97074 Würzburg, Germany

² Max Planck Institute for the Physics of Complex Systems, 01187 Dresden, Germany

³ Perimeter Institute for Theoretical Physics,
Waterloo, Ontario N2L 2Y5, Canada

⁴ Würzburg-Dresden Cluster of Excellence ct.qmat,
Am Hubland, 97074 Würzburg, Germany

*tp1-alf@uni-wuerzburg.de

September 18, 2026

Abstract

The *Algorithms for Lattice Fermions* package provides a general code for the finite-temperature and projective auxiliary-field quantum Monte Carlo algorithm. The code is engineered to be able to simulate any model that can be written in terms of sums of single-body operators, of squares of single-body operators and single-body operators coupled to a bosonic field with given dynamics. The package includes five pre-defined model classes: SU(N) Kondo, SU(N) Hubbard, SU(N) t-V and SU(N) models with long range Coulomb repulsion on honeycomb, square and N-leg lattices, as well as Z_2 unconstrained lattice gauge theories coupled to fermionic and Z_2 matter. An implementation of the stochastic Maximum Entropy method is also provided. One can download the code from GitHub at <https://github.com/ALF-QMC/ALF/tree/master> and sign in to file issues.

Contents

1	Introduction	5
1.1	Motivation	5
1.2	Definition of the Hamiltonian	6
1.3	Outline and What is new	8
2	Auxiliary Field Quantum Monte Carlo: finite temperature	9
2.1	Formulation of the method	9
2.1.1	The partition function	10
2.1.2	Observables	11
2.1.3	Reweighting and the sign problem	13
2.2	Sequential updating schemes	14
2.2.1	Space-time sequential single spin flips: discrete fields	15
2.2.1.1	Sampling of e^{-S_0}	15
2.2.2	Space-time sequential single spin flips: Continuous fields	15

2.2.2.1	Box distribution	16
2.2.2.2	Metropolis assisted Langevin algorithm (MALA)	16
2.2.3	Time sequential, global in space updates.	17
2.2.3.1	User-defined global moves on a single time slice: Metropolis	17
2.2.3.2	User defined global moves on a single time slice: MALA	18
2.2.4	Combining various sequential updating schemes.	18
2.3	Global updates in time and space	19
2.3.1	User specified global moves in space and time	19
2.3.1.1	Metropolis	19
2.3.1.2	MALA	20
2.3.2	Parallel tempering	20
2.3.3	Langevin dynamics	21
2.3.4	Hybrid Monte dynamics	26
2.3.4.1	The leap-frog integrator	28
2.3.4.2	Specific Implementation	28
2.4	The Trotter error and checkerboard decomposition	29
2.4.1	Asymmetric Trotter decomposition	29
2.4.2	Symmetric Trotter decomposition	30
2.4.3	The Symm flag	32
2.5	Stabilization - a peculiarity of the BSS algorithm	32
3	Auxiliary Field Quantum Monte Carlo: projective algorithm	34
3.1	Specification of the trial wave function	35
3.2	Some technical aspects of the projective code.	36
3.3	Comparison of finite and projective codes.	36
4	Monte Carlo sampling	36
4.1	The Jackknife resampling method	39
4.2	An explicit example of error estimation	39
4.3	Pseudocode description	41
5	Data Structures and Input/Output	42
5.1	The Operator type	43
5.2	Handling of the fields: the Fields type	43
5.3	The Lattice and Unit_cell types	44
5.4	The observable types Obser_Vec, Obser_Local and Obser_Latt	45
5.4.1	Scalar observables	46
5.4.2	Local observables	46
5.4.3	Equal-time and time-displaced correlation functions	47
5.5	The WaveFunction type	47
5.6	Specification of the Hamiltonian: the Hamiltonian module	48
5.6.1	Flavor symmetries	50
5.6.2	Time-dependent Hamiltonians	51
5.7	File structure	52
5.7.1	Input files	53
5.7.2	Output files – observables	63
6	Using the Code	65
6.1	Quick Start	65
6.2	Compiling and running	66
6.3	Error analysis	68

6.4	Parameter optimization	71
6.5	Parallel parameters feature	72
6.6	Convert plain text bins to HDF5	73
7	The plain vanilla Hubbard model on the square lattice	74
7.1	Defining the parameters	74
7.2	Setting the Hamiltonian: <code>Ham_set</code>	75
7.3	The lattice: <code>Ham_latt</code>	75
7.4	The hopping: <code>Ham_hop</code>	75
7.5	The interaction: <code>Ham_V</code>	76
7.6	The trial wave function: <code>Ham_Trial</code>	77
7.7	Observables	78
7.7.1	Allocating space for the observables: <code>Alloc_obs</code>	78
7.7.2	Measuring equal-time observables: <code>Obser</code>	79
7.7.3	Measuring time-displaced observables: <code>ObserT</code>	80
7.8	Flavor symmetries	80
7.9	Numerical precision	81
7.10	Running the code and testing	81
8	Predefined Structures	82
8.1	Predefined lattices	82
8.1.1	Square lattice, Fig. 9(a)	84
8.1.2	Bilayer Square lattice, Fig. 9(b)	85
8.1.3	N-leg Ladder lattice, Fig. 9(c)	85
8.1.4	Honeycomb lattice, Fig. 9(d)	85
8.1.5	Bilayer Honeycomb lattice, Fig. 9(e)	85
8.1.6	Triangular lattice, Fig. 9(f)	86
8.1.7	Kagome lattice, Fig. 9(g)	86
8.1.8	π -Flux lattice (deprecated)	86
8.2	Generic hopping matrices on Bravais lattices	87
8.2.1	Setting up the hopping matrix: the <code>Hopping_Matrix_type</code>	87
8.2.2	An example: nearest neighbor hopping on the honeycomb lattice	90
8.2.3	Predefined hoppings	90
8.3	Predefined interaction vertices	92
8.3.1	SU(N) Hubbard interaction	93
8.3.2	M_z -Hubbard interaction	93
8.3.3	SU(N) V -interaction	94
8.3.4	Fermion-Ising coupling	94
8.3.5	Long-Range Coulomb repulsion	94
8.3.6	J_z - J_z interaction	95
8.3.7	J_a - J_b interaction	95
8.4	Predefined observables	96
8.4.1	Equal-time SU(N) spin-spin correlations	96
8.4.2	Equal-time spin correlations	98
8.4.3	Equal-time Green function	98
8.4.4	Equal-time density-density correlations	98
8.4.5	Time-displaced Green function	99
8.4.6	Time-displaced SU(N) spin-spin correlations	99
8.4.7	Time-displaced spin correlations	99
8.4.8	Time-displaced density-density correlations	99
8.4.9	Dimer-Dimer correlations	100

8.4.10	Bond-Bond correlations	100
8.4.11	Expectation value of perfect squares	101
8.4.12	Cotunneling for Kondo models	101
8.4.13	Rényi Entropy	102
8.5	Predefined trial wave functions	103
8.5.1	Square	103
8.5.2	Honeycomb	103
8.5.3	N-leg ladder	103
8.5.4	Bilayer square	104
8.5.5	Bilayer honeycomb	104
9	Model Classes	104
9.1	SU(N) Hubbard models <code>Hamiltonian_Hubbard_smod.F90</code>	104
9.2	SU(N) t-V models <code>Hamiltonian_tV_smod.F90</code>	106
9.3	SU(N) Kondo lattice models <code>Hamiltonian_Kondo_smod.F90</code>	107
9.4	Models with long range Coulomb interactions <code>Hamiltonian_LRC_smod.F90</code>	110
9.5	$\mathbb{Z}_2$ lattice gauge theories coupled to fermion and $\mathbb{Z}_2$ matter <code>Hamiltonian_Z2_smod.F90</code>	112
9.5.1	Projective approach	114
9.5.2	Observables	115
9.5.3	A test case: $\mathbb{Z}_2$ slave spin formulation of the SU(2) Hubbard model	115
9.6	Spin Peierls <code>Hamiltonian_Spin_Peierls_smod.F90</code>	116
10	Maximum Entropy	117
10.1	Quick Start	118
10.2	General setup	119
10.3	Single-particle quantities: <code>Channel=P</code>	122
10.4	Single-particle quantities with particle-hole symmetry: <code>Channel=P_PH</code>	123
10.5	Particle-hole quantities: <code>Channel=PH</code>	123
10.6	Optical conductivity: <code>Channel=PH_C</code>	125
10.7	Particle-Particle quantities: <code>Channel=PP</code>	125
10.8	Zero-temperature, projective code: <code>Channel=T0</code>	125
10.9	Integration over the spectral function: QFI and $\frac{\partial I}{\partial V}$	126
10.10	Dynamics of the one-dimensional half-filled Hubbard model	127
11	Conclusions and Future Directions	127
	Acknowledgments	128
A	Practical implementation of Wick decomposition of $2n$-point correlation functions of two imaginary times	129
B	Performance, memory requirements and parallelization	130
C	Licenses and Copyrights	132
	References	132

1 Introduction

1.1 Motivation

The aim of the ALF project is to provide a general formulation of the auxiliary-field QMC method that enables one to promptly play with different model Hamiltonians at minimal programming cost. The package also comes with a number of predefined Hamiltonians aimed at producing benchmark results.

The auxiliary-field quantum Monte Carlo (QMC) approach is the algorithm of choice to simulate thermodynamic properties of a variety of correlated electron systems in the solid state and beyond [1–6]. Apart from the physics of the canonical Hubbard model [7,8], the topics one can investigate in detail include correlation effects in the bulk and on surfaces of topological insulators [9–12], quantum phase transitions between Dirac fermions and insulators [13–20], deconfined quantum critical points [18,21–24], constrained and unconstrained lattice gauge theories [21,25–30], heavy fermion systems [31–36], nematic [37,38] and magnetic [39,40] quantum phase transitions in metals, antiferromagnetism in metals [41], superconductivity in spin-orbit split and in topological flat bands [42–44], SU(N) symmetric models [45–50], long-ranged Coulomb interactions in graphene systems [51–55], cold atomic gases [56], low energy nuclear physics [57] that may require formulations in the canonical ensemble [58,59], entanglement entropies and spectra [60–66], electron-phonon systems [67–69], Landau level regularization of continuum theories [70,71], Yukawa SYK models [72] and even spin systems [73] among others. This ever-growing list of topics is based on algorithmic progress and on recent symmetry-related insights [74–77] that lead to formulations free of the negative sign problem for a number of model systems with very rich phase diagrams.

Auxiliary-field methods can be formulated in a number of very different ways. The fields define the configuration space $\mathcal{C}$. They can stem from the Hubbard-Stratonovich (HS) [78] transformation required to decouple the many-body interacting term into a sum of non-interacting problems, or they can correspond to bosonic modes with predefined dynamics such as phonons or gauge fields. In all cases, the result is that the grand-canonical partition function takes the form

$$Z = \text{Tr} \left(e^{-\beta \hat{\mathcal{H}}} \right) = \sum_{\mathcal{C}} e^{-S(\mathcal{C})}, \quad (1)$$

where β corresponds to the inverse temperature and S is the action of non-interacting fermions subject to a space-time fluctuating auxiliary field. The high-dimensional integration over the fields is carried out stochastically. In this formulation of many-body quantum systems, there is no reason for the action to be a real number. Thereby $e^{-S(\mathcal{C})}$ cannot be interpreted as a weight. To circumvent this problem one can adopt re-weighting schemes and sample $|e^{-S(\mathcal{C})}|$. This invariably leads to the so-called *negative sign problem*, with the associated exponential computational scaling in system size and inverse temperature [79]. The sign problem is formulation dependent and, as mentioned above, there has been tremendous progress at identifying an increasing number of models not affected by the negative sign problem which cover a rich domain of collective emergent phenomena. For continuous fields, the stochastic integrations can be carried out with Langevin dynamics or hybrid methods [80]. However, for many problems one can get away with discrete fields [81]. In this case, Monte Carlo importance sampling will often be put to use [82]. We note that due to the non-locality of the fermion determinant (see below), cluster updates, such as in the loop or stochastic series expansion algorithms for quantum spin systems [83–85], are hard to formulate for this class of problems. The search for efficient updating schemes that quickly wander through the configuration space defines ongoing challenges.

Formulations differ not only in the choice of the fields, continuous or discrete, and sampling strategy, but also by the formulation of the action itself. For a given field configuration,

integrating out fermionic degrees of freedom generically leads to a fermionic determinant of dimension βN where N is the volume of the system. Working with this determinant leads to the Hirsch-Fye approach [86] and the computational effort scales¹ as $\mathcal{O}(\beta N)^3$. The Hirsch-Fye algorithm is the method of choice for impurity problems, but has in general been outperformed by a class of so-called continuous-time quantum Monte Carlo approaches [87–89]. One key advantage of continuous-time methods is being action based, allowing one to better handle the retarded interactions obtained when integrating out fermion or boson baths. However, in high dimensions or at low temperatures, the cubic scaling originating from the fermionic determinant is expensive. To circumvent this, the hybrid Monte-Carlo approach [5, 90, 91] expresses the fermionic determinant in terms of a Gaussian integral thereby introducing a new variable in the Monte Carlo integration. The resulting algorithm is the method of choice for lattice gauge theories in 3+1 dimensions and has been used to provide *ab initio* estimates of light hadron masses starting from quantum chromodynamics [92].

The approach we adopt lies between the above two *extremes*. We keep the fermionic determinant, but formulate the problem so as to work only with $N \times N$ matrices. This Blankenbecler, Scalapino, Sugar (BSS) algorithm scales linearly in imaginary time β , but remains cubic in the volume N . Furthermore, the algorithm can be formulated either in a projective manner [3, 4], adequate to obtain zero temperature properties in the canonical ensemble, or at finite temperatures, in the grand-canonical ensemble [2]. The formulations of the auxiliary-field approach can be generalized to time dependent Hamiltonians. This allows for great flexibility and in particular to use the projective algorithm as a variational approach [93]. In this documentation we summarize the essential aspects of the auxiliary-field QMC approach, and refer the reader to Refs. [6, 94] for complete reviews.

1.2 Definition of the Hamiltonian

The first and most fundamental part of the project is to define a general Hamiltonian which can accommodate a large class of models. Our approach is to express the model as a sum of one-body terms, a sum of two-body terms each written as a perfect square of a one body term, as well as a one-body term coupled to a bosonic field with dynamics to be specified by the user. Writing the interaction in terms of sums of perfect squares allows us to use generic forms of discrete approximations to the HS transformation [95, 96]. Symmetry considerations are imperative to increase the speed of the code. We therefore include a *color* index reflecting an underlying SU(N) color symmetry as well as a *flavor* index reflecting the fact that after the HS transformation, the fermionic determinant is block diagonal in this index.

The class of solvable models includes Hamiltonians $\hat{\mathcal{H}}$ that have the following general

¹Here we implicitly assume the absence of negative sign problem.

form:

$$\hat{\mathcal{H}} = \hat{\mathcal{H}}_T + \hat{\mathcal{H}}_V + \hat{\mathcal{H}}_I + \hat{\mathcal{H}}_Q + \hat{\mathcal{H}}_{0,I}, \text{ where} \quad (2)$$

$$\hat{\mathcal{H}}_T = \sum_{k=1}^{M_T} \sum_{\sigma=1}^{N_{\text{col}}} \sum_{s=1}^{N_{\text{fl}}} \sum_{x,y}^{N_{\text{dim}}} \hat{c}_{x\sigma s}^\dagger T_{xy}^{(ks)} \hat{c}_{y\sigma s} \equiv \sum_{k=1}^{M_T} \hat{T}^{(k)}, \quad (3)$$

$$\hat{\mathcal{H}}_V = \sum_{k=1}^{M_V} U_k \left\{ \sum_{\sigma=1}^{N_{\text{col}}} \sum_{s=1}^{N_{\text{fl}}} \left[\left(\sum_{x,y}^{N_{\text{dim}}} \hat{c}_{x\sigma s}^\dagger V_{xy}^{(ks)} \hat{c}_{y\sigma s} \right) + \alpha_{ks} \right] \right\}^2 \equiv \sum_{k=1}^{M_V} U_k (\hat{V}^{(k)})^2, \quad (4)$$

$$\hat{\mathcal{H}}_I = \sum_{k=1}^{M_I} \hat{Z}_k \left(\sum_{\sigma=1}^{N_{\text{col}}} \sum_{s=1}^{N_{\text{fl}}} \sum_{x,y}^{N_{\text{dim}}} \hat{c}_{x\sigma s}^\dagger I_{xy}^{(ks)} \hat{c}_{y\sigma s} \right) \equiv \sum_{k=1}^{M_I} \hat{Z}_k \hat{I}^{(k)}, \quad (5)$$

$$\hat{\mathcal{H}}_Q = \sum_{k=1}^{M_Q} \tilde{U}_k (1 + \hat{Q}_k) \left\{ \sum_{\sigma=1}^{N_{\text{col}}} \sum_{s=1}^{N_{\text{fl}}} \left[\left(\sum_{x,y}^{N_{\text{dim}}} \hat{c}_{x\sigma s}^\dagger \tilde{V}_{xy}^{(ks)} \hat{c}_{y\sigma s} \right) + \tilde{\alpha}_{ks} \right] \right\}^2. \quad (6)$$

The indices and symbols used above have the following meaning:

- The number of fermion *flavors* is set by N_{fl} . After the HS transformation, the action will be block diagonal in the flavor index.
- The number of fermion *colors* is set² by N_{col} . The Hamiltonian is invariant under $\text{SU}(N_{\text{col}})$ rotations.
- N_{dim} is the total number of spacial vertices: $N_{\text{dim}} = N_{\text{unit-cell}} N_{\text{orbital}}$, where $N_{\text{unit-cell}}$ is the number of unit cells of the underlying Bravais lattice and N_{orbital} is the number of orbitals per unit cell.
- The indices x and y label lattice sites where $x, y = 1, \dots, N_{\text{dim}}$.
- Therefore, the matrices $\mathbf{T}^{(ks)}$, $\mathbf{V}^{(ks)}$ and $\mathbf{I}^{(ks)}$ are of dimension $N_{\text{dim}} \times N_{\text{dim}}$.
- The number of interaction terms is labeled by M_V and M_I . $M_T > 1$ would allow for a checkerboard decomposition.
- $\hat{c}_{y\sigma s}^\dagger$ is a second-quantized operator that creates an electron in a Wannier state centered around lattice site y , with color σ , and flavor index s . The operators satisfy the anti-commutation relations:

$$\{\hat{c}_{y\sigma s}^\dagger, \hat{c}_{y'\sigma's'}\} = \delta_{yy'} \delta_{ss'} \delta_{\sigma\sigma'}, \text{ and } \{\hat{c}_{y\sigma s}, \hat{c}_{y'\sigma's'}\} = 0. \quad (7)$$

- α_{ks} is a complex number.

The bosonic part of the general Hamiltonian (2) is $\hat{\mathcal{H}}_{0,I}$ and has the following properties:

- $\hat{Z}_k$ couples to a general one-body term and $\hat{Q}_k$ to a two-body operator. We will work in a basis where these operators are diagonal: $\hat{Z}_k|\phi\rangle = \phi_k|\phi\rangle$, $\hat{Q}_k|\varphi\rangle = \varphi_k|\varphi\rangle$. ϕ_k is a real number or an Ising variable. Hence $\hat{Z}_k$ can correspond to the Pauli matrix $\hat{\sigma}_z$ or to the position operator. The same applies for φ_k .
- The dynamics of the bosonic fields is given by $\hat{\mathcal{H}}_{0,I}$. This term is not specified here; it has to be specified by the user and becomes relevant when the Monte Carlo update probability is computed in the code

²Note that in the code $N_{\text{col}} \equiv N_{\text{SUN}}$.

Note that the matrices $T^{(ks)}$, $V^{(ks)}$, $\tilde{V}^{(ks)}$ and $I^{(ks)}$ explicitly depend on the flavor index s but not on the color index σ . The color index σ only appears in the second quantized operators such that the Hamiltonian is manifestly $SU(N_{\text{col}})$ symmetric. We also require the matrices $T^{(ks)}$, $V^{(ks)}$, $\tilde{V}^{(ks)}$ and $I^{(ks)}$ to be Hermitian. In the description of the code will drop the $\mathcal{H}_Q$ term since it merely obscures the notation without adding any new aspects. This feature of the code will be used in the spin-Peierls Hamiltonian example 9.6.

1.3 Outline and What is new

In order to use the program, a minimal understanding of the algorithm is necessary. Its code is written in Fortran, according to the 2008 standard, and natively uses MPI (MPI 3.0 compliant implementation needed) for parallel runs on supercomputing systems. In this documentation we aim to present in enough detail both the algorithm and its implementation to allow the user to confidently use and modify the program.

In Sec. 2, we summarize the steps required to formulate the many-body, imaginary-time propagation in terms of a sum over HS and bosonic fields of one-body, imaginary-time propagators. To simulate a model not already included in ALF, the user has to provide this one-body, imaginary-time propagator for a given configuration of HS and bosonic fields. In this section we also touch on how to compute observables and on how we deal with the negative sign problem. Since version 2.0, ALF has a number of new updating schemes. The package comes with the possibility to implement global updates in space and time or only in space; we provide parallel-tempering, Langevin dynamics, Hybrid Monte Carlo options, and it is possible to implement symmetric Trotter decompositions. At the end of the section we comment on the issue of stabilization for the finite temperature code.

In Sec. 3, we describe the projective version of the algorithm, constructed to produce ground state properties. One can very easily switch between projective and finite temperature codes, but a trial wave function must be provided for the projective algorithm.

One of the key challenges in Monte Carlo methods is to adequately evaluate the stochastic error. In Sec. 4 we provide an explicit example of how to correctly estimate the error.

Section 5 is devoted to the data structures that are needed to implement the model, as well as to the input and output file structure. The data structures include an `Operator` type to optimally work with sparse Hermitian matrices, a `Lattice` type to define one- and two-dimensional Bravais lattices, a generic `Fields` type for the auxiliary fields, two `Observable` types to handle scalar observables (e.g., total energy) and equal-time or time-displaced two-point correlation functions (e.g., spin-spin correlations) and finally a `Wavefunction` type to define the trial wave function in the projective code. At the end of this section we comment on the file structure.

In Sec. 6 we provide details on running the code using the shell. As an alternative the user can download a separate project, [pyALF](#) that provides a convenient python interface as well as Jupyter notebooks.

The package has a set of predefined structures that allow easy reuse of lattices, observables, interactions and trial wave functions. Although convenient, this extra layer of abstraction might render ALF harder to modify. To circumvent this we make available an implementation of a plain vanilla Hubbard model on the square lattice (see Sec. 7) that shows explicitly how to implement this basic model without making use of predefined structures. We believe that this is a good starting point to modify a Hamiltonian from scratch, as exemplified in the package's [Tutorial](#). Yet another possible starting point is provided by the template Hamiltonian `Hamiltonian_##NAME##_smod.F90`

Sec. 8 introduces the sets of predefined lattices, hopping matrices, interactions, observables and trial wave functions available. The goal here is to provide a library to facilitate

implementation of new Hamiltonians.

The package comes with a set of Hamiltonians, described in Sec. 9, which includes: (i) SU(N) Hubbard models allowing pinning fields [13], (ii) SU(N) t-V models, (iii) SU(N) Kondo lattice models, (iv) Models with long ranged coulomb interactions, (v) Generic $\mathbb{Z}_2$ lattice gauge theories coupled to $\mathbb{Z}_2$ matter and fermions and (vi) models with spin-Peierls interactions. These model classes are built on the predefined structures.

In Sec. 10 we describe how to use our implementation of the stochastic analytical continuation [97, 98] as well as the maximum entropy method [99, 100]. These methods allow one to obtain spectral functions from imaginary-time correlation functions measured in the QMC simulations. One can switch between both methods by merely changing a parameter in the input file.

Finally, in Sec. 11 we list a number of features being considered for future releases of the ALF package.

2 Auxiliary Field Quantum Monte Carlo: finite temperature

We start this section by deriving the detailed form of the partition function and outlining the computation of observables (Sec. 2.1.1 - 2.1.3). Next, we present a number of update strategies, namely local updates, global updates, parallel tempering and Langevin dynamics (Sec. 2.2). We then discuss the Trotter error, both for symmetric and asymmetric decompositions (Sec. 2.4) and, finally, we describe the measures we have implemented to make the code numerically stable (Sec. 2.5).

2.1 Formulation of the method

Our aim is to compute observables for the general Hamiltonian (2) in thermodynamic equilibrium as described by the grand-canonical ensemble. We show below how the grand-canonical partition function can be rewritten as

$$Z = \text{Tr} \left(e^{-\beta \hat{\mathcal{H}}} \right) = \sum_C e^{-S(C)} + \mathcal{O}(\Delta\tau^2), \quad (8)$$

and define the space of configurations C . Note that the chemical potential term is already included in the definition of the one-body term $\hat{\mathcal{H}}_T$, see Eq. (3), of the general Hamiltonian. The essential ingredients of the auxiliary-field quantum Monte Carlo implementation in the ALF package are the following:

- We discretize the imaginary time propagation: $\beta = \Delta\tau L_{\text{Trotter}}$. Generically this introduces a systematic Trotter error of $\mathcal{O}(\Delta\tau)^2$ [101]. We note that there has been considerable effort at getting rid of the Trotter systematic error and to formulate a genuine continuous-time BSS algorithm [102]. To date, efforts in this direction that are based on a CT-AUX type formulation [103, 104] face two issues. The first one is that they are restricted to a class of models with Hubbard-type interactions

$$(\hat{n}_i - 1)^2 = (\hat{n}_i - 1)^4, \quad (9)$$

in order for the basic CT-AUX equation [105],

$$1 + \frac{U}{K} (\hat{n}_i - 1)^2 = \frac{1}{2} \sum_{s=\pm 1} e^{as(\hat{n}_i - 1)} \quad \text{with} \quad \frac{U}{K} = \cosh(\alpha) - 1 \quad \text{and} \quad K \in \mathbb{R}, \quad (10)$$

to hold. The second issue is that it is hard to formulate a computationally efficient algorithm. Given this situation, if eliminating the Trotter systematic error is required, it

turns out that extrapolating to small imaginary-time steps using the multi-grid method [106–108] is a more efficient scheme.

There has also been progress in efficient continuous-time methods using techniques that draw from the Stochastic Series Expansion [109] which can be combined with fermion bag ideas [110]. However, these techniques are even more restricted to a specific class of Hamiltonians, those that can be expressed as sums of exponentiated fermionic bilinear terms $\hat{H} = \sum_i \hat{h}^{(i)}$, where

$$\hat{h}^{(i)} = -\gamma^{(i)} e^{\sum_{jk} \alpha_{jk}^{(i)} \hat{c}_j^\dagger \hat{c}_k + \text{H.c.}} \quad (11)$$

Stabilization can also be costly depending on the parameters, particularly for large α values [111].

- Having isolated the two-body term, we apply Gauß-Hermite quadrature [112] to the continuous HS transform and obtain the discrete HS transformation [95, 96]:

$$e^{\Delta\tau\lambda\hat{A}^2} = \frac{1}{4} \sum_{l=\pm 1, \pm 2} \gamma(l) e^{\sqrt{\Delta\tau\lambda}\eta(l)\hat{A}} + \mathcal{O}((\Delta\tau\lambda)^4), \quad (12)$$

where the fields η and γ take the values:

$$\begin{aligned} \gamma(\pm 1) &= 1 + \sqrt{6}/3, & \eta(\pm 1) &= \pm \sqrt{2(3 - \sqrt{6})}, \\ \gamma(\pm 2) &= 1 - \sqrt{6}/3, & \eta(\pm 2) &= \pm \sqrt{2(3 + \sqrt{6})}. \end{aligned} \quad (13)$$

Since the Trotter error is already of order $(\Delta\tau^2)$ per time slice, this transformation is next to exact. One can relate the expectation value of the field $\eta(l)$ to the operator $\hat{A}$ by noting that:

$$\begin{aligned} \frac{1}{4} \sum_{l=\pm 1, \pm 2} \gamma(l) e^{\sqrt{\Delta\tau\lambda}\eta(l)\hat{A}} \left(\frac{\eta(l)}{2\sqrt{\Delta\tau\lambda}} \right) &= e^{\Delta\tau\lambda\hat{A}^2} \hat{A} + \mathcal{O}((\Delta\tau\lambda)^3) \text{ and} \\ \frac{1}{4} \sum_{l=\pm 1, \pm 2} \gamma(l) e^{\sqrt{\Delta\tau\lambda}\eta(l)\hat{A}} \left(\frac{(\eta(l))^2 - 2}{4\Delta\tau\lambda} \right) &= e^{\Delta\tau\lambda\hat{A}^2} \hat{A}^2 + \mathcal{O}((\Delta\tau\lambda)^2). \end{aligned} \quad (14)$$

- $\hat{Z}_k$ in Eq. (5) can stand for a variety of operators, such as the Pauli matrix $\hat{\sigma}_z$ – in which case the Ising spins take the values $s_k = \pm 1$ – or the position operator – such that $\hat{Z}_k|\phi\rangle = \phi_k|\phi\rangle$, with ϕ_k a real number.
- From the above it follows that the Monte Carlo configuration space C is given by the combined spaces of bosonic configurations and of HS discrete field configurations:

$$C = \{ \phi_{i,\tau}, l_{j,\tau} \text{ with } i = 1 \cdots M_I, j = 1 \cdots M_V, \tau = 1 \cdots L_{\text{Trotter}} \}. \quad (15)$$

Here, the HS fields take the values $l_{j,\tau} = \pm 2, \pm 1$ and $\phi_{i,\tau}$ may, for instance, be a continuous real field or, if $\hat{Z}_k = \hat{\sigma}_z$, be restricted to ± 1 .

2.1.1 The partition function

In this section and in what follows, we will set $\tilde{U}_k = 0$. An explicit example in which this feature is used can be found in Sec. 9.6 where we use to ALF-library to simulate the spin-Peierls Heisenberg Hamiltonian. With the above, the partition function of the model (2) can

be written as follows.

$$\begin{aligned}
Z &= \text{Tr} \left(e^{-\beta \hat{H}} \right) \\
&= \text{Tr} \left[e^{-\Delta\tau \hat{H}_{0,I}} \prod_{k=1}^{M_V} e^{-\Delta\tau U_k (\hat{V}^{(k)})^2} \prod_{k=1}^{M_I} e^{-\Delta\tau \hat{\sigma}_k \hat{I}^{(k)}} \prod_{k=1}^{M_T} e^{-\Delta\tau \hat{T}^{(k)}} \right]^{L_{\text{Trotter}}} + \mathcal{O}(\Delta\tau^2) \\
&= \sum_C \left(\prod_{k=1}^{M_V} \prod_{\tau=1}^{L_{\text{Trotter}}} \gamma_{k,\tau} \right) e^{-S_0(\{s_{i,\tau}\})} \times \\
&\quad \text{Tr}_F \left\{ \prod_{\tau=1}^{L_{\text{Trotter}}} \left[\prod_{k=1}^{M_V} e^{\sqrt{-\Delta\tau} U_k \eta_{k,\tau} \hat{V}^{(k)}} \prod_{k=1}^{M_I} e^{-\Delta\tau s_{k,\tau} \hat{I}^{(k)}} \prod_{k=1}^{M_T} e^{-\Delta\tau \hat{T}^{(k)}} \right] \right\} + \mathcal{O}(\Delta\tau^2). \quad (16)
\end{aligned}$$

In the above, the trace Tr runs over the bosonic and fermionic degrees of freedom, and Tr_F only over the fermionic Fock space. $S_0(\{s_{i,\tau}\})$ is the action corresponding to the bosonic Hamiltonian, and is only dependent on the bosonic fields so that it can be pulled out of the fermionic trace. We have adopted the shorthand notation $\eta_{k,\tau} \equiv \eta(l_{k,\tau})$ and $\gamma_{k,\tau} \equiv \gamma(l_{k,\tau})$. At this point, and since for a given configuration C we are dealing with a free propagation, we can integrate out the fermions to obtain a determinant:

$$\begin{aligned}
\text{Tr}_F \left\{ \prod_{\tau=1}^{L_{\text{Trotter}}} \left[\prod_{k=1}^{M_V} e^{\sqrt{-\Delta\tau} U_k \eta_{k,\tau} \hat{V}^{(k)}} \prod_{k=1}^{M_I} e^{-\Delta\tau s_{k,\tau} \hat{I}^{(k)}} \prod_{k=1}^{M_T} e^{-\Delta\tau \hat{T}^{(k)}} \right] \right\} = \\
\prod_{s=1}^{N_{\text{fl}}} \left[e^{\sum_{k=1}^{M_V} \sum_{\tau=1}^{L_{\text{Trotter}}} \sqrt{-\Delta\tau} U_k \alpha_{k,s} \eta_{k,\tau}} \right]^{N_{\text{col}}} \times \\
\prod_{s=1}^{N_{\text{fl}}} \left[\det \left(\mathbb{1} + \prod_{\tau=1}^{L_{\text{Trotter}}} \prod_{k=1}^{M_V} e^{\sqrt{-\Delta\tau} U_k \eta_{k,\tau} \mathbf{V}^{(ks)}} \prod_{k=1}^{M_I} e^{-\Delta\tau s_{k,\tau} \mathbf{I}^{(ks)}} \prod_{k=1}^{M_T} e^{-\Delta\tau \mathbf{T}^{(ks)}} \right) \right]^{N_{\text{col}}}, \quad (17)
\end{aligned}$$

where the matrices $\mathbf{T}^{(ks)}$, $\mathbf{V}^{(ks)}$, and $\mathbf{I}^{(ks)}$ define the Hamiltonian [Eq. (2) - (5)]. All in all, the partition function is given by:

$$\begin{aligned}
Z &= \sum_C e^{-S_0(\{s_{i,\tau}\})} \left(\prod_{k=1}^{M_V} \prod_{\tau=1}^{L_{\text{Trotter}}} \gamma_{k,\tau} \right) e^{N_{\text{col}} \sum_{s=1}^{N_{\text{fl}}} \sum_{k=1}^{M_V} \sum_{\tau=1}^{L_{\text{Trotter}}} \sqrt{-\Delta\tau} U_k \alpha_{k,s} \eta_{k,\tau}} \times \prod_{s=1}^{N_{\text{fl}}} \left[\det \left(\mathbb{1} \right. \right. \\
&\quad \left. \left. + \prod_{\tau=1}^{L_{\text{Trotter}}} \prod_{k=1}^{M_V} e^{\sqrt{-\Delta\tau} U_k \eta_{k,\tau} \mathbf{V}^{(ks)}} \prod_{k=1}^{M_I} e^{-\Delta\tau s_{k,\tau} \mathbf{I}^{(ks)}} \prod_{k=1}^{M_T} e^{-\Delta\tau \mathbf{T}^{(ks)}} \right) \right]^{N_{\text{col}}} + \mathcal{O}(\Delta\tau^2) \\
&\equiv \sum_C e^{-S(C)} + \mathcal{O}(\Delta\tau^2). \quad (18)
\end{aligned}$$

In the above, one notices that the weight factorizes in the flavor index. The color index raises the determinant to the power N_{col} . This corresponds to an explicit $\text{SU}(N_{\text{col}})$ symmetry for each configuration. This symmetry is manifest in the fact that the single particle Green functions are color independent, again for each given configuration C .

2.1.2 Observables

In the auxiliary-field QMC approach, the single-particle Green function plays a crucial role. It determines the Monte Carlo dynamics and is used to compute observables. Consider the observable:

$$\langle \hat{O} \rangle = \frac{\text{Tr} [e^{-\beta \hat{H}} \hat{O}]}{\text{Tr} [e^{-\beta \hat{H}}]} = \sum_C P(C) \langle \hat{O} \rangle_C, \quad \text{where } P(C) = \frac{e^{-S(C)}}{\sum_C e^{-S(C)}} \quad (19)$$

and $\langle\langle\hat{O}\rangle\rangle_C$ denotes the observed value of $\hat{O}$ for a given configuration C . For a given configuration C one can use Wick's theorem to compute $\langle\langle\hat{O}\rangle\rangle_C$ from the knowledge of the single-particle Green function:

$$G(x, \sigma, s, \tau | x', \sigma', s', \tau') = \langle\langle \mathcal{T} \hat{c}_{x\sigma s}(\tau) \hat{c}_{x'\sigma's'}^\dagger(\tau') \rangle\rangle_C, \quad (20)$$

where $\mathcal{T}$ denotes the imaginary-time ordering operator. The corresponding equal-time quantity reads

$$G(x, \sigma, s, \tau | x', \sigma', s', \tau) = \langle\langle \hat{c}_{x\sigma s}(\tau) \hat{c}_{x'\sigma's'}^\dagger(\tau) \rangle\rangle_C. \quad (21)$$

Since, for a given HS field, translation invariance in imaginary-time is broken, the Green function has an explicit τ and τ' dependence. On the other hand it is diagonal in the flavor index, and independent of the color index. The latter reflects the explicit SU(N) color symmetry present at the level of individual HS configurations. As an example, one can show that the equal-time Green function at $\tau = 0$ reads [6]:

$$G(x, \sigma, s, 0 | x', \sigma, s, 0) = \left(\mathbb{1} + \prod_{\tau=1}^{L_{\text{Trotter}}} \mathbf{B}_{\tau}^{(s)} \right)_{x, x'}^{-1} \quad (22)$$

with

$$\mathbf{B}_{\tau}^{(s)} = \prod_{k=1}^{M_V} e^{\sqrt{-\Delta\tau} U_k \eta_{k,\tau} V^{(ks)}} \prod_{k=1}^{M_I} e^{-\Delta\tau s_{k,\tau} I^{(ks)}} \prod_{k=1}^{M_T} e^{-\Delta\tau T^{(ks)}}. \quad (23)$$

To compute equal-time, as well as time-displaced observables, one can make use of Wick's theorem. A convenient formulation of this theorem for QMC simulations reads:

$$\begin{aligned} & \langle\langle \mathcal{T} \hat{c}_{\underline{x}_1}^\dagger(\tau_1) \hat{c}_{\underline{x}'_1}(\tau'_1) \cdots \hat{c}_{\underline{x}_n}^\dagger(\tau_n) \hat{c}_{\underline{x}'_n}(\tau'_n) \rangle\rangle_C = \\ & \det \begin{bmatrix} \langle\langle \mathcal{T} \hat{c}_{\underline{x}_1}^\dagger(\tau_1) \hat{c}_{\underline{x}'_1}(\tau'_1) \rangle\rangle_C & \langle\langle \mathcal{T} \hat{c}_{\underline{x}_1}^\dagger(\tau_1) \hat{c}_{\underline{x}'_2}(\tau'_2) \rangle\rangle_C & \cdots & \langle\langle \mathcal{T} \hat{c}_{\underline{x}_1}^\dagger(\tau_1) \hat{c}_{\underline{x}'_n}(\tau'_n) \rangle\rangle_C \\ \langle\langle \mathcal{T} \hat{c}_{\underline{x}_2}^\dagger(\tau_2) \hat{c}_{\underline{x}'_1}(\tau'_1) \rangle\rangle_C & \langle\langle \mathcal{T} \hat{c}_{\underline{x}_2}^\dagger(\tau_2) \hat{c}_{\underline{x}'_2}(\tau'_2) \rangle\rangle_C & \cdots & \langle\langle \mathcal{T} \hat{c}_{\underline{x}_2}^\dagger(\tau_2) \hat{c}_{\underline{x}'_n}(\tau'_n) \rangle\rangle_C \\ \vdots & \vdots & \ddots & \vdots \\ \langle\langle \mathcal{T} \hat{c}_{\underline{x}_n}^\dagger(\tau_n) \hat{c}_{\underline{x}'_1}(\tau'_1) \rangle\rangle_C & \langle\langle \mathcal{T} \hat{c}_{\underline{x}_n}^\dagger(\tau_n) \hat{c}_{\underline{x}'_2}(\tau'_2) \rangle\rangle_C & \cdots & \langle\langle \mathcal{T} \hat{c}_{\underline{x}_n}^\dagger(\tau_n) \hat{c}_{\underline{x}'_n}(\tau'_n) \rangle\rangle_C \end{bmatrix}. \end{aligned} \quad (24)$$

Here, we have defined the super-index $\underline{x} = \{x, \sigma, s\}$.

Wick's theorem can be also used to express a reduced density matrix, i.e., the density matrix for a subsystem, in terms of its correlations [113]. Within the framework of Auxiliary-Field QMC, this allows to express a reduced density matrix $\hat{\rho}_A$ for a subsystem A as [60]

$$\hat{\rho}_A = \sum_C P(C) \det(\mathbb{1} - G_A(\tau_0; C)) e^{-c_{\underline{x}, \underline{x}'}^{(A)} H_{\underline{x}, \underline{x}'}^{(A)}}, \quad H^{(A)} \equiv \ln \left\{ \left[(G_A(\tau_0; C))^T \right]^{-1} - \mathbb{1} \right\}, \quad (25)$$

where $G_A(\tau_0; C)$ is the equal-time Green's function matrix restricted on the subsystem A and at a given time-slice τ_0 . In Eq. (25) an implicit summation over repeated indexes $\underline{x}, \underline{x}' \in A$ is assumed. Interestingly, Eq. (25) holds also when A is the entire system: in this case, it provides an alternative expression for the density matrix, or the (normalized) partition function, as a superposition of Gaussian operators. Eq. (25) is the starting point for computing the entanglement Hamiltonian [64] and the Rényi entropies [60, 62, 63]. A short review on various computational approaches to quantum entanglement in interacting fermionic models can be found in Ref. [66]. ALF provides predefined observables to compute the second Rényi entropy and its associated mutual information, see Sec. 8.4.13.

In Sec. 8.4 we describe the equal-time and time-displaced correlation functions that come predefined in ALF. Using the above formulation of Wick's theorem, arbitrary correlation functions can be computed (see Appendix A). We note, however, that the program is limited to the calculation of observables that contain only two different imaginary times.

2.1.3 Reweighting and the sign problem

In general, the action $S(C)$ will be complex, thereby inhibiting a direct Monte Carlo sampling of $P(C)$. This leads to the infamous sign problem. The sign problem is formulation dependent and as noted above, much progress has been made at understanding the class of models that can be formulated without encountering this problem [74–77]. When the average sign is not too small, we can nevertheless compute observables within a reweighting scheme. Here we adopt the following scheme. First note that the partition function is real such that:

$$Z = \sum_C e^{-S(C)} = \sum_C \overline{e^{-S(C)}} = \sum_C \text{Re}[e^{-S(C)}]. \quad (26)$$

Thereby³ and with the definition

$$\text{sgn}(C) = \frac{\text{Re}[e^{-S(C)}]}{|\text{Re}[e^{-S(C)}]|}, \quad (27)$$

the computation of the observable [Eq. (19)] is re-expressed as follows:

$$\begin{aligned} \langle \hat{O} \rangle &= \frac{\sum_C e^{-S(C)} \langle \hat{O} \rangle_C}{\sum_C e^{-S(C)}} \\ &= \frac{\sum_C \text{Re}[e^{-S(C)}] \frac{e^{-S(C)}}{\text{Re}[e^{-S(C)}]} \langle \hat{O} \rangle_C}{\sum_C \text{Re}[e^{-S(C)}]} \\ &= \frac{\left\{ \sum_C |\text{Re}[e^{-S(C)}]| \text{sgn}(C) \frac{e^{-S(C)}}{\text{Re}[e^{-S(C)}]} \langle \hat{O} \rangle_C \right\} / \sum_C |\text{Re}[e^{-S(C)}]|}{\left\{ \sum_C |\text{Re}[e^{-S(C)}]| \text{sgn}(C) \right\} / \sum_C |\text{Re}[e^{-S(C)}]|} \\ &= \frac{\left\langle \text{sgn} \frac{e^{-S}}{\text{Re}[e^{-S}]} \langle \hat{O} \rangle \right\rangle_{\bar{P}}}{\langle \text{sgn} \rangle_{\bar{P}}}. \end{aligned} \quad (28)$$

The average sign is

$$\langle \text{sgn} \rangle_{\bar{P}} = \frac{\sum_C |\text{Re}[e^{-S(C)}]| \text{sgn}(C)}{\sum_C |\text{Re}[e^{-S(C)}]|}, \quad (29)$$

and we have $\langle \text{sgn} \rangle_{\bar{P}} \in \mathbb{R}$ per definition. The Monte Carlo simulation samples the probability distribution

$$\bar{P}(C) = \frac{|\text{Re}[e^{-S(C)}]|}{\sum_C |\text{Re}[e^{-S(C)}]|}. \quad (30)$$

such that the nominator and denominator of Eq. (28) can be computed.

Notice that, for the Langevin updating scheme with variable Langevin time step, a straightforward generalization of the equations above is used, see Sec. 2.3.3.

The negative sign problem is still an issue because the average sign is a ratio of two partition functions and one can argue that

$$\langle \text{sgn} \rangle_{\bar{P}} \propto e^{-\Delta N \beta}, \quad (31)$$

where Δ is an intensive positive quantity and $N\beta$ denotes the Euclidean volume. In a Monte Carlo simulation the error scales as $1/\sqrt{T_{\text{CPU}}}$ where T_{CPU} corresponds to the computational

³The attentive reader will have noticed that for arbitrary Trotter decompositions, the imaginary time propagator is not necessarily Hermitian. Thereby, the above equation is correct only up to corrections stemming from the controlled Trotter systematic error.

time. Since the error on the average sign has to be much smaller than the average sign itself, one sees that:

$$T_{\text{CPU}} \gg e^{2\Delta N\beta}. \quad (32)$$

Two comments are in order. First, the presence of a sign problem invariably leads to an exponential increase of CPU time as a function of the Euclidean volume. And second, Δ is formulation dependent. For instance, at finite doping, the SU(2) invariant formulation of the Hubbard model presented in Sec. 9.1 has a much more severe sign problem than the formulation (presented in the same section) where the HS field couples to the z -component of the magnetization. Optimization schemes minimize Δ have been put forward in [114, 115].

2.2 Sequential updating schemes

The program allows for different types of updating schemes, which are described below and summarized in Tab. 1. Except for Langevin dynamics, for a given configuration C , we propose a new one, C' , with a given probability $T_0(C \rightarrow C')$ and accept it according to the Metropolis-Hastings acceptance-rejection probability,

$$P(C \rightarrow C') = \min\left(1, \frac{T_0(C' \rightarrow C)W(C')}{T_0(C \rightarrow C')W(C)}\right), \quad (33)$$

to guarantee the stationarity condition. Here, $W(C) = |\text{Re}[e^{-S(C)}]|$.

Predicting how efficient a certain Monte Carlo update scheme will turn out to be for a given simulation is very hard, so one must typically resort to testing to find out which option produces best results. Methods to optimize the acceptance of global moves include Hybrid Monte Carlo [80] as well as self-learning techniques [116, 117]. Langevin dynamics stands apart, and as we will see does not depend on the Metropolis-Hastings acceptance-rejection scheme.

Updating schemes	Method	Section
Sequential space-time discrete fields	Metropolis, spin flip	2.2.1, 2.2.1.1
Sequential space-time continuous fields	Metropolis, box move	2.2.2.1
Sequential space-time continuous fields	Metropolis assisted Langevin	2.2.2.2
Sequential time, global space	Metropolis, User defined	2.2.3.1
Sequential time, global space	Metropolis assisted Langevin	2.2.3.2
Sequential space-time, global space	Combinations of moves	2.2.4
Global space-time, discrete/continuous	User defined, Metropolis	2.3.1.1
Global space-time, continuous	User defined, Metropolis assisted Langevin	2.3.1.2
Global space-time, continuous	Parallel tempering	2.3.2
Global space-time, continuous	Langevin dynamics	2.3.3
Global space-time, continuous	Hybrid Monte Carlo	2.3.4

Table 1: Summary of possible updating schemes. Aside from the parallel tempering schemes, the user can test and compare all these schemes using our implementation of the Hubbard model. Here we recommend using a one-dimensional lattice with open boundary conditions and a Hubbard-Stratonovitch decoupling that couples to the magnetization. As mentioned in Sec. 2.3.3, this choice guarantees the absence of zero in the fermion determinant such that all updating schemes will work out of the box and can be compared.

2.2.1 Space-time sequential single spin flips: discrete fields

The program adopts per default a sequential, single spin-flip strategy. It will visit sequentially each HS field in the space-time operator list and propose a spin flip. Consider the Ising spin $s_{i,\tau}$. By default (`Propose_S0=.false.`), we will flip it with probability 1, such that for this local move the proposal matrix is symmetric. If we are considering the HS field $l_{i,\tau}$ we will propose with probability 1/3 one of the other three possible fields. For a continuous field, we modify it with a box distribution of width `Amplitude` centered around the origin. The default value of `Amplitude` is set to unity. These updating rules are defined in the `Fields_mod.F90` module (see Sec. 5.2). Again, for these local moves, the proposal matrix is symmetric. Hence in all cases we will accept or reject the move according to

$$P(C \rightarrow C') = \min\left(1, \frac{W(C')}{W(C)}\right). \quad (34)$$

This default updating scheme can be overruled by, e.g., setting `Global_tau_moves` to `.true.` and not setting `Nt_sequential_start` and `Nt_sequential_end` (see Sec. 5.7.1). It is also worth noting that this type of sequential spin-flip updating does not satisfy detailed balance, but rather the more fundamental stationarity condition [82].

2.2.1.1 Sampling of e^{-S_0}

The package can also propose single spin-flip updates according to a non-vanishing free bosonic action $S_0(C)$. This sampling scheme is used if the logical variable `Propose_S0` is set to `.true.`. As mentioned previously, this option only holds for Ising variables.

Consider an Ising spin at space-time i, τ in the configuration C . Flipping this spin generates the configuration C' and we propose this move according to

$$T_0(C \rightarrow C') = \frac{e^{-S_0(C')}}{e^{-S_0(C')} + e^{-S_0(C)}} = 1 - \frac{1}{1 + e^{-S_0(C')}/e^{-S_0(C)}}. \quad (35)$$

Note that the function `S0` in the `Hamiltonian_Hubbard_include.h` module computes precisely the ratio $e^{-S_0(C')}/e^{-S_0(C)}$, therefore $T_0(C \rightarrow C')$ is obtained without any additional calculation. The proposed move is accepted with the probability:

$$P(C \rightarrow C') = \min\left(1, \frac{e^{-S_0(C)}W(C')}{e^{-S_0(C')}W(C)}\right). \quad (36)$$

Note that, as can be seen from Eq. (18), the bare action $S_0(C)$ determining the dynamics of the bosonic configuration in the absence of coupling to the fermions does not enter the Metropolis acceptance-rejection step.

As mentioned above, this sequential updating scheme is space-time is the default and requires the following settings in the

```
&VAR_QMC
.....
Sequential          = .T.  ! Flag for sequential moves
Propose_S0          = .F.  ! Whether to propose moves according to e^{-S0}
.....
/
```

2.2.2 Space-time sequential single spin flips: Continuous fields

If the field is a continuous real variable, `type=3`, (See Sec. 5.2) then the program provides two types of updating schemes.

2.2.2.1 Box distribution

The first one is the default and consists in proposing a new value for the field according to a box distribution of width `Amplitude` centered around the origin. This updating scheme is symmetric, and the move is accepted with probability $P(C \rightarrow C') = \min\left(1, \frac{W(C')}{W(C)}\right)$. This is the default updating scheme for continuous fields.

2.2.2.2 Metropolis assisted Langevin algorithm (MALA)

This updating scheme is based on the Langevin equation that we will discuss in Sec. 2.3.3. For a given field $s_{n,\tau}$ (See. Eq. 23), we propose a new value of the field according to a single step of the discretized Langevin equation just for this field:

$$s'_{n,\tau} = s_{n,\tau} - \frac{\partial S}{\partial s_{n,\tau}} \delta t + \sqrt{2\delta t} \eta \quad (37)$$

where η is a Gaussian random variable with zero mean and unit variance, and δt is the time step of the Langevin dynamics. This updating scheme is not symmetric, and the move is accepted with probability the Metropolis-Hastings acceptance-rejection probability,

$$P(C \rightarrow C') = \min\left(1, \frac{T_0(C' \rightarrow C)W(C')}{T_0(C \rightarrow C')W(C)}\right), \quad (38)$$

where $T_0(C \rightarrow C')$ is the probability to propose the move $C \rightarrow C'$ and reads:

$$T_0(C \rightarrow C') = \frac{1}{\sqrt{2\pi\delta t}} e^{-\frac{(s'_{n,\tau} - s_{n,\tau} + \frac{\partial S}{\partial s_{n,\tau}} \delta t)^2}{2\delta t}}. \quad (39)$$

The user can control the time step δt by setting the variable `Delta_t_MALA_sequential` in the input file. The maximum value of the force $\frac{\partial S}{\partial s_{n,\tau}}$ is controlled by the variable `Max_Force_MALA_sequential`, which is used to avoid proposing moves with very low acceptance. The force is then defined as $\text{sign}(\frac{\partial S}{\partial s_{n,\tau}}) \min(|\frac{\partial S}{\partial s_{n,\tau}}|, \text{Max_Force_MALA_sequential})$.

```
&VAR_QMC
.....
Sequential          = .T.          ! Flag for sequential moves
Propose_S0           = .F.          ! Whether to propose moves according to e^{-S0}
Amplitude            = 0.5D0        ! Amplitude of the box distribution
Sequential_MALA      = .true.       ! MALA sequential moves for continuous fields
Max_Force_MALA_sequential = 1.0     ! Maximum value of the force
Delta_t_MALA_sequential = 0.1      ! Time step for the MALA
.....
/
```

We note that we have two routines to compute the force of the bosonic part of the action, S_0 . For the Langevin dynamics described in Sec. 2.3.3 the force for each field has to be computed. This is done in the routine `Ham_Langevin_HMC_S0` that returns the two-dimension array of forces. For the MALA sequential moves, we only need the force for a single field, which is computed in the routine `Ham_Langevin_HMC_S0_single`. If this routine is not provided by the user, the code will call `Ham_Langevin_HMC_S0` and extract the force for the given field. This is computationally expensive, such that a warning is printed on the standard output at execution. The following routine must be provided by the user before this option can be used:

```
Subroutine Ham_Langevin_HMC_S0(Forces_0)
  Real (Kind=Kind(0.d0)), Intent(inout), allocatable :: Forces_0(:, :)
  ! Forces_0(n,nt) = \frac{\partial S_0}{\partial s_{n,\tau}}.
  ! Here n is the index of the field in the operator list and nt is the time slice.
End Subroutine Ham_Langevin_HMC_S0
```

2.2.3 Time sequential, global in space updates.

2.2.3.1 User-defined global moves on a single time slice: Metropolis

This option allows one to carry out user-defined global moves on a single time slice. This option is enabled by setting the logical variable `Global_tau_moves` to `.true.`. Recall that the propagation over a time step $\Delta\tau$ (see Eq. 23) can be written as:

$$e^{-V_{M_I+M_V}(s_{M_I+M_V,\tau})} \dots e^{-V_1(s_1,\tau)} \prod_{k=1}^{M_T} e^{-\Delta\tau T^{(k)}}, \quad (40)$$

where $e^{-V_n(s_n)}$ denotes one element of the operator list containing the HS fields. One can provide an interval of indices, `[Nt_sequential_start, Nt_sequential_end]`, in which the operators will be updated sequentially. Setting `Nt_sequential_start = 1` and `Nt_sequential_end =  $M_I + M_V$` reproduces the sequential single spin flip strategy of the above section.

The variable `N_Global_tau` sets the number of global moves carried out on each time slice `ntau`. Each global move is generated in the routine `Global_move_tau`, which is provided by the user in the Hamiltonian file. In order to define this move, one specifies the following variables:

- `Flip_length`: An integer stipulating the number of spins to be flipped.
- `Flip_list(1:Flip_length)`: Integer array containing the indices of the operators to be flipped.
- `Flip_value(1:Flip_length)`: `Flip_value(n)` is an integer containing the new value of the HS field for the operator `Flip_list(n)`.
- `T0_Proposal_ratio`: Real number containing the quotient

$$\frac{T_0(C' \rightarrow C)}{T_0(C \rightarrow C')}, \quad (41)$$

where C' denotes the new configuration obtained by flipping the spins specified in the `Flip_list` array. Since we allow for a stochastic generation of the global move, it may very well be that no change is proposed. In this case, `T0_Proposal_ratio` takes the value 0 upon exit of the routine `Global_move_tau` and no update is carried out.

- `S0_ratio`: Real number containing the ratio $e^{-S_0(C')}/e^{-S_0(C)}$.

Specifically, the routine `Global_move_tau` must be provided by the user and has the following structure:

```
Subroutine Global_move_tau(T0_Proposal_ratio, S0_ratio, &
&
& Flip_list, Flip_length, Flip_value, ntau)

Real (Kind = Kind(0.d0)), INTENT(OUT) :: T0_Proposal_ratio, S0_ratio
Integer, INTENT(OUT) :: Flip_list(:)
Complex (Kind = Kind(0.d0)), INTENT(OUT) :: Flip_value(:)
Integer, INTENT(OUT) :: Flip_length
Integer, INTENT(IN) :: ntau

end Subroutine Global_move_tau
```

The subroutine `Override_global_tau_sampling_parameters` can be used to overrule the default values of the parameters controlling the global moves on a single time slice, which are set in the input file.

```
Subroutine Override_global_tau_sampling_parameters(Nt_sequential_start,Nt_sequential_end,&
& N_Global_tau, N_Global_tau_MALA)
  Integer, Intent(OUT) :: N_Global_tau, Nt_sequential_start, Nt_sequential_end
end Subroutine Override_global_tau_sampling_parameters
```

As an example, one can run the Hubbard Hamiltonian with the parameters:

```
&VAR_QMC
.....
Sequential      = .T.      ! Flag for sequential moves
Global_tau_moves = .T.      ! Global moves on a time slice
N_Global_tau = 1           ! Number of global moves will be set
                        ! to the length of OP_V by the override routine
.....
```

Here we randomly chose a vertex and update the field.

2.2.3.2 User defined global moves on a single time slice: MALA

This option will carry out a global move on a single time slice using the Metropolis assisted Langevin algorithm. To use this option, the

```
&VAR_QMC
.....
Sequential      = .T.      ! Flag for sequential moves
Global_tau_MALA_moves = .T. ! Global MALA moves on a single time slice
N_Global_tau_MALA = 1       ! Number of global MALA moves will be set
Delta_t_MALA_global_tau = 0.1 ! Time step for the MALA global-tau move
Max_Force_MALA_global_tau = 1.0 ! Maximum force for the MALA global-tau move
.....
```

The user can override the number of `N_Global_tau_MALA` by using the aforementioned `Override_global_tau_sampling_parameters` routine. The routine:

```
subroutine Global_MALA_move_tau(Flip_list, Flip_length,ntau)
  Integer, Intent(OUT) :: Flip_list(:)
  Integer, Intent(OUT) :: Flip_length
  Integer, Intent(IN) :: ntau
  ! Fields nsigma%f(Flip_list(1),ntau) ...
  ! nsigma%f(Flip_list(Flip_Length),ntau) will be updated.
end Subroutine Global_MALA_move_tau
```

Allows to specify the list of field to be updated in the MALA move. By default, all the field on the time slice are visited and a warning is issued on the standard output.

2.2.4 Combining various sequential updating schemes.

The user can combine the various sequential updating schemes described above. For example one can carry out a number of sequential single spin-flip sweeps, followed by a number of global ones. The range of the sequential sweeps are specified by the variables `Nt_sequential_start` and `Nt_sequential_end`, while the number of global moves is set by `N_Global_tau`. For example, the parameter file:

```
&VAR_QMC
.....
Sequential      = .T. ! Sequential moves
Propose_S0      = .F. ! Proposes single spin flip moves with probability exp(-S0)
Nt_sequential_start = 1 ! Starting index of the sequential moves on a time slice
Nt_sequential_end   = 3 ! Ending index of the sequential moves on a time slice
Sequential_MALA    = .T. ! MALA sequential moves for continuous fields
```

```

Max_Force_MALA_sequential = 1.0 ! Max force for the MALA sequential moves
Delta_t_MALA_sequential   = 0.1 ! Time step for the MALA sequential moves
Global_tau_moves = .true.    ! Global moves on a time slice
N_Global_tau = 6             ! Number of global moves on a time slice
.....

```

Will carry out sequential single spin-flip moves on the first three operators of the operator list using the MALA updating scheme 2.2.2.2, followed by `N_Global_tau=6` global moves on the time slice.

2.3 Global updates in time and space

2.3.1 User specified global moves in space and time

2.3.1.1 Metropolis

The code allows for global updates as well. The user must then provide two additional functions `Global_move` and `Get_Delta_S0_global( Nsigma_old )`.

The subroutine `Global_move(T0_Proposal_ratio,nsigma_old,size_clust)` proposes a global move. Its single input is the variable `nsigma_old` of type `Field` (see Section 5.2) that contains the full configuration C stored in `nsigma_old%f(M_V + M_I, Ltrot)`. On output, the new configuration C' , determined by the user, is stored in the two-dimensional array `nsigma`, which is a global variable declared in the Hamiltonian module. Like for the global move in space (Sec. 2.2.3.1), `T0_Proposal_ratio` contains the proposal ratio $\frac{T_0(C' \rightarrow C)}{T_0(C \rightarrow C')}$. Since we allow for a stochastic generation of the global move, it may very well be that no change is proposed. In this case, `T0_Proposal_ratio` takes the value 0 upon exit, and `nsigma=nsigma_old`. The real-valued `size_clust` gives the size of the proposed move (e.g., $\frac{\text{Number of flipped spins}}{\text{Total number of spins}}$). This is used to calculate the average sizes of proposed and accepted moves, which are printed in the info file. The variable `size_clust` is not necessary for the simulation, but may help the user to estimate the effectiveness of the global update.

In order to compute the acceptance-rejection ratio, the user must also provide a function `Get_Delta_S0_global(nsigma_old)` that computes the difference $\Delta S_0 = S_0(C) - S_0(C')$. Again, the configuration C' is given by the field `nsigma`.

The variable `N_Global` determines the number of global updates performed per sweep. Note that global updates are expensive, since they require a complete recalculation of the weight.

Practically one will combine global update with local ones. In the implementation of the Hubbard model, the parameter file:

```

&VAR_QMC
.....
Sequential      = .T.      ! Flag for sequential moves
Global_moves    = .T.      ! Flag for global moves in space and time
N_Global        = 6        ! Number of global moves in space and time
.....
/

```

will carry out 6 global moves in space and time at the end of each sweep. A possible choice for the global move is to randomly choose a lattice point and change the sign of the field at this point for all time slices. The routine that will do this global update reads:

```

Subroutine Global_move(T0_Proposal_ratio, nsigma_old, size_clust)
  Real (Kind = Kind(0.d0)), INTENT(OUT) :: T0_Proposal_ratio, size_clust
  Type(Field), INTENT(IN) :: nsigma_old

  size_clust = Ltrot

```

```

n = nranf(Size(Op_V,1))
nsigma%f(n,:) = - nsigma_old%f(n,:)
T0_Proposal_ratio = 1
end Subroutine Global_move

```

2.3.1.2 MALA

Generically, user defined global moves have a poor acceptance rate. For continuous variables, one can improve this by adopting the Metropolis assisted Langevin algorithm (MALA). Since the move is user defined, one has to provide a function that tags the fields that be updated during the global MALA move. This function is specified in the Hamiltonian, and takes the following form:

```

Subroutine Global_MALA_move(Flip_list)
  Integer                                , INTENT(OUT) :: Flip_list(size(Op_V,1),Ltrot)
  ....
  Flip_list(n,nt) = 1  ! Field will be updated
  Flip_list(n,nt) = 0  ! Field will not be updated
  ....
end Subroutine Global_MALA_move

```

The default implementation sets all entries of the Flip_list to unity.

To activate this type of move, one needs to modify the following in the parameter file:

```

&VAR_QMC
.....
Sequential                = .T.  ! Flag for sequential moves
.....
Global_MALA_moves          = .T.  ! Allows for global MALA updates
N_Global_MALA_sweeps       = 1    ! Number of global MALA sweeps
Delta_t_MALA_global        = 0.1  ! Time step for global MALA updates
Max_Force_MALA_global      = 5.d0 ! Maximum force for global MALA updates
.....
/

```

2.3.2 Parallel tempering

Exchange Monte Carlo [118], or parallel tempering [119], is a possible route to overcome sampling issues in parts of the parameter space. Let h be a parameter which one can vary without altering the configuration space $\{C\}$ and let us assume that for some values of h one encounters sampling problems. For example, in the realm of spin glasses, h could correspond to the inverse temperature. Here at high temperatures the phase space is easily sampled, but at low temperatures simulations get stuck in local minima. For quantum systems, h could trigger a quantum phase transition where sampling issues are encountered, for example, in the ordered phase and not in the disordered one. As its name suggests, parallel tempering carries out in parallel simulations at consecutive values of h : $h_1, h_2, \dots, h_n$, with $h_1 < h_2 < \dots < h_n$. One will sample the extended ensemble:

$$P([h_1, C_1], [h_2, C_2], \dots, [h_n, C_n]) = \frac{W(h_1, C_1)W(h_2, C_2) \cdots W(h_n, C_n)}{\sum_{C_1, C_2, \dots, C_n} W(h_1, C_1)W(h_2, C_2) \cdots W(h_n, C_n)}, \quad (42)$$

where $W(h, C)$ corresponds to the weight for a given value of h and configuration C . Clearly, one can sample $P([h_1, C_1], [h_2, C_2], \dots, [h_n, C_n])$ by carrying out n independent runs. However, parallel tempering includes the following exchange step:

$$[h_1, C_1], \dots, [h_i, C_i], [h_{i+1}, C_{i+1}], \dots, [h_n, C_n] \rightarrow [h_1, C_1], \dots, [h_i, C_{i+1}], [h_{i+1}, C_i], \dots, [h_n, C_n] \quad (43)$$

which, for a symmetric proposal matrix, will be accepted with probability

$$\min\left(1, \frac{W(h_i, C_{i+1})W(h_{i+1}, C_i)}{W(h_i, C_i)W(h_{i+1}, C_{i+1})}\right). \quad (44)$$

In this way a configuration can meander in parameter space h and explore regions where ergodicity is not an issue. In the context of spin-glasses, a low temperature configuration, stuck in a local minima, can heat up, overcome the potential barrier and then cool down again.

A judicious choice of the values h_i is important to obtain a good acceptance rate for the exchange step. With $W(h, C) = e^{-S(h, C)}$, the distribution of the action S reads:

$$\mathcal{P}(h, S) = \sum_C P(h, C) \delta(S(h, C) - S). \quad (45)$$

A given exchange step can only be accepted if the distributions $\mathcal{P}(h, S)$ and $\mathcal{P}(h + \Delta h, S)$ overlap. For $\langle S \rangle_h < \langle S \rangle_{h+\Delta h}$ one can formulate this requirement as:

$$\langle S \rangle_h + \langle \Delta S \rangle_h \simeq \langle S \rangle_{h+\Delta h} - \langle \Delta S \rangle_{h+\Delta h}, \text{ with } \langle \Delta S \rangle_h = \sqrt{\langle (S - \langle S \rangle_h)^2 \rangle_h}. \quad (46)$$

Assuming $\langle \Delta S \rangle_{h+\Delta h} \simeq \langle \Delta S \rangle_h$ and expanding in Δh one obtains:

$$\Delta h \simeq \frac{2\langle \Delta S \rangle_h}{\partial \langle S \rangle_h / \partial h}. \quad (47)$$

The above equation becomes transparent for classical systems with $S(h, C) = hH(C)$. In this case, the above equation reads:

$$\Delta h \simeq 2h \frac{\sqrt{c}}{c + h\langle H \rangle_h}, \text{ with } c = h^2 \langle (H - \langle H \rangle_h)^2 \rangle_h. \quad (48)$$

Several comments are in order:

- i) Let us identify h with the inverse temperature such that c corresponds to the specific heat. This quantity is extensive, as well as the energy, such that $\Delta h \simeq 1/\sqrt{N}$ where N is the system size.
- ii) Near a phase transition the specific heat can diverge, and h must be chosen with particular care.
- iii) Since the action is formulation dependent, also the acceptance rate of the exchange move equally depend upon the formulation.

The quantum Monte Carlo code in the ALF project carries out parallel-tempering runs when the script `configure.sh` is called with the argument `Tempering` before compilation, see Sec. 6.2.

2.3.3 Langevin dynamics

For models that include continuous real fields $\mathbf{s} \equiv \{s_{k,\tau}\}$ there is the option of using Langevin dynamics for the updating scheme, by setting the variable `Langevin` to `.true..` This corresponds to a stochastic differential equation for the fields. They acquire a discrete Langevin time t_l with step width δt_l and satisfy the stochastic differential equation

$$\mathbf{s}(t_l + \delta t_l) = \mathbf{s}(t_l) - Q \frac{\partial S(\mathbf{s}(t_l))}{\partial \mathbf{s}(t_l)} \delta t_l + \sqrt{2\delta t_l Q} \boldsymbol{\eta}(t_l). \quad (49)$$

Here, $\eta(t_l)$ are independent Gaussian stochastic variables satisfying:

$$\langle \eta_{k,\tau}(t_l) \rangle_\eta = 0 \quad \text{and} \quad \langle \eta_{k,\tau}(t_l) \eta_{k',\tau'}(t'_l) \rangle_\eta = \delta_{k,k'} \delta_{\tau,\tau'} \delta_{t_l,t'_l}, \quad (50)$$

$S(s(t_l))$ is an arbitrary real action and Q is an arbitrary positive definite matrix. By default Q is equal to the identity matrix, but a proper choice can help accelerate the update scheme, as we discuss below. We refer the reader to Ref. [120] for an in-depth introduction to stochastic differential equations. To see that the above indeed produces the desired probability distribution in the long Langevin time limit, we can transform the Langevin equation into the corresponding Fokker-Plank one. Let $P(s, t_l)$ be the distribution of fields at Langevin time t_l . Then,

$$P(s, t_l + \delta t_l) = \int Ds' P(s', t_l) \left\langle \delta \left(s - \left[s' - Q \frac{\partial S(s')}{\partial s'} \delta t_l + \sqrt{2\delta t_l Q} \eta(t_l) \right] \right) \right\rangle_\eta, \quad (51)$$

where δ corresponds to the $L_{\text{trotter}} M_I$ dimensional Dirac δ -function. Taylor expanding up to order δt_l and averaging over the stochastic variable yields:

$$P(s, t_l + \delta t_l) = \int Ds' P(s', t_l) \left(\delta(s' - s) - \frac{\partial}{\partial s'} \delta(s' - s) Q \frac{\partial S(s')}{\partial s'} \delta t_l + \frac{\partial}{\partial s'} Q \frac{\partial}{\partial s'} \delta(s' - s) \delta t_l \right) + \mathcal{O}(\delta t_l^2). \quad (52)$$

Partial integration and taking the limit of infinitesimal time steps gives the Fokker-Plank equation

$$\frac{\partial}{\partial t_l} P(s, t_l) = \frac{\partial}{\partial s} \left(P(s, t_l) Q \frac{\partial S(s)}{\partial s} + Q \frac{\partial P(s, t_l)}{\partial s} \right). \quad (53)$$

The stationary, $\frac{\partial}{\partial t_l} P(s, t_l) = 0$, normalizable, solution to the above equation corresponds to the desired probability distribution:

$$P(s) = \frac{e^{-S(s)}}{\int Ds e^{-S(s)}}. \quad (54)$$

Taking into account a potential negative sign problem, the action for our general model reads:

$$\bar{S}(C) = -\ln |\text{Re} \{ e^{-S(C)} \}| \quad (55)$$

where $S(C)$ is defined in Eq. (18). Hence,

$$\frac{\partial \bar{S}(C)}{\partial s_{k,\tau}} = \frac{1}{\text{Re} \{ e^{i\phi(C)} \}} \text{Re} \left\{ e^{i\phi(C)} \frac{\partial S(C)}{\partial s_{k,\tau}} \right\} \quad (56)$$

with

$$e^{i\phi(C)} = \frac{e^{-S(C)}}{|e^{-S(C)}|} \quad (57)$$

corresponding to the variable PHASE in the ALF-package.

Therefore, to formulate the Langevin dynamics we need to estimate the forces:

$$\frac{\partial S(C)}{\partial s_{k,\tau}} = \frac{\partial S_0(C)}{\partial s_{k,\tau}} + \frac{\partial S^F(C)}{\partial s_{k,\tau}}, \quad (58)$$

with the fermionic part of the action being

$$S^F(C) = -\ln \left\{ \left(\prod_{k=1}^{M_V} \prod_{\tau=1}^{L_{\text{Trotter}}} \gamma_{k,\tau} \right) e^{\sum_{s=1}^{N_{\text{fl}}} \sum_{k=1}^{M_V} \sum_{\tau=1}^{L_{\text{Trotter}}} \sqrt{-\Delta\tau U_k} \alpha_{k,s} \eta_{k,\tau}} \right. \\ \left. \times \prod_{s=1}^{N_{\text{fl}}} \left[\det \left(\mathbb{1} + \prod_{\tau=1}^{L_{\text{Trotter}}} \prod_{k=1}^{M_V} e^{\sqrt{-\Delta\tau U_k} \eta_{k,\tau} V^{(ks)}} \prod_{k=1}^{M_I} e^{-\Delta\tau s_{k,\tau} I^{(ks)}} \prod_{k=1}^{M_T} e^{-\Delta\tau T^{(ks)}} \right) \right]^{N_{\text{col}}} \right\}. \quad (59)$$

The forces must be bounded for Langevin dynamics to work well. If this condition is violated the results produced by the code are *not reliable*.

One possible source of divergence is the determinant in the fermionic action. Zeros lead to unbounded forces and, in order to mitigate this problem, we adopt a variable time step. The user provides an upper bound to the fermion force, `Max_Force` and, if the maximal force in a configuration, `Max_Force_Conf`, is larger than `Max_Force`, then the time step is rescaled as

$$\tilde{\delta t}_l = \frac{\text{Max_Force}}{\text{Max_Force_Conf}} * \delta t_l. \quad (60)$$

With the adaptive time step, averages are computed as:

$$\langle \hat{O} \rangle = \frac{\sum_n (\tilde{\delta t}_l)_n \text{sgn}(C_n) \frac{e^{-S(C_n)}}{\text{Re}[e^{-S(C_n)}]} \langle \hat{O} \rangle_{(C_n)}}{\sum_n (\tilde{\delta t}_l)_n \text{sgn}(C_n)}. \quad (61)$$

where $\text{sgn}(C_n)$ is defined in Eq. (27). In this context the adaptive time step corresponds to the variable `Mc_step_weight` required for the measurement routines (see Sec. 5.4).

A possible way to reduce autocorrelation times is to employ Fourier acceleration [121,122]. As we see from Eq. (54), the choice of the matrix Q does not alter the probability distribution obtained from the Langevin equation. The main idea of Fourier acceleration is to exploit this freedom and use Q to enhance (reduce) the Langevin time step δt_l of slow (fast) modes of the fields s [123]. The modified Langevin equation reads:

$$s(t_l + \delta t_l) = s(t_l) - \hat{F}^{-1} \left[Q \hat{F} \left[\frac{\partial S(s(t_l))}{\partial s(t_l)} \right] \delta t_l + \sqrt{2\delta t_l} Q \hat{F} [\eta(t_l)] \right], \quad (62)$$

with $\hat{F}$ being a transformation to independent modes of the field. This generically corresponds to a Fourier transform, thus the notation. Currently, Fourier acceleration is not implemented in ALF, but can be included by the user.

In order to use Langevin dynamics the user also has to provide the Langevin time step `Delta_t_Langevin_HMC`, the maximal force `Max_Force`, set `Global_update_scheme=Langevin` in the parameter file. Furthermore, the forces $\frac{\partial S_0(C)}{\partial s_{k,\tau}}$ are to be specified in the routine `Ham_Langevin_HMC_S0` of the Hamiltonian files. The Langevin update for a general Hamiltonian is carried out in the module `Langevin_HMC_mod.F90`. In particular the fermion forces,

$$\frac{\partial S^F(C)}{\partial s_{k,\tau}} = \Delta\tau N_{\text{col}} \sum_{s=1}^{N_{\text{fl}}} \text{Tr} [I^{(ks)} (\mathbb{1} - G^{(s)}(k, \tau))], \quad (63)$$

are computed in this module. In the above, we introduce a Green function that depends on the time slice τ and the interaction term k to which the corresponding field $s_{k,\tau}$ belongs:

$$G_{x,y}^{(s)}(k, \tau) = \frac{\text{Tr} [\hat{U}_{(s)}^<(k, \tau) \hat{c}_{x,s} \hat{c}_{y,s}^\dagger \hat{U}_{(s)}^>(k, \tau)]}{\text{Tr} [\hat{U}_{(s)}^<(k, \tau) \hat{U}_{(s)}^>(k, \tau)]}, \quad (64)$$

where the following definitions are used

$$\hat{U}_{(s)}^<(k', \tau') = \prod_{\tau=\tau'+1}^{L_{\text{Trotter}}} (\hat{U}_{(s)}(\tau)) \prod_{k=1}^{M_V} e^{\sqrt{-\Delta\tau} U_k \eta_{k,\tau'} \hat{c}_s^\dagger V^{(ks)} \hat{c}_s} \prod_{k=k'+1}^{M_I} e^{-\Delta\tau s_{k,\tau'} \hat{c}_s^\dagger I^{(ks)} \hat{c}_s}, \quad (65)$$

$$\hat{U}_{(s)}^>(k', \tau') = \prod_{k=1}^{k'} e^{-\Delta\tau s_{k,\tau'} \hat{c}_s^\dagger I^{(ks)} \hat{c}_s} \prod_{k=1}^{M_T} e^{-\Delta\tau \hat{c}_s^\dagger T^{(ks)} \hat{c}_s} \prod_{\tau=1}^{\tau'-1} (\hat{U}_{(s)}(\tau)), \quad (66)$$

$$\hat{U}_{(s)}(\tau) = \prod_{k=1}^{M_V} e^{\sqrt{-\Delta\tau} U_k \eta_{k,\tau} \hat{c}_s^\dagger V^{(ks)} \hat{c}_s} \prod_{k=1}^{M_I} e^{-\Delta\tau s_{k,\tau} \hat{c}_s^\dagger I^{(ks)} \hat{c}_s} \prod_{k=1}^{M_T} e^{-\Delta\tau \hat{c}_s^\dagger T^{(ks)} \hat{c}_s}. \quad (67)$$

The vector $\hat{c}_{x,s}^\dagger$ contains all fermionic operators $\hat{c}_{x,s}^\dagger$ of flavor s .

During each Langevin step, all fields are updated and the Langevin time is incremented by $\delta\tilde{t}_l$. At the end of a run, the mean and maximal forces encountered during the run are printed out in the info file.

The great advantage of the Langevin updating scheme is the absence of update rejection, meaning that all fields are updated at each step. As mentioned above, the price we pay for using Langevin dynamics is ensuring that forces show no singularities. Two other potential issues should be highlighted:

- Langevin dynamics is carried out at a finite Langevin time step, thereby introducing a further source of systematic error.
- The factor $\sqrt{2\delta\tilde{t}_l}$ multiplying the stochastic variable makes the noise dominant on short time scales. On these time scales Langevin dynamics essentially corresponds to a random walk. This has the advantage of allowing one to circumvent potential barriers, but may render the updating scheme less efficient than the hybrid molecular dynamics approach.

Example - Hubbard chain at half-filling

Let us consider a 6-site Hubbard chain at half-filling with $U/t = 4$ and $\beta t = 4$. The Hubbard interaction can be decoupled using a continuous HS transformation, where we introduce a real auxiliary field $s_{i,\tau}$ for every lattice site i and time slice τ . When the HS fields are coupled to the z -component of the magnetization (see Sec. 9.1), the partition function can be written as

$$Z = \int \left(\prod_{\tau=1}^{L_{\text{Trotter}}} \prod_{i=1}^{N_{\text{unit-cell}}} \frac{ds_{i,\tau}}{\sqrt{2\pi}} e^{-\frac{1}{2}s_{i,\tau}^2} \right) \times \prod_{s=\uparrow,\downarrow} \det \left(\mathbb{1} + \prod_{\tau=1}^{L_{\text{Trotter}}} \prod_{i=1}^{N_{\text{unit-cell}}} \left(e^{-\sqrt{\Delta\tau} U s_{i,\tau} V^{(is)}} \right) e^{-\Delta\tau T} \right) + \mathcal{O}(\Delta\tau^2). \quad (68)$$

The flavor-dependent interaction matrices have only one non-vanishing entry each:

$$V_{x,y}^{(i,s=\uparrow)} = \delta_{x,y} \delta_{x,i} \quad \text{and} \quad V_{x,y}^{(i,s=\downarrow)} = -\delta_{x,y} \delta_{x,i}.$$

The forces of the Hubbard model are given by:

$$\frac{\partial S(C)}{\partial s_{i,\tau}} = s_{i,\tau} - \sqrt{\Delta\tau} U \sum_{s=\uparrow,\downarrow} \text{Tr} [V^{(is)} (\mathbb{1} - \mathbf{G}^{(s)}(i, \tau))], \quad (69)$$

where the Green function is defined by Eq. (64) with

$$\hat{U}_{(s)}^<(i', \tau') = \prod_{\tau=\tau'+1}^{L_{\text{Trotter}}} (\hat{U}_{(s)}(\tau)) \prod_{i=i'+1}^{N_{\text{unit-cell}}} e^{-\sqrt{\Delta\tau} \bar{U}_{s_{i,\tau}} \hat{c}_s^\dagger V^{(is)} \hat{c}_s}, \quad (70)$$

$$\hat{U}_{(s)}^>(i', \tau') = \prod_{i=1}^{i'} \left(e^{-\sqrt{\Delta\tau} \bar{U}_{s_{i,\tau}} \hat{c}_s^\dagger V^{(is)} \hat{c}_s} \right) e^{-\Delta\tau \hat{c}_s^\dagger T \hat{c}_s} \prod_{\tau=1}^{\tau'-1} (\hat{U}_{(s)}(\tau)), \quad (71)$$

$$\hat{U}_{(s)}(\tau) = \prod_{i=1}^{N_{\text{unit-cell}}} \left(e^{-\sqrt{\Delta\tau} \bar{U}_{s_{i,\tau}} \hat{c}_s^\dagger V^{(is)} \hat{c}_s} \right) e^{-\Delta\tau \hat{c}_s^\dagger T \hat{c}_s}. \quad (72)$$

One can show that for periodic boundary conditions the forces are not bounded and to make sure that the program does not crash we set `Max_Force = 1.5`.

The results are: the reference, discrete-variable code gives

$$\langle \hat{H} \rangle = -3.4684 \pm 0.0007, \quad (73)$$

while the Langevin code at $\delta t_l = 0.001$ yields

$$\langle \hat{H} \rangle = -3.457 \pm 0.010 \quad (74)$$

and at $\delta t_l = 0.01$

$$\langle \hat{H} \rangle = -3.495 \pm 0.007. \quad (75)$$

At $\delta t_l = 0.001$ the maximal force that occurred during the run was 112, whereas at $\delta t_l = 0.01$ it grew to 524. In both cases the average force was given by 0.45. For larger values of δt_l the maximal force grows and the fluctuations on the energy become larger (for instance, $\langle \hat{H} \rangle = -3.718439 \pm 0.206469$ at $\delta t_l = 0.02$; for this parameter set the maximal force we encountered during the run was of 1658).

Controlling Langevin dynamics when the action has logarithmic divergences is a challenge, and it is not a given that the results are satisfactory. For our specific problem we can solve this issue by considering open boundary conditions. Following an argument put forward in [89], we can show, using world lines, that the determinant is always positive. In this case the action does not have logarithmic divergences and the Langevin dynamics works beautifully well, see Fig. 1.

Practically to start the Langevin dynamic run the user will have to provide the function

```
Subroutine Ham_Langevin_HMC_S0(Forces_0)
  Real (Kind=Kind(0.d0)), Intent(inout), allocatable :: Forces_0(:, :)
  ! Forces_0(n,nt) = \frac{\partial S_0}{\partial s_{\{n,\tau\}}}.
  ! Here n is the index of the field in the operator list and nt is the time slice.
End Subroutine Ham_Langevin_HMC_S0
```

and in the parameter file chose the option:

```
&VAR_QMC
.....
Langevin           = .T.      ! Langevin update
Delta_t_Langevin_HMC = 0.01    ! Default time step for Langevin and HMC updates.
Max_Force          = 5.0      ! Max Force for Langevin
.....
/
```

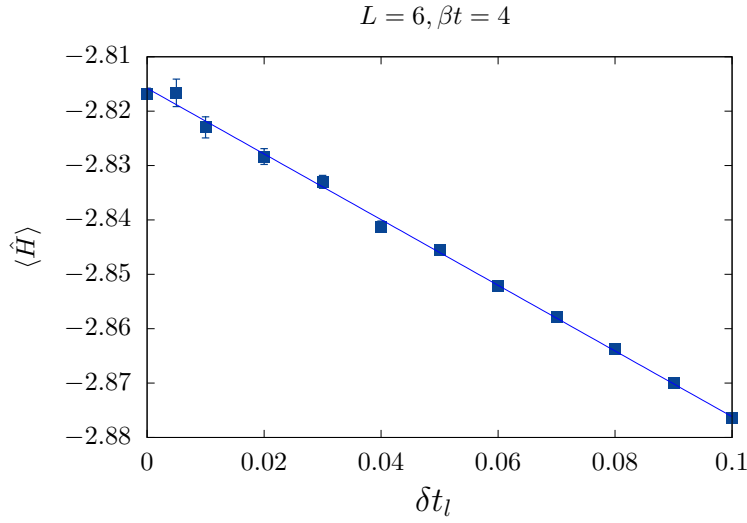


Figure 1: Total energy for the 6-site Hubbard chain at $U/t = 4$, $\beta t = 4$ and with open boundary conditions. For this system it can be shown that the determinant is always positive, so that no singularities occur in the action and, consequently, the Langevin dynamics works very well. The reference data point at $\delta t_l = 0$ comes from the discrete field code for the field coupled to the z-component of the magnetization and reads -2.8169 ± 0.0013 , while the extrapolated value is -2.8176 ± 0.0010 . Throughout the runs the maximal force remained below the threshold of 1.5. The displayed data has been produced by the pyALF script [Langevin.py](#).

2.3.4 Hybrid Monte dynamics

Hybrid molecular dynamics circumvents some drawbacks of Langevin dynamics. It does not introduce a systematic error and does not boil down to a random walk at small time steps. The approach is based on the Metropolis-Hastings importance sampling formula. Let C and C' be configurations in the *Monte Carlo* space. The probability of accepting a move from C to C' is given by

$$P(C \rightarrow C') = \max\left(\frac{T_0(C' \rightarrow C)P(C')}{T_0(C \rightarrow C')P(C)}, 1\right) \quad (76)$$

where $T_0(C' \rightarrow C)$ is the probability of *proposing* a move from C' to C . In the Monte Carlo approach, we will iterate the above procedure so as to generate a time series of configurations C_m . Provided that we are able to reach all configurations in the Monte Carlo space from any starting configuration, then,

$$\lim_{n \rightarrow \infty} \frac{1}{n} \sum_{m=1}^n \delta_{C_m, C} = P(C). \quad (77)$$

Ideally one would like to propose global, ergodic, moves that satisfy $P(C \rightarrow C') = 1$ and thereby *hope* to have *small* autocorrelation times. This is a property of cluster algorithms such as the loop [6], SSE [84] or Wolff [124] algorithms.

We will start by expanding the configuration space to $C = \{\mathbf{p}, \mathbf{s}\}$ and define the Hamiltonian

$$H(\mathbf{p}, \mathbf{s}) = \frac{\mathbf{p}^T M^{-1} \mathbf{p}}{2} + S(\mathbf{s}). \quad (78)$$

$\mathbf{p}$ and $\mathbf{s}$ are canonical conjugate. Clearly,

$$\langle \hat{O} \rangle = \frac{\int D\mathbf{s} e^{-S(\mathbf{s})} \langle \hat{O} \rangle_{\mathbf{s}}}{\int D\mathbf{s} e^{-S(\mathbf{s})}} = \frac{\int D\mathbf{s} D\mathbf{p} e^{-H(\mathbf{p}, \mathbf{s})} \langle \hat{O} \rangle_{\mathbf{s}}}{\int D\mathbf{s} D\mathbf{p} e^{-H(\mathbf{p}, \mathbf{s})}}. \quad (79)$$

and in the hybrid molecular dynamics scheme we sample

$$P(\mathbf{p}, \mathbf{s}) = \frac{e^{-H(\mathbf{p}, \mathbf{s})}}{\int D\mathbf{s} D\mathbf{p} e^{-H(\mathbf{p}, \mathbf{s})}}. \quad (80)$$

The mass matrix has to be positive definite and can be used to speed up simulations by assigning a larger (smaller) mass to fast (slow) modes. Since M is positive and real it can be written as $M^{-1} = B^T B$ with sole restriction on B being that $\text{Ker}(B) = \{0\}$. The action of B or B^T on a vector of e.g. momenta has to be provided by the user. To this purpose we have included a routine `Apply_B_HMC_base` in the base Hamiltonian. By default $M = B = 1$.

Hybrid molecular dynamics consists of two steps.

Step 1: Updating the momenta $\mathbf{p}$

Here we choose:

$$T_0(C' = \{\mathbf{p}', \mathbf{s}\} \rightarrow C = \{\mathbf{p}, \mathbf{s}\}) = \frac{e^{-\mathbf{p}^2}}{\int d\mathbf{p} e^{-\mathbf{p}^2}} \quad (81)$$

such that $P(C \rightarrow C') = 1$.

Step 2: Updating the positions $\mathbf{s}$

This step is numerically expensive and uses Hamiltonian equations of motions,

$$\dot{\mathbf{p}} = -\frac{\partial H}{\partial \mathbf{s}} \quad \text{and} \quad \dot{\mathbf{s}} = \frac{\partial H}{\partial \mathbf{p}} \quad (82)$$

that conserve energy, H , for time independent Hamiltonians. As for the Langevin dynamics, the fields acquire an additional time index, t_m , and $\dot{\mathbf{s}} = \frac{d\mathbf{s}}{dt_m}$. We can propagate the fields over a given molecular dynamics time interval, T_M , to obtain:

$$\{\mathbf{p}, \mathbf{s}\}(t_m + T_m) = U_{T_m}^H[\{\mathbf{p}, \mathbf{s}\}(t_m)] \quad (83)$$

where $U_{T_m}^H[\{\mathbf{p}, \mathbf{s}\}(t_m)]$ propagates the initial state $\{\mathbf{p}, \mathbf{s}\}(t_m)$ with Hamiltonian dynamics for a time interval T_m . The Hamiltonian equations of motion are time reversal symmetric, and according to Liouville's theorem conserve volumes in phase space. Thereby,

$$\frac{T_0(\{\mathbf{p}, \mathbf{s}\}(t_m + T_m) \rightarrow \{\mathbf{p}, \mathbf{s}\}(t_m)) e^{-H(\{\mathbf{p}, \mathbf{s}\}(t_m + T_m))}}{T_0(\{\mathbf{p}, \mathbf{s}\}(t_m) \rightarrow \{\mathbf{p}, \mathbf{s}\}(t_m + T_m)) e^{-H(\{\mathbf{p}, \mathbf{s}\}(t_m))}} = 1, \quad (84)$$

and the acceptance will be of unity. Clearly this corresponds to the ideal case, and in practice the integration will be carried out with a finite time step such that the energy will not be conserved exactly and the acceptance will not be of unity. Provided that we choose an integrator that is time reversal symmetric (see below) then the Monte Carlo acceptance rejection step will cure this systematic error. The acceptance-rejection step of the molecular dynamics trajectory is the reason why this updating scheme is coined *hybrid* molecular dynamics. The algorithm then proceeds by iterating step 1 followed by step 2.

In our implementation of the Hybrid Molecular dynamics approach, the forces are computed exactly. This stands in contrast to the algorithm generically referred to as Hybrid Monte Carlo where the forces are computed stochastically. The reader is referred to Refs. [5, 91] for further readings.

2.3.4.1 The leap-frog integrator

In practice one will adopt an integrator that conserves time reversal symmetry such as the Leapfrog algorithm. Our Hamiltonian can be split into $H_1 = \mathbf{p}^2/2$ and $H_2 = S(\mathbf{s})$. Propagating with H_1 only allows for an exact solution since in this case $\mathbf{p}$ is constant and $\mathbf{s}(t) = \mathbf{s}(t = t_0) + (t - t_0)\mathbf{p}$. Similarly for H_2 , $\mathbf{s}$ is constant and $\mathbf{p}(t) = \mathbf{p}(t = t_0) - (t - t_0)\frac{\partial S(\mathbf{s})}{\partial \mathbf{s}}$. Hence both for H_1 and H_2 the propagation can be carried out exactly such that time reversal symmetry and Liouville's theorem hold. In very much the same manner as for the symmetric Trotter decomposition, the leapfrog approach carries out a δt_m time interval propagation of the full Hamiltonian $H = H_1 + H_2$ as:

$$U_{\delta t_m}^H = U_{\delta t_m/2}^{H_1} \circ U_{\delta t_m}^{H_2} \circ U_{\delta t_m/2}^{H_1} + \mathcal{O}(\delta t_m^2). \quad (85)$$

Clearly time reversal is satisfied and because of this property the error contains only even powers of the time step. The energy $H = H_1 + H_2$ will however not be conserved exactly such that, as mentioned above, the molecular dynamics trajectory will be accepted according to:

$$\begin{aligned} & \max \left(\frac{T_0(\{\mathbf{p}, \mathbf{s}\}(t_m + T_m) \rightarrow \{\mathbf{p}, \mathbf{s}\}(t_m)) e^{-H(\{\mathbf{p}, \mathbf{s}\}(t_m + T_m))}}{T_0(\{\mathbf{p}, \mathbf{s}\}(t_m) \rightarrow \{\mathbf{p}, \mathbf{s}\}(t_m + T_m)) e^{-H(\{\mathbf{p}, \mathbf{s}\}(t_m))}}, 1 \right) \\ &= \max \left(\frac{e^{-H(\{\mathbf{p}, \mathbf{s}\}(t_m + T_m))}}{e^{-H(\{\mathbf{p}, \mathbf{s}\}(t_m))}}, 1 \right). \end{aligned} \quad (86)$$

2.3.4.2 Specific Implementation

To use the hybrid molecular dynamics update, the user has to set `HMC=.true.` in the parameter file. Clearly we also have to use continuous Hubbard-Stratonovitch fields such that `OP_V%type=3`. The time step for the leap-frog integrator is given by the same variable as for the Langevin dynamics, `Delta_t_Langevin_HMC`, and the total number of leap-frog molecular dynamics steps is given by the variable `Leapfrog_steps`. Thereby $t_m = \text{Delta_t_Langevin_HMC} \times \text{Leapfrog_steps}$. Molecular dynamics can be understood a clever way to propose a global update, that will be accepted or rejected by a Metropolis acceptance rejection step. Hence it can be combined with other moves such as our standard sequential updating scheme. In the code, this will happen provide that the variable `Sequential=.true.`. If both the variables `HMC=.true.` and `Sequential=.true.` then `N_HMC_sweeps` will correspond to the number of hybrid molecular dynamics carried out between a sequential sweep. In order to address potential ergodicity issues [91] it is crucial to integrate both sequential and hybrid molecular dynamics moves, rather than relying solely on hybrid molecular dynamics [125].

To test the code, we have carried out high precision calculations of the 6-sites Hubbard chain with open boundary conditions (see Sec. 2.3.3) at $\beta t = 4$. Using *only* hybrid molecular dynamics updates we obtain: $\langle \hat{H} \rangle = -2.81715 \pm 0.00023$. This compares very well with the discrete field sequential updating run: $\langle \hat{H} \rangle = -2.81706 \pm 0.00013$.

Specifically to start the above HMC run, the user again has to define the subroutine `Ham_Langevin_HMC_S0` in the base Hamiltonian module that computes the derivative of the bosonic part action with respect to the fields and in the parameter file chose the option:

```
&VAR_QMC
.....
Sequential      = .F.      ! Sequential moves
.....
HMC              = .T.      ! Enable hybrid molecular dynamics updates
Delta_t_Langevin_HMC = 0.5    ! Default time step for Langevin and HMC updates.
N_HMC_sweeps     = 1        ! # of HMC updates between sequential ones
Leapfrog_steps   = 100     ! Number of leapfrog iterations
```

```
.....
/
```

For this parameter choice, since `Sequential = .F.`, the code will carry out only hybrid molecular dynamics updates. The user can of course choose to carry out both sequential and hybrid molecular dynamics updates by setting `Sequential = .T.` and choosing a finite value for `N_HMC_sweeps`. It is also interesting to note that HMC time step can be chosen much larger than for the Langevin dynamics. The Langevin integrator suffers from systematic errors in the time step whereas the HMC integrator does not.

2.4 The Trotter error and checkerboard decomposition

2.4.1 Asymmetric Trotter decomposition

In practice, many applications are carried out at finite imaginary time steps, and it is important to understand the consequences of the Trotter error. How does it scale with system size and what symmetries does it break? In particular, when investigating a critical point, one should determine whether the potential symmetry breaking associated with the Trotter decomposition generates relevant operators.

To best describe the workings of the ALF code, we divide the Hamiltonian into hopping terms $\hat{\mathcal{H}}_T$ and interaction terms $\hat{\mathcal{H}}_V + \hat{\mathcal{H}}_I + \hat{\mathcal{H}}_{0,I}$. Let

$$\hat{\mathcal{H}}_T = \sum_{i=1}^{N_T} \sum_{k \in \mathcal{S}_i^T} \hat{T}^{(k)} \equiv \sum_{i=1}^{N_T} \hat{T}_i. \quad (87)$$

Here the decomposition follows the rule that if k and k' belong to the same set $\mathcal{S}_i^T$ then $[\hat{T}^{(k)}, \hat{T}^{(k')}] = 0$. An important case to consider is that of the checkerboard decomposition. For the square lattice we can decouple the nearest neighbor hopping into $N_T = 4$ groups, each consisting of two site hopping processes. This type of checkerboard decomposition is activated for a set of predefined lattices by setting the flag `Checkerboard` to `.true.`. We will carry out the same separation for the interaction:

$$\hat{\mathcal{H}}_V + \hat{\mathcal{H}}_I + \hat{\mathcal{H}}_{0,I} = \sum_{i=1}^{N_I} \hat{O}_i, \quad (88)$$

where each $\hat{O}_i$ contains a set of commuting terms. For instance, for the Hubbard model, the above reduces to $U \sum_i \hat{n}_{i,\uparrow} \hat{n}_{i,\downarrow}$ such that $N_I = 1$ and $\hat{O}_1 = U \sum_i \hat{n}_{i,\uparrow} \hat{n}_{i,\downarrow}$.

The default Trotter decomposition in the ALF code is based on the equation:

$$e^{-\Delta\tau(\hat{A}+\hat{B})} = e^{-\Delta\tau\hat{A}}e^{-\Delta\tau\hat{B}} + \frac{\Delta\tau^2}{2} [\hat{B}, \hat{A}] + \mathcal{O}(\Delta\tau^3). \quad (89)$$

Using iteratively the above the single time step is given by:

$$e^{-\Delta\tau\mathcal{H}} = \prod_{i=1}^{N_O} e^{-\Delta\tau\hat{O}_i} \prod_{j=1}^{N_T} e^{-\Delta\tau\hat{T}_j} + \frac{\Delta\tau^2}{2} \underbrace{\left(\sum_{i=1}^{N_O} \sum_{j=1}^{N_T} [\hat{T}_j, \hat{O}_i] + \sum_{j'}^{N_T-1} [\hat{T}_{j'}, \hat{T}_{j'}^>] + \sum_{i'=1}^{N_O-1} [\hat{O}_{i'}, \hat{O}_{i'}^>] \right)}_{\equiv \Delta\tau\hat{\lambda}_1} + \mathcal{O}(\Delta\tau^3). \quad (90)$$

In the above, we have introduced the shorthand notation

$$\hat{T}_n^> = \sum_{j=n+1}^{N_T} \hat{T}_j \quad \text{and} \quad \hat{O}_n^> = \sum_{j=n+1}^{N_O} \hat{O}_j. \quad (91)$$

The full propagation then reads

$$\begin{aligned} \hat{U}_{\text{Approx}} &= \left(\prod_{i=1}^{N_O} e^{-\Delta\tau \hat{O}_i} \prod_{j=1}^{N_T} e^{-\Delta\tau \hat{T}_j} \right)^{L_{\text{Trotter}}} = e^{-\beta(\hat{H} + \hat{\lambda}_1)} + \mathcal{O}(\Delta\tau^2) \\ &= e^{-\beta\hat{H}} - \int_0^\beta d\tau e^{-(\beta-\tau)\hat{H}} \hat{\lambda}_1 e^{-\tau\hat{H}} + \mathcal{O}(\Delta\tau^2). \end{aligned} \quad (92)$$

The last step follows from time-dependent perturbation theory. The following comments are in order:

- The error is anti-Hermitian since $\hat{\lambda}_1^\dagger = -\hat{\lambda}_1$. As a consequence, if all the operators as well as the quantity being measured are simultaneously real representable, then the prefactor of the linear in $\Delta\tau$ error vanishes since it ultimately corresponds to computing the trace of an anti-symmetric matrix. This *lucky* cancellation was put forward in Ref. [101]. Hence, under this assumption – which is certainly valid for the Hubbard model considered in Fig. 2 – the systematic error is of order $\Delta\tau^2$.
- The biggest drawback of the above decomposition is that the imaginary-time propagation is not Hermitian. This can lead to acausal features in imaginary-time correlation functions [126]. To be more precise, the eigenvalues of $H_{\text{Approx}} = -\frac{1}{\beta} \log U_{\text{Approx}}$ need not be real and thus imaginary-time displaced correlation functions may oscillate as a function of imaginary time. This is shown in Fig. 2(a) that plots the absolute value of local time-displaced Green function for the Honeycomb lattice at $U/t = 2$. Sign changes of this quantity involve zeros that, on the considered log-scale, correspond to negative divergences. As detailed in [112], using the non-symmetric Trotter decomposition leads to an additional non-hermitian second-order error in the measurement of observables O that is proportional to $[T, [T, O]]$. As we see next, these issues can be solved by considering a symmetric Trotter decomposition.

2.4.2 Symmetric Trotter decomposition

To address the issue described above, the ALF package provides the possibility of using a symmetric Trotter decomposition,

$$e^{-\Delta\tau(\hat{A}+\hat{B})} = e^{-\Delta\tau\hat{A}/2} e^{-\Delta\tau\hat{B}} e^{-\Delta\tau\hat{A}/2} + \frac{\Delta\tau^3}{12} [2\hat{A} + \hat{B}, [\hat{B}, \hat{A}]] + \mathcal{O}(\Delta\tau^5), \quad (93)$$

by setting the `Symm` flag to `.true..` Before we apply the expression above to a time step, let us write

$$e^{-\Delta\tau\mathcal{H}} = e^{-\frac{\Delta\tau}{2} \sum_{j=1}^{N_T} \hat{T}_j} e^{-\Delta\tau \sum_{i=1}^{N_O} \hat{O}_i} e^{-\frac{\Delta\tau}{2} \sum_{j=1}^{N_T} \hat{T}_j} + \underbrace{\frac{\Delta\tau^3}{12} [2\hat{T}_0^> + \hat{O}_0^>, [\hat{O}_0^>, \hat{T}_0^>]]}_{\equiv \Delta\tau \hat{\lambda}_{TO}} + \mathcal{O}(\Delta\tau^5). \quad (94)$$

Then,

$$e^{-\Delta\tau \sum_i^{N_I} \hat{O}_i} = \left(\prod_{i=1}^{N_O-1} e^{-\frac{\Delta\tau}{2} \hat{O}_i} \right) e^{-\Delta\tau \hat{O}_{N_O}} \left(\prod_{i=N_O-1}^1 e^{-\frac{\Delta\tau}{2} \hat{O}_i} \right) + \underbrace{\frac{\Delta\tau^3}{12} \sum_{i=1}^{N_O-1} [2\hat{O}_i + \hat{O}_i^>, [\hat{O}_i^>, \hat{O}_i]]}_{\equiv \Delta\tau \hat{\lambda}_O} + \mathcal{O}(\Delta\tau^5), \quad (95)$$

$$e^{-\frac{\Delta\tau}{2} \sum_j^{N_T} \hat{T}_j} = \left(\prod_{j=1}^{N_T-1} e^{-\frac{\Delta\tau}{4} \hat{T}_j} \right) e^{-\frac{\Delta\tau}{2} \hat{T}_{N_T}} \left(\prod_{j=N_T-1}^1 e^{-\frac{\Delta\tau}{4} \hat{T}_j} \right) + \underbrace{\frac{\Delta\tau^3}{96} \sum_{j=1}^{N_T-1} [2\hat{T}_j + \hat{T}_j^>, [\hat{T}_j^>, \hat{T}_j]]}_{\equiv \Delta\tau \hat{\lambda}_T} + \mathcal{O}(\Delta\tau^5) \quad (96)$$

and we can derive a closed equation for the free energy density:

$$f_{\text{Approx}} = -\frac{1}{\beta V} \log \text{Tr} \left[\left(\prod_{j=1}^{N_T-1} e^{-\frac{\Delta\tau}{4} \hat{T}_j} \right) e^{-\frac{\Delta\tau}{2} \hat{T}_{N_T}} \left(\prod_{j=N_T-1}^1 e^{-\frac{\Delta\tau}{4} \hat{T}_j} \right) \times \left(\prod_{i=1}^{N_O-1} e^{-\frac{\Delta\tau}{2} \hat{O}_i} \right) e^{-\Delta\tau \hat{O}_{N_O}} \left(\prod_{i=N_O-1}^1 e^{-\frac{\Delta\tau}{2} \hat{O}_i} \right) \times \left(\prod_{j=1}^{N_T-1} e^{-\frac{\Delta\tau}{4} \hat{T}_j} \right) e^{-\frac{\Delta\tau}{2} \hat{T}_{N_T}} \left(\prod_{j=N_T-1}^1 e^{-\frac{\Delta\tau}{4} \hat{T}_j} \right) \right]^{L_{\text{Trotter}}} \\ = f - \frac{1}{V} \langle \hat{\lambda}_{TO} + \hat{\lambda}_O + 2\hat{\lambda}_T \rangle + \mathcal{O}(\Delta\tau^4). \quad (97)$$

The following comments are in order:

- The approximate imaginary-time propagation from which the f_{Approx} is derived is Hermitian. Hence no spurious effects in imaginary-time correlation functions are to be expected. This is clearly shown in Fig. 2(a).
- In Fig. 2(b) we have used the ALF-library with `Symm=.true.` with and without checkerboard decomposition. We still expect the systematic error to be of order $\Delta\tau^2$. However its prefactor is much smaller than that of the aforementioned anti-symmetric decomposition.
- We have taken the burden to evaluate explicitly the prefactor of the $\Delta\tau^2$ error on the free energy density. One can see that for Hamiltonians that are sums of local operators, the quantity $\langle \hat{\lambda}_{TO} + \hat{\lambda}_O + 2\hat{\lambda}_T \rangle$ scales as the volume V of the system, such that the systematic error on the free energy density (and on correlation functions that can be computed by adding source terms) will be volume independent. For model Hamiltonians that are not sums of local terms, care must be taken. A conservative upper bound on the error is $\langle \hat{\lambda}_{TO} + \hat{\lambda}_O + 2\hat{\lambda}_T \rangle \propto \Delta\tau^2 V^3$, which means that, in order to maintain a constant systematic error for the free energy density, we have to keep $\Delta\tau V$ constant. Such a situation has been observed in Ref. [71].

Alternative symmetric second order methods as well as the issues with decompositions of higher order have been detailed in [112].

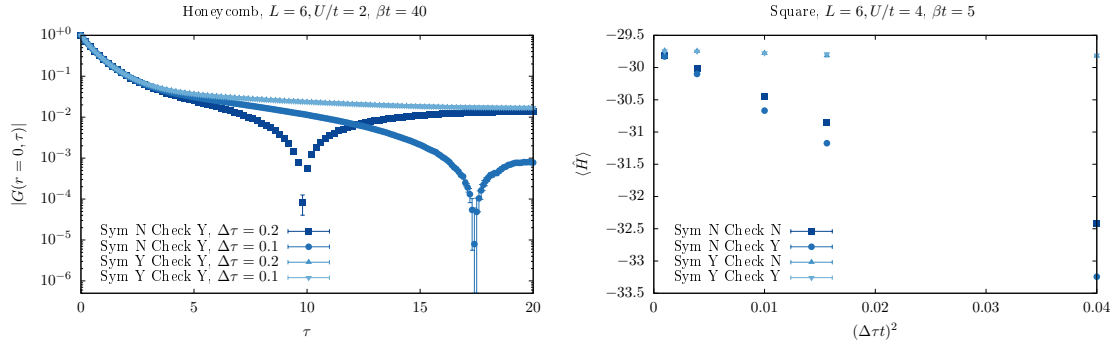


Figure 2: Analysis of Trotter systematic error. Left: We consider a 6×6 Hubbard model on the Honeycomb lattice, $U/t = 2$, half-band filling, inverse temperature $\beta t = 40$, and we have used an HS transformation that couples to the density. The figure plots the local-time displaced Green function. Right: Here we consider the 6×6 Hubbard model at $U/t = 4$, half-band filling, inverse temperature $\beta t = 5$, and we have used the HS transformation that couples to the z -component of spin. We provide data for the four combinations of the logical variables `Symm` and `Checkerboard`, where `Symm=.true. (.false.)` indicates a symmetric (asymmetric) Trotter decomposition has been used, and `Checkerboard=.true. (.false.)` that the checkerboard decomposition for the hopping matrix has (not) been used. The large deviations between different choices of `Symm` are here $\sim [T, [T, H]]$ as detailed in [112].

2.4.3 The `Symm` flag

If the `Symm` flag is set to true, then the program will automatically – for the set of predefined lattices and models – use the symmetric $\Delta\tau$ time step of the interaction and hopping terms. However, the specification of the interaction in the symmetric form of Eq. 97 has to be carried out by the user in the Hamiltonian module.

To save CPU time when the `Symm` flag is on we carry out the following approximation:

$$\left[\left(\prod_{j=1}^{N_T-1} e^{-\frac{\Delta\tau}{4} \hat{T}_j} \right) e^{-\frac{\Delta\tau}{2} \hat{T}_{N_T}} \left(\prod_{j=N_T-1}^1 e^{-\frac{\Delta\tau}{4} \hat{T}_j} \right) \right]^2 \simeq \left(\prod_{j=1}^{N_T-1} e^{-\frac{\Delta\tau}{2} \hat{T}_j} \right) e^{-\Delta\tau \hat{T}_{N_T}} \left(\prod_{j=N_T-1}^1 e^{-\frac{\Delta\tau}{2} \hat{T}_j} \right). \quad (98)$$

The above is consistent with the overall precision of the Trotter decomposition and more importantly conserves the Hermiticity of the propagation.

2.5 Stabilization - a peculiarity of the BSS algorithm

From the partition function in Eq. (18) it can be seen that, for the calculation of the Monte Carlo weight and of the observables, a long product of matrix exponentials has to be formed. In addition to that, we need to be able to extract the single-particle Green function for a given flavor index at, say, time slice $\tau = 0$. As mentioned above (cf. Eq. (22)), this quantity is given by:

$$\mathbf{G} = \left(\mathbb{1} + \prod_{\tau=1}^{L_{\text{Trotter}}} \mathbf{B}_{\tau} \right)^{-1}, \quad (99)$$

which can be recast as the more familiar linear algebra problem of finding a solution for the linear system

$$\left(\mathbb{1} + \prod_{\tau=1}^{L_{\text{Trotter}}} \mathbf{B}_{\tau} \right) x = b. \quad (100)$$

The matrices $\mathbf{B}_{\tau} \in \mathbb{C}^{n \times n}$ depend on the lattice size as well as other physical parameters that can be chosen such that a matrix norm of $\mathbf{B}_{\tau}$ can be unbound in magnitude. From standard perturbation theory for linear systems, the computed solution $\tilde{x}$ would contain a relative error

$$\frac{|\tilde{x} - x|}{|x|} = \mathcal{O} \left(\epsilon \kappa_p \left(\mathbb{1} + \prod_{\tau=1}^{L_{\text{Trotter}}} \mathbf{B}_{\tau} \right) \right), \quad (101)$$

where ϵ denotes the machine precision, which is 2^{-53} for IEEE double-precision numbers, and $\kappa_p(\mathbf{M})$ is the condition number of the matrix $\mathbf{M}$ with respect to the matrix p -norm. Due to $\prod_{\tau} \mathbf{B}_{\tau}$ containing exponentially large and small scales, as can be seen in Eq. (18), a straightforward inversion is completely ill-suited, in that its condition number would grow exponentially with increasing inverse temperature, rendering the computed solution $\tilde{x}$ meaningless.

In order to circumvent this, more sophisticated methods have to be employed. In the realm of the BSS algorithm there has been a long history [4, 94, 127–130] of using various matrix factorization techniques. The predominant techniques are either based on the singular value decomposition (SVD) or on techniques using the QR decomposition. The default stabilization strategy in the auxiliary-field QMC implementation of the ALF package is to form a product of QR-decompositions, which is proven to be weakly backwards stable [129]. While algorithms using the SVD can provide higher stability, though at a higher cost, we note that great care has to be taken in the choice of the algorithm, which has to achieve a high relative accuracy [131, 132].

As a first step we assume that, for a given integer NWrap , the multiplication of NWrap $\mathbf{B}$ matrices has an acceptable condition number and, for simplicity, that L_{Trotter} is divisible by NWrap . We can then write:

$$\mathbf{G} = \left(\mathbb{1} + \prod_{i=1}^{\frac{L_{\text{Trotter}}}{\text{NWrap}}} \underbrace{\prod_{\tau=1}^{\text{NWrap}} \mathbf{B}_{(i-1) \cdot \text{NWrap} + \tau}}_{\equiv \mathbf{B}_i} \right)^{-1}. \quad (102)$$

The key idea is to efficiently separate the scales of a matrix from the orthogonal part of a matrix. This can be achieved with the QR decomposition of a matrix $\mathbf{A}$ in the form $\mathbf{A}_i = \mathbf{Q}_i \mathbf{R}_i$. The matrix $\mathbf{Q}_i$ is unitary and hence in the usual 2-norm it satisfies $\kappa_2(\mathbf{Q}_i) = 1$. To get a handle on the condition number of $\mathbf{R}_i$ we consider the diagonal matrix

$$(\mathbf{D}_i)_{n,n} = |(\mathbf{R}_i)_{n,n}| \quad (103)$$

and set $\tilde{\mathbf{R}}_i = \mathbf{D}_i^{-1} \mathbf{R}_i$, which gives the decomposition

$$\mathbf{A}_i = \mathbf{Q}_i \mathbf{D}_i \tilde{\mathbf{R}}_i. \quad (104)$$

The matrix $\mathbf{D}_i$ now contains the row norms of the original $\mathbf{R}_i$ matrix and hence attempts to separate off the total scales of the problem from $\mathbf{R}_i$. This is similar in spirit to the so-called matrix equilibration which tries to improve the condition number of a matrix through suitably chosen column and row scalings. Due to a theorem by van der Sluis [133] we know that the choice in Eq. (103) is almost optimal among all diagonal matrices $\mathbf{D}$ from the space of diagonal matrices $\mathcal{D}$, in the sense that

$$\kappa_p((\mathbf{D}_i)^{-1} \mathbf{R}_i) \leq n^{1/p} \min_{\mathbf{D} \in \mathcal{D}} \kappa_p(\mathbf{D}^{-1} \mathbf{R}_i).$$

Now, given an initial decomposition $A_{j-1} = \prod_i B_i = Q_{j-1} D_{j-1} T_{j-1}$, an update $B_j A_{j-1}$ is formed in the following three steps:

1. Form $M_j = (B_j Q_{j-1}) D_{j-1}$. Note the parentheses.
2. Do a QR decomposition of $M_j = Q_j D_j R_j$. This gives the final Q_j and D_j .
3. Form the updated T matrices $T_j = R_j T_{j-1}$.

This is a stable but expensive method for calculating the matrix product. Here is where NWrap comes into play: it specifies the number of plain multiplications performed between the QR decompositions just described, so that $NWrap = 1$ corresponds to always performing QR decompositions whereas larger values define longer intervals where no QR decomposition will be performed. Whenever we perform a stabilization, we compare the old result (fast updates) with the new one (recalculated from the QR stabilized matrices). The difference is documented as the stability, both for the Green function and for the sign (of the determinant). The effectiveness of the stabilization *has* to be judged for every simulation from the output file info (Sec. 5.7.2). For most simulations there are two values to look out for:

- Precision Green
- Precision Phase

The Green function, as well as the average phase, are usually numbers with a magnitude of $\mathcal{O}(1)$. For that reason we recommend that $NWrap$ is chosen such that the mean precision is of the order of 10^{-8} or better (for further recommendations see Sec. 6.4). We include typical values of Precision Phase and of the mean and the maximal values of Precision Green in the example simulations discussed in Sec. 7.9.

3 Auxiliary Field Quantum Monte Carlo: projective algorithm

The projective approach is the method of choice if one is interested in ground-state properties. The starting point is a pair of trial wave functions, $|\Psi_{T,L/R}\rangle$, that are not orthogonal to the ground state $|\Psi_0\rangle$:

$$\langle \Psi_{T,L/R} | \Psi_0 \rangle \neq 0. \quad (105)$$

The ground-state expectation value of any observable $\hat{O}$ can then be computed by propagation along the imaginary time axis:

$$\frac{\langle \Psi_0 | \hat{O} | \Psi_0 \rangle}{\langle \Psi_0 | \Psi_0 \rangle} = \lim_{\theta \rightarrow \infty} \frac{\langle \Psi_{T,L} | e^{-\theta \hat{H}} e^{-(\beta-\tau) \hat{H}} \hat{O} e^{-\tau \hat{H}} e^{-\theta \hat{H}} | \Psi_{T,R} \rangle}{\langle \Psi_{T,L} | e^{-(2\theta+\beta) \hat{H}} | \Psi_{T,R} \rangle}, \quad (106)$$

where β defines the imaginary time range where observables (time displaced and equal time) are measured and τ varies from 0 to β in the calculation of time-displace observables. The simulations are carried out at large but finite values of θ so as to guarantee convergence to the ground state within the statistical uncertainty. The trial wave functions are determined up to a phase, and the program uses this gauge choice to guarantee that

$$\langle \Psi_{T,L} | \Psi_{T,R} \rangle > 0. \quad (107)$$

In order to use the projective version of the code, the model's namespace in the parameter file must set `projector=.true.` and specify the value of the projection parameter `Theta`, as well as the imaginary time interval `Beta` in which observables are measured.

Note that time-displaced correlation functions are computed for a τ ranging from 0 to β . The implicit assumption in this formulation is that the projection parameter `Theta` suffices to reach the ground state. Since the computational time scales linearly with `Theta` large projections parameters are computationally not expensive.

3.1 Specification of the trial wave function

For each flavor, one needs to specify a left and a right trial wave function. In the ALF, they are assumed to be the ground state of single-particle trial Hamiltonians $\hat{H}_{T,L/R}$ and hence correspond to a single Slater determinant each. More specifically, we consider a single-particle Hamiltonian with the same symmetries, color and flavor, as the original Hamiltonian:

$$\hat{H}_{T,L/R} = \sum_{\sigma=1}^{N_{\text{col}}} \sum_{s=1}^{N_{\text{fl}}} \sum_{x,y}^{N_{\text{dim}}} \hat{c}_{x\sigma s}^\dagger h_{xy}^{(s,L/R)} \hat{c}_{y\sigma s}. \quad (108)$$

Ordering the eigenvalues of the Hamiltonian in ascending order yields the ground state

$$|\Psi_{T,L/R}\rangle = \prod_{\sigma=1}^{N_{\text{col}}} \prod_{s=1}^{N_{\text{fl}}} \prod_{n=1}^{N_{\text{part},s}} \left(\sum_{x=1}^{N_{\text{dim}}} \hat{c}_{x\sigma s}^\dagger U_{x,n}^{(s,L/R)} \right) |0\rangle, \quad (109)$$

where

$$U^{\dagger,(s,L/R)} h^{(s,L/R)} U^{(s,L/R)} = \text{Diag} \left(\epsilon_1^{(s,L/R)}, \dots, \epsilon_{N_{\text{dim}}}^{(s,L/R)} \right). \quad (110)$$

The trial wave function is hence completely defined by the set of orthogonal vectors $U_{x,n}^{(s,L/R)}$ for n ranging from 1 to the number of particles in each flavor sector, $N_{\text{part},s}$. This information is stored in the `WaveFunction` type defined in the module `WaveFunction_mod` (see Sec. 5.5). Note that, owing to the $\text{SU}(N_{\text{col}})$ symmetry, the color index is not necessary to define the trial wave function. The user will have to specify the trial wave function in the following way:

```
Do s = 1, N_fl
  Do x = 1, Ndim
    Do n = 1, N_part(s)
      WF_L(s)%P(x,n) = U_{x,n}^{(s,L)}
      WF_R(s)%P(x,n) = U_{x,n}^{(s,R)}
    Enddo
  Enddo
Enddo
```

In the above `WF_L` and `WF_R` are `WaveFunction` arrays of length N_{fl} . ALF comes with a set of predefined trial wave functions, see Sec. 8.5.

Generically, the unitary matrix will be generated by a diagonalization routine such that if the ground state for the given particle number is degenerate, the trial wave function has a degree of ambiguity and does not necessarily share the symmetries of the Hamiltonian $\hat{H}_{T,L/R}$. Since symmetries are the key for guaranteeing the absence of the negative sign problem, violating them in the choice of the trial wave function can very well lead to a sign problem. It is hence recommended to define the trial Hamiltonians $\hat{H}_{T,L/R}$ such that the ground state for the given particle number is non-degenerate. That can be checked using the value of `WF_L/R(s)%Degen`, which stores the energy difference between the last occupied and first unoccupied single particle state. If this value is greater than zero, then the trial wave function is non-degenerate and hence has all the symmetry properties of the trial Hamiltonians, $\hat{H}_{T,L/R}$. When the projector variable is set to `.true.`, this quantity is listed in the `info` file.

3.2 Some technical aspects of the projective code.

If one is interested solely in zero-temperature properties, the projective code offers many advantages. This comes from the related facts that the Green function matrix is a projector, and that scales can be omitted.

In the projective algorithm, it is known [6] that

$$G(x, \sigma, s, \tau | x', \sigma, s, \tau) = \left[1 - U_{(s)}^>(\tau) \left(U_{(s)}^<(\tau) U_{(s)}^>(\tau) \right)^{-1} U_{(s)}^<(\tau) \right]_{x, x'} \quad (111)$$

with

$$U_{(s)}^>(\tau) = \prod_{\tau'=1}^{\tau} \mathbf{B}_{\tau'}^{(s)} P_{\tau'}^{(s), R} \quad \text{and} \quad U_{(s)}^<(\tau) = P^{(s), L, \dagger} \prod_{\tau'=L_{\text{Trotter}}}^{\tau+1} \mathbf{B}_{\tau'}^{(s)}, \quad (112)$$

where $\mathbf{B}_{\tau}^{(s)}$ is given by Eq. (23) and $P^{(s), L/R}$ correspond to the $N_{\text{dim}} \times N_{\text{part}, s}$ submatrices of $U_{(s)}^{(s), L/R}$. To see that scales can be omitted, we carry out a singular value decomposition:

$$U_{(s)}^>(\tau) = \tilde{U}_{(s)}^>(\tau) d^> v^> \quad \text{and} \quad U_{(s)}^<(\tau) = v^< d^< \tilde{U}_{(s)}^<(\tau) \quad (113)$$

such that $\tilde{U}_{(s)}^>(\tau)$ corresponds to a set of column-wise orthogonal vectors. It can be readily seen that scales can be omitted, since

$$G(x, \sigma, s, \tau | x', \sigma, s, \tau) = \left[1 - \tilde{U}_{(s)}^>(\tau) \left(\tilde{U}_{(s)}^<(\tau) \tilde{U}_{(s)}^>(\tau) \right)^{-1} \tilde{U}_{(s)}^<(\tau) \right]_{x, x'}. \quad (114)$$

Hence, stabilization is never an issue for the projective code, and arbitrarily large projection parameters can be reached.

The form of the Green function matrix implies that it is a projector: $G^2 = G$. This property has been used in Ref. [134] to very efficiently compute imaginary-time-displaced correlation functions.

3.3 Comparison of finite and projective codes.

The finite temperature code operates in the grand canonical ensemble, whereas in the projective approach the particle number is fixed. On finite lattices, the comparison between both approaches can only be made at a temperature scale below which a finite-sized charge gap emerges. In Fig. 3 we consider a semi-metallic phase as realized by the Hubbard model on the Honeycomb lattice at $U/t = 2$. It is evident that, at a scale below which charge fluctuations are suppressed, both algorithms yield identical results.

4 Monte Carlo sampling

Error estimates in Monte Carlo simulations are based on the central limit theorem [136] and can be a delicate matter, especially as it requires independent measurements and a finite variance. In this section we give examples of the care that must be taken to satisfy these requirements when using a Monte Carlo code. This is part of the common lore of the field and we cover them briefly in this text. For a deeper understanding of the inherent issues of Markov-chain Monte Carlo methods we refer the reader to the pedagogical introduction in chapter 1.3.5 of Krauth [137], the overview article of Sokal [82], the more specialized literature by Geyer [138] and chapter 6.3 of Neal [139].

In general, one distinguishes local from global updates. As the name suggest, the local update corresponds to a small change of the configuration, e.g., a single spin flip of one of the

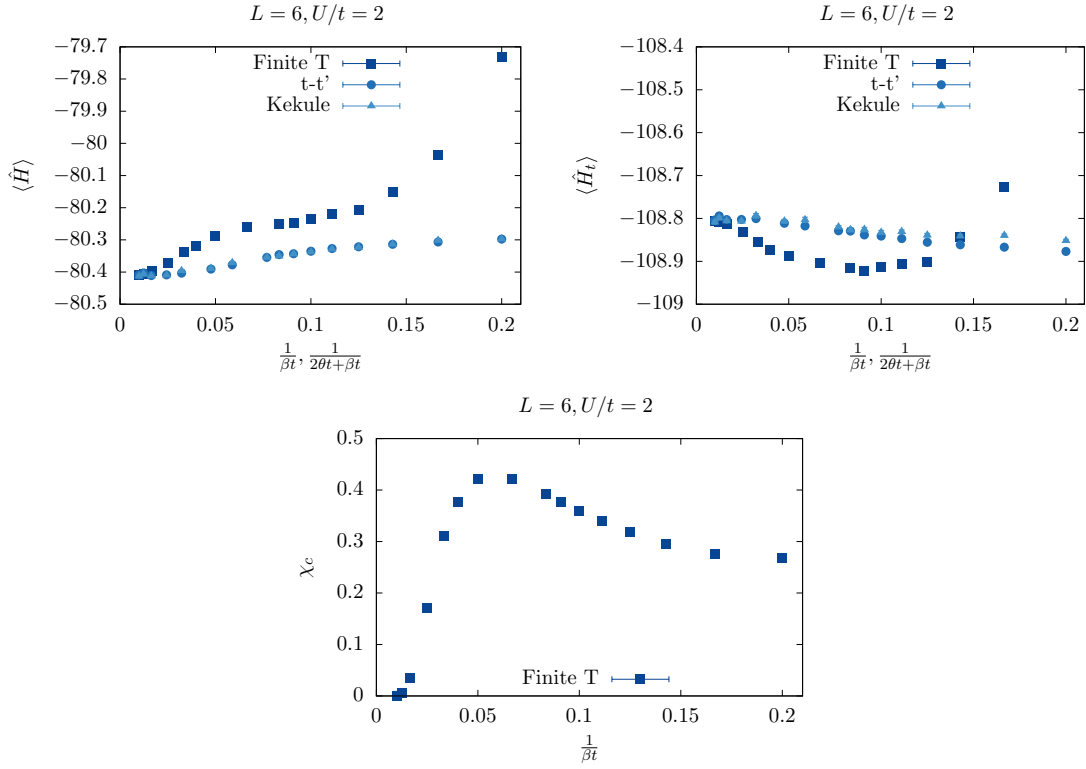


Figure 3: Comparison between the finite-temperature and projective codes for the Hubbard model on a 6×6 Honeycomb lattice at $U/t = 2$ and with periodic boundary conditions. For the projective code (blue and black symbols) $\beta t = 1$ is fixed, while θ is varied. In all cases we have $\Delta\tau t = 0.1$, no checkerboard decomposition, and a symmetric Trotter decomposition. For this lattice size and choice of boundary conditions, the non-interacting ground state is degenerate, since the Dirac points belong to the discrete set of crystal momenta. In order to generate the trial wave function we have lifted this degeneracy by either including a Kékulé mass term [46] that breaks translation symmetry (blue symbols), or by adding a next-next nearest neighbor hopping (black symbols) that breaks the symmetry nematically and shifts the Dirac points away from the zone boundary [135]. As apparent, both choices of trial wave functions yield the same answer, which compares very well with the finite temperature code at temperature scales below the finite-size charge gap.

$L_{\text{Trotter}}(M_I + M_V)$ field entries (see Sec. 2.2), whereas a global update changes a significant part of the configuration. The default update scheme of the ALF implementation are local updates, such that there is a minimum number of moves required for generating an independent configuration. The associated time scale is called the autocorrelation time, T_{auto} , and is generically dependent upon the choice of the observables.

We call a *sweep* a sequential propagation from $\tau = 0$ to $\tau = L_{\text{Trotter}}$ and back, such that each field is visited twice in each sweep. A single sweep will generically not suffice to produce an independent configuration. In fact, the autocorrelation time T_{auto} characterizes the required time scale to generate independent values of $\langle\langle\hat{O}\rangle\rangle_C$ for the observable O . This has several consequences for the Monte Carlo simulation:

- First of all, we start from a randomly chosen field configuration, such that one has to invest a time of *at least* one T_{auto} , but typically many more, in order to generate relevant, equilibrated configurations before reliable measurements are possible. This phase of the simulation is known as the warm-up or burn-in phase. In order to keep the code as flexible as possible (as different simulations might have different autocorrelation times), measurements are taken from the very beginning and, in the analysis phase, the parameter `n_skip` controls the number of initial bins that are ignored.
- Second, our implementation averages over bins with `NSWEEPS` measurements before storing the results on disk. The error analysis requires statistically independent bins in order to generate reliable confidence estimates. If the bins are too small (averaged over a period shorter than T_{auto}), then the error bars are typically underestimated. Most of the time, however, the autocorrelation time is unknown before the simulation is started and, sometimes, single runs long enough to generate appropriately sized bins are not feasible. For this reason, we provide a rebinning facility controlled by the parameter `N_rebin` that specifies the number of bins recombined into each new bin during the error analysis. One can test the suitability of a given bin size by verifying whether an increase in size changes the error estimate (For an explicit example, see Sec. 4.2 and the appendix of Ref. [94]).
- The `N_rebin` variable can be used to control a further issue. The distribution of the Monte Carlo estimates $\langle\langle\hat{O}\rangle\rangle_C$ is unknown, while a result in the form (mean $\pm$ error) assumes a Gaussian distribution. Every distribution with a finite variance turns into a Gaussian one once it is folded often enough (central limit theorem). Due to the internal averaging (folding) within one bin, many observables are already quite Gaussian. Otherwise one can increase `N_rebin` further, even if the bins are already independent [140].
- The last issue we mention concerns time-displaced correlation functions. Even if the configurations are independent, the fields within the configuration are still correlated. Hence, the data for $S_{\alpha,\beta}(\mathbf{k}, \tau)$ [see Sec. 5.4; Eq. (138)] and $S_{\alpha,\beta}(\mathbf{k}, \tau + \Delta\tau)$ are also correlated. Setting the switch `N_Cov=1` triggers the calculation of the covariance matrix in addition to the usual error analysis. The covariance is defined by

$$\text{COV}_{\tau\tau'} = \frac{1}{N_{\text{Bin}}} \langle (S_{\alpha,\beta}(\mathbf{k}, \tau) - \langle S_{\alpha,\beta}(\mathbf{k}, \tau) \rangle) (S_{\alpha,\beta}(\mathbf{k}, \tau') - \langle S_{\alpha,\beta}(\mathbf{k}, \tau') \rangle) \rangle. \quad (115)$$

An example where this information is necessary is the calculation of mass gaps extracted by fitting the tail of the time-displaced correlation function. Omitting the covariance matrix will underestimate the error.

4.1 The Jackknife resampling method

For each observable $\hat{A}, \hat{B}, \hat{C} \dots$ the Monte Carlo program computes a data set of N_{Bin} (ideally) independent values where for each observable the measurements belong to the same statistical distribution. In the general case, we would like to evaluate a function of expectation values, $f(\langle \hat{A} \rangle, \langle \hat{B} \rangle, \langle \hat{C} \rangle \dots)$ – see for example the expression (28) for the observable including reweighting – and are interested in the statistical estimates of its mean value and the standard error of the mean. A numerical method for the statistical analysis of a given function f which properly handles error propagation and correlations among the observables is the Jackknife method, which is, like the related Bootstrap method, a resampling scheme [141]. Here we briefly review the *delete-1 Jackknife* scheme, which consists in generating N_{bin} new data sets of size $N_{\text{bin}} - 1$ by consecutively removing one data value from the original set. By $A_{(i)}$ we denote the arithmetic mean for the observable $\hat{A}$, without the i -th data value A_i , namely

$$A_{(i)} \equiv \frac{1}{N_{\text{Bin}} - 1} \sum_{k=1, k \neq i}^{N_{\text{Bin}}} A_k. \quad (116)$$

As the corresponding quantity for the function $f(\langle \hat{A} \rangle, \langle \hat{B} \rangle, \langle \hat{C} \rangle \dots)$, we define

$$f_{(i)}(\langle \hat{A} \rangle, \langle \hat{B} \rangle, \langle \hat{C} \rangle \dots) \equiv f(A_{(i)}, B_{(i)}, C_{(i)} \dots). \quad (117)$$

Following the convention in the literature, we will denote the final Jackknife estimate of the mean by $f_{(\cdot)}$ and its standard error by Δf . The Jackknife mean is given by

$$f_{(\cdot)}(\langle \hat{A} \rangle, \langle \hat{B} \rangle, \langle \hat{C} \rangle \dots) = \frac{1}{N_{\text{Bin}}} \sum_{i=1}^{N_{\text{Bin}}} f_{(i)}(\langle \hat{A} \rangle, \langle \hat{B} \rangle, \langle \hat{C} \rangle \dots), \quad (118)$$

and the standard error, including bias correction, is given by

$$(\Delta f)^2 = \frac{N_{\text{Bin}} - 1}{N_{\text{Bin}}} \sum_{i=1}^{N_{\text{Bin}}} [f_{(i)}(\langle \hat{A} \rangle, \langle \hat{B} \rangle, \langle \hat{C} \rangle \dots) - f_{(\cdot)}(\langle \hat{A} \rangle, \langle \hat{B} \rangle, \langle \hat{C} \rangle \dots)]^2. \quad (119)$$

For $f = \langle \hat{A} \rangle$, the equations (118) and (119) reduce to the plain sample average and the standard, bias-corrected, estimate of the error.

4.2 An explicit example of error estimation

In the following we use one of our examples, the Hubbard model on a square lattice in the M_z HS decoupling (see Sec. 9.1), to show explicitly how to estimate errors. We show as well that the autocorrelation time is dependent on the choice of observable. In fact, different observables within the same run can have different autocorrelation times and, of course, this time scale depends on the parameter choice. Hence, the user has to check autocorrelations of individual observables for each simulation! Typical regions of the phase diagram that require special attention are critical points where length scales diverge.

In order to determine the autocorrelation time, we calculate the correlation function

$$S_{\hat{O}}(t_{\text{Auto}}) = \sum_{i=1}^{N_{\text{Bin}} - t_{\text{Auto}}} \frac{(O_i - \langle \hat{O} \rangle)(O_{i+t_{\text{Auto}}} - \langle \hat{O} \rangle)}{(O_i - \langle \hat{O} \rangle)(O_i - \langle \hat{O} \rangle)}, \quad (120)$$

where O_i refers to the Monte Carlo estimate of the observable $\hat{O}$ in the i^{th} bin. This function typically shows an exponential decay and the decay rate defines the autocorrelation time. Fig-

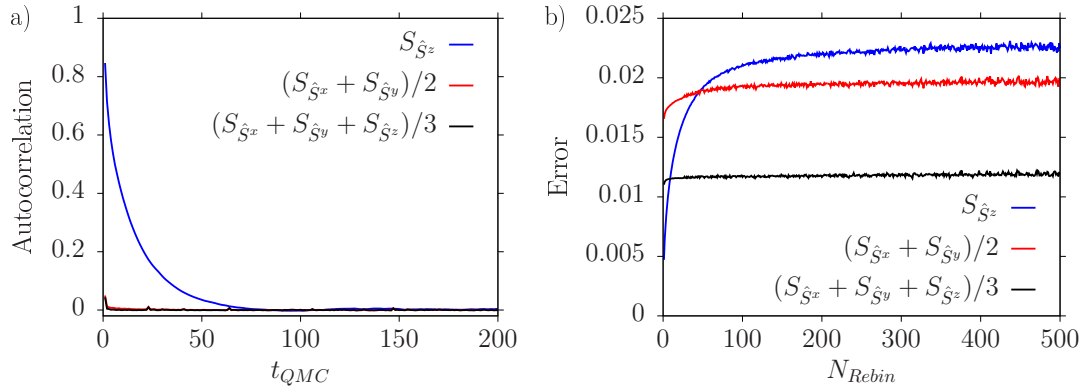


Figure 4: The autocorrelation function $S_{\hat{O}}(t_{\text{Auto}})$ (a) and the scaling of the error with effective bin size (b) of three equal-time, spin-spin correlation functions $\hat{O}$ of the Hubbard model in the M_z decoupling (see Sec. 9.1). Simulations were done on a 6×6 square lattice, with $U/t = 4$ and $\beta t = 6$. We used $N_{\text{auto}} = 500$ (see Sec. 6) and a total of approximately one million bins. The original bin contained only one sweep and we calculated around one million bins on a single core. The different autocorrelation times for the xy -plane compared to the z -direction can be detected from the decay rate of the autocorrelation function (a) and from the point where saturation of the error sets in (b), which defines the required effective bin size for independent measurements. The improved estimator $(S_{\hat{S}_x} + S_{\hat{S}_y} + S_{\hat{S}_z})/3$ appears to have the smallest autocorrelation time, as argued in the text.

Figure 4(a) shows the autocorrelation functions $S_{\hat{O}}(t_{\text{Auto}})$ for three spin-spin-correlation functions [Eq. (138)] at momentum $\mathbf{k} = (\pi, \pi)$ and at $\tau = 0$:

$\hat{O} = S_{\hat{S}_z}$ for the z spin direction, $\hat{O} = (S_{\hat{S}_x} + S_{\hat{S}_y})/2$ for the xy plane, and $\hat{O} = (S_{\hat{S}_x} + S_{\hat{S}_y} + S_{\hat{S}_z})/3$ for the total spin. The Hubbard model has an $SU(2)$ spin symmetry. However, we chose a HS field which couples to the z -component of the magnetization, M_z , such that each individual configuration breaks this symmetry. Of course, after Monte Carlo averaging one expects restoration of the symmetry. The model, on bipartite lattices, shows spontaneous spin-symmetry breaking at $T = 0$ and in the thermodynamic limit. At finite temperatures, and within the so-called renormalized classical regime, quantum antiferromagnets have a length scale that diverges exponentially with decreasing temperatures [142]. The parameter set chosen for Fig. 4 is non-trivial in the sense that it places the Hubbard model in this renormalized classical regime where the correlation length is substantial. Figure 4 clearly shows a very short autocorrelation time for the xy -plane whereas we detect a considerably longer autocorrelation time for the z -direction. This is a direct consequence of the *long* magnetic length scale and the chosen decoupling. The physical reason for the long autocorrelation time corresponds to the restoration of the $SU(2)$ spin symmetry. This insight can be used to define an improved, $SU(2)$ symmetric estimator for the spin-spin correlation function, namely $(S_{\hat{S}_x} + S_{\hat{S}_y} + S_{\hat{S}_z})/3$. Thereby, global spin rotations are no longer an issue and this improved estimator shows the shortest autocorrelation time, as can be clearly seen in Fig. 4(b). Other ways to tackle large autocorrelations are global updates and parallel tempering.

A simple method to obtain estimates of the mean and its standard error from the time series of Monte Carlo samples is provided by the aforementioned facility of rebinning. Also known in the literature as rebatching, it consists in aggregating a fixed number N_{rebin} of adjacent original bins into a new effective bin. In addition to measuring the decay rate of the autocorrelation function (Eq. (120)), a measure for the autocorrelation time can be also

obtained by the rebinning method. For a comparison to other methods of estimating the auto-correlation time we refer the reader to the literature [138, 139, 143]. A reliable error analysis requires independent bins, otherwise the error is typically underestimated. This behavior is observed in Fig. 4 (b), where the effective bin size is systematically increased by rebinning. If the effective bin size is smaller than the autocorrelation time the error will be underestimated. When the effective bin size becomes larger than the autocorrelation time, converging behavior sets in and the error estimate becomes reliable.

4.3 Pseudocode description

The Monte Carlo algorithm as implemented in ALF is summarized in Alg. 1. Key control variables include:

- Projector** Uses (=true) the projective instead of finite- T algorithm (see Sec. 3)
- L_τ Measures ($L_{\text{tau}}=1$) time-displaced observables (see Sec. 2.1.2)
- Tempering** Runs (=true) in parallel tempering mode (see Table 1)
- Global_moves** Carries out (=true) global moves in a single time slice (see Table 1)
- Sequential** Carries out (=true) sequential, single spin-flip updates (see Table 1)
- Langevin** Uses (=true) Langevin dynamics instead of sequential (see Table 1)

Per default, the finite-temperature algorithm is used, $L_{\text{tau}}=0$, and the updating used is Sequential (i.e., Global_moves, Tempering and Langevin default values are all `.false.`).

Algorithm 1 Basic structure of the QMC implementation in Prog/main.f90

```

  ▶ INITIALIZATION
1: call Fields_Init                                ▶ Set the auxiliary fields
2: call Alloc_Ham                                ▶ Select Hamiltonian based on ham_name in parameters
3: call ham%Ham_Set                                ▶ Set the Hamiltonian and the lattice
4: call Nsigma%in                                ▶ Read in an auxiliary-field configuration or generate it randomly
5: for n = L_Trotter to 1 do                        ▶ Fill the storage needed for the first actual MC sweep
6:   call Wrapul                                ▶ Compute propagation matrices and store them at stabilization points
7: end for

  ▶ MONTE CARLO RUN
8: for nbc = 1 to NBin do                        ▶ Loop over bins. The bin defines the unit of Monte Carlo time
9:   for nsw = 1 to NSweep do                    ▶ Loop over sweeps. Each sweep updates twice (upward and
                                                downward in imaginary time) the space-time lattice of auxiliary fields
10:    if Tempering then
11:      call Exchange_Step                        ▶ Perform exchange step in a parallel tempering run
12:    end if
13:    if Global_moves then
14:      call Global_Updates                        ▶ Perform chosen global updates
15:    end if
16:    if Langevin then
17:      call Langevin_update                        ▶ UPDATE AND MEASURE equal-time observables
18:      if Lτ == 1 then
19:        if Projector then
20:          call Tau_p                                ▶ MEASURE time-displaced observables (projective code)
21:        else
22:          call Tau_m                                ▶ MEASURE time-displaced observables (finite temperature)
23:        end if

```

```

24:         end if
25:     end if (Langevin)
26:     if Sequential then
27:         ▷ UPWARD SWEEP
28:         for  $n_\tau = 1$  to  $L_{\text{Trotter}}$  do
29:             call Wrapgrup ▷ PROPAGATE Green function from  $n_\tau - 1$  to  $n_\tau$ , and compute
30:                 its new estimate at  $n_\tau$ , using sequential updates
31:             if  $n_\tau ==$  stabilization point in imaginary time then ▷ STABILIZE
32:                 call Wrapur ▷ Propagate from previous stabilization point to  $n_\tau$ 
33:                 ▷ Storage management:
34:                 – Read from storage: propagation from  $L_{\text{Trotter}}$  to  $n_\tau$ 
35:                 – Write to storage: the just computed propagation
36:                 call CGR ▷ Recalculate the Green function at time  $n_\tau$  in a stable way
37:                 call Control_PrecisionG ▷ Compare propagated and recalculated Greens
38:             end if
39:             if  $n_\tau \in [\text{Lobs\_st}, \text{Lobs\_en}]$  then
40:                 call ham%Obser ▷ MEASURE the equal-time observables
41:             end if
42:         end for
43:         ▷ DOWNWARD SWEEP
44:         for  $n_\tau = L_{\text{Trotter}}$  to 1 do
45:             ▷ Same steps as for the upward sweep (propagation and estimate update, stabilization,
46:                 equal-time measurements) now downwards in imaginary time
47:             if Projector and  $L_\tau == 1$  and
48:                  $n_\tau =$  stabilization point in imaginary time and
49:                 the projection time  $\theta$  is within the measurement interval then
50:                 call Tau_p ▷ MEASURE time-displaced observables (projective code)
51:             end if
52:         end for
53:         ▷ MEASURE time-displaced observables (finite temperature)
54:         if  $L_\tau == 1$  and not Projector then
55:             call Tau_m
56:         end if
57:     end if (Sequential)
58: end for (Sweeps)
59: call ham%Pr_obs ▷ Calculate and write to disk measurement averages for current bin
60: call Nsigma%out ▷ Write auxiliary field configuration to disk
61: end for (Bins)

```

5 Data Structures and Input/Output

To manipulate the relevant physical quantities in a general model, we define a set of corresponding data types. The `Operator` type (Sec. 5.1) is used to specify the interaction as well as the hopping. The handling of the fields is taken care of by the `Fields` type (Sec. 5.2). To

define a Bravais lattice as well as a unit cell we introduce the `Lattice` and `Unit_cell` types (Sec. 5.3). General scalar, equal-time, and time-displaced correlation functions are handled by the `Observable` type (Sec. 5.4). For the projective code, we provide a `WaveFunction` type (Sec. 5.5) to specify the left and right trial wave functions. The Hamiltonian is then specified in the `Hamiltonian` module (Sec. 5.6), making use of the aforementioned types.

5.1 The Operator type

The fundamental data structure in the code is the `Operator`. It is implemented as a Fortran derived data type designed to efficiently define the Hamiltonian (2).

Let the matrix X of dimension $N_{\text{dim}} \times N_{\text{dim}}$ stand for any of the typically sparse, Hermitian matrices $T^{(ks)}$, $V^{(ks)}$ and $I^{(ks)}$ that define the Hamiltonian. Furthermore, let $\{z_1, \dots, z_N\}$ denote the subset of N indices such that

$$X_{x,y} \begin{cases} \neq 0 & \text{if } x, y \in \{z_1, \dots, z_N\} \\ = 0 & \text{otherwise.} \end{cases} \quad (121)$$

Usually, we have $N \ll N_{\text{dim}}$. We define the $N \times N_{\text{dim}}$ matrices P as

$$P_{i,x} = \delta_{z_i,x}, \quad (122)$$

where $i \in [1, \dots, N]$ and $x \in [1, \dots, N_{\text{dim}}]$. The matrix P selects the non-vanishing entries of X , which are contained in the rank- N matrix O defined by:

$$X = P^T O P, \quad (123)$$

and

$$X_{x,y} = \sum_{i,j} P_{i,x} O_{i,j} P_{j,y} = \sum_{i,j} \delta_{z_i,x} O_{i,j} \delta_{z_j,y}. \quad (124)$$

Since the P matrices have only one non-vanishing entry per column, they can conveniently be stored as a vector P , with entries

$$P_i = z_i. \quad (125)$$

There are many useful identities which emerge from this structure. For example:

$$e^X = e^{P^T O P} = \sum_{n=0}^{\infty} \frac{(P^T O P)^n}{n!} = \mathbb{1} + P^T (e^O - \mathbb{1}) P, \quad (126)$$

since

$$P P^T = \mathbb{1}_{N \times N}. \quad (127)$$

In the code, we define a structure called `Operator` that makes use of the properties described above. This type `Operator` bundles the several components, listed in Table 2 and described in the remaining of this section, that are needed to define and use an operator matrix in the program.

5.2 Handling of the fields: the `Fields` type

The partition function (see Sec. 2.1) consists of terms which, in general, can be written as $\gamma e^{g\phi^X}$, where X denotes an arbitrary operator, g is a constant, and γ and ϕ are functions of a complex field f . The ALF includes three different types of fields:

t=1 This type is for an Ising field, therefore $\gamma = 1$ and $\phi = \text{Re } f = \pm 1$,

Variable	Type	Description
Op_X%N	int	Effective dimension N
Op_X%O	cmplx	Matrix O of dimension $N \times N$
Op_X%P	int	Matrix P encoded as a vector of dimension N
Op_X%g	cmplx	Coupling strength g
Op_X%g_t	cmplx	Time dependent coupling strength $g(\tau)$
Op_X%alpha	cmplx	Constant α
Op_X%type	int	Sets the type of HS transformation (1: Ising; 2: discrete HS for perfect-square term; 3: continuous real field)
Op_X%diag	logical	True if O is diagonal
Op_X%U	cmplx	Matrix containing the eigenvectors of O
Op_X%E	dble	Eigenvalues of O
Op_X%N_non_zero	int	Number of non-vanishing eigenvalues of O
Op_X%M_exp	cmplx	Stores $M_exp(:, :, s) = e^{g\phi(s, type)O(:, :, s)}$
Op_X%E_exp	cmplx	Stores $E_exp(:, s) = e^{g\phi(s, type)E(:, s)}$

Table 2: Member variables of the Operator type. In the left column, the letter X is a placeholder for the letters T and V, indicating hopping and interaction operators, respectively. The highlighted variables must be specified by the user. M_exp and E_exp are allocated only if type = 1, 2.

t=2 This type is for the generic HS transformation of Eq. (12) where $\gamma \equiv \gamma(\text{Ref})$ and $\phi = \eta(\text{Ref})$ with $\text{Ref} = \pm 1, \pm 2$ [see Eq. (13)],

t=3 This type is for continuous fields, i.e., $\gamma = 1$ and $\phi = \text{Ref} \in \mathbb{R}$.

t=4 For this type both real and imaginary part of the field ϕ is used. It is limited to the form $\gamma \equiv \gamma(\text{Ref})$ and $\phi = \sqrt{1 + \text{Im}f} \eta(\text{Ref})$, with $\text{Ref} = \pm 1, \pm 2$ and $\text{Im}f \in \mathbb{R}$. This type is useful for simulating terms of the form: $g(1 + \hat{Z})\hat{O}^2$ where in a basis where $\hat{Z}|f\rangle = \text{Im}f|f\rangle$ with $\text{Im}f \in \mathbb{R}$.

For such auxiliary fields a dedicated type Fields is defined, whose components, listed in Table 3, include the variables Field%f and Field%t, which store the field values and types, respectively, and functions such as Field%flip, which flips the field values randomly. Before using this variable type, the routine Fields_init(Amplitude) should be called (its argument is optional and the default value is of unity (see Sec. 2.2.1), in order for internal variables such as η and γ [see Eq. (13)] to be initialized.

5.3 The Lattice and Unit_cell types

ALF's lattice module can generate one- and two-dimensional Bravais lattices. Both the lattice and the unit cell are defined in the module lattices_v3_mod.F90 and their components are detailed in Tables 4 and 5. As its name suggest the module Predefined_Latt_mod.F90 also provides predefined lattices as described in Sec. 8.1. The user who wishes to define his/her own lattice has to specify: 1) unit vectors $\mathbf{a}_1$ and $\mathbf{a}_2$, 2) the size and shape of the lattice, characterized by the vectors $\mathbf{L}_1$ and $\mathbf{L}_2$ and 3) the unit cell characterized by the number of orbitals and their positions. The coordination number of the lattice is specified in the Unit_cell data type. The lattice is placed on a torus (periodic boundary conditions):

$$\hat{c}_{i+L_1} = \hat{c}_{i+L_2} = \hat{c}_i. \quad (128)$$

The function call

```
Call Make_Lattice( L1, L2, a1, a2, Latt )
```

generates the lattice `Latt` of type `Lattice`. The reciprocal lattice vectors $\mathbf{g}_j$ are defined by:

$$\mathbf{a}_i \cdot \mathbf{g}_j = 2\pi \delta_{i,j}, \quad (129)$$

and the Brillouin zone BZ corresponds to the Wigner-Seitz cell of the lattice. With $\mathbf{k} = \sum_i \alpha_i \mathbf{g}_i$, the k-space quantization follows from:

$$\begin{bmatrix} \mathbf{L}_1 \cdot \mathbf{g}_1 & \mathbf{L}_1 \cdot \mathbf{g}_2 \\ \mathbf{L}_2 \cdot \mathbf{g}_1 & \mathbf{L}_2 \cdot \mathbf{g}_2 \end{bmatrix} \begin{bmatrix} \alpha_1 \\ \alpha_2 \end{bmatrix} = 2\pi \begin{bmatrix} n \\ m \end{bmatrix} \quad (130)$$

such that

$$\mathbf{k} = n\mathbf{b}_1 + m\mathbf{b}_2, \text{ with} \quad (131)$$

$$\begin{aligned} \mathbf{b}_1 &= \frac{2\pi}{(\mathbf{L}_1 \cdot \mathbf{g}_1)(\mathbf{L}_2 \cdot \mathbf{g}_2) - (\mathbf{L}_1 \cdot \mathbf{g}_2)(\mathbf{L}_2 \cdot \mathbf{g}_1)} [(\mathbf{L}_2 \cdot \mathbf{g}_2)\mathbf{g}_1 - (\mathbf{L}_2 \cdot \mathbf{g}_1)\mathbf{g}_2], \\ \mathbf{b}_2 &= \frac{2\pi}{(\mathbf{L}_1 \cdot \mathbf{g}_1)(\mathbf{L}_2 \cdot \mathbf{g}_2) - (\mathbf{L}_1 \cdot \mathbf{g}_2)(\mathbf{L}_2 \cdot \mathbf{g}_1)} [(\mathbf{L}_1 \cdot \mathbf{g}_1)\mathbf{g}_2 - (\mathbf{L}_1 \cdot \mathbf{g}_2)\mathbf{g}_1]. \end{aligned} \quad (132)$$

The `Lattice` module also handles the Fourier transformation. For example, the subroutine `Fourier_R_to_K` carries out the transformation:

$$S(\mathbf{k}, :, :, :) = \frac{1}{N_{\text{unit-cell}}} \sum_{i,j} e^{-i\mathbf{k} \cdot (\mathbf{i}-\mathbf{j})} S(\mathbf{i}-\mathbf{j}, :, :, :) \quad (133)$$

and `Fourier_K_to_R` the inverse Fourier transform

$$S(\mathbf{r}, :, :, :) = \frac{1}{N_{\text{unit-cell}}} \sum_{\mathbf{k} \in BZ} e^{i\mathbf{k} \cdot \mathbf{r}} S(\mathbf{k}, :, :, :). \quad (134)$$

In the above, the unspecified dimensions of the structure factor can refer to imaginary-time and orbital indices.

The position of an orbital i is given by $\mathbf{R}_i + \boldsymbol{\delta}_i$. $\mathbf{R}_i$ is a point of the Bravais lattice that defines a unit cell, and $\boldsymbol{\delta}_i$ labels the orbital in the unit cell. This information is stored in the array `Unit_cell%Orb_pos` detailed in Table 5.

The total number of orbitals is then given by `Ndim=Lattice%N*Unit_cell%Norb`. To keep track of the orbital and unit cell structure, it is useful to define arrays `List(Ndim,2)` and `Inv_list(Latt%N, Unit_cell%Norb)`. For a superindex $x = (i, n)$ labeling the unit cell, i , and the orbital, n , of a site on the lattice, we have `List(x,1)=i`, `List(x,2)=n` and `Inv_list(i,n)=x`.

5.4 The observable types `Obser_Vec`, `Obser_Local` and `Obser_Latt`

Our definition of the model includes observables [Eq. (28)]. We define two observable types: `Obser_vec` for an array of *scalar* observables such as the energy, and `Obser_Latt` for correlation functions that have the lattice symmetry. In the latter case, translation symmetry can be used to provide improved estimators and to reduce the size of the output. We also obtain improved estimators by taking measurements in the imaginary-time interval `[LOBS_ST, LOBS_EN]` (see the parameter file in Sec. 5.7.1) thereby exploiting the invariance under translation in imaginary-time. Note that the translation symmetries in space and in time are *broken* for a given configuration C but restored by the Monte Carlo sampling. In general, the user defines

size and number of bins in the parameter file, each bin containing a given amount of sweeps. Within a sweep we run sequentially through the HS and bosonic fields, from time slice 1 to time slice L_{Trotter} and back. The results of each bin are written to a file and analyzed at the end of the run.

To accomplish the reweighting of observables (see Sec. 2.1.3), for each configuration the measured value of an observable is multiplied by the factors ZS and ZP:

$$\text{ZS} = \text{sgn}(C), \quad (135)$$

$$\text{ZP} = \frac{e^{-S(C)}}{\text{Re}[e^{-S(C)}]}. \quad (136)$$

They are computed from the Monte Carlo phase of a configuration,

$$\text{phase} = \frac{e^{-S(C)}}{|e^{-S(C)}|}, \quad (137)$$

which is provided by the main program. Note that each observable structure also includes the average sign [Eq. (29)].

5.4.1 Scalar observables

Scalar observables are stored in the data type `Obser_vec`, described in Table 6. Consider a variable `Obs` of type `Obser_vec`. At the beginning of each bin, a call to `Obser_Vec_Init` in the module `observables_mod.F90` will set `Obs%N=0`, `Obs%Ave_sign=0` and `Obs%Obs_vec(:)=0`. Each time the main program calls the routine `Obser` in the Hamiltonian module, the counter `Obs%N` is incremented by one, the sign [see Eq. (27)] is accumulated in the variable `Obs%Ave_sign`, and the desired observables (multiplied by the sign and $\frac{e^{-S(C)}}{\text{Re}[e^{-S(C)}]}$, see Sec. 2.1.2) are accumulated in the vector `Obs%Obs_vec`. At the end of the bin, a call to `Print_bin_Vec` in module `observables_mod.F90` will append the result of the bin in the file `File_Vec_scal`. Note that this subroutine will automatically append the suffix `_scal` to the filename `File_Vec`. This suffix is important to facilitate automatic analyses of the data at the end of the run. Furthermore, the file `File_Vec_scal_info` is created (if it does not exist yet), which contains a string that specifies how to analyze the observable and an optional description.

5.4.2 Local observables

Local observables are stored in the data type `Obser_Local` described in Table 7. This data structure is useful for dealing with local observables of the form $\hat{O}_{i,\alpha}$, where i is a lattice site and α an orbital index. No translation symmetry is implied such that it is possible to use it in conjunction with pinning fields [13]. Note that this routine is only implemented for equal-time correlation functions.

Consider a variable `Obs` of type `Obser_local`. At the beginning of each bin a call to `Obser_Latt_Init` in the module `observables_mod.F90` will initialize the elements of `Obs` to zero. Each time the main program calls the `Obser` routines at time slice τ one accumulates $\langle\langle \hat{O}_{i,\alpha}(\tau) \rangle\rangle_C \frac{e^{-S(C)}}{\text{Re}[e^{-S(C)}]} \text{sgn}(C)$ in `Obs%Obs_latt(i,1, $\alpha$ )`. At the end of each bin, a call to `Print_bin_Local` in the module `observables_mod.F90` will append the result of the bin in the specified file `Obs%File_Latt`. The routine does not carry out a Fourier transformation and prints the data in real space. We have adopted the following naming conventions: For equal-time observables, defined by having the second dimension of the array `Obs%Obs_local(i, $\tau$ , $\alpha$ )` set to unity, the routine `Print_bin_Latt` attaches the suffix `_eq` to `Obs%File_Latt`. Time displaced local observables are not yet implemented. Furthermore, `Print_bin_Latt` will create a corresponding info file with suffix `_eq_info`, if not already present. The info

file contains the channel, number of imaginary time steps, length of one imaginary time step, unit cell and the vectors defining the Bravais lattice.

5.4.3 Equal-time and time-displaced correlation functions

The data type `Obser_latt` (see Table 8) is useful for dealing with both equal-time and imaginary-time-displaced correlation functions of the form:

$$S_{\alpha,\beta}(\mathbf{k}, \tau) = \frac{1}{N_{\text{unit-cell}}} \sum_{i,j} e^{-i\mathbf{k} \cdot (\mathbf{i}-\mathbf{j})} (\langle \hat{O}_{i,\alpha}(\tau) \hat{O}_{j,\beta} \rangle - \langle \hat{O}_{i,\alpha} \rangle \langle \hat{O}_{j,\beta} \rangle), \quad (138)$$

where α and β are orbital indices and $\mathbf{i}$ and $\mathbf{j}$ lattice positions. Here, translation symmetry of the Bravais lattice is explicitly taken into account. The correlation function splits in a correlated part $S_{\alpha,\beta}^{(\text{corr})}(\mathbf{k}, \tau)$ and a background part $S_{\alpha,\beta}^{(\text{back})}(\mathbf{k})$:

$$S_{\alpha,\beta}^{(\text{corr})}(\mathbf{k}, \tau) = \frac{1}{N_{\text{unit-cell}}} \sum_{i,j} e^{-i\mathbf{k} \cdot (\mathbf{i}-\mathbf{j})} \langle \hat{O}_{i,\alpha}(\tau) \hat{O}_{j,\beta} \rangle, \quad (139)$$

$$\begin{aligned} S_{\alpha,\beta}^{(\text{back})}(\mathbf{k}) &= \frac{1}{N_{\text{unit-cell}}} \sum_{i,j} e^{-i\mathbf{k} \cdot (\mathbf{i}-\mathbf{j})} \langle \hat{O}_{i,\alpha} \rangle \langle \hat{O}_{j,\beta} \rangle \\ &= N_{\text{unit-cell}} \langle \hat{O}_{\alpha} \rangle \langle \hat{O}_{\beta} \rangle \delta(\mathbf{k}), \end{aligned} \quad (140)$$

where translation invariance in space and time has been exploited to obtain the last line. The background part depends only on the expectation value $\langle \hat{O}_{\alpha} \rangle$, for which we use the following estimator

$$\langle \hat{O}_{\alpha} \rangle \equiv \frac{1}{N_{\text{unit-cell}}} \sum_i \langle \hat{O}_{i,\alpha} \rangle. \quad (141)$$

Consider a variable `Obs` of type `Obser_latt`. At the beginning of each bin a call to `Obser_Latt_Init` in the module `observables_mod.F90` will initialize the elements of `Obs` to zero. Each time the main program calls the `Obser` or `ObserT` routines one accumulates $\langle \langle \hat{O}_{i,\alpha}(\tau) \hat{O}_{j,\beta} \rangle \rangle_C \frac{e^{-S(C)}}{\text{Re}[e^{-S(C)}]} \text{sgn}(C)$ in `Obs%Obs_latt(i-j, τ, α, β)` and $\langle \langle \hat{O}_{i,\alpha} \rangle \rangle_C \frac{e^{-S(C)}}{\text{Re}[e^{-S(C)}]} \cdot \text{sgn}(C)$ in `Obs%Obs_latt0(α)`. At the end of each bin, a call to `Print_bin_Latt` in the module `observables_mod.F90` will append the result of the bin in the specified file `Obs%File_Latt`. Note that the routine `Print_bin_Latt` carries out the Fourier transformation and prints the results in k -space. We have adopted the following naming conventions. For equal-time observables, defined by having the second dimension of the array `Obs%Obs_latt(i-j, τ, α, β)` set to unity, the routine `Print_bin_Latt` attaches the suffix `_eq` to `Obs%File_Latt`. For time-displaced correlation functions we use the suffix `_tau`. Furthermore, `Print_bin_Latt` will create a corresponding info file with suffix `_eq_info` or `_tau_info`, if not already present. The info file contains the channel, number of imaginary time steps, length of one imaginary time step, unit cell and the vectors defining the Bravais lattice.

5.5 The WaveFunction type

The projective algorithm (Sec. 3) requires a pair of trial wave functions, $|\Psi_{T,L/R}\rangle$, for which there is the dedicated `WaveFunction` type, defined in the module `WaveFunction_mod` as described in Table 9.

The module `WaveFunction_mod` also includes the routine `WF_overlap(WF_L, WF_R, Z_norm)` for normalizing the right trial wave function `WF_R` by the factor `Z_norm`, such that $\langle \Psi_{T,L} | \Psi_{T,R} \rangle = 1$.

5.6 Specification of the Hamiltonian: the Hamiltonian module

The module `Hamiltonian_main` in `Prog/Hamiltonian_main_mod.F90` defines the interface for all model-specific variables and subroutines needed by the Monte Carlo algorithm, like the hopping, the interaction, the observables, the trial wave function, and optionally updating schemes (see Sec. 2.2). All Hamiltonians (which is the term we are using for an encapsulated model definition) are derived from this main Hamiltonian. In order to implement a new user-defined Hamiltonian, one only has to set up a single submodule of the module `Hamiltonian_main`. Accordingly, this documentation focuses almost entirely on this module and how to derive a new model from it. The remaining parts of the code may hence be treated as a black box.

Table 10 shows all variables declared in `Hamiltonian_main`, they fully define the model. Note that the procedures listed in Table 11 are part of the variable `ham`.

To define a new Hamiltonian called *New_model*, one has to do two things:

1. Add a new line *New_model* to the file `Prog/Hamiltonians.list`
2. Write the new submodule in `Prog/Hamiltonians/Hamiltonian_New_model_smod.F90`

In this new submodule the user can redefine the procedures listed in Table 11, those have to be bound to a new type, which is derived from the Hamiltonian object `ham_base`. The submodule has access to all variables defined in `Hamiltonian_main`, while all variables defined in the submodule are encapsulated. To expose the new Hamiltonian, the user has to define

```
module Subroutine Ham_Alloc_New_model
  allocate(ham_New_model::ham)
end Subroutine Ham_Alloc_New_model
```

where `ham_New_model` is the name of the new type derived from `ham_base`. The rest of the linking is done automatically through the entry in `Prog/Hamiltonians.list`.

Hamiltonian variables to be read in through the parameters file should be written in a specific format, since they will be parsed at compile time and subroutines for reading from parameters file and writing the HDF5 file will be automatically generated. For each namelist, there has to be block of this form:

```
!#PARAMETERS START# <namelist_name>
<var1_type> :: <var1_name> = <var1_default> ! <var1_description>
<var2_type> :: <var2_name> = <var2_default> ! <var2_description>
...
!#PARAMETERS END#
```

Each of those “namelist specifications” starts with a line containing `#PARAMETERSSTART#` and end with a line containing `#PARAMETERSEND#`. The namelist name has to be written after `#PARAMETERSSTART#` on the same line. The variable type specification `<varX_type>` should be either `real`, `integer`, `character` or `logical`, declared as `real(Kind=Kind(0.d0))`, `integer`, `character(len=64)` or `logical` respectively. Each variable needs to have a default value. The description of the parameters is optional. A variable can be commented out, but will still be parsed to be read from parameters. This is to facilitate reading of variables that are already defined in `Hamiltonian_main`, e.g. `N_SUN`.

For example, a namelist called `my_parameter_list` containing `N_SUN` and `Beta` could look like:

```
!#PARAMETERS START# my_parameter_list
!Integer :: N_SUN = 2
real(Kind=Kind(0.d0)) :: Beta = 5.d0 ! Inverse temperature
!#PARAMETERS END#
```

The parsing script `parse_ham.py` in `Prog/` has the option `--test_file` for testing the namelist specifications, e.g. calling:

```
./parse_ham.py --test_file Hamiltonians/Hamiltonian_New_model_smod.F90
```

prints out the results from parsing for manual checking. We recommend doing this after every change in the namelist.

During compilation, the file `Hamiltonian_New_model_read_write_parameters.F90` containing the subroutines `read_parameters` and `write_parameters_hdf5` is generated automatically. The former subroutine can be called in `ham_set`, while the latter has to be bound to `ham_New_model` through:

```
#ifdef HDF5
    procedure, nopass :: write_parameters_hdf5
#endif
```

To help creating a new Hamiltonian, we provide a template `Prog/Hamiltonians/Hamiltonian_##NAME##_smod.F90`, which can be copied to `Prog/Hamiltonians/Hamiltonian_New_model_smod.F90` before being modified. To simplify the implementation of a new Hamiltonian, ALF comes with a set of predefined structures (Sec. 8) which the user can combine together or use as templates.

In order to specify a Hamiltonian, we have to set the matrix representation of the imaginary-time propagators, $e^{-\Delta\tau T^{(ks)}}$, $e^{\sqrt{-\Delta\tau U_k} \eta_{k\tau} V^{(ks)}}$ and $e^{-\Delta\tau s_{k\tau} I^{(ks)}}$, that appear in the partition function (18). For each pair of indices (k, s) , these terms have the general form

$$\text{Matrix Exponential} = e^{g \phi(\text{type})X}. \quad (142)$$

In case of the perfect-square term, we additionally have to set the constant α , see the definition of the operators $\hat{V}^{(k)}$ in Eq. (4). The data structures which hold all the above information are variables of the type `Operator` (see Table 2). For each pair of indices (k, s) , we store the following parameters in an `Operator` variable:

- $\mathbf{P}$ and $\mathbf{O}$ defining the matrix X [see Eq. (123)],
- the constants g, α ,
- optionally: the type `type` of the discrete fields ϕ .

The latter parameter can take one of three values: Ising (1), discrete HS (2), and real (3), as detailed in Sec. 5.2. Note that we have dropped the color index σ , since the implementation uses the $SU(N_{\text{col}})$ invariance of the Hamiltonian.

Accordingly, the following data structures fully describe the Hamiltonian (2):

- For the hopping Hamiltonian (3), we have to set the exponentiated hopping matrices $e^{-\Delta\tau T^{(ks)}}$:

In this case $X^{(ks)} = T^{(ks)}$, and a single variable `Op_T` describes the operator matrix

$$\left(\sum_{x,y}^{N_{\text{dim}}} \hat{c}_{xs}^\dagger T_{xy}^{(ks)} \hat{c}_{ys} \right), \quad (143)$$

where $k = [1, M_T]$ and $s = [1, N_{\text{fl}}]$. In the notation of the general expression (142), we set $g = -\Delta\tau$ (and $\alpha = 0$). In case of the hopping matrix, the type variable takes its default value `Op_T%type = 0`. All in all, the corresponding array of structure variables is `Op_T(M_T, N_fl)`.

- For the interaction Hamiltonian (4), which is of perfect-square type, we have to set the exponentiated matrices $e^{\sqrt{-\Delta\tau}U_k\eta_{k\tau}V^{(ks)}}$:

In this case, $X = V^{(ks)}$ and a single variable `Op_V` describes the operator matrix:

$$\left[\left(\sum_{x,y}^{N_{\text{dim}}} \hat{c}_{xs}^\dagger V_{x,y}^{(ks)} \hat{c}_{ys} \right) + \alpha_{ks} \right], \quad (144)$$

where $k = [1, M_V]$ and $s = [1, N_{\text{fl}}]$, $g = \sqrt{-\Delta\tau U_k}$ and $\alpha = \alpha_{ks}$. The discrete HS decomposition which is used for the perfect-square interaction, is selected by setting the type variable to `Op_V%type = 2`. All in all, the required structure variables `Op_V` are defined using the array `Op_V(M_V, N_{\text{fl}})`.

- For the bosonic interaction Hamiltonian (5), we have to set the exponentiated matrices $e^{-\Delta\tau s_{k\tau}I^{(ks)}}$:

In this case, $X = I^{(k,s)}$ and a single variable `Op_V` then describes the operator matrix:

$$\left(\sum_{x,y}^{N_{\text{dim}}} \hat{c}_{xs}^\dagger I_{xy}^{(ks)} \hat{c}_{ys} \right), \quad (145)$$

where $k = [1, M_I]$ and $s = [1, N_{\text{fl}}]$ and $g = -\Delta\tau$ (and $\alpha = 0$). If this operator couples to an Ising field, we specify the type variable `Op_V%type=1`. On the other hand, if it couples to a scalar field (i.e. real number) then we specify `Op_V%type=3`. All in all, the required structure variables are contained in the array `Op_V(M_I, N_{\text{fl}})`.

- In case of a full interaction [perfect-square term (4) and bosonic term (5)], we define the corresponding doubled array `Op_V(M_V+M_I, N_{\text{fl}})` and set the variables separately for both ranges of the array according to the above.

5.6.1 Flavor symmetries

This code allows the use of time-reversal or particle-hole symmetry to accelerate the algorithm by only explicitly calculating a subset of flavors and reconstructing the complement by symmetry. Here, a pair of flavors ($n_{\text{fl}}, \bar{n}_{\text{fl}}$) are related by a unitary or anti-unitary symmetry, including particle-hole transformations, such that

$$\mathcal{U}^{-1} \text{Op_V}(i, n_{\text{fl}}) \mathcal{U} = \text{Op_V}(i, \bar{n}_{\text{fl}}); \quad \mathcal{U}^{-1} \text{Op_T}(i, n_{\text{fl}}) \mathcal{U} = \text{Op_T}(i, \bar{n}_{\text{fl}}) \quad (146)$$

for any given i . Note that `Op_V(i, n_{\text{fl}})` includes a constant shift α to absorb contributions from the commutator in case of particle-hole symmetries. For example, the particle-hole symmetry requires a non-zero shift $\alpha = 1/2$ in the M_z decoupling of the Hubbard interaction to map the upspin to the downspin. This acceleration is activated by allocating `Calc_Fl` in `Ham_set` and setting the ‘active’ flavors n_{fl} to `.True.` and the symmetry-related flavors $\bar{n}_{\text{fl}}$ to `.False..`

This symmetry allows one to reconstruct one flavor, say $\bar{n}_{\text{fl}}$, from the other, n_{fl} . For unitary symmetries, the weight is given by $Z(\bar{n}_{\text{fl}}) = Z(n_{\text{fl}})$, while anti-unitary symmetries lead to $Z(\bar{n}_{\text{fl}}) = Z(n_{\text{fl}})^*$. This relation has to be provided by the user in `weight_reconstruction`. Up to entry, all weights $Z(n_{\text{fl}})$ have been explicitly calculated by ALF and the user has to fill all ‘inactive’ flavors $\bar{n}_{\text{fl}}$ of the array Z . Similarly, the subroutines `GR_reconstruction` and `GRT_reconstruction` have to be overridden to provide the symmetry reconstruction of the ‘inactive’ flavors $\bar{n}_{\text{fl}}$ of the equal-time and time-displaced Green functions out of n_{fl} , respectively.

Finally we note that if the projective algorithm is used then the trial wave function also has to satisfy the aforementioned symmetry. In particular, assume that the trial wave function

corresponds to the ground state of a single particle Hamiltonian, $\hat{H}_T(n_{\text{fl}})$, then we will require that

$$\mathcal{U}^{-1} \hat{H}_T(n_{\text{fl}}) \mathcal{U} = \hat{H}_T(\bar{n}_{\text{fl}}) \quad (147)$$

5.6.2 Time-dependent Hamiltonians

The Hamiltonian can also include an imaginary time dependency. In that case the imaginary time evolution is given by

$$\begin{aligned} Z \equiv \text{Tr} \left(\mathcal{T} e^{-\int_0^\beta d\tau \hat{\mathcal{H}}(\tau)} \right) = \\ \sum_{\mathcal{C}} e^{-S_0(\{s_{i,\tau}\})} \left(\prod_{k=1}^{M_V} \prod_{\tau=1}^{L_{\text{Trotter}}} \gamma_{k,\tau} \right) e^{N_{\text{col}} \sum_{s=1}^{N_{\text{fl}}} \sum_{k=1}^{M_V} \sum_{\tau=1}^{L_{\text{Trotter}}} \sqrt{-\Delta\tau g_{k,\tau}^V U_k \alpha_{k,s} \eta_{k,\tau}} \\ \times \prod_{s=1}^{N_{\text{fl}}} \left[\det \left(\mathbb{1} + \prod_{\tau=1}^{L_{\text{Trotter}}} \mathbf{B}_{\tau}^{(s)} \right) \right]^{N_{\text{col}}} + \mathcal{O}(\Delta\tau^2)} \end{aligned} \quad (148)$$

with

$$\mathbf{B}_{\tau}^{(s)} = \prod_{k=1}^{M_V} e^{\sqrt{-\Delta\tau g_{k,\tau}^V U_k} \eta_{k,\tau} \mathbf{V}^{(ks)}} \prod_{k=1}^{M_I} e^{-\Delta\tau g_{k,\tau}^I s_{k,\tau} \mathbf{I}^{(ks)}} \prod_{k=1}^{M_T} e^{-\Delta\tau g_{k,\tau}^T T^{(ks)}} \quad (149)$$

In order to recover the partition function of a time independent Hamiltonian, given in Eq. (18), one has to set $g_{k,\tau}^X \rightarrow 1$ with $X = V, I, T$.

By default a constant coupling strength g is assumed. In order to include an imaginary time dependency for a given operator the array `Op_X%g_t(Ltrot)` has to be allocated.

In thermodynamic equilibrium, the Hamiltonian is time independent, and the imaginary time propagation is by definition Hermitian. In this case, the asymmetric and symmetric Trotter decomposition differ by symmetry: for the symmetric (asymmetric) decomposition, the imaginary time propagator is (non-) hermitian. For time dependent Hamiltonians, this distinction cannot be made since generically,

$$\mathcal{T} e^{-\int_0^\beta d\tau \hat{\mathcal{H}}(\tau)} \neq \left(\mathcal{T} e^{-\int_0^\beta d\tau \hat{\mathcal{H}}(\tau)} \right)^\dagger \quad (150)$$

Nevertheless, to guarantee consistency with the rest of the code, we generalize the Trotter decompositions presented in Sec. 2.4 as follows. The operators $\hat{T}_i$ and $\hat{O}_i$ defined in Eq. (87) and (88) acquire an imaginary time index that encodes the time dependence of the coupling constants as explicitly shown above. Since we only allow for a time variation of the coupling constants, the very same Trotter decomposition that we use to define $\hat{T}_i$ can be used to for time dependent problems.

Asymmetric Trotter The asymmetric Trotter decomposition, estimates the imaginary time propagation as

$$\hat{U}_{\text{Approx}} = \prod_{\tau=1}^{L_{\text{Trotter}}} \left(\prod_{i=1}^{N_O} e^{-\Delta\tau \hat{O}_{i,\tau}} \prod_{j=1}^{N_T} e^{-\Delta\tau \hat{T}_{j,\tau}} \right). \quad (151)$$

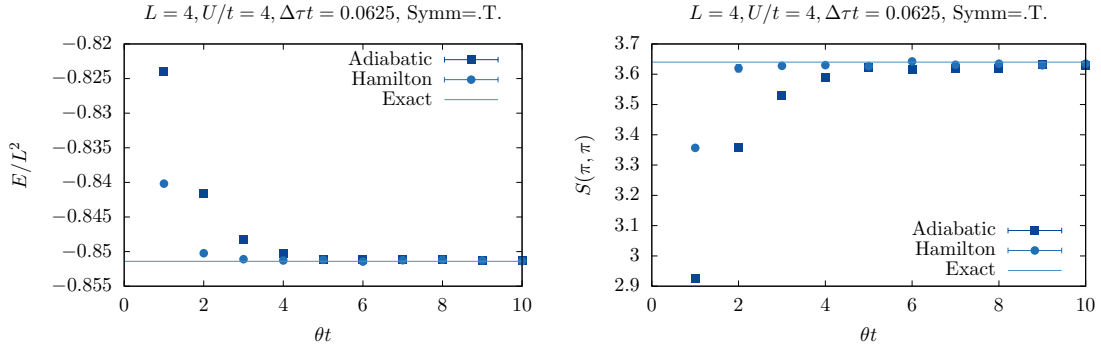


Figure 5: Simulations of the Hubbard model with adiabatic (Fig. 8) switching and Hamilton propagation. Left: Energy. Right: Spin-spin correlations at wave vector (π, π) . The Exact results stem from [144].

Symmetric Trotter For the symmetric Trotter, we use

$$\hat{U}_{\text{Approx}} = \prod_{\tau=1}^{L_{\text{Trotter}}} \left[\left(\prod_{j=1}^{N_T-1} e^{-\frac{\Delta\tau}{4} \hat{T}_{j,\tau}} \right) e^{-\frac{\Delta\tau}{2} \hat{T}_{N_T,\tau}} \left(\prod_{j=N_T-1}^1 e^{-\frac{\Delta\tau}{4} \hat{T}_{j,\tau}} \right) \times \right. \\ \left. \left(\prod_{i=1}^{N_O-1} e^{-\frac{\Delta\tau}{2} \hat{O}_{i,\tau}} \right) e^{-\Delta\tau \hat{O}_{N_O,\tau}} \left(\prod_{i=N_O-1}^1 e^{-\frac{\Delta\tau}{2} \hat{O}_{i,\tau}} \right) \times \right. \\ \left. \left(\prod_{j=1}^{N_T-1} e^{-\frac{\Delta\tau}{4} \hat{T}_{j,\tau}} \right) e^{-\frac{\Delta\tau}{2} \hat{T}_{N_T,\tau}} \left(\prod_{j=N_T-1}^1 e^{-\frac{\Delta\tau}{4} \hat{T}_{j,\tau}} \right) \right]$$

As for the imaginary time independent Hamiltonians, we save computational resources by approximating:

$$\left(\prod_{j=1}^{N_T-1} e^{-\frac{\Delta\tau}{4} \hat{T}_{j,\tau+1}} \right) e^{-\frac{\Delta\tau}{2} \hat{T}_{N_T,\tau+1}} \left(\prod_{j=N_T-1}^1 e^{-\frac{\Delta\tau}{4} \hat{T}_{j,\tau+1}} \right) \left(\prod_{j=1}^{N_T-1} e^{-\frac{\Delta\tau}{4} \hat{T}_{j,\tau}} \right) e^{-\frac{\Delta\tau}{2} \hat{T}_{N_T,\tau}} \left(\prod_{j=N_T-1}^1 e^{-\frac{\Delta\tau}{4} \hat{T}_{j,\tau}} \right) \simeq \\ \left(\prod_{j=1}^{N_T-1} e^{-\frac{\Delta\tau}{4} (\hat{T}_{j,\tau+1} + \hat{T}_{j,\tau})} \right) e^{-\frac{\Delta\tau}{2} (\hat{T}_{N_T,\tau+1} + \hat{T}_{N_T,\tau})} \left(\prod_{j=N_T-1}^1 e^{-\frac{\Delta\tau}{4} (\hat{T}_{j,\tau+1} + \hat{T}_{j,\tau})} \right) \quad (152)$$

We conclude this section by comparing the generic projector QMC code where the imaginary time propagation is carried out with the Hamiltonian and the adiabatic scheme in which the interaction is adiabatically switched on according to Fig. 8.

5.7 File structure

The code package, summarized in Table 12, consists of the program directories `Prog/`, `Libraries/`, `Analysis/`, and the directory `Scripts_and_Parameters_files/`, which contains supporting scripts and, in its subdirectory `Start`, the input files necessary for a run, described in the Sec. 5.7.1 as well as `Mathematica/` that contains Mathematica notebooks to evaluate higher order correlation functions with Wicks theorem as described in Appendix A. The routines available in the directory `Analysis/` are described in Sec. 6.3, and the testsuite in Sec. 6.2.

Below we describe the structure of ALF's input and output files. Notice that the input/output files for the Analysis routines are described in Sec. 6.3.

5.7.1 Input files

The package's two input files are described in Table 13. The parameter file Start/parameters has the following form – using as an example the Hubbard model on a square lattice (see Sec. 9.1 for the general SU(N) Hubbard and Sec. 7 for a detailed walk-through on its plain vanilla version):

```

=====
!   Input variables for a general ALF run
=====
&VAR_ham_name           !! Use Hamiltonian defined in
ham_name = "Hubbard"     ! Prog/Hamiltonians/Hamiltonian_{ham_name}_smod.F90
/

&VAR_lattice            !! Parameters defining the specific lattice and base model
L1      = 6              ! Length in direction a_1
L2      = 6              ! Length in direction a_2
Lattice_type = "Square"  ! Sets a_1 = (1,0), a_2=(0,1), Norb=1, N_coord=2
Model    = "Hubbard"     ! Sets the Hubbard model, to be specified in &VAR_Hubbard
/

&VAR_Model_Generic      !! Common model parameters
Checkerboard = .T.       ! Whether checkerboard decomposition is used
Symm         = .T.       ! Whether symmetrization takes place
N_SUN        = 2         ! Number of colors
N_FL         = 1         ! Number of flavors
Phi_X        = 0.d0      ! Twist along the L_1 direction, in units of the flux quanta
Phi_Y        = 0.d0      ! Twist along the L_2 direction, in units of the flux quanta
Bulk         = .T.       ! Twist as a vector potential (.T.); at the boundary (.F.)
N_Phi        = 0         ! Total number of flux quanta traversing the lattice
Dtau         = 0.1d0     ! Thereby Ltrot=Beta/dtau
Beta         = 5.d0      ! Inverse temperature
Projector    = .F.       ! Whether the projective algorithm is used
Theta        = 10.d0     ! Projection parameter
/

&VAR_QMC               !! Variables for the QMC run
Nwrap         = 10       ! Stabilization. Green functions will be computed from
                        ! scratch after each time interval Nwrap*Dtau
NSweep        = 20       ! Number of sweeps
NBin          = 5        ! Number of bins
Ltau          = 1        ! 1 to calculate time-displaced Green functions; 0 otherwise
LOBS_ST       = 0        ! Start measurements at time slice LOBS_ST
LOBS_EN       = 0        ! End measurements at time slice LOBS_EN
CPU_MAX       = 0.0      ! Code stops after CPU_MAX hours, if 0 or not
                        ! specified, the code stops after Nbin bins
Propose_S0    = .F.      ! Proposes single spin flip moves with probability exp(-S0)
Global_moves  = .F.      ! Allows for global moves in space and time
N_Global      = 1        ! Number of global moves per sweep
Global_tau_moves = .F.   ! Allows for global moves on a single time slice.
N_Global_tau  = 1        ! Number of global moves that will be carried out on a
                        ! single time slice
Sequential    = .T.      ! Flag for sequential moves
Nt_sequential_start = 0   ! One can combine sequential & global moves on a time slice
Nt_sequential_end   = -1  ! The program then carries out sequential local moves in the
                        ! range [Nt_sequential_start, Nt_sequential_end] followed by
                        ! N_Global_tau global moves
Langevin      = .F.      ! Langevin update
Delta_t_Langevin_HMC = 0.01 ! Default time step for Langevin and HMC updates
Max_Force     = 1.5      ! Max force for Langevin
HMC           = .F.      ! Flag for hybrid molecular dynamics update
N_HMC_sweeps  = 1        ! Number of hybrid molecular dynamic updates between

```

```

! sequential sweep
Leapfrog_steps      = 0      ! Number of leapfrog iterations
Amplitude           = 1.0    ! Width of the box distribution for update of
                             ! type t=3,4 fields. Defaults to 1.0.

Sequential_MALA      = .F.    ! Sequential MALA updates. Requires continuous fields
Delta_t_MALA_sequential = 0.1d0 ! Time step for sequential MALA updates
Max_Force_MALA_sequential = 1.d0 ! Maximum force for sequential MALA updates

Global_tau_MALA_moves = .F.    ! Allows for global tau MALA updates
N_Global_tau_MALA      = 0      ! Number of global tau MALA moves
Delta_t_MALA_global_tau = 0.1d0 ! Time step for global tau MALA updates
Max_Force_MALA_global_tau = 1.d0 ! Maximum force for global tau MALA updates

Global_MALA_moves     = .F.    ! Allows for global MALA updates
N_Global_MALA_sweeps   = 1      ! Number of global MALA sweeps
Delta_t_MALA_global    = 0.1d0 ! Time step for global MALA updates
Max_Force_MALA_global  = 1.d0 ! Maximum force for global MALA updates
/

&VAR_errors           !! Variables for analysis programs
n_skip = 1             ! Number of bins that to be skipped
N_rebin = 1            ! Rebinning
N_Cov = 0              ! If set to 1 covariance computed for non-equal-time
                             ! correlation functions
N_auto = 0             ! If > 0 triggers calculation of autocorrelation
N_Back = 1             ! If set to 1, subtract background in correlation functions
Extended_zone = .F.    ! Allows for Fourier analysis in extended zone scheme
N_BZ_Zones = 1         ! Number of Brillouin zones that will be covered if extended
                             ! zone scheme is activated. Default = Norb
/

&VAR_TEMP             !! Variables for parallel tempering
N_exchange_steps      = 6      ! Number of exchange moves [see Eq. (43)]
N_Tempering_frequency = 10     ! The frequency in units of sweeps at which the
                             ! exchange moves are carried out
mpi_per_parameter_set = 2      ! Number of mpi-processes per parameter set
Tempering_calc_det     = .T.    ! Specifies whether the fermion weight has to be taken
                             ! into account while tempering. The default is .true.,
                             ! and it can be set to .F. if the parameters that
                             ! get varied only enter the free bosonic action S_0
/

&VAR_Max_Stoch        !! Variables for Maximum entropy
                             ! The attribute Stoch means that this parameter is
                             ! only required for the stochastic approach.
Ngamma = 400           ! Number of delta-functions for parametrization (Stoch)
Om_st = -10.d0         ! Frequency range lower bound
Om_en = 10.d0          ! Frequency range upper bound
NDis = 2000            ! Number of boxes for histogram, frequency discretization.
Nbins = 250            ! Number of bins for Monte Carlo (Stoch)
Nsweeps = 70           ! Number of sweeps per bin (Stoch)
NWarm = 20             ! The Nwarm first bins will be omitted (Stoch)
N_alpha = 14           ! Number of temperatures (Stoch)
alpha_st = 1.d0        ! Smallest inverse temperature increment for inverse (Stoch)
R = 1.2d0              ! temperature (see above) (Stoch)
Checkpoint = .F.       ! Whether to produce dump files, allowing the simulation
                             ! to be resumed later on (Stoch)
Tolerance = 0.1d0      ! Data points for which the relative error exceeds the
                             ! tolerance threshold will be omitted.
Stochastic = .T.       ! If true, stochastic maxent is called
                             ! If false, classic maxent is called

```

```
/
&VAR_Hubbard          !! Variables for the specific model
Mz                    = .T.      ! When true, sets the M_z-Hubbard model: Nf=2, demands that
                                ! N_sun is even, HS field couples to the z-component of
                                ! magnetization; otherwise, HS field couples to the density
Continuous = .F.      ! Uses (T: continuous; F: discrete) HS transformation
ham_T          = 1.d0   ! Hopping parameter
ham_chem       = 0.d0   ! Chemical potential
ham_U          = 4.d0   ! Hubbard interaction
ham_T2         = 1.d0   ! For bilayer systems
ham_U2         = 4.d0   ! For bilayer systems
ham_Tperp      = 1.d0   ! For bilayer systems
/
```

Component	Description	
Variable	Type	
Field%t(1:n_op)	int	Sets the HS transformation type (1: Ising; 2: discrete HS for perfect-square term; 3: continuous real field). The index runs through the operator sequence
Field%f(1:n_op, 1:Ltrot)	cmplx	Defines the auxiliary fields. The first index runs through the operator sequence and the second through the time slices. For $t=1$, $f = \pm 1$; for $t=2$, $f = \pm 1, \pm 2$; for $t=3$, $f \in \mathbb{R}$; and for $t=4$, $f \in \mathbb{C}$, $\text{Real}(f) = \pm 1, \pm 2$, $\text{Im}(f) \in \mathbb{R}$
Field%flip_protocol(1:n_op)	int	Gives the possibility to choose the flipping protocol of the field for type $t=4$ fields. 1: real or imaginary parts of the field are updated with 0.5 probability (default) 2: both real and imaginary parts are updated 3: only real part 4: only imaginary part.
amplitude	dble	Random flip width for fields of types $t=3, 4$, defaults to 1
Method(arguments)		
Field%make(n_op, Ltrot)		Reserves memory for the field
Field%clear()		Clears field from memory
Field%set()		Sets a random configuration
Field%flip(n, nt)		Flips the field values randomly for field n on time slice nt . For $t=1$ it flips the sign of the Ising spin. For $t=2$ it randomly choose one of the three other values of l . For $t=3$, $f = f + \text{amplitude} * (\text{ranf}() - 1/2)$. For $t=4$, the real part is flipped as for $t=2$, and the imaginary part as for $t=3$. The specifics of the flip is encoded in the flip_protocol variable described above.
Field%phi(n, nt)		Returns ϕ for the n -th operator at the time slice nt . For $t=1$, $3 \phi = f$; for $t=2 \phi = \eta(\text{Re } f)$; and for $t=4 \phi = \sqrt{1 + \text{Im } f} \eta(\text{Re } f)$
Field%gamma(n, nt)		Returns γ for the n -th operator at the time slice nt For $t=1, 3 \gamma = 1$; and for $t=2, 4 \gamma = \gamma(\text{Re } f)$
Field%i(n, nt)		Returns Field%f rounded to nearest integer (if $t=1$ or 2)
Field%in(Group_Comm, In_field)		If the file <code>confin_np</code> (or <code>confin_np.h5</code>) exists it reads the field configuration from this file. Otherwise if <code>In_field</code> is present it sets the fields to <code>In_field</code> . If both <code>confin_np(.h5)</code> and <code>In_field</code> are not provided it sets a random field by calling <code>Field%set()</code> . Here <code>np</code> is the rank number of the process
Field%out(Group_Comm)		Writes out the field configuration

Table 3: Components of a variable of type `Fields` named `Field`. The routine `Fields_init(del)` should be called before the use of this variable type, since it initializes necessary internal variables such as $\eta(l)$, $\gamma(l)$ [see Eq. (13)]. Note that `del` and `amplitude` are private variables of the `fields` module. The integers `n_op` and `Ltrot` are the number of interacting operators per time slice and time slices, respectively, `Group_Comm` (integer) is an MPI communicator defined by the main program, and the optional `In_field` stores the initial field configuration.

Variable	Type	Description
Latt%a1_p, Latt%a2_p	dbl	Unit vectors $\mathbf{a}_1, \mathbf{a}_2$
Latt%L1_p, Latt%L2_p	dbl	Vectors $\mathbf{L}_1, \mathbf{L}_2$ that define the topology of the lattice Tilted lattices are thereby possible to implement
Latt%N	int	Number of lattice points, $N_{\text{unit-cell}}$
Latt%list	int	Maps each lattice point $i = 1, \dots, N_{\text{unit-cell}}$ to a real space vector denoting the position of the unit cell: $\mathbf{R}_i = \text{list}(i, 1)\mathbf{a}_1 + \text{list}(i, 2)\mathbf{a}_2 \equiv i_1\mathbf{a}_1 + i_2\mathbf{a}_2$
Latt%invlist	int	Return lattice point from position: $\text{Invlist}(i_1, i_2) = i$
Latt%nnlist	int	Nearest neighbor indices: $j = \text{nnlist}(i, n_1, n_2)$, $ n_1 , n_2 \leq \text{nnlist_range}$, $\mathbf{R}_j = \mathbf{R}_i + n_1\mathbf{a}_1 + n_2\mathbf{a}_2$
Latt%imj	int	$\mathbf{R}_{\text{imj}(i,j)} = \mathbf{R}_i - \mathbf{R}_j$, with $\text{imj}, i, j \in 1, \dots, N_{\text{unit-cell}}$
Latt%BZ1_p, Latt%BZ2_p	dbl	Reciprocal space vectors $\mathbf{g}_i$ [See Eq. (129)]
Latt%b1_p, Latt%b2_p	dbl	k -quantization [See Eq. (132)]
Latt%listk	int	Maps each reciprocal lattice point $k = 1, \dots, N_{\text{unit-cell}}$ to a reciprocal space vector $\mathbf{k}_k = \text{listk}(k, 1)\mathbf{b}_1 + \text{listk}(k, 2)\mathbf{b}_2 \equiv k_1\mathbf{b}_1 + k_2\mathbf{b}_2$
Latt%invlistk	int	$\text{Invlistk}(k_1, k_2) = k$
Latt%b1_perp_p, Latt%b2_perp_p	dbl	Orthonormal vectors to $\mathbf{b}_i$ (for internal use)

Table 4: Components of the Lattice type for two-dimensional lattices using as example the default lattice name Latt. The highlighted variables must be specified by the user. Other components of Lattice are generated upon calling: `Call Make_Lattice(L1, L2, a1, a2, Latt, nnlist_range)`. The variable `nnlist_range` is optional and defaults to unity.

Variable	Type	Description
Norb	int	Number of orbitals
N_coord	int	Coordination number
Orb_pos(1..Norb, 2[3])	dbl	Orbitals' positions, measured from the lattice site

Table 5: Components of an instance Latt_unit of the Unit_cell type. The highlighted variables have to be specified by the user. Note that for bilayer lattices the second index of the Orb_pos array ranges from 1 to 3.

Variable	Type	Description	Contribution
N	int	Number of measurements	+1
Ave_sign	dble	Cumulated average sign [Eq. (29)]	$\text{sgn}(C)$
Obs_vec(:)	cmplx	Cumulated vector of observables [Eq. (28)]	$\langle \langle \hat{O}(\cdot) \rangle \rangle_C \frac{e^{-S(C)}}{\text{Re}[e^{-S(C)}]} \text{sgn}(C)$
File_Vec	char	Name of output file	
analysis_mode	char	How to analyze the observable Default value: “identity”	
description(:)	char	Optional description. Arbitrary number of 64-character lines	

Table 6: Components of a variable of type `Obser_vec`. The contribution listed is that of each configuration C .

Variable	Type	Description	Contribution
Obs%N	int	Number of measurements	+1
Obs%Ave_sign	dble	Cumulated sign [Eq. (29)]	$\text{sgn}(C)$
Obs%Obs_latt(i, τ, α)	cmplx	Cumulated correlation function	$\langle \langle \hat{O}_{i,\alpha}(\tau) \rangle \rangle_C \times \frac{e^{-S(C)}}{\text{Re}[e^{-S(C)}]} \text{sgn}(C)$
Obs%File_Latt	char	Name of output file	
Obs%Latt	Lattice*	Bravais lattice [Tab. 4]	
Obs%Latt_unit	Unit_cell*	Unit cell [Tab. 5]	
Obs%dtau	dble	Imaginary time step	
Obs%Channel	char	Channel for Maximum Entropy	

Table 7: Components of a variable of type `Obser_local` named `Obs`. Be aware: The types marked with asterisks, *, are actually pointers; when the subroutine `Obser_Latt_make` creates an observable `Obs`, the variables `Latt` and `Latt_unit` do not get copied but a pointer to them is stored, implying that their modification after the creation of `Obs` still affects the observable. Each observable can have its own lattice and unit-cell that can differ from the lattice and unit cell associated to the Hamiltonian. For example, when simulation a two-dimensional Hubbard model, one can define an observable on a one-dimensional sub-lattice. We refer the reader to Sec. 8.4 for further discussion on this option.

Variable	Type	Description	Contribution
Obs%N	int	Number of measurements	+1
Obs%Ave_sign	dble	Cumulated sign [Eq. (29)]	$\text{sgn}(C)$
Obs%Obs_latt($i-j, \tau, \alpha, \beta$)	cmplx	Cumulated correlation function [Eq. (28)]	$\langle \langle \hat{O}_{i,\alpha}(\tau) \hat{O}_{j,\beta} \rangle \rangle_C \times \frac{e^{-S(C)}}{\text{Re}[e^{-S(C)}]} \text{sgn}(C)$
Obs%Obs_latt0(α)	cmplx	Cumulated expected value [Eq. (28)]	$\langle \langle \hat{O}_{i,\alpha} \rangle \rangle_C \times \frac{e^{-S(C)}}{\text{Re}[e^{-S(C)}]} \text{sgn}(C)$
Obs%File_Latt	char	Name of output file	
Obs%Latt	Lattice*	Bravais lattice [Tab. 4]	
Obs%Latt_unit	Unit_cell*	Unit cell [Tab. 5]	
Obs%dtau	dble	Imaginary time step	
Obs%Channel	char	Channel for Maximum Entropy	

Table 8: Components of a variable of type Obser_latt named Obs. Be aware: The types marked with asterisks, *, are actually pointers; when the subroutine Obser_Latt_make creates an observable Obs, the variables Latt and Latt_unit do not get copied but a pointer to them is stored, implying that their modification after the creation of Obs still affects the observable. Each observable can have its own lattice and unit-cell that can differ from the lattice and unit cell associated to the Hamiltonian. For example, when simulation a two-dimensional Hubbard model, one can define an observable on a one-dimensional sub-lattice. We refer the reader to Sec. 8.4 for further discussion on this option.

Variable	Type	Description
WF%P(:, :)	cmplx	P is an $N_{\text{dim}} \times N_{\text{part}}$ matrix, where N_{part} is the number of particles
WF%Degen	dble	It stores the energy difference between the last occupied and first unoccupied single particle state and can be used to check for degeneracy

Table 9: Components of a variable of type WaveFunction named WF.

Public Variable	Type	Description
ham	class(ham_base)	Hamiltonian object. All model dependent procedures are attached to this variable (see Table 11).
Op_V	Operator	Interaction
Op_T	Operator	Hopping
WF_L	WaveFunction	Left trial wave function
WF_R	WaveFunction	Right trial wave function
nsigma	Fields	Fields
Ndim	int	Number of sites
N_Fl	int	Number of flavors
N_SUN	int	Number of colors
Ltrot	int	Total number of trotter silces
Thtrot	int	Number of trotter slices reserved for projection
Projector	logical	Enable projector code
Group_Comm	int	Group communicator for MPI
Symm	logical	Symmetric trotter
Calc_Fl	logical	Explicitly calculation of flavors (optional)
Private Variable	Type	Description
Obs_scal	Obser_Vec	Storage for measured scalar observables
Obs_eq	Obser_Latt	Storage for measured equal time correlations
Obs_tau	Obser_Latt	Storage for measured time displaced correlations

Table 10: List of the public and private variables declared in the module Hamiltonian. The highlighted variables have to be set in the subroutine ham_set.

Procedure	Description	Section
Ham_Set	Reads in model and lattice parameters from the file parameters. Sets the Hamiltonian, which is commonly split up into subroutines Ham_Latt, Ham_Hop, Ham_V and Ham_Trial Ham_Latt: Sets the Lattice and the Unit_cell as well as the the arrays List and Inv_list required for multiorbital problems Ham_hop: Sets the hopping term $\hat{H}_T$ (i.e., operator Op_T) by calling Op_make and Op_set Ham_V: Sets the interaction term $\hat{H}_V$ (i.e., operator Op_V) by calling Op_make and Op_set Ham_Trial: Sets the trial wave function for the projective code $ \Psi_{T,L/R}\rangle$ specified by the Wavefunction type	5.6, 9 5.3, 7.3 8.1 5.1, 7.4, 8.2 5.1, 7.5, 8.3 5.5, 7.6, 8.5
Alloc_obs	Assigns memory storage to the observable	5.4, 7.7.1
Obser	Computes the scalar and equal-time observables	5.4, 7.7.2, 8.4
ObserT	Computes time-displaced correlation functions	5.4, 7.7.3, 8.4
S0	Returns the ratio $e^{S_0(C')}/e^{-S_0(C)}$ for a single spin flip	2.2.1.1
Global_move_tau	Generates a global move on a given time slice τ . This routine is only called if Global_tau_moves=True and N_Global_tau>0	2.2.3.1
Override_global_tau_sampling_parameters	Allows setting global_tau parameters at run time	2.2.3.1
Hamiltonian_set_nsigma	Sets the initial field configuration. This routine is to be modified if one wants to specify the initial configuration. By default the initial configuration is assumed to be random	
Global_move	Handles global moves in time and space	2.3
Get_Delta_S0_global	Computes $\Delta S_0 = S_0(C) - S_0(C')$ for a global move	2.3
Init_obs	Initializes the observables to zero. Usually, this doesn't have to be modified.	
Pr_obs	Writes the observables to disk by calling Print_bin of the Observables module. Usually this does not have to be modified	
weight_reconstruction	Reconstructs of the weight for 'inactive' flavors	5.6.1
GR_reconstruction	Reconstructs the Green function GR for the 'inactive' flavors	5.6.1
GRT_reconstruction	Reconstructs the time-displaced Green functions GOT and GT0	5.6.1

Table 11: Typebound procedures bound to type ham_base. To define a new model, at least Ham_Set has to be overloaded in the Hamiltonian submodule. For measurements Alloc_obs, Obser (and ObserT for time displaced observables) are necessary. The other procedures are needed for optional features.

Directory	Description
Prog/	Main program and subroutines
Libraries/	Collection of mathematical routines
Analysis/	Routines for error analysis
Scripts_and_Parameters_files/	Helper scripts and the Start/ directory, which contains the files required to start a run
Documentation/	This documentation
Mathematica/	Mathematica notebooks to evaluate higher order correlation functions with Wicks theorem
testsuite/	An automatic test suite for various parts of the code

Table 12: Overview of the directories included in the ALF package.

File	Description
parameters	Defines which Hamiltonian to use and sets the parameters for: lattice, model, QMC process, and error analysis
seeds	List of integer numbers to initialize the random number generator and to start a simulation from scratch

Table 13: Overview of the input files required for a simulation, which can be found in the subdirectory Scripts_and_Parameters_files/Start/.

The program allows for a number of different updating schemes. If no other variables are specified in the VAR_QMC name space, then the program will run in its default mode, namely the sequential single spin-flip mode. In particular, note that if `Nt_sequential_start` and `Nt_sequential_end` are not specified and that the variable `Global_tau_moves` is set to true, then the program will carry out only global moves, by setting `Nt_sequential_start=1` and `Nt_sequential_end=0`.

5.7.2 Output files – observables

The standard output files are listed in Table 14 and Table 15 for plain-text and HDF5 output, respectively. Notice that, besides these files, which contain direct QMC outputs, ALF can also produce a number of analysis output files, discussed in Sec. 6.3.

For converting plain-text output to HDF5, see Sec. 6.6.

The output of the measured data is organized in bins. One bin corresponds to the arithmetic average over a fixed number of individual measurements which depends on the chosen measurement interval [`LOBS_ST`, `LOBS_EN`] on the imaginary-time axis and on the number `NSweep` of Monte Carlo sweeps. If the user runs an MPI parallelized version of the code, the average also extends over the number of MPI threads.

File	Description
<code>info</code>	Summary after completion of the simulation, including parameters of the model and the QMC run and simulation metrics (precision, acceptance rate, wallclock time)
<code>X_scal</code>	Results of equal-time measurements of scalar observables. The placeholder X stands for the observables <code>Kin</code> , <code>Pot</code> , <code>Part</code> , and <code>Ener</code>
<code>X_scal_info</code>	Info on how to analyze the observable and optionally a description.
<code>Y_eq, Y_tau, Y_local</code>	Results of equal-time and time-displaced measurements of correlation functions. The placeholder Y stands for <code>Green</code> , <code>SpinZ</code> , <code>SpinXY</code> , <code>Den</code> , etc.
<code>Y_eq_info, Y_tau_info, Y_local</code>	Additional info, like Bravais lattice and unit cell, for equal-time and time-displaced observables
<code>confout_<thread#></code>	Output files (one per MPI instance) for the HS and bosonic configuration

Table 14: Overview of the standard output files if compiled without HDF5. See Sec. 5.4 for the definitions of observables and correlation functions and Table 15 for HDF5 output.

The formatting of a single bin's output depends on the observable type, `Obs_vec` or `Obs_Latt`:

- Observables of type `Obs_vec`: For each additional bin, a single new line is added to the output file. In case of an observable with `N_size` components, the formatting is

```
N_size+1 <measured value, 1> ... <measured value, N_size> <measured sign>
```

The counter variable `N_size+1` refers to the number of measurements per line, including the phase measurement. This format is required by the error analysis routine (see

File	Description
info	Same as in Tab. 14
data.h5	Contains the same information as the scalar, equal-time correlation and time-displaced correlation operators as in Tab. 14, but in one single HDF5 file. This file also includes all Hamiltonian parameters defined as specified in Sec. 5.6 (see also Fig. 6). Note: The parameter names in the HDF5 file are all lower case
confout_<thread#>.h5	Output files (one per MPI instance) for the HS and bosonic configuration, in HDF5 format

Table 15: Overview of the standard output files if compiled with HDF5. See Sec. 5.4 for the definitions of observables and correlation functions.

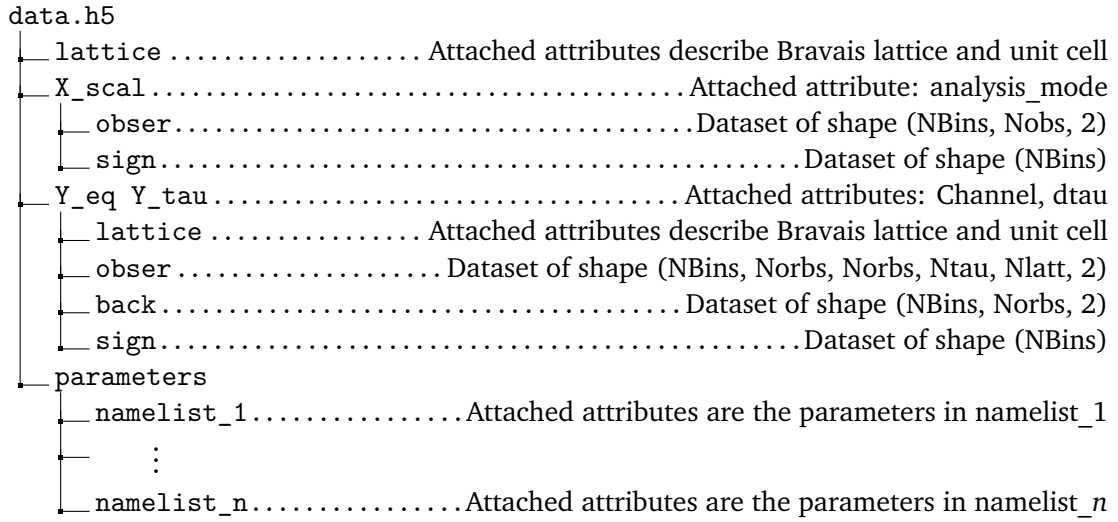


Figure 6: Structure of HDF5 output file data.h5. In parameters all n namelists connected with the simulated Hamiltonian can be found.

Sec. 6.3). Scalar observables like kinetic energy, potential energy, total energy and particle number are treated as a vector of size $N_size=1$.

- Observables of type `Obs_Latt`: For each additional bin, a new data block is added to the output file. The block consists of the expectation values [Eq. (141)] contributing to the background part [Eq. (140)] of the correlation function, and the correlated part [Eq. (139)] of the correlation function. For imaginary-time displaced correlation functions, the formatting of the block is given by:

```

<measured sign> <N_orbital> <N_unit_cell> <N_time_slices> <dtau> <Channel>
do alpha = 1, N_orbital
  <O_alpha>
enddo
do i = 1, N_unit_cell
  <reciprocal lattice vector k(i)>
  do tau = 1, N_time_slices
    do alpha = 1, N_orbital
      do beta = 1, N_orbital
        <S_alpha_beta^(corr)(k(i), tau)>
      enddo
    enddo
  enddo
enddo

```

```

        enddo
    enddo
enddo

```

The same block structure is used for equal-time correlation functions, except for the entries `<N_time_slices>`, `<dtau>` and `<Channel>`, which are then omitted. Using this structure for the bins as input, the full correlation function $S_{\alpha,\beta}(\mathbf{k}, \tau)$ [Eq. (138)] is then calculated by calling the error analysis routine (see Sec. 6.3).

6 Using the Code

In this section we describe the steps for compiling and running the code from the shell, and describe how to search for optimal parameter values as well as how to perform the error analysis of the data.

The source code of ALF dev. is available at <https://github.com/ALF-QMC/ALF/tree/master/> and can be cloned with git or downloaded from the repository (make sure to choose the appropriate release).

A Python interface, **pyALF**, is also available and can be found, together with a number of Jupyter notebooks exploring the interface’s capabilities, at <https://github.com/ALF-QMC/pyALF/tree/master/>, with the documentation <https://alf.physik.uni-wuerzburg.de/pyalf-doc/>. This interface facilitates setting up simple runs and is ideal for setting benchmarks and getting acquainted with ALF. Some of pyALF’s notebooks form the core of the introductory part of the [ALF Tutorial](#), where pyALF’s usage is described in more detail.

We start out by providing step-by-step instructions that allow a first-time user to go from zero to performing a simulation and reading out their first measurement using ALF.

6.1 Quick Start

The aim of this section is to provide a fruitful and stress-free first contact with the package. Ideally, it should be possible to copy and paste the instructions below to a Debian/Ubuntu-based Linux shell without further thought⁴. Explanations and further options and details are found in the remaining sections and in the [Tutorial](#).

Prerequisites: You should have access to a shell and the permissions to install – or have already installed – the numerical packages Lapack and Blas, a Fortran compiler, Python, and the tools make and git. Alternatively, one could use [this Docker image](#), which has ALF, pyALF and a Jupyter server pre-installed.

The following commands can be executed in a Debian-based shell⁵ in order to install ALF dev. and its dependencies, run a demonstration simulation and output one of the measurements performed:

```

sudo apt-get update
sudo apt-get install gfortran liblapack-dev python3 make git
git clone -b master https://github.com/ALF-QMC/ALF.git
cd ALF
source configure.sh GNU noMPI
make clean
make
cp -r ./Scripts_and_Parameters_files/Start ./Run && cd ./Run/

```

⁴For other systems and distributions see the package’s [README](#).

⁵Avoid folder names containing spaces, which are not supported.

```
$ALF_DIR/Prog/ALF.out
$ALF_DIR/Analysis/ana.out Ener_scal
cat Ener_scalJ
```

The last command will output a few lines, including one similar to:

```
1 -0.29821914139694449E+002 0.13032001866398857E+000
```

which is listing the internal energy of the system and its error.

6.2 Compiling and running

The necessary environment variables and the directives for compiling the code are set by the script `configure.sh`:

```
source configure.sh [MACHINE] [MODE] [STAB]
```

If run with no arguments, it lists the available options and sets a generic, serial GNU compiler with minimal flags `-cpp -O3 -ffree-line-length-none -ffast-math`. The predefined machine configurations and parallelization modes available, as well as the options for stabilization schemes for the matrix multiplications (see Sec. 2.5) are shown in Table 16. The stabilization scheme choice, in particular, is critical for performance and is discussed further in Sec. 6.4.

In order to compile the libraries, the analysis routines and the QMC program at once, just execute the single command:

```
make
```

Related auxiliary directories, object files and executables can be removed by executing the command `make clean`. The accompanying Makefile also provides rules for compiling and cleaning up the library, the analysis routines and the QMC program separately.

A suite of tests for individual parts of the code (subroutines, functions, operations, etc.) is available at the directory `testsuite`. The tests can be run by executing the following sequence of commands (the script `configure.sh` sets environment variables as described above):

```
source configure.sh GNU Devel noMPI
make all
mkdir testsuite/tests
cd testsuite/tests
cmake ..
make
ctest -VV -O log.txt
```

which will output test results and total success rate.

Starting a simulation

In order to start a simulation from scratch, the following files have to be present: `parameters` and `seeds` (see Sec. 5.7.1). To run serial simulation, issue the command

```
$ALF_DIR/Prog/ALF.out
```

⁶A known issue with the alternative Intel Fortran compiler `ifort` is the handling of automatic, temporary arrays which `ifort` allocates on the stack. For large system sizes and/or low temperatures this may lead to a runtime error. One solution is to demand allocation of arrays above a certain size on the heap instead of the stack. This is accomplished by the `ifort` compiler flag `-heap-arrays [n]` where `[n]` is the minimal size (in kilobytes, for example `n=1024`) of arrays that are allocated on the heap.

⁷Supercomputer at the Leibniz Supercomputing Centre.

⁸Supercomputer at the Jülich Supercomputing Centre.

Argument	Selected feature
MACHINE	
GNU	GNU compiler (gfortran/mpifort) for a generic machine (<i>default</i>)
Intel	Intel compiler (ifort/mpiifort) for a generic machine ⁶
PGI	PGI compiler (pgfortran/mpifort) for a generic machine
SuperMUC-NG	Intel compiler (mpiifort) and loads modules for SuperMUC-NG ⁷
JUWELS	Intel compiler (mpiifort) and loads modules for JUWELS ⁸
MODE	
noMPI Serial	No parallelization
MPI	MPI parallelization (<i>default</i> – if a machine is selected)
Tempering	Parallel tempering (Sec. 2.3.2) and the required MPI as well
PARALLEL_-PARAMS	Run multiple independent simulations in parallel (Sec. 6.5)
STAB	
STAB1	Simplest stabilization: UDV (QR-, not SVD-based) decompositions
STAB2	QR-based UDV decompositions with additional normalizations
STAB3	Latest, additionally separates large and small scales (<i>default</i>)
LOG	Logarithmic storage for internal scales, increases accessible ranges
OTHER SWITCHES	
Devel	Compile with additional flags for development and debugging
HDF5	Compile with HDF5 – automatically downloads and installs HDF5 if not present
NO-INTERACTIVE	Do not ask for user confirmation during execution of this script
Environment Variables	These are not arguments, but rather environment variables that will be recognized by <code>configure.sh</code>
ALF_FLAGS_EXT	Hand these additional flags to the compiler
ALF_HDF5_DIR	ALF usually self-compiles HDF5 and stores the library in subdirectories of ALF/HDF5, this behavior can be changed by setting the environment variable ALF_HDF5_DIR

Table 16: Available arguments for the script `configure.sh`, called before compilation of the package: predefined machines, parallelization modes, and stabilization schemes (see also Sec. 6.4).

In order to run with MPI parallelization, the appropriate MPI execution command should be called. For instance, a program compiled with OpenMPI can be run in parallel by issuing

```
orterun -np <number of processes> $ALF_DIR/Prog/ALF.out
```

The environment variable `ALF_SHM_CHUNK_SIZE_GB` can be used to reduce the program's memory footprint by sharing memory between MPI processes on the same node. The variable, a positive real number, defines the chunk size of the shared memory objects in units of GB. Typical values are 1.0 or 2.0 GB, but larger values can be used, if otherwise the total number of MPI communicators so large as to trigger MPI error messages. If `ALF_SHM_CHUNK_SIZE_GB` is not defined or set to values smaller than one, then the memory is not shared between MPI

processes, which is the default behavior.

To restart the code using the configuration from a previous simulation as a starting point, first run the script `out_to_in.sh`, which copies outputted field configurations into input files, before calling the ALF executable. This file is located in the directory `$ALF_DIR/Scripts_and_Parameters_files/Start/`

Notice that, when compiled with DHF5 the code checks whether the parameters stored in existing data files have the same values as those in the parameter file and exit with an error when they do not.

6.3 Error analysis

The ALF package includes the analysis programs `ana.out` for plain text bins and `ana_hdf5.out` for bins in HDF5 format. They perform the same simple error analysis and correlation function calculations on the three observable types. To perform an error analysis based on the Jackknife resampling method [141] (Sec. 4.1) of the Monte Carlo bins for a list of plain-text observables run

```
$ALF_DIR/Analysis/ana.out <list of files>
```

or run

```
$ALF_DIR/Analysis/ana.out *
```

for all observables.

For analyzing observables stored in the HDF5 file `data.h5`, run

```
$ALF_DIR/Analysis/ana_hdf5.out <list of observables>
```

or run

```
$ALF_DIR/Analysis/ana_hdf5.out
```

for all observables.

The programs `ana.out` and `ana_hdf5.out` are based on the included module `ana_mod`, which provides subroutines for reading and analyzing ALF Monte Carlo bins, that can be used to implement more specialized analysis. The three high-level analysis routines employed by `ana_mod` are listed in Table 17. The files taken as input, as well as the output files are listed in Table 18.

The error analysis is based on the central limit theorem, which requires bins to be statistically independent, and also the existence of a well-defined variance for the observable under consideration (see Sec. 4). The former will be the case if bins are longer than the autocorrelation time – autocorrelation functions are computed by setting the parameter `N_auto` to a nonzero value – which has to be checked by the user. In the parameter file described in Sec. 5.7.1, the user can specify how many initial bins should be omitted (variable `n_skip`). This number should be comparable to the autocorrelation time. The rebinning variable `N_rebin` will merge `N_rebin` bins into a single new bin. If the autocorrelation time is smaller than the effective bin size, the error should become independent of the bin size and thereby of the variable `N_rebin`. The analysis output files listed in Table 18 and are formatted in the following way:

- For the scalar quantities X , the output files `X_scalJ` have the following formatting:

```
Effective number of bins, and bins: <N_bin - N_skip>/<N_rebin> <N_bin>
OBS : 1    <mean(X)>      <error(X)>
OBS : 2    <mean(sign)>   <error(sign)>
```

- For the equal-time correlation functions in real space, Y , the formatting of the output files `Y_eqJR` follows the structure:

Program	Description
cov_vec(name)	The bin file name, which should have suffix <code>_scal</code> , is read in, and the corresponding file with suffix <code>_scalJ</code> is produced. It contains the result of the Jackknife rebinning analysis (see Sec. 4)
cov_eq(name)	The bin file name, which should have suffix <code>_eq</code> , is read in, and the corresponding files with suffix <code>_eqJR</code> and <code>_eqJK</code> are produced. They correspond to correlation functions in real and Fourier space, respectively
cov_tau(name)	The bin file name, which should have suffix <code>_tau</code> , is read in, and the directories <code>X_kx_ky</code> are produced for all <code>kx</code> and <code>ky</code> greater or equal to zero. Here <code>X</code> is a place holder from Green, SpinXY, etc., as specified in <code>Alloc_obs(Ltau)</code> (See section 7.7.1). Each directory contains a file <code>g_dat</code> containing the time-displaced correlation function traced over the orbitals. It also contains the covariance matrix if <code>N_cov</code> is set to unity in the parameter file (see Sec. 5.7.1). Besides, a directory <code>X_R0</code> for the local time displaced correlation function is generated. For particle-hole, imaginary-time correlation functions (<code>Channel = "PH"</code>) such as spin and charge, we use the fact that these correlation functions are symmetric around $\tau = \beta/2$ so that we can define an improved estimator by averaging over τ and $\beta - \tau$

Table 17: Overview of analysis subroutines called within `ana.out` and `ana_hdf5.out`.

```

do i = 1, N_unit_cell
  <r_x(i)>   <r_y(i)>
  do alpha = 1, N_orbital
    do beta = 1, N_orbital
      alpha  beta  Re<mean(Y)>  Re<error(Y)>  Im<mean(Y)>  Im<error(Y)>
    enddo
  enddo
enddo

```

where Re and Im refer to the real and imaginary part, respectively.

- The format for equal time correlations in Fourier space depends upon the boolean variable `Extended_Zone`. For `Extended_Zone=.F.`, the format reads:

```

do i = 1, N_unit_cell
  <k_x(i)>   <k_y(i)>
  do alpha = 1, N_orbital
    do beta = 1, N_orbital
      alpha  beta  Re<mean(Y)>  Re<error(Y)>  Im<mean(Y)>  Im<error(Y)>
    enddo
  enddo
enddo

```

where Re and Im refer to the real and imaginary part, respectively. The Fourier transformation is defined in Eq. 133.

For `Extended_Zone=.T.`, the format reads:

```

do n = 0, N_BZ_Zones -1
  do m = 0, N_BZ_Zones -1

```

File	Description
Input	
parameters	Includes error analysis variables N_skip, N_rebin, and N_Cov (see Sec. 5.7.1)
X_scal, Y_eq, Y_tau	Monte Carlo bins (see Table 14)
Output	
X_scalJ	Jackknife mean and error of X, where X stands for Kin, Pot, Part, or Ener
Y_eqJR and Y_eqJK	Jackknife mean and error of Y, which stands for Green, SpinZ, SpinXY, or Den. The suffixes R and K refer to real and reciprocal space, respectively
Y_eq_eJK	Jackknife mean and error of Y, which stands for Green, SpinZ, SpinXY, or Den. The file contains the analysis in the extended zone scheme (See Eq. 153)
Y_R0/g_R0	Time-resolved and spatially local Jackknife mean and error of Y, where Y stands for Green, SpinZ, SpinXY, and Den
Y_kx_ky/g_kx_ky	Time resolved and $\mathbf{k}$ -dependent Jackknife mean and error of Y, where Y stands for Green, SpinZ, SpinXY, and Den
Y_e_kx_ky/g_kx_ky	Time resolved and $\mathbf{k}$ -dependent Jackknife mean and error of Y, where Y stands for Green, SpinZ, SpinXY, and Den. The file contains the analysis in the extended zone scheme (See Eq. 153)
Part_scal_Auto	Autocorrelation functions $S_{\hat{O}}(t_{\text{Auto}})$ in the range $t_{\text{Auto}} = [0, N_{\text{auto}}]$ for the observable $\hat{O}$

Table 18: Standard input and output files of the error analysis program ana.out.

```

do i = 1, N_unit_cell
  <k_x(i) + n b_1 + m b_2 > <k_y(i) + n b_1 + m b_2 > &
  & Re<mean(Y)> Re<error(Y)> Im<mean(Y)> Im<error(Y)>
enddo
enddo
enddo

```

In this case, the Fourier transformation is defined as:

$$S(\mathbf{k} + n\mathbf{b}_1 + m\mathbf{b}_2) = \sum_{\alpha, \beta} e^{-i(\mathbf{k} + n\mathbf{b}_1 + m\mathbf{b}_2) \cdot (\mathbf{r}_\alpha - \mathbf{r}_\beta)} S_{\alpha, \beta}(\mathbf{k}). \quad (153)$$

The positions of the orbitals, $\mathbf{r}_\alpha$ within the unit cell are read in from the file Y_info, $\mathbf{b}_1$, $\mathbf{b}_2$, are the reciprocal lattice vectors and the Fourier transform is defined in equation Eq. 133. The naming of the output files equally depends upon the boolean variable Extended_Zone (see table 18).

- The imaginary-time displaced correlation functions Y are written to the output files g_R0 inside folders Y_R0, when measured locally in space; and to the output files g_kx_ky inside folders Y_kx_ky when they are measured $\mathbf{k}$ -resolved (where $\mathbf{k} = (kx, ky)$). The first line of the file contains the number of imaginary times, the effective number of bins, β , the number of orbitals and the channel. Both output files have the following formatting:

```
do i = 0, Ltau
```

```
tau(i)    <mean( Tr[Y] )>    <error( Tr[Y])>
enddo
```

where Tr corresponds to the trace over the orbital degrees of freedom. For particle-hole quantities at finite temperature, τ runs from 0 to $\beta/2$. In all other cases it runs from 0 to β . The above is valid for `Extended_Zone=.F..` If `Extended_Zone=.T.` then $\text{Tr}[Y]$ is replaced by the Fourier transform in the extended zone scheme as defined above, but for time-displaced correlation functions.

- The file `Y_tauJK` contains the susceptibilities defined as:

$$\chi(\mathbf{q}) = \sum_{n,n'=1}^{\text{Norb}} \int_0^\beta d\tau (\langle Y_n(\mathbf{q}, \tau) Y_{n'}(-\mathbf{q}, 0) \rangle - \langle Y_n(\mathbf{q}) \rangle \langle Y_{n'}(-\mathbf{q}) \rangle \delta_{\mathbf{q},0}) \quad (154)$$

The output file has the following formatting:

```
do i = 0, Ltau
  q_x, q_y, <mean(Real(chi(q)) )>, <error(Real(chi(q)))>, &
    & <mean(Im (chi(q)) )>, <error(lmi (chi(q)))>
enddo
```

- Setting the parameter `N_auto` to a finite value triggers the computation of autocorrelation functions $S_{\hat{O}}(t_{\text{Auto}})$ in the range $t_{\text{Auto}} = [0, N_{\text{Auto}}]$. The output is written to the file `Part_scal_Auto`, where the data is organized in three columns:

```
t_Auto    S_O(t_Auto)    error
```

Since these computations are quite time consuming and require many Monte Carlo bins, our default is `N_auto=0`.

6.4 Parameter optimization

The finite-temperature, auxiliary-field QMC algorithm is known to be numerically unstable, as discussed in Sec. 2.5. The numerical instabilities arise from the imaginary-time propagation, which invariably leads to exponentially small and exponentially large scales. As shown in Ref. [6], scales can be omitted in the ground state algorithm – thus rendering it very stable – but have to be taken into account in the finite-temperature code.

Numerical stabilization of the code is a delicate procedure that has been pioneered in Ref. [2] for the finite-temperature algorithm and in Refs. [3, 4] for the zero-temperature, projective algorithm. It is important to be aware of the fragility of the numerical stabilization and that there is no guarantee that it will work for a given model. It is therefore crucial to always check the file `info`, which, apart from runtime data, contains important information concerning the stability of the code, in particular `Precision Green`. If the numerical stabilization fails, one possible measure is to reduce the value of the parameter `Nwrap` in the parameter file, which will however also impact performance – see Table. 19 for further optimization tips for the Monte Carlo algorithm (Sec. 4). Typical values for the numerical precision ALF can achieve can be found in Sec. 9.1.

In particular, for the stabilization of the involved matrix multiplications we rely on routines from LAPACK. Notice that results are very likely to change depending on the specific implementation of the library used⁹. In order to deal with this possibility, we offer a simple baseline which can be used as a quick check as to whether results depend on the library used for linear algebra routines. Namely, we have included QR-decomposition related routines of

⁹The linked library should implement at least the LAPACK-3.4.0 interface.

Element	Suggestion
Precision Green, Precision Phase	Should be found to be <i>small</i> , of order $< 10^{-8}$ (see Sec. 2.5)
theta	Should be <i>large</i> enough to guarantee convergence to ground state
dtau	Should be set <i>small</i> enough to limit Trotter errors
Nwrap	Should be set <i>small</i> enough to keep Precisions small
Nsweep	Should be set <i>large</i> enough for bins to be of the order of the auto-correlation time
Nbin	Should be set <i>large</i> enough to provide desired statistics
nskip	Should be set <i>large</i> enough to allow for equilibration ($\sim$ autocorrelation time)
Nrebin	Can be set to 1 when Nsweep is large enough; otherwise, and for testing, larger values can be used
Stabilization scheme	Use the default STAB3 – newest and fastest, if it works for your model; alternatives are: STAB1 – simplest, for reference only; STAB2 – with additional normalizations; and LOG – for dealing with more extreme scales (see also Tab. 16)
Parallelism	For some models and systems, restricting parallelism in your BLAS library can improve performance: for OpenBLAS try setting OPENBLAS_NUM_THREADS=1 in the shell
ALF_SHM_CHUNK_ SIZE_GB	An environment variable that sets the chunk size in GBs for the memory shared between different MPI processes on the same computing node. By default it is zero (i.e., no sharing), but can be set to, e.g., 1.0 or 2.0 GB or larger if, for instance, the total number of MPI communicators is so large as to trigger MPI error messages.

Table 19: Rules of thumb for obtaining best results and performance from ALF. It is important to fine tune the parameters to the specific model under consideration and perform sanity checks throughout. Most suggestions can severely impact performance and numerical stability if overdone.

the LAPACK-3.7.0 reference implementation from <http://www.netlib.org/lapack/>, which you can use by running the script `configure.sh`, (described in Sec. 6), with the flag STAB1 and recompiling ALF¹⁰. The stabilization flags available are described in Tables 16 and 19. The performance of the package is further discussed in Sec. B.

6.5 Parallel parameters feature

With the compile-time option `PARALLEL_PARAMS` (or equivalently `PP`), ALF can run multiple independent simulations in parallel. This feature is meant for utilizing the high core counts of modern HPC systems in cases where a normal job size might be overkill for a single simulation. The basic folder structure is the same as in Parallel Tempering, as sketched in Fig. 7: The root directory contains the files `parameters`, `seeds` and directories `Temp_0` to `Temp_{N-1}`, where N is the number of parameter sets, i.e. different simulations. Contrary to Parallel Tempering, in Parallel Parameters mode only `mpi_per_parameter_set` from the namelist `VAR_TEMP` is read from `./parameters`. This variable is defined as the number of available MPI processes

¹⁰This flag may trigger compiling issues, in particular, the Intel ifort compiler version 10.1 fails for all optimization levels.

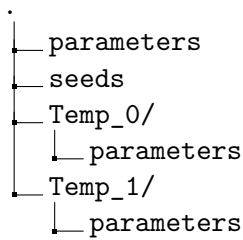


Figure 7: Folder structure for a parallel parameters (or Parallel Tempering) simulation with 2 parameter sets.

divided by N . All the rest of the parameters are read from `Temp_*/parameters`, which allows to simulate completely different parameters and even different Hamiltonians.

6.6 Convert plain text bins to HDF5

The Analysis folder also contains programs for converting plain text bins to HDF5, writing the bins from files ending in `_scal`, `_eq`, or `_tau` to `data.h5`, thereby reducing file sizes and increasing compatibility with pyALF.

There are a number of scripts and program:

- `convert_scal.out`, `convert_latt.out`, `convert_local.out`: Writing the contents of a `_scal` observable, a lattice type observables (that is `_eq`, and `_tau`), or a local observable (that is `_local`, and `_localtau`) to an HDF5 file. E.g. writing bins from `some_obs_scal`, `some_obs_eq`, `some_obs_tau`, `some_obs_local` and `some_obs_localtau` to `data.h5` is done by executing.

```

$ALF_DIR/Analysis/convert_scal.out some_obs_scal data.h5
$ALF_DIR/Analysis/convert_latt.out some_obs_eq data.h5
$ALF_DIR/Analysis/convert_latt.out some_obs_tau data.h5
$ALF_DIR/Analysis/convert_local.out some_obs_local data.h5
$ALF_DIR/Analysis/convert_local.out some_obs_localtau data.h5

```

- `convert_bins.py`:
 - Automatically combines file parameters and the Hamiltonian used in the simulation to determine Hamiltonian-specific simulation parameters and write them into HDF5 file.
 - Calls `convert_scal.out`, `convert_latt.out`, and `convert_local.out` for all observables.
 - Moves the original plain text bins to a subdirectory `old_bins` or optionally removes them.

You can also get a help text by executing:

```
$ALF_DIR/Analysis/convert_bins.py --help
```

One can automatically convert a big number of simulations by executing,

```
find . -name "$ref" -execdir ${ALF_DIR}/Analysis/copy_parameters.py \;
```

where `$ref` has to be the name of an observable produced by the simulation, e.g. `Ener_scal`.

- `convert_batch.sh`: Executes the command above.

7 The plain vanilla Hubbard model on the square lattice

All the data structures necessary to implement a given model have been introduced in the previous sections. Here we show how to implement a new model based on the example of the Hubbard model.

As stated in Sec. 5.6, for defining a new Hamiltonian calle *New_model* one needs to

1. Add a new line containing *New_model* to the file `Prog/Hamiltonians.list`
2. Write the corresponding new submodule in `Prog/Hamiltonians/Hamiltonian_New_model_smod.F90`

Here our *New_model* will be `Hubbard_Plain_Vanilla`. There is a template `Prog/Hamiltonians/Hamiltonian_##NAME##_smod.F90` that can be used for creating a new model.

To get a valid Hamiltonian, one has to specify its parameters, the lattice, the hopping, the interaction, the trial wave function (if required), and the observables. Consider the *plain vanilla* Hubbard model written as:

$$\mathcal{H} = -t \sum_{\langle i,j \rangle, \sigma=\uparrow, \downarrow} (\hat{c}_{i,\sigma}^\dagger \hat{c}_{j,\sigma} + \text{H.c.}) - \frac{U}{2} \sum_i [\hat{c}_{i,\uparrow}^\dagger \hat{c}_{i,\uparrow} - \hat{c}_{i,\downarrow}^\dagger \hat{c}_{i,\downarrow}]^2 - \mu \sum_{i,\sigma} \hat{c}_{i,\sigma}^\dagger \hat{c}_{i,\sigma}. \quad (155)$$

Here $\langle i, j \rangle$ denotes nearest neighbors. We can make contact with the general form of the Hamiltonian [see Eq. (2)] by setting: $N_{\text{fl}} = 2$, $N_{\text{col}} \equiv N_{\text{SUN}} = 1$, $M_T = 1$,

$$T_{xy}^{(ks)} = \begin{cases} -t & \text{if } x, y \text{ are nearest neighbors} \\ -\mu & \text{if } x = y \\ 0 & \text{otherwise,} \end{cases} \quad (156)$$

$M_V = N_{\text{unit-cell}}$, $U_k = \frac{U}{2}$, $V_{xy}^{(k,s=1)} = \delta_{x,y} \delta_{x,k}$, $V_{xy}^{(k,s=2)} = -\delta_{x,y} \delta_{x,k}$, $\alpha_{ks} = 0$ and $M_I = 0$. The coupling of the HS fields to the z -component of the magnetization breaks the $SU(2)$ spin symmetry. Nevertheless, the z -component of the spin remains a good quantum number such that the imaginary-time propagator – for a given HS field – is block diagonal in this quantum number. This corresponds to the flavor index running from 1 to 2, labeling spin up and spin down degrees of freedom. We note that in this formulation the hopping matrix can be flavor dependent such that a Zeeman magnetic field can be introduced. If the chemical potential is set to zero, this will not generate a negative sign problem [74, 145, 146]. The code that we describe below can be found in the submodule `Prog/Hamiltonians/Hamiltonian_Hubbard_Plain_Vanilla_smod.F90`.

7.1 Defining the parameters

Defining the parameters as specified in Sec. 5.6, we arrive at:

```
!#PARAMETERS START# VAR_lattice
Character (len=64) :: Model = '' ! Value irrelevant
Character (len=64) :: Lattice_type = 'Square' ! Possible Values: 'Square'
Integer :: L1 = 4 ! Length in direction a_1
Integer :: L2 = 4 ! Length in direction a_2
```

```

!#PARAMETERS END#

!#PARAMETERS START# VAR_Hubbard_Plain_Vanilla
!Integer      :: N_SUN = 2
real(Kind=Kind(0.d0)) :: ham_T = 1.d0      ! Hopping parameter
real(Kind=Kind(0.d0)) :: Ham_chem = 0.d0    ! Chemical potential
real(Kind=Kind(0.d0)) :: Ham_U = 4.d0      ! Hubbard interaction
real(Kind=Kind(0.d0)) :: Dtau = 0.1d0      ! Thereby Ltrot=Beta/dtau
real(Kind=Kind(0.d0)) :: Beta = 5.d0       ! Inverse temperature
!logical      :: Projector = .false. ! Whether the projective algorithm is used
logical        :: Adiabatic = .false. ! If true and if the projector is true
                                           ! the Hubbard interaction will be ramped
                                           ! from 0 to Ham_U in the time interval
                                           ! 0 to theta
real(Kind=Kind(0.d0)) :: Theta = 5.d0      ! Projection parameter
!logical      :: Symm = .false. ! Whether symmetrization takes place
Integer       :: N_part = -1             ! Number of particles in trial wave function.
                                           ! If N_part < 0 -> N_part = L1*L2/2

!#PARAMETERS END#

```

We can test the correct formatting of the parameters by calling:

```
../parse_ham.py --test_file Hamiltonian_Hubbard_Plain_Vanilla_smod.F90
```

7.2 Setting the Hamiltonian: Ham_set

The main program will call the subroutine Ham_set in the submodule Hamiltonian_Hubbard_Plain_Vanilla_smod.F90 which specify the model. The routine Ham_set will first read the parameter file parameters (see Sec. 5.7.1) Call read_parameters; then set the lattice: Call Ham_latt; set the hopping: Call Ham_hop; set the interaction: call Ham_V; and if required, set the trial wave function: call Ham_trial. In the subroutine Ham_set one will equally have to specify if a symmetry relates different flavors. This functionality is described in Sec. 5.6.1 and one enables it by allocating tine array Calc_Fl.

7.3 The lattice: Ham_latt

The routine, which sets the square lattice, reads:

```

a1_p(1) = 1.0 ; a1_p(2) = 0.d0
a2_p(1) = 0.0 ; a2_p(2) = 1.d0
L1_p    = dble(L1)*a1_p
L2_p    = dble(L2)*a2_p
Call Make_Lattice(L1_p, L2_p, a1_p, a2_p, Latt)
Latt_unit%Norb = 1
Latt_unit%N_coord = 2
allocate(Latt_unit%Orb_pos_p(Latt_unit%Norb,2))
Latt_unit%Orb_pos_p(1, :) = [0.d0, 0.d0]
Ndim = Latt%N*Latt_unit%Norb

```

In its last line, the routine sets the total number of single particle states per flavor and color: $N_{\text{dim}} = L_{\text{att}} \times N_{\text{unit}} \times N_{\text{orb}}$.

7.4 The hopping: Ham_hop

The hopping matrix is implemented as follows. We allocate an array of dimension $1 \times N_{\text{fl}}$ of type operator called Op_T and set the dimension for the hopping matrix to $N = N_{\text{dim}}$. The operator allocation and initialization is performed by the subroutine Op_make:

```
do nf = 1, N_FL
  call Op_make(Op_T(1,nf), Ndim)
enddo
```

Since the hopping does not break down into small blocks, we have $P = 1$ and

```
Do nf = 1, N_FL
  Do i = 1, Latt%N
    Op_T(1,nf)%P(i) = i
  Enddo
Enddo
```

We set the hopping matrix with

```
Do nf = 1, N_FL
  Do I = 1, Latt%N
    Ix = Latt%nnlist(I,1,0)
    Iy = Latt%nnlist(I,0,1)
    Op_T(1,nf)%O(I, Ix) = cmplx(-Ham_T, 0.d0, kind(0.D0))
    Op_T(1,nf)%O(Ix, I) = cmplx(-Ham_T, 0.d0, kind(0.D0))
    Op_T(1,nf)%O(I, Iy) = cmplx(-Ham_T, 0.d0, kind(0.D0))
    Op_T(1,nf)%O(Iy, I) = cmplx(-Ham_T, 0.d0, kind(0.D0))
    Op_T(1,nf)%O(I, I) = cmplx(-Ham_chem, 0.d0, kind(0.D0))
  Enddo
  Op_T(1,nf)%g = -Dtau
  Op_T(1,nf)%alpha = cmplx(0.d0,0.d0, kind(0.D0))
  Call Op_set(Op_T(1,nf))
Enddo
```

Here, the integer function `Latt%nnlist(I,n,m)` is defined in the lattice module and returns the index of the lattice site $I + na_1 + ma_2$. Note that periodic boundary conditions are already taken into account. The hopping parameter `Ham_T`, as well as the chemical potential `Ham_chem` are read from the parameter file. To completely define the hopping we further set: `Op_T(1,nf)%g = -Dtau`, `Op_T(1,nf)%alpha = cmplx(0.d0,0.d0,kind(0.D0))` and call the routine `Op_set(Op_T(1,nf))` so as to generate the unitary transformation and eigenvalues as specified in Table 2. Recall that for the hopping, the variable `Op_set(Op_T(1,nf))%type` takes its default value of 0. Finally, note that, although a checkerboard decomposition is not used here, it can be implemented by considering a larger number of sparse hopping matrices.

7.5 The interaction: `Ham_V`

To implement the interaction, we allocate an array of `Operator` type. The array is called `Op_V` and has dimensions $N_{\text{dim}} \times N_{\text{fl}} = N_{\text{dim}} \times 2$. We set the dimension for the interaction term to $N = 1$, and allocate and initialize this array of type `Operator` by repeatedly calling the subroutine `Op_make`. Here we also allow for the possibility of switching on adiabatically the Hubbard interaction provided that `Adiabatic=.true.` and that we are working with the projector method. The protocol we use is depicted in Fig. 8. Note that to trigger the time dependence `Op_V%g_t` has to be allocated, and the index of the complex valued one-dimensional array has to run from 1 to `Ltrot`.

```
Allocate(Op_V(Ndim,N_FL))
do nf = 1, N_FL
  do i = 1, Ndim
    Call Op_make(Op_V(i,nf), 1)
  enddo
enddo
Do nf = 1, N_FL
  X = 1.d0
```

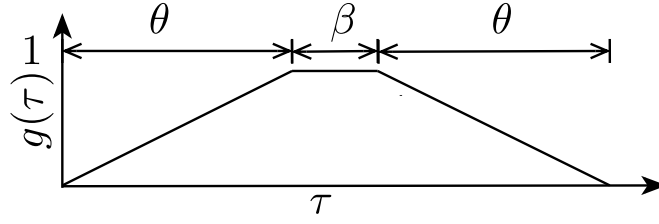


Figure 8: Protocol for wrapping up the Hubbard U-term for in the projective algorithm. This approach has been put forward as a method to alleviate the negative sign problem [147].

```

if (nf == 2) X = -1.d0
Do i = 1,Ndim
  Op_V(i,nf)%P(1) = I
  Op_V(i,nf)%O(1,1) = cmplx(1.d0, 0.d0, kind(0.D0))
  If (Adiabatic) then
    Allocate(OP_V(i,nf)%g_t(Ltrot))
    Op_V(i,nf)%g_t = X*SQRT(CMPLX(DTAU*ham_U/2.d0, 0.D0, kind(0.D0)))
    do nt = 1, Throt
      Op_V(i,nf)%g_t(nt) = X*SQRT(CMPLX(DTAU*dble(nt)/dble(thtrot)* &
        & ham_U/2.d0, 0.D0, kind(0.D0)))
      Op_V(i,nf)%g_t(Ltrot-(nt-1)) = X*SQRT(CMPLX(DTAU*dble(nt)/dble(thtrot)* &
        & ham_U/2.d0, 0.D0, kind(0.D0)))
    enddo
  else
    Op_V(i,nf)%g = X*SQRT(CMPLX(DTAU*ham_U/2.d0, 0.D0, kind(0.D0)))
  endif
  Op_V(i,nf)%alpha = cmplx(-0.5d0, 0.d0, kind(0.D0))
  Op_V(i,nf)%type = 2
  Call Op_set( Op_V(i,nf) )
Enddo
Enddo

```

If The code above makes it explicit that there is a sign difference between the coupling of the HS field in the two flavor sectors. Hence, $\text{Op_V}(i, \text{nf})$ encodes $e^{X\sqrt{\Delta\tau U/2}(\hat{c}_{i,\text{nf}}^\dagger \hat{c}_{i,\text{nf}} + \alpha)}$ with $X = 1$ for $\text{nf} = 1$ and $X = -1$ for $\text{nf} = 2$. Strictly speaking X can be omitted. However, it is required when using the flavor symmetry option in the presence of particle-hole symmetry (see Sec. 5.6.1). If the variable `Adiabatic=.true.` and `Projector=.true.` then the imaginary time profile of the Hubbard interaction is given by Fig. 8

7.6 The trial wave function: `Ham_Trial`

As argued in Sec. 3.1, it is useful to generate the trial wave function from a non-interacting trial Hamiltonian. Here we will use the same left and right flavor-independent trial wave functions that correspond to the ground state of:

$$\hat{H}_T = -t \sum_i \left[(1 + (-1)^{i_x + i_y} \delta) \hat{c}_i^\dagger \hat{c}_{i+a_x} + (1 - \delta) \hat{c}_i^\dagger \hat{c}_{i+a_y} + \text{H.c.} \right] \equiv \sum_{i,j} \hat{c}_i^\dagger h_{i,j} \hat{c}_j. \quad (157)$$

For the half-filled case, the dimerization $\delta = 0^+$ opens up a gap at half-filling, thus generating the desired non-degenerate trial wave function that has the same symmetries (particle-hole for instance) as the trial Hamiltonian.

Diagonalization of $h_{i,j}$, $U^\dagger h U = \text{Diag}(\epsilon_1, \dots, \epsilon_{N_{\text{dim}}})$ with $\epsilon_i < \epsilon_j$ for $i < j$, allows us to define the trial wave function. In particular, for the half-filled case, we set

```

Do s = 1, N_fl
  Do x = 1, Ndim
    Do n = 1, N_part
      WF_L(s)%P(x,n) = Ux,n
      WF_R(s)%P(x,n) = Ux,n
    Enddo
  Enddo
Enddo

```

with $N_{\text{part}} = N_{\text{dim}}/2$. The variable `Degen` belonging to the `WaveFunction` type is given by $\text{Degen} = \epsilon_{N_{\text{part}}+1} - \epsilon_{N_{\text{part}}}$. This quantity should be greater than zero for non-degenerate trial wave functions.

7.7 Observables

At this point, all the information for starting the simulation has been provided. The code will sequentially go through the operator list `Op_V` and update the fields. Between time slices `LOBS_ST` and `LOBS_EN` the main program will call the routine `Obser (GR, Phase, Ntau)`, which handles equal-time correlation functions, and, if `Ltau=1`, the routine `ObserT (NT, GTO, GOT, G00, GTT, PHASE)` which handles imaginary-time displaced correlation functions.

Both `Obser` and `ObserT` should be provided by the user, who can either implement themselves the observables they want to compute or use the predefined structures of Chap. 8. Here we describe how to proceed in order to define an observable.

7.7.1 Allocating space for the observables: `Alloc_obs(Ltau)`

For four scalar or vector observables, the user will have to declare the following:

```

Allocate ( Obs_scal(4) )
Do I = 1, Size(Obs_scal,1)
  select case (I)
    case (1)
      N = 2; Filename = "Kin"
    case (2)
      N = 1; Filename = "Pot"
    case (3)
      N = 1; Filename = "Part"
    case (4)
      N = 1; Filename = "Ener"
    case default
      Write(6,*) ' Error in Alloc_obs '
  end select
  Call Obser_Vec_make(Obs_scal(I), N, Filename)
enddo

```

Here, `Obs_scal(1)` contains a vector of two observables so as to account for the x - and y -components of the kinetic energy, for example.

For equal-time correlation functions we allocate `Obs_eq` of type `Obser_Latt`. Here we include the calculation of spin-spin and density-density correlation functions alongside equal-time Green functions.

```

Allocate ( Obs_eq(5) )
Do I = 1, Size(Obs_eq,1)
  select case (I)
    case (1)
      Filename = "Green"
    case (2)
      Filename = "SpinZ"

```

```

case (3)
  Filename = "SpinXY"
case (4)
  Filename = "SpinT"
case (5)
  Filename = "Den"
case default
  Write(6,*) "Error in Alloc_obs"
end select
Nt = 1
Channel = "--"
Call Obser_Latt_make(Obs_eq(I), Nt, Filename, Latt, Latt_unit, Channel, dtau)
Enddo

```

Be aware that `Obser_Latt_make` does not copy the Bravais lattice `Latt` and unit cell `Latt_unit`, but links them through pointers to be more memory efficient. One can have different lattices attached to different observables by declaring additional instances of `Type(Lattice)` and `Type(Unit_cell)`. For equal-time correlation functions, we set `Nt = 1` and `Channel` specification is not necessary.

If `Ltau = 1`, then the code allocates space for time displaced quantities. The same structure as for equal-time correlation functions is used, albeit with `Nt = Ltrot + 1` and the channel should be set. With `Channel="PH"`, for instance, the analysis algorithm assumes the observable to be particle-hole symmetric. For more details on this parameter, see Sec. 10.

At the beginning of each bin, the main program will set the bin observables to zero by calling the routine `Init_obs(Ltau)`. The user does not have to edit this routine.

7.7.2 Measuring equal-time observables: `Obser(GR,Phase,Ntau)`

Having allocated the necessary memory, we proceed to define the observables. The equal-time Green function,

$$GR(x, y, \sigma) = \langle \hat{c}_{x,\sigma} \hat{c}_{y,\sigma}^\dagger \rangle, \quad (158)$$

the phase factor `phase` [Eq. (137)], and time slice `Ntau` are provided by the main program.

Here, `x` and `y` label both unit cell as well as the orbital within the unit cell. For the Hubbard model described here, `x` corresponds to the unit cell. The Green function does not depend on the color index, and is diagonal in flavor. For the $SU(2)$ symmetric implementation there is only one flavor, $\sigma = 1$ and the Green function is independent on the spin index. This renders the calculation of the observables particularly easy.

An explicit calculation of the potential energy $\langle U \sum_i \hat{n}_{i,\uparrow} \hat{n}_{i,\downarrow} \rangle$ reads

```

Obs_scal(2)%N      = Obs_scal(2)%N + 1
Obs_scal(2)%Ave_sign = Obs_scal(2)%Ave_sign + Real(ZS,kind(0.d0))
Do i = 1,Ndim
  Obs_scal(2)%Obs_vec(1) = Obs_scal(2)%Obs_vec(1) + (1-GR(i,i,1))*(1-GR(i,i,2))*Ham_U*ZS*ZP
Enddo

```

Here $ZS = \text{sgn}(C)$ [see Eq. (27)], $ZP = \frac{e^{-S(C)}}{\text{Re}[e^{-S(C)}]}$ [see Eq. (137)] and `Ham_U` corresponds to the Hubbard U term.

Equal-time correlations are also computed in this routine. As an explicit example, we consider the equal-time density-density correlation:

$$\langle \hat{n}_i \hat{n}_j \rangle - \langle \hat{n}_i \rangle \langle \hat{n}_j \rangle, \quad (159)$$

with

$$\hat{n}_i = \sum_{\sigma} \hat{c}_{i,\sigma}^\dagger \hat{c}_{i,\sigma}. \quad (160)$$

For the calculation of such quantities, it is convenient to define:

$$\text{GRC}(x, y, s) = \delta_{x,y} - \text{GR}(y, x, s) \quad (161)$$

such that $\text{GRC}(x, y, s)$ corresponds to $\langle\langle \hat{c}_{x,s}^\dagger \hat{c}_{y,s} \rangle\rangle$. In the program code, the calculation of the equal-time density-density correlation function looks as follows:

```
Obs_eq(4)%N = Obs_eq(4)%N + 1      ! Even if it is redundant, each observable
                                   ! carries its own counter and sign.
Obs_eq(4)%Ave_sign = Obs_eq(4)%Ave_sign + Real(ZS,kind(0.d0))
Do I = 1, Ndim
  Do J = 1, Ndim
    imj = latt%imj(I, J)
    Obs_eq(4)%Obs_Latt(imj, 1, 1, 1) = Obs_eq(4)%Obs_Latt(imj, 1, 1, 1) + &
      &      ( (GRC(I, I, 1) + GRC(I, I, 2)) * (GRC(J, J, 1) + GRC(J, J, 2)) ) + &
      &      GRC(I, J, 1) * GR(I, J, 1) + GRC(I, J, 2) * GR(I, J, 2) ) * ZP * ZS
  Enddo
  Obs_eq(4)%Obs_Latt0(1) = Obs_eq(4)%Obs_Latt0(1) + (GRC(I, I, 1) + GRC(I, I, 2)) * ZP * ZS
Enddo
```

At the end of each bin the main program calls the routine `Pr_obs(LTAU)`. This routine appends the result for the current bins to the corresponding file, with the appropriate suffix.

7.7.3 Measuring time-displaced observables: `ObserT(NT, GT0, GOT, G00, GTT, PHASE)`

This subroutine is called by the main program at the beginning of each sweep, provided that `LTAU` is set to 1. The variable `NT` runs from 0 to `Ltrot` and denotes the imaginary time difference. For a given time displacement, the main program provides:

$$\begin{aligned} \text{GT0}(x, y, s) &= \langle\langle \hat{c}_{x,s}(Nt\Delta\tau) \hat{c}_{y,s}^\dagger(0) \rangle\rangle = \langle\langle \mathcal{T} \hat{c}_{x,s}(Nt\Delta\tau) \hat{c}_{y,s}^\dagger(0) \rangle\rangle \\ \text{GOT}(x, y, s) &= -\langle\langle \hat{c}_{y,s}^\dagger(Nt\Delta\tau) \hat{c}_{x,s}(0) \rangle\rangle = \langle\langle \mathcal{T} \hat{c}_{x,s}(0) \hat{c}_{y,s}^\dagger(Nt\Delta\tau) \rangle\rangle \\ \text{G00}(x, y, s) &= \langle\langle \hat{c}_{x,s}(0) \hat{c}_{y,s}^\dagger(0) \rangle\rangle \\ \text{GTT}(x, y, s) &= \langle\langle \hat{c}_{x,s}(Nt\Delta\tau) \hat{c}_{y,s}^\dagger(Nt\Delta\tau) \rangle\rangle. \end{aligned} \quad (162)$$

In the above we have omitted the color index since the Green functions are color independent. The time-displaced spin-spin correlations $4\langle\langle \hat{S}_i^z(\tau) \hat{S}_j^z(0) \rangle\rangle$ are then given by:

$$\begin{aligned} 4\langle\langle \hat{S}_i^z(\tau) \hat{S}_j^z(0) \rangle\rangle &= (\text{GTT}(I, I, 1) - \text{GTT}(I, I, 2)) * (\text{G00}(J, J, 1) - \text{G00}(J, J, 2)) \\ &\quad - \text{GOT}(J, I, 1) * \text{GT0}(I, J, 1) - \text{GOT}(J, I, 2) * \text{GT0}(I, J, 2) \end{aligned} \quad (163)$$

The handling of time-displaced correlation functions is identical to that of equal-time correlations.

7.8 Flavor symmetries: `weight_reconstruction(weight)`, `GR_reconstruction(GR)`, and `GRT_reconstruction(GT0, GOT)`

At zero chemical potential, and for repulsive interactions, the plain-vanilla Hubbard model enjoys a partial particle-hole symmetry which, for each HS field configuration, maps one flavor (i.e. spin sector) onto the other:

$$\hat{P}_Z \hat{c}_{i,\downarrow}^\dagger \hat{P}^{-1} = z^* e^{i\mathbf{Q} \cdot \mathbf{r}_i} \hat{c}_{i,\uparrow}. \quad (164)$$

Here $\mathbf{Q}$ is the antiferromagnetic wave vector. Note that in the presence of an orbital magnetic field, or of twisted boundary conditions that couples symmetrically to the flavor (spin) degree

of freedom, the anti-unitarity of the transformation is required. As a consequence of this symmetry, for a given HS field configuration the following holds for the equal-time Green function.

$$\begin{aligned} G_{00}(\mathbf{x}, \mathbf{y}, \uparrow) &= \langle \langle \hat{c}_{\mathbf{x}, \uparrow}(0) \hat{c}_{\mathbf{y}, \uparrow}^\dagger(0) \rangle \rangle = e^{i(\mathbf{y}-\mathbf{x}) \cdot \mathbf{Q}} \overline{\langle \langle \hat{c}_{\mathbf{x}, \downarrow}^\dagger(0) \hat{c}_{\mathbf{y}, \uparrow}(0) \rangle \rangle} \\ &= \delta_{\mathbf{x}, \mathbf{y}} - e^{i(\mathbf{y}-\mathbf{x}) \cdot \mathbf{Q}} \overline{G_{00}(\mathbf{y}, \mathbf{x}, \downarrow)} \end{aligned} \quad (165)$$

For the attractive Hubbard model $U < 0$, the up and down sectors are related by time reversal symmetry:

$$\hat{T} \begin{pmatrix} \hat{c}_{i, \uparrow} \\ \hat{c}_{i, \downarrow} \end{pmatrix} \hat{T}^{-1} = z^* i \sigma_y \begin{pmatrix} \hat{c}_{i, \uparrow} \\ \hat{c}_{i, \downarrow} \end{pmatrix} \quad (166)$$

Of course, we have assumed that the hopping remains invariant under time reversal. As a consequence of this symmetry:

$$G_{00}(\mathbf{x}, \mathbf{y}, \uparrow) = \overline{G_{00}(\mathbf{x}, \mathbf{y}, \downarrow)} \quad (167)$$

The usage of the flavor symmetry is described in Sec. 5.6.1. Only one flavor has to be computed and the routines `GR_reconstruction(GR)`, and `GRT_reconstruction(GT0, GT)` reconstruct, respectively, the equal-time and time-displaced Green functions for one flavor given the other. Hence we gain a factor two in computing time. We note that since both symmetries are anti-unitary, the weights between the two sectors are related by a complex conjugation. This is specified in the routine `weight_reconstruction(weight)`.

7.9 Numerical precision

Information on the numerical stability is included in the following lines of the corresponding file `info`. For a *short* simulation on a 4×4 lattice at $U/t = 4$ and $\beta t = 10$ we obtain

```
Precision Green   Mean, Max :   5.0823874429126405E-011   5.8621144596315844E-006
Precision Phase   Max       :   0.00000000000000000
Precision tau     Mean, Max :   1.5929357848647394E-011   1.0985132530727526E-005
```

showing the mean and maximum difference between the *wrapped* and from scratched computed equal and time-displaced Green functions [6]. A stable code should produce results where the mean difference is smaller than the stochastic error. The above example shows a very stable simulation since the Green function is of order one.

7.10 Running the code and testing

To test the code, one can carry out high precision simulations. After compilation, the executable `ALF.out` is found in the directory `$ALF_DIR/Prog/` and can be run from any directory containing the files `parameters` and `seeds` (See Sec. 5.7).

Alternatively, as we do bellow, it may be convenient to use `pyALF` to compile and run the code, especially when using one of the scripts or notebooks available.

One-dimensional case

The `pyALF` python script `Hubbard_Plain_Vanilla.py` runs the projective version of the code for the four-site Hubbard model. At $\theta t = 10$, $\Delta \tau t = 0.05$ with the symmetric Trotter decomposition, we obtain after 40 bins of 2000 sweeps each the total energy:

$$\langle \hat{H} \rangle = -2.103750 \pm 0.004825,$$

and the exact result is

$$\langle \hat{H} \rangle_{\text{Exact}} = -2.100396.$$

Two-dimensional case

For the two-dimensional case, with similar parameters, we obtain the results listed in Table 20. The exact results stem from Ref. [144] and the slight discrepancies from the exact results can

	QMC	Exact
Total energy	-13.618 ± 0.002	-13.6224
$\mathbf{Q} = (\pi, \pi)$ spin correlations	3.630 ± 0.006	3.64

Table 20: Test results for the Hubbard_Plain_Vanilla code on a two-dimensional lattice with default parameters.

be assigned to the finite value of $\Delta\tau$. Note that all the simulations were carried out with the default value of the Hubbard interaction, $U/t = 4$.

8 Predefined Structures

The ALF package includes predefined structures, which the user can combine together or use as templates for defining new ones. Using the data types defined in the Sec. 5 the following modules are available:

- lattices and unit cells – `Predefined_Latt_mod.F90`
- hopping Hamiltonians – `Predefined_Hop_mod.F90`
- interaction Hamiltonians – `Predefined_Int_mod.F90`
- observables – `Predefined_Obs_mod.F90`
- trial wave functions – `Predefined_Trial_mod.F90`

which we describe in the remaining of this section.

8.1 Predefined lattices

The types `Lattice` and `Unit_cell`, described in Section 5.3, allow us to define arbitrary one- and two-dimensional Bravais lattices. The subroutine `Predefined_Latt` provides some of the most common lattices, as described below.

The subroutine is called as:

```
Predefined_Latt(Lattice_type, L1, L2, Ndim, List, Invlist, Latt, Latt_Unit)
```

which returns a lattice of size $L1 \times L2$ of the given `Lattice_type`, as detailed in Table 21. Notice that the orbital position `Latt_Unit%Orb_pos_p(1, :)` is set to zero unless otherwise specified.

In order to easily keep track of the orbital and unit cell, `List` and `Invlist` make use of a super-index, defined as shown below:

```
nc = 0
Do I = 1, Latt%N
  Do no = 1, Norb
    nc = nc + 1
    List(nc, 1) = I
    List(nc, 2) = no
```

! Super-index labeling unit cell and orbital
! Unit-cell index
! Orbital index
! Unit-cell of super index nc
! Orbital of super index nc

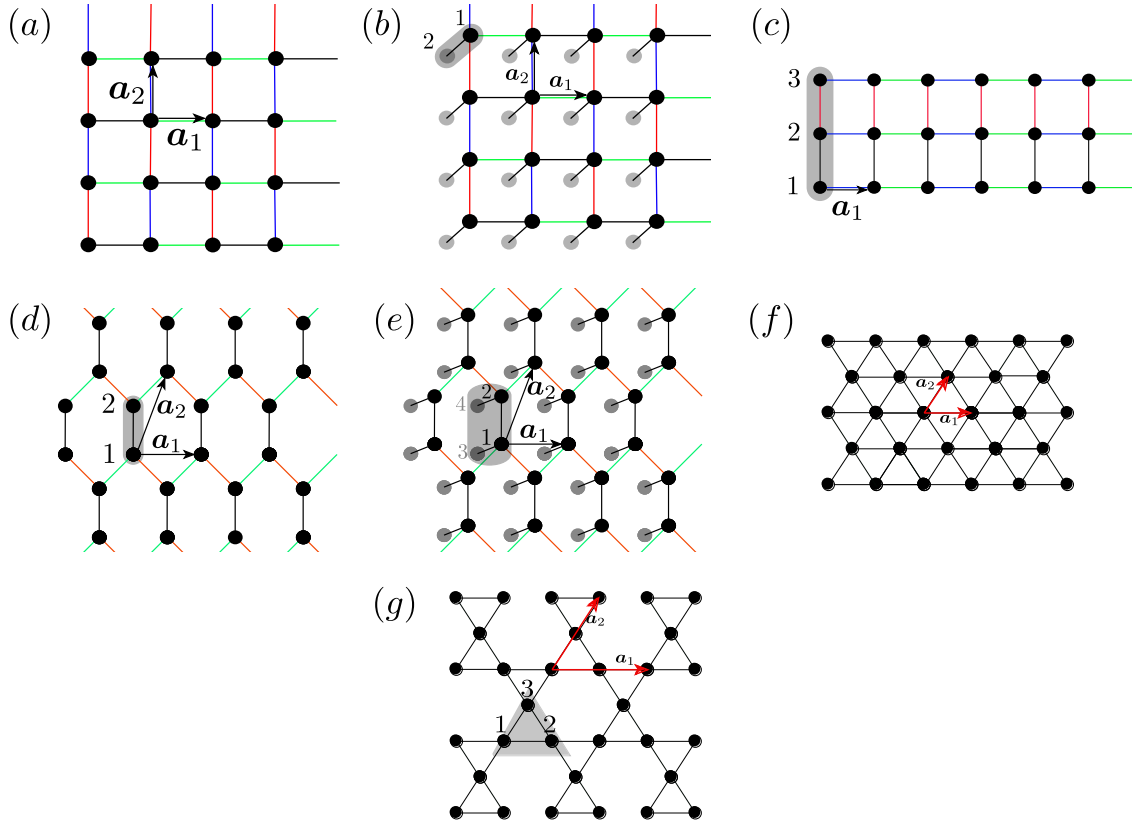


Figure 9: Predefined lattices in ALF: (a) square, (b) bilayer square, (c) 3-leg ladder, (d) honeycomb, (e) bilayer honeycomb, (f) triangular and (g) kagome. Nontrivial unit cells are shown as gray regions, while gray sites belong to the second layer in bilayer systems. The links between the orbitals denote the hopping matrix elements and we have assumed, for the purpose of the plot, the absence of hopping in the second layer for bilayer systems. The color coding of the links denotes the checkerboard decomposition.

Argument	Type	Role	Description
Lattice_type	char	Input	Lattice configuration, which can take the values: - Square - Honeycomb - Pi_Flux (deprecated) - N_leg_ladder - Bilayer_square - Bilayer_honeycomb
L1, L2	int	Input	Lattice sizes (set L2=1 for 1D lattices)
Ndim	int	Output	Total number of orbitals
List	int	Output	For every site index $I \in [1, Ndim]$, stores the corresponding lattice position, $List(I, 1)$, and the (local) orbital index, $List(I, 2)$
Invlist	int	Output	For every lattice_position $\in [1, Latt\%N]$ and orbital $\in [1, Norb]$ stores the corresponding site index $I(lattice_position, orbital)$
Latt	Lattice	Output	Sets the lattice
Latt_Unit	Unit_cell	Output	Sets the unit cell

Table 21: Arguments of the subroutine Predefined_Latt. Note that the Pi_Flux lattice is deprecated, since it can be emulated with the Square lattice with half a flux quanta piercing each plaquette.

```

      Invlist(I,no) = nc                      ! Super-index for given unit cell and orbital
    Enddo
  Enddo

```

With the above-defined lists one can run through all the orbitals while keeping track of the unit-cell and orbital index. We note that when translation symmetry is completely absent one can work with a single unit cell, and the number of orbitals will then correspond to the number of lattice sites.

8.1.1 Square lattice, Fig. 9(a)

The choice `Lattice_type = "Square"` sets $\mathbf{a}_1 = (1, 0)$ and $\mathbf{a}_2 = (0, 1)$ and for an $L_1 \times L_2$ lattice $\mathbf{L}_1 = L_1 \mathbf{a}_1$ and $\mathbf{L}_2 = L_2 \mathbf{a}_2$:

```

Latt_Unit%N_coord = 2
Latt_Unit%Norb    = 1
Latt_Unit%Orb_pos_p(1,:) = 0.d0
a1_p(1) = 1.0 ; a1_p(2) = 0.d0
a2_p(1) = 0.0 ; a2_p(2) = 1.d0
L1_p    = dble(L1)*a1_p
L2_p    = dble(L2)*a2_p
Call Make_Lattice( L1_p, L2_p, a1_p, a2_p, Latt )

```

Also, the number of orbitals per unit cell is given by $NORB=1$ such that $N_{dim} \equiv N_{unit-cell} \cdot NORB = Latt\%N \cdot NORB$, since $N_{unit-cell} = Latt\%N$.

8.1.2 Bilayer Square lattice, Fig. 9(b)

The "Bilayer_square" configuration sets:

```
Latt_Unit%Norb      = 2
Latt_Unit%N_coord   = 2
do no = 1,2
  Latt_Unit%Orb_pos_p(no,1) = 0.d0
  Latt_Unit%Orb_pos_p(no,2) = 0.d0
  Latt_Unit%Orb_pos_p(no,3) = real(1-no,kind(0.d0))
enddo
Call Make_Lattice( L1_p, L2_p, a1_p, a2_p, Latt )
Latt%a1_p(1) = 1.0 ; Latt%a1_p(2) = 0.d0
Latt%a2_p(1) = 0.0 ; Latt%a2_p(2) = 1.d0
Latt%L1_p    = dble(L1)*a1_p
Latt%L2_p    = dble(L2)*a2_p
```

8.1.3 N-leg Ladder lattice, Fig. 9(c)

The "N_leg_ladder" configuration sets:

```
Latt_Unit%Norb      = L2
Latt_Unit%N_coord   = 1
do no = 1,L2
  Latt_Unit%Orb_pos_p(no,1) = 0.d0
  Latt_Unit%Orb_pos_p(no,2) = real(no-1,kind(0.d0))
enddo
a1_p(1) = 1.0 ; a1_p(2) = 0.d0
a2_p(1) = 0.0 ; a2_p(2) = 1.d0
L1_p    = dble(L1)*a1_p
L2_p    = a2_p
Call Make_Lattice( L1_p, L2_p, a1_p, a2_p, Latt )
```

8.1.4 Honeycomb lattice, Fig. 9(d)

In order to carry out simulations on the Honeycomb lattice, which is a triangular Bravais lattice with two orbitals per unit cell, choose Lattice_type="Honeycomb", which sets

```
a1_p(1) = 1.D0 ; a1_p(2) = 0.d0
a2_p(1) = 0.5D0 ; a2_p(2) = sqrt(3.D0)/2.D0
L1_p    = Dble(L1) * a1_p
L2_p    = dble(L2) * a2_p
Call Make_Lattice( L1_p, L2_p, a1_p, a2_p, Latt )
Latt_Unit%Norb      = 2
Latt_Unit%N_coord   = 3
Latt_Unit%Orb_pos_p(1,:) = 0.d0
Latt_Unit%Orb_pos_p(2,:) = (a2_p(:) - 0.5D0*a1_p(:)) * 2.D0/3.D0
```

The coordination number of this lattice is $N_{\text{coord}}=3$ and the number of orbitals per unit cell, $N_{\text{ORB}}=2$. The total number of orbitals is therefore $N_{\text{dim}}=Latt\%N*N_{\text{ORB}}$.

8.1.5 Bilayer Honeycomb lattice, Fig. 9(e)

The "Bilayer_honeycomb" configuration sets:

```
Latt_Unit%Norb      = 4
Latt_Unit%N_coord   = 3
Latt_unit%Orb_pos_p = 0.d0
do n = 1,2
  Latt_Unit%Orb_pos_p(1,n) = 0.d0
```

```

Latt_Unit%Orb_pos_p(2,n) = (a2_p(n) - 0.5D0*a1_p(n)) * 2.D0/3.D0
Latt_Unit%Orb_pos_p(3,n) = 0.d0
Latt_Unit%Orb_pos_p(4,n) = (a2_p(n) - 0.5D0*a1_p(n)) * 2.D0/3.D0
enddo
Latt_Unit%Orb_pos_p(3,3) = -1.d0
Latt_Unit%Orb_pos_p(4,3) = -1.d0
a1_p(1) = 1.D0 ; a1_p(2) = 0.d0
a2_p(1) = 0.5D0 ; a2_p(2) = sqrt(3.D0)/2.D0
L1_p = dble(L1)*a1_p
L2_p = dble(L2)*a2_p
Call Make_Lattice( L1_p, L2_p, a1_p, a2_p, Latt )

```

8.1.6 Triangular lattice, Fig. 9(f)

The "Triangular" configuration sets:

```

Latt_Unit%Norb = 1
Latt_Unit%N_coord = 3
a1_p(1) = 1.D0 ; a1_p(2) = 0.d0
a2_p(1) = 0.5D0 ; a2_p(2) = sqrt(3.D0)/2.D0
Allocate (Latt_Unit%Orb_pos_p(1,2))
Latt_Unit%Orb_pos_p = 0.d0
L1_p = dble(L1) * a1_p
L2_p = dble(L2) * a2_p
Call Make_Lattice( L1_p, L2_p, a1_p, a2_p, Latt )

```

This lattice is only implemented for the Hubbard model.

8.1.7 Kagome lattice, Fig. 9(g)

The "Kagome" configuration sets:

```

Latt_Unit%Norb = 3
Latt_Unit%N_coord = 4
a1_p(1) = 1.D0 ; a1_p(2) = 0.d0
a2_p(1) = 0.5D0 ; a2_p(2) = sqrt(3.D0)/2.D0
Allocate (Latt_Unit%Orb_pos_p(3,2))
Latt_Unit%Orb_pos_p = 0.d0
Latt_Unit%Orb_pos_p(2,:) = a1_p(:)/2.d0
Latt_Unit%Orb_pos_p(3,:) = a2_p(:)/2.d0
L1_p = dble(L1) * a1_p
L2_p = dble(L2) * a2_p
Call Make_Lattice( L1_p, L2_p, a1_p, a2_p, Latt )

```

This lattice is only implemented for the Hubbard model.

8.1.8 π -Flux lattice (deprecated)

The "Pi_Flux" lattice has been deprecated, since it can be emulated with the Square lattice with half a flux quanta piercing each plaquette. Nonetheless, the configuration is still available, and sets:

```

Latt_Unit%Norb = 2
Latt_Unit%N_coord = 4
a1_p(1) = 1.D0 ; a1_p(2) = 1.d0
a2_p(1) = 1.D0 ; a2_p(2) = -1.d0
Latt_Unit%Orb_pos_p(1,:) = 0.d0
Latt_Unit%Orb_pos_p(2,:) = (a1_p(:) - a2_p(:))/2.d0
L1_p = dble(L1) * (a1_p - a2_p)/2.d0
L2_p = dble(L2) * (a1_p + a2_p)/2.d0
Call Make_Lattice( L1_p, L2_p, a1_p, a2_p, Latt )

```

8.2 Generic hopping matrices on Bravais lattices

The module `Predefined_Hopping` provides a generic way to specify a hopping matrix on a multi-orbital Bravais lattice. The only assumption that we make is translation symmetry. We allow for twisted boundary conditions in the L_1 and L_2 lattice directions. The twist is given by `Phi_X` and `Phi_Y` respectively. If the flag `bulk=.true.`, then the twist is implemented with a vector potential. Otherwise, if `bulk=.false.`, the twist is imposed at the boundary. The routine also accounts for the inclusion of a total number of N_{Phi} flux quanta traversing the lattice. All phase factors mentioned above can be flavor dependent. Finally, the checkerboard decomposition can also be specified in this module.

8.2.1 Setting up the hopping matrix: the `Hopping_Matrix_type`

All information for setting up a generic hopping matrix on a lattice, including the checkerboard decomposition, is specified in the `Hopping_Matrix_type` type, which we describe in the remaining of this section. The information stored in this type (see Table 22) fully defines the array of operator type `OP_T` that accounts for the single particle propagation in one time step, from which the kinetic energy can be derived as well.

Generic hopping matrices

The generic Hopping Hamiltonian reads:

$$\hat{H}_T = \sum_{(i,\delta),(j,\delta'),s,\sigma} T_{(i,\delta),(j,\delta'),s,\sigma}^{(s)} \hat{c}_{(i,\delta),s,\sigma}^\dagger e^{\frac{2\pi i}{\Phi_0} \int_{i+\delta}^{j+\delta'} A^{(s)}(l) dl} \hat{c}_{(j,\delta'),s,\sigma} \quad (168)$$

with boundary conditions

$$\hat{c}_{(i+L_i,\delta),s,\sigma}^\dagger = e^{-2\pi i \frac{\Phi_i^{(s)}}{\Phi_0}} e^{\frac{2\pi i}{\Phi_0} \chi_{L_i}^{(s)}(i+\delta)} \hat{c}_{(i,\delta),s,\sigma}^\dagger. \quad (169)$$

Here i labels the unit cell and δ the orbital. Both the twist and vector potential can have a flavor dependency. These and the other components of the generic Hopping Hamiltonian are described below. For now onwards we will mostly omit the flavor index s .

Phase factors. The vector potential accounts for an orbital magnetic field in the z direction that is implemented in the Landau gauge: $\mathbf{A}(\mathbf{x}) = -B(y, 0, 0)$ with $\mathbf{x} = (x, y, z)$. Φ_0 corresponds to the flux quanta and the scalar function χ is defined through:

$$\mathbf{A}(\mathbf{x} + \mathbf{L}_i) = \mathbf{A}(\mathbf{x}) + \nabla \chi_{L_i}(\mathbf{x}). \quad (170)$$

Provided that the bare hopping Hamiltonian, T (i.e., without phases, see Eq. (176)), is invariant under lattice translations, $\hat{H}_T$ commutes with magnetic translations that satisfy the algebra:

$$\hat{T}_a \hat{T}_b = e^{\frac{2\pi i}{\Phi_0} \mathbf{B} \cdot (\mathbf{a} \times \mathbf{b})} \hat{T}_b \hat{T}_a. \quad (171)$$

On the torus, the uniqueness of the wave functions requires that $\hat{T}_{L_1} \hat{T}_{L_2} = \hat{T}_{L_2} \hat{T}_{L_1}$ such that

$$\frac{\mathbf{B} \cdot (\mathbf{a} \times \mathbf{b})}{\Phi_0} = N_\Phi \quad (172)$$

with N_Φ an integer. The variable `N_Phi`, specified in the parameter file, denotes the number of flux quanta piercing the lattice. The variables `Phi_X` and `Phi_Y` also in the parameter file denote the twists – in units of the flux quanta – along the L_1 and L_2 directions. There are

gauge equivalent ways to insert the twist in the boundary conditions. In the above we have inserted the twist as a boundary condition such that for example setting $\text{Phi_1}=0.5$ corresponds to anti-periodic boundary conditions along the L_1 axis. Alternatively we can consider the Hamiltonian:

$$\hat{H}_T = \sum_{(i,\delta),(j,\delta'),s,\sigma} T_{(i,\delta),(j,\delta'),s,\sigma}^{(s)} \tilde{c}_{(i,\delta),s,\sigma}^\dagger e^{\frac{2\pi i}{\Phi_0} \int_{i+\delta}^{j+\delta'} (A(l)+A_\phi) dl} \tilde{c}_{(j,\delta'),s,\sigma} \quad (173)$$

with boundary conditions

$$\tilde{c}_{(i+L_i,\delta),s,\sigma}^\dagger = e^{\frac{2\pi i}{\Phi_0} \chi_{L_i}(i+\delta)} \tilde{c}_{(i,\delta),s,\sigma}^\dagger. \quad (174)$$

Here

$$A_\phi = \frac{\phi_1 |a_1|}{2\pi |L_1|} b_1 + \frac{\phi_2 |a_2|}{2\pi |L_2|} b_2 \quad (175)$$

and b_i corresponds to the reciprocal lattice vectors satisfying $a_i \cdot b_j = 2\pi \delta_{i,j}$. The logical variable `bulk` chooses between these two gauge equivalent ways of inserting the twist angle. If `bulk=.true.` then we use periodic boundary conditions – in the absence of an orbital field – otherwise twisted boundaries are used. The above phase factors are computed in the module function:

```
complex function Generic_hopping(i, no_i, n_1, n_2, no_j, N_Phi, Phi_1, Phi_2, Bulk,
                                Latt, Latt_Unit)
```

which returns the phase factor involved in the hopping of a hole from lattice site $i + \delta_{\text{no}_i}$ to $i + n_1 a_1 + n_2 a_2 + \delta_{\text{no}_j}$. Here δ_{no_i} is the position of the no_i orbital in the unit cell i . The information for the phases is encoded in the type `Hopping_matrix_type`.

The Hopping matrix elements. The hopping matrix is specified assuming only translation invariance. (The point group symmetry of the lattice can be broken.) That is, we assume that for each flavor index:

$$T_{(i,\delta),(i+n_1 a_1 + n_2 a_2, \delta')}^{(s)} = T_{(0,\delta),(n_1 a_1 + n_2 a_2, \delta')}^{(s)}. \quad (176)$$

The right hand side of the above equation is given the type `Hopping_matrix_type`.

The checkerboard decomposition. Aside from the hopping phases and hopping matrix elements, the `Hopping_matrix_type` type contains information concerning the checkerboard decomposition. In Eq. (87) we wrote the hopping Hamiltonian as:

$$\hat{\mathcal{H}}_T = \sum_{i=1}^{N_T} \sum_{k \in \mathcal{S}_i^T} \hat{T}^{(k)}, \quad (177)$$

with the rule that if k and k' belong to the same set $\mathcal{S}_i^T$ then $[\hat{T}^{(k)}, \hat{T}^{(k')}] = 0$. In the checkerboard decomposition, $\hat{T}^{(k)}$ corresponds to hopping on a bond. The checkerboard decomposition depends on the lattice type, as well as on the hopping matrix elements. The required information is stored in `Hopping_matrix_type`. In this data type, `N_FAM` corresponds to the number of sets (or families) (N_T in the above equation). `L_FAM(1:N_FAM)` corresponds to the number of bonds in the set, and finally, `LIST_FAM(1:N_FAM, 1:max(L_FAM(:)), 2)` contains information concerning the two legs of the bonds. Finally, to be able to generate the imaginary time step of length $\Delta\tau$ we have to know by which fraction of $\Delta\tau$ we have to propagate each set. This information is given in the array `Prop_Fam`.

As an example we can consider the three-leg ladder lattice of Figure 9(c). Here the number of sets (or families) `N_FAM` is equal to four, corresponding to the red, green, black and blue bonds. It is clear from the figure that bonds in a given set do not have common legs, so that hopping instances on the bonds of a given set commute.

Usage: the Hopping_Matrix_type

There are N_{bonds} hopping matrix elements emanating from a given unit cell, defined so that looping over all of the elements does not overcount the bonds. For each bond, the array `List` contains the full information to define the RHS of Eq. (176). The hopping amplitudes are stored in the array `T` and the local potentials in the array `T_loc` (See Table 22). The `Hopping_Matrix_type` type also contains the information for the checkerboard decomposition.

Variable	Type	Description
<code>N_bonds</code>	<code>int</code>	Number of hopping matrix elements within and emanating from a unit cell
<code>List(N_bonds,4)</code>	<code>int</code>	$\text{List}(\bullet,1) = \delta$ $\text{List}(\bullet,2) = \delta'$ $\text{List}(\bullet,3) = n_1$ $\text{List}(\bullet,4) = n_2$
<code>T(N_bonds)</code>	<code>cmplx</code>	Hopping amplitude
<code>T_loc(Norb)</code>	<code>cmplx</code>	On site potentials (e.g., chemical potential, Zeeman field)
<code>N_Phi</code>	<code>int</code>	Number of flux quanta piercing the lattice
<code>Phi_X</code>	<code>dbble</code>	Twist in $\mathbf{a}_1$ direction
<code>Phi_Y</code>	<code>dbble</code>	Twist in $\mathbf{a}_2$ direction
<code>Bulk</code>	<code>logical</code>	Twist as vector potential (T) or boundary condition (F)
<code>N_Fam</code>	<code>int</code>	Number of sets, N_T in Eq. (87)
<code>L_Fam(N_FAM)</code>	<code>int</code>	Number of bonds per set $\mathcal{S}^T$
<code>List_Fam(N_FAM,max(L_FAM(:)),2)</code>	<code>int</code>	$\text{List_Fam}(\bullet,\bullet,1) = \text{Unit cell}$ $\text{List_Fam}(\bullet,\bullet,2) = \text{Bond number}$
<code>Prop_Fam(N_FAM)</code>	<code>dbble</code>	The fraction of $\Delta\tau$ with which the set will be propagated

Table 22: Public member variables of the `Hopping_Matrix_type` type.

The data in the `Hopping_matrix_type` type suffices to uniquely define the unit step propagation for the kinetic energy, and for any combinations of the Checkerboard and Symm options (see Sec. 2.4). The propagation is set through the call:

```
Call Predefined_Hoppings_set_OPT(Hopping_Matrix, List, Invlist, Latt, Latt_unit, Dtau,
                                Checkerboard, Symm, OP_T,
                                pinned_vertices, pinning_factor, pinning_offset)
```

in which the operator array `OP_T(*,N_FL)` is allocated and defined. In the simplest case, where no checkerboard is used, the array's first dimension is unity.

This routine allows optional arguments to allow for pinning: that is, modifying existing matrix elements in the hopping matrix. The syntax is the following:

- `pinned_vertices[1:N_pin, 1:2]` contains the index of the two legs `pinned_vertices[i, 1]` and `pinned_vertices[i, 2]` of the i -th vertex to be pinned, where N_{pin} is the number of pinned vertices. If `pinned_vertices[i, 1] = pinned_vertices[i, 2]` then the pinning refers to a site and not to a bond.
- `pinning_factor[1:N_pin, 1:N_fl]` contains the multiplicative factor for each flavor. Meaning that the matrix element gets multiplied by this factor.

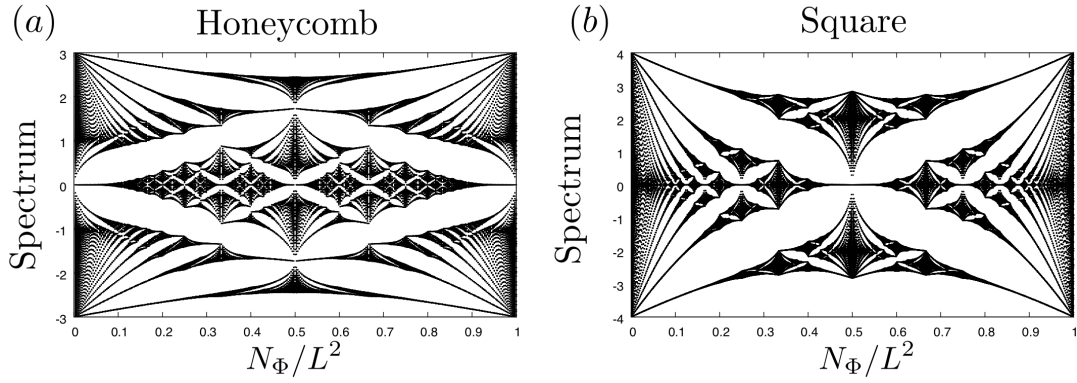


Figure 10: The single particle spectrum of the tight binding model on the honeycomb (a) and square (b) lattices as a function of the flux N_Φ . This corresponds to the well known Hofstadter butterflies.

- `pinning_offset[1:N_pin, 1:N_fl]` contains the offset for each flavor. Meaning that this value gets added to the matrix element after multiplication by the pinning factor.

An explicit example of how to use the pinning option is describe in Sec. 9.1.

The data in the `Hopping_matrix_type` type equally suffices to compute the kinetic energy. This is carried out in the routine `Predefined_Hoppings_Compute_Kin`.

8.2.2 An example: nearest neighbor hopping on the honeycomb lattice

For the honeycomb lattice of Fig. 9(d) the number of bond within and emanating from a unit cell is $N_{\text{bonds}} = 3$. The list array of the `Hopping_matrix_type` reads:

```
list(1,1) = 1; list(1,2) = 2; list(1,3) = 0; list(1,4) = 0 ! Intra unit-cell hopping
list(2,1) = 2; list(2,2) = 1; list(2,3) = 0; list(2,4) = 1 ! Inter unit-cell hopping
list(3,1) = 1; list(3,2) = 2; list(3,3) = 1; list(3,4) = -1 ! Inter unit-cell hopping
T(1) = -1.0; T(2) = -1.0; T(3) = -1.0 ! Hopping
T_loc(1) = 0.0; T_loc(2) = 0.0 ! Chemical potential
```

In the last two lines, we have set the hopping matrix element for each bond to -1 and the chemical potential to zero. The fields, can then be specified with the variables N_{phi} , Phi_x , Phi_y . Setting the twists, Phi_x , Phi_y to zero and looping over N_{phi} from $1 \cdots L^2$ produces the single particle spectrum of Fig. 10(a).

For the honeycomb lattice the checkerboard decomposition for the nearest neighbor hopping consists of three sets: $N_{\text{Fam}} = 3$ each of length corresponding to the number of unit cells. In Fig. 9(d) these sets are denoted by different colors. In the code, the elements of the sets are specified as:

```
do I = 1,Latt%N
  do nf = 1,N_FAM
    List_Fam(nf,I,1) = I ! Unit cell
    List_Fam(nf,I,2) = nf ! The bond
  enddo
enddo
```

8.2.3 Predefined hoppings

The module provides hopping and checkerboard decompositions, defining a `Hopping_Matrix` (an array of length N_{FL} of type `Hopping_Matrix_type`, see Sec. 8.2.1) for each of the fol-

lowing predefined lattices.

Square

The call:

```
Call Set_Default_hopping_parameters_square(Hopping_Matrix, T_vec, Chem_vec, Phi_X_vec,
                                           Phi_Y_vec, Bulk, N_Phi_vec, N_FL, List, Invlist, Latt, Latt_unit)
```

defines the Hopping_Matrix for the square lattice:

$$\hat{H}_T = \sum_{i,\sigma,s} \left(\left[\sum_{\delta=\{a_1,a_2\}} -t^{(s)} \hat{c}_{i,s,\sigma}^\dagger e^{\frac{2\pi i}{\Phi_0} \int_i^{i+\delta} A^{(s)}(l) dl} \hat{c}_{i+\delta,s,\sigma} + \text{H.c.} \right] - \mu^{(s)} \hat{c}_{i,s,\sigma}^\dagger \hat{c}_{i,s,\sigma} \right). \quad (178)$$

The vectors T_vec and Chem_vec have length N_FL and specify the hopping and the chemical potentials, while the vectors Phi_X_vec, Phi_Y_vec and N_Phi_vec, also of length N_FL, define the vector potential.

Honeycomb

The call:

```
Call Set_Default_hopping_parameters_honeycomb(Hopping_Matrix, T_vec, Chem_vec, Phi_X_vec,
                                                Phi_Y_vec, Bulk, N_Phi_vec, N_FL, List, Invlist, Latt, Latt_unit)
```

defines the Hopping_Matrix for the honeycomb lattice:

$$\hat{H}_T = \sum_{i,\sigma,s} \left(\sum_{\delta=\{\delta_1,\delta_2,\delta_3\}} -t^{(s)} \hat{c}_{i,s,\sigma}^\dagger e^{\frac{2\pi i}{\Phi_0} \int_i^{i+\delta} A^{(s)}(l) dl} \hat{c}_{i+\delta,s,\sigma} + \text{H.c.} \right) + \sum_{i,\sigma,s} -\mu^{(s)} \left(\hat{c}_{i,s,\sigma}^\dagger \hat{c}_{i,s,\sigma} + \hat{c}_{i+\delta_1,s,\sigma}^\dagger \hat{c}_{i+\delta_1,s,\sigma} \right), \quad (179)$$

where the T_vec and Chem_vec have length N_FL and specify the hopping and the chemical potentials, while the vectors Phi_X_vec, Phi_Y_vec and N_Phi_vec, also of length N_FL, define the vector potential. Here i runs over sublattice A, and $i + \delta$ over the three nearest neighbors of site i .

Square bilayer

The call:

```
Call Set_Default_hopping_parameters_Bilayer_square(Hopping_Matrix, T1_vec, T2_vec,
                                                    Tperp_vec, Chem_vec, Phi_X_vec, Phi_Y_vec, Bulk, N_Phi_vec, N_FL, List, Invlist,
                                                    Latt, Latt_unit)
```

defines the Hopping_Matrix for the bilayer square lattice:

$$\hat{H}_T = \sum_{i,\sigma,s,n} \left(\left[\sum_{\delta=\{a_1,a_2\}} -t_n^{(s)} \hat{c}_{i,s,\sigma,n}^\dagger e^{\frac{2\pi i}{\Phi_0} \int_i^{i+\delta} A^{(s)}(l) dl} \hat{c}_{i+\delta,s,\sigma,n} + \text{H.c.} \right] - \mu^{(s)} \hat{c}_{i,s,\sigma,n}^\dagger \hat{c}_{i,s,\sigma,n} \right) + \sum_{i,\sigma,s} -t_\perp^{(s)} \left(\hat{c}_{i,s,\sigma,1}^\dagger \hat{c}_{i,s,\sigma,2} + \text{H.c.} \right), \quad (180)$$

where the additional index n labels the layers.

Honeycomb bilayer

The call:

```
Call Set_Default_hopping_parameters_Bilayer_honeycomb(Hopping_Matrix, T1_vec, T2_vec,
    Tperp_vec, Chem_vec, Phi_X_vec, Phi_Y_vec, Bulk, N_Phi_vec, N_FL, List, Invlist,
    Latt, Latt_unit)
```

defines the Hopping_Matrix for the bilayer honeycomb lattice:

$$\begin{aligned} \hat{H}_T = & \sum_{i,\sigma,s,n} \left(\sum_{\delta=\{\delta_1,\delta_2,\delta_3\}} -t_n^{(s)} \hat{c}_{i,s,\sigma,n}^\dagger e^{\frac{2\pi i}{\Phi_0} \int_i^{i+\delta} A^{(s)}(l) dl} \hat{c}_{i+\delta,s,\sigma,n} + \text{H.c.} \right) \\ & + \sum_{i,\sigma,s} -t_\perp^{(s)} \left(\hat{c}_{i,s,\sigma,1}^\dagger \hat{c}_{i,s,\sigma,2} + \hat{c}_{i+\delta_1,s,\sigma,1}^\dagger \hat{c}_{i+\delta_1,s,\sigma,2} + \text{H.c.} \right) \\ & + \sum_{i,\sigma,s,n} -\mu^{(s)} \left(\hat{c}_{i,s,\sigma,n}^\dagger \hat{c}_{i,s,\sigma,n} + \hat{c}_{i+\delta_1,s,\sigma,n}^\dagger \hat{c}_{i+\delta_1,s,\sigma,n} \right) \end{aligned} \quad (181)$$

Here, the additional index n labels the layer. i runs over the unit cells and $\delta = \{\delta_1, \delta_2, \delta_3\}$ over the three nearest neighbors.

N-leg ladder

The call:

```
Call Set_Default_hopping_parameters_n_leg_ladder(Hopping_Matrix, T_vec, Tperp_vec,
    Chem_vec, Phi_X_vec, Phi_Y_vec, Bulk, N_Phi_vec, N_FL, List, Invlist, Latt, Latt_unit)
```

defines the Hopping_Matrix for the the N-leg ladder lattice:

$$\begin{aligned} \hat{H}_T = & \sum_{i,\sigma,s} \sum_{n=1}^{\text{Norb}} \left(-t^{(s)} \hat{c}_{i,s,\sigma,n}^\dagger e^{\frac{2\pi i}{\Phi_0} \int_i^{i+a_1} A^{(s)}(l) dl} \hat{c}_{i+a_1,s,\sigma,n} + \text{H.c.} - \mu^{(s)} \hat{c}_{i,s,\sigma,n}^\dagger \hat{c}_{i,s,\sigma,n} \right) \\ & + \sum_{i,\sigma,s} \sum_{n=1}^{\text{Norb}-1} -t_\perp^{(s)} \left(\hat{c}_{i+\delta_1,s,\sigma,n}^\dagger e^{\frac{2\pi i}{\Phi_0} \int_{(n-1)a_2}^{(n)a_2} A^{(s)}(l) dl} \hat{c}_{i+\delta_1,s,\sigma,n+1} + \text{H.c.} \right). \end{aligned} \quad (182)$$

Here, the additional index n defines the orbital. Note that this lattice has open boundary conditions in the a_2 direction.

Triangular and Kagome

The very same structure as above applies for the Kagome and triangular lattices.

8.3 Predefined interaction vertices

In its most general form, an interaction Hamiltonian, expressed in terms of sums of perfect squares, can be written, as presented in Section 1, as a sum of M_V vertices:

$$\begin{aligned} \hat{\mathcal{H}}_V = & \sum_{k=1}^{M_V} U_k \left\{ \sum_{\sigma=1}^{N_{\text{col}}} \sum_{s=1}^{N_{\text{fl}}} \left[\left(\sum_{x,y}^{N_{\text{dim}}} \hat{c}_{x\sigma s}^\dagger V_{xy}^{(ks)} \hat{c}_{y\sigma s} \right) + \alpha_{ks} \right] \right\}^2 \equiv \sum_{k=1}^{M_V} U_k (\hat{V}^{(k)})^2 \\ & \equiv \sum_{k=1}^{M_V} \hat{\mathcal{H}}_V^{(k)}, \end{aligned} \quad (4)$$

which are encoded in one or more variables of type `Operator`, described in Sec. 5.1. We often use arrays of `Operator` type, which should be initialized by repeatedly calling the subroutine `Op_make`.

The module `Predefined_Int_mod.F90` implements some of the most common of such interaction vertices $\hat{\mathcal{H}}_V^{(k)}$, as detailed in the remainder of this section, where we drop the superscript (k) when unambiguous.

8.3.1 SU(N) Hubbard interaction

The SU(N) Hubbard interaction on a given site i is given by

$$\hat{\mathcal{H}}_{V,i} = + \frac{U}{N_{\text{col}}} \left[\sum_{\sigma=1}^{N_{\text{col}}} (\hat{c}_{i\sigma}^\dagger \hat{c}_{i\sigma} - 1/2) \right]^2. \quad (183)$$

Assuming that no other term in the Hamiltonian breaks the SU(N) color symmetry, then this interaction term conveniently corresponds to a single operator, obtained by calling, for each of the N_{dim} sites i :

```
Call Predefined_Int_U_SUN(OP, I, N_SUN, DTAU, U)
```

which defines:

```
Op%P(1) = I
Op%O(1,1) = cmplx(1.d0, 0.d0, kind(0.DO))
Op%alpha = cmplx(-0.5d0, 0.d0, kind(0.DO))
Op%g = SQRT(CMPLX(-DTAU*U/(DBLE(N_SUN)), 0.DO, kind(0.DO)))
Op%type = 2
```

To relate to Eq. (4), we have $V_{xy}^{(is)} = \delta_{x,y} \delta_{x,i}$, $\alpha_{is} = -\frac{1}{2}$ and $U_k = \frac{U}{N_{\text{col}}}$. Here the flavor index, s , plays no role.

8.3.2 M_z -Hubbard interaction

```
Call Predefined_Int_U_MZ(OP_up, OP_do, I, DTAU, U)
```

The M_z -Hubbard interaction is given by

$$\hat{\mathcal{H}}_V = -\frac{U}{2} \sum_i [\hat{c}_{i\uparrow}^\dagger \hat{c}_{i\uparrow} - \hat{c}_{i\downarrow}^\dagger \hat{c}_{i\downarrow}]^2, \quad (184)$$

which corresponds to the general form of Eq. (4) by setting: $N_{\text{fl}} = 2$, $N_{\text{col}} \equiv N_{\text{SUN}} = 1$, $M_V = N_{\text{unit-cell}}$, $U_k = \frac{U}{2}$, $V_{xy}^{(i,s=1)} = \delta_{x,y} \delta_{x,i}$, $V_{xy}^{(i,s=2)} = -\delta_{x,y} \delta_{x,i}$, and $\alpha_{is} = 0$; and which is defined in the subroutine `Predefined_Int_U_MZ` by two operators:

```
Op_up%P(1) = I
Op_up%O(1,1) = cmplx(1.d0, 0.d0, kind(0.DO))
Op_up%alpha = cmplx(0.d0, 0.d0, kind(0.DO))
Op_up%g = SQRT(CMPLX(DTAU*U/2.d0, 0.DO, kind(0.DO)))
Op_up%type = 2

Op_do%P(1) = I
Op_do%O(1,1) = cmplx(1.d0, 0.d0, kind(0.DO))
Op_do%alpha = cmplx(0.d0, 0.d0, kind(0.DO))
Op_do%g = -SQRT(CMPLX(DTAU*U/2.d0, 0.DO, kind(0.DO)))
Op_do%type = 2
```

8.3.3 SU(N) V-interaction

Call Predefined_Int_V_SUN(OP, I, J, N_SUN, DTAU, V)

The interaction term of the generalized t-V model, given by

$$\hat{\mathcal{H}}_{V,i,j} = -\frac{V}{N_{\text{col}}} \left[\sum_{\sigma=1}^{N_{\text{col}}} \left(\hat{c}_{i\sigma}^\dagger \hat{c}_{j\sigma} + \hat{c}_{j\sigma}^\dagger \hat{c}_{i\sigma} \right) \right]^2, \quad (185)$$

is coded in the subroutine Predefined_Int_V_SUN by a single symmetric operator:

```
Op%P(1) = I
Op%P(2) = J
Op%O(1,2) = cmplx(1.d0 ,0.d0, kind(0.D0))
Op%O(2,1) = cmplx(1.d0 ,0.d0, kind(0.D0))
Op%g      = SQRT(CMPLX(DTAU*V/real(N_SUN,kind(0.d0)), 0.D0, kind(0.D0)))
Op%alpha  = cmplx(0.d0, 0.d0, kind(0.D0))
Op%type   = 2
```

8.3.4 Fermion-Ising coupling

Call Predefined_Int_Ising_SUN(OP, I, J, DTAU, XI)

The interaction between the Ising and a fermion degree of freedom, given by

$$\hat{\mathcal{H}}_{V,i,j} = \hat{Z}_{i,j} \xi \sum_{\sigma=1}^{N_{\text{col}}} \left(\hat{c}_{i\sigma}^\dagger \hat{c}_{j\sigma} + \hat{c}_{j\sigma}^\dagger \hat{c}_{i\sigma} \right), \quad (186)$$

where ξ determines the coupling strength, is implemented in the subroutine Predefined_Int_Ising_SUN:

```
Op%P(1) = I
Op%P(2) = J
Op%O(1,2) = cmplx(1.d0 ,0.d0, kind(0.D0))
Op%O(2,1) = cmplx(1.d0 ,0.d0, kind(0.D0))
Op%g      = cmplx(-DTAU*XI,0.D0,kind(0.D0))
Op%alpha  = cmplx(0.d0,0.d0, kind(0.D0))
Op%type   = 1
```

8.3.5 Long-Range Coulomb repulsion

Call Predefined_Int_LRC(OP, I, DTAU)

The Long-Range Coulomb (LRC) interaction can be written as

$$\hat{\mathcal{H}}_V = \frac{1}{N} \sum_{i,j} \left(\hat{n}_i - \frac{N}{2} \right) V_{i,j} \left(\hat{n}_j - \frac{N}{2} \right), \quad (187)$$

where

$$\hat{n}_i = \sum_{\sigma=1}^N \hat{c}_{i,\sigma}^\dagger \hat{c}_{i,\sigma} \quad (188)$$

and i corresponds to a super-index labelling the unit cell and orbital.

The code uses the following HS decomposition:

$$e^{-\Delta\tau\hat{H}_{V,k}} = \int \prod_i d\phi_i e^{-\frac{N\Delta\tau}{4}\phi_i V_{i,j}^{-1}\phi_j - \sum_i i\Delta\tau\phi_i(\hat{n}_i - \frac{N}{2})}. \quad (189)$$

The above holds only provided that the matrix V is positive definite and the implementation follows Ref. [51].

The LRC interaction is implemented in the subroutine `Predefined_Int_LRC`:

```
Op%P(1) = I
Op%O(1,1) = cmplx(1.d0, 0.d0, kind(0.D0))
Op%alpha = cmplx(-0.5d0, 0.d0, kind(0.D0))
Op%g = cmplx(0.d0, DTAU, kind(0.D0))
Op%type = 3
```

8.3.6 J_z - J_z interaction

```
Call Predefined_Int_Jz(OP_up, Op_do, I, J, DTAU, Jz)
```

Another predefined vertex is:

$$\hat{\mathcal{H}}_{V,i,j} = -\frac{|J_z|}{2} \left(S_i^z - \text{sgn}|J_z| S_j^z \right)^2 = J_z S_i^z S_j^z - \frac{|J_z|}{2} (S_i^z)^2 - \frac{|J_z|}{2} (S_j^z)^2 \quad (190)$$

which, if particle fluctuations are frozen on the i and j sites, then $(S_i^z)^2 = 1/4$ and the interaction corresponds to a J_z - J_z ferromagnetic or antiferromagnetic coupling.

The implementation of the interaction in `Predefined_Int_Jz` defines two operators:

```
Op_up%P(1) = I
Op_up%P(2) = J
Op_up%O(1,1) = cmplx(1.d0, 0.d0, kind(0.D0))
Op_up%O(2,2) = cmplx(-Jz/Abs(Jz), 0.d0, kind(0.D0))
Op_up%alpha = cmplx(0.d0, 0.d0, kind(0.D0))
Op_up%g = SQRT(CMPLX(DTAU*Jz/8.d0, 0.d0, kind(0.D0)))
Op_up%type = 2

Op_do%P(1) = I
Op_do%P(2) = J
Op_do%O(1,1) = cmplx(1.d0, 0.d0, kind(0.d0))
Op_do%O(2,2) = cmplx(-Jz/Abs(Jz), 0.d0, kind(0.d0))
Op_do%alpha = cmplx(0.d0, 0.d0, kind(0.d0))
Op_do%g = -SQRT(CMPLX(DTAU*Jz/8.d0, 0.d0, kind(0.d0)))
Op_do%type = 2
```

8.3.7 J_a - J_b interaction

```
Call Predefined_Int_SaSb(OP, a, b, I1_up, I2_up, I1_do, I2_do, J, dtau, gauge_phi)
```

Sets a spin-spin interaction term of the form $JS_a S_b$. Here, a and b are characters that can be 'x', 'y' or 'z'.

The integers `I1_up`, `I2_up`, `I1_do` and `I2_do` are the indices of the sites involved in the interaction, namely of $c_{i1,\uparrow}^\dagger$, $c_{i2,\uparrow}$, $c_{i1,\downarrow}^\dagger$, $c_{i,\downarrow}$.

The term includes a gauge factor g , which can possibly be used to alleviate the sign problem. The gauge factor is a pure phase factor, defined as $g = e^{i\phi}$ with $\phi = \text{gauge_phi} \cdot \pi$.

The spin operators are represented in terms of fermionic operators as follows: $S_i^\alpha = \frac{1}{2}(c_i^\dagger \sigma^\alpha c_i)$. Here, $c_i^\dagger = (c_{i,\uparrow}^\dagger, c_{i,\downarrow}^\dagger)$ is a spinor of fermionic creation operators and σ^α are the Pauli matrices.

All in all, the interaction term is given by:

$$\hat{\mathcal{H}}_V = \frac{g|J|}{8} \left(S_i^a + \bar{g} \frac{J}{|J|} S_j^b \right)^2 = S_i^a S_j^b + \text{const}, \quad (191)$$

where the last equality assumes that all charges fluctuations are frozen out, e.g. by a large Hubbard U . Thereby, $c_{i,\uparrow}^\dagger c_{i,\uparrow} + c_{i,\downarrow}^\dagger c_{i,\downarrow} = 1$.

8.4 Predefined observables

The types `Obser_Vec` and `Obser_Latt` described in Section 5.4 handle arrays of scalar observables and correlation functions with lattice symmetry respectively. The module `Predefined_Obs` provides a set of standard equal-time and time-displaced observables, as described below. It contains procedures and functions. Procedures provide a complete handling of the observable structure. That is, they take care, for example, of incrementing the counter and of the average sign. On the other hand, functions only provide the Wick decomposition result, and the handling of the observable structure is left to the user.

The predefined measurements methods take as input Green functions `GR`, `GTO`, `GOT`, `G00`, and `GTT`, defined in Sec. 7.7.2 and 7.7.3, as well as `N_SUN`, time slice `Ntau`, lattice information, and so on – see Table 23.

As mentioned in Sec. 5.4 and summarized in Table 8 the lattice and unit-cell objects that are passed in the following routines, are tied to the specific observable and not to the Hamiltonian. For example, the Hamiltonian can be defined on a bilayer-honeycomb lattice, with associated *hamiltonian* lattice `Latt_ham`, unit-cell `Latt_unit_ham`, and lists `list_ham`, `invlist_ham`, whereas the observable can be defined on a one dimensional *observable* lattice `Latt` with a two orbitals per unit cell `Unit_cell`. Recall that the total number of orbitals reads

$$\text{Ndim} = \text{Latt_ham}\%N * \text{Latt_unit_ham}\%\text{Norb}$$

and that the Green functions have dimension $\text{Ndim} \times \text{Ndim} \times \text{N_fl}$. The array `list(Ndim, 2)` is defined as follows:

$$\text{list}(n, 1) = \begin{cases} 0 & \text{if } n \text{ is not an orbital of the } \textit{observable} \text{ lattice} \\ i_{\text{unit}} & \text{otherwise. } i_{\text{unit}} \text{ corresponds to the unit cell of the } \textit{observable} \text{ lattice} \end{cases} \quad (192)$$

$$\text{list}(n, 2) = \begin{cases} 0 & \text{if } n \text{ is not an orbital of the } \textit{observable} \text{ lattice} \\ i_{\text{orb}} & \text{otherwise. } i_{\text{orb}} \text{ corresponds to the orbital of the } \textit{observable} \text{ lattice} \end{cases} \quad (193)$$

8.4.1 Equal-time SU(N) spin-spin correlations

A measurement of SU(N) spin-spin correlations can be obtained through:

```
Call Predefined_Obs_eq_SpinSUN_measure(Latt, Latt_unit, List, GR, GRC, N_SUN, ZS, ZP, Obs)
```

If `N_FL = 1` then this routine returns

$$\text{Obs}(i-j, n_i, n_j) = \frac{2N}{N^2-1} \sum_{a=1}^{N^2-1} \langle \langle \hat{c}_{i,n_i}^\dagger T^a \hat{c}_{i,n_i} \hat{c}_{j,n_j}^\dagger T^a \hat{c}_{j,n_j} \rangle \rangle_C, \quad (194)$$

where T^a are the generators of SU(N) satisfying the normalization conditions $\text{Tr}[T^a T^b] = \delta_{a,b}/2$, $\text{Tr}[T^a] = 0$, $\hat{c}_{j,n_j}^\dagger = (\hat{c}_{j,n_j,1}^\dagger, \dots, \hat{c}_{j,n_j,N}^\dagger)$ is an N-flavored spinor, j corresponds to the unit-cell index and n_j labels the orbital.¹¹

¹¹Recall that $\langle \langle \bullet \rangle \rangle_C$ corresponds to the expectation value of an observable for a given configuration of fields C and not to the connected part of the observable, see Eq. (19).

Argument	Type	Description
Latt	Lattice	Lattice as a variable of type Lattice, see Sec. 5.3
Latt_Unit	Unit_cell	Unit cell as a variable of type Unit_cell, see Sec. 5.3
List(Ndim,2)	int	For every site index I, stores the corresponding lattice position, List(I,1), and the (local) orbital index, List(I,2)
NT	int	Imaginary time τ
GR(Ndim,Ndim,N_FL)	cmplx	Equal-time Green function $GR(i,j,s) = \langle c_{i,s} c_{j,s}^\dagger \rangle$
GRC(Ndim,Ndim,N_FL)	cmplx	$GRC(i,j,s) = \langle c_{i,s}^\dagger c_{j,s} \rangle = \delta_{i,j} - GR(j,i,s)$
GTO(Ndim,Ndim,N_FL)	cmplx	Time-displaced Green function $\langle \langle \mathcal{T} \hat{c}_{i,s}(\tau) \hat{c}_{j,s}^\dagger(0) \rangle \rangle$
GOT(Ndim,Ndim,N_FL)	cmplx	Time-displaced Green function $\langle \langle \mathcal{T} \hat{c}_{i,s}(0) \hat{c}_{j,s}^\dagger(\tau) \rangle \rangle$
GOO(Ndim,Ndim,N_FL)	cmplx	Time-displaced Green function $\langle \langle \mathcal{T} \hat{c}_{i,s}(0) \hat{c}_{j,s}^\dagger(0) \rangle \rangle$
GTT(Ndim,Ndim,N_FL)	cmplx	Time-displaced Green function $\langle \langle \mathcal{T} \hat{c}_{i,s}(\tau) \hat{c}_{j,s}^\dagger(\tau) \rangle \rangle$
N_SUN	int	Number of fermion colors N_{col}
ZS	cmplx	$ZS = \text{sgn}(C)$, see Sec. 5.4
ZP	cmplx	$ZP = e^{-S(C)} / \text{Re}[e^{-S(C)}]$, see Sec. 5.4
Obs	Obser_Latt	Output: one or more measurement result

Table 23: Arguments taken by the subroutines in the module Predefined_Obs. Note that a given method makes use of only a subset of this list, as described in this section. Note also that we use the superindex $i = (i, n_i)$ where i denotes the unit cell and n_i the orbital.

Using Wick's theorem, valid for a given configuration of fields, we obtain

$$\text{Obs} = \frac{2N}{N^2 - 1} \sum_{a=1}^{N^2-1} \sum_{\alpha, \beta, \gamma, \delta=1}^N T_{\alpha, \beta}^a T_{\gamma, \delta}^a \times \left(\langle \langle \hat{c}_{i, n_i, \alpha}^\dagger \hat{c}_{i, n_i, \beta} \rangle \rangle_C \langle \langle \hat{c}_{j, n_j, \gamma}^\dagger \hat{c}_{j, n_j, \delta} \rangle \rangle_C + \langle \langle \hat{c}_{i, n_i, \alpha}^\dagger \hat{c}_{j, n_j, \delta} \rangle \rangle_C \langle \langle \hat{c}_{i, n_i, \beta} \hat{c}_{j, n_j, \gamma}^\dagger \rangle \rangle_C \right). \quad (195)$$

For this SU(N) symmetric code, the Green function is diagonal in the spin index and spin independent:

$$\langle \langle \hat{c}_{i, n_i, \alpha}^\dagger \hat{c}_{j, n_j, \beta} \rangle \rangle_C = \delta_{\alpha, \beta} \langle \langle \hat{c}_{i, n_i}^\dagger \hat{c}_{j, n_j} \rangle \rangle_C. \quad (196)$$

Hence,

$$\begin{aligned} \text{Obs} &= \frac{2N}{N^2 - 1} \sum_{a=1}^{N^2-1} \left([\text{Tr} T^a]^2 \langle \langle \hat{c}_{i, n_i}^\dagger \hat{c}_{i, n_i} \rangle \rangle_C \langle \langle \hat{c}_{j, n_j}^\dagger \hat{c}_{j, n_j} \rangle \rangle_C \right. \\ &\quad \left. + \text{Tr}[T^a T^a] \langle \langle \hat{c}_{i, n_i}^\dagger \hat{c}_{j, n_j} \rangle \rangle_C \langle \langle \hat{c}_{i, n_i} \hat{c}_{j, n_j}^\dagger \rangle \rangle_C \right) \\ &= N \langle \langle \hat{c}_{i, n_i}^\dagger \hat{c}_{j, n_j} \rangle \rangle_C \langle \langle \hat{c}_{i, n_i} \hat{c}_{j, n_j}^\dagger \rangle \rangle_C. \end{aligned} \quad (197)$$

Note that we can also define the generators of SU(N) as

$$\hat{S}_\nu^\mu(x) = \hat{c}_{x, \mu}^\dagger \hat{c}_{x, \nu} - \delta_{\mu, \nu} \frac{1}{N} \sum_{\alpha=1}^N \hat{c}_{x, \alpha}^\dagger \hat{c}_{x, \alpha}. \quad (198)$$

With this definition, the spin-spin correlations read:

$$\sum_{\mu, \nu=1}^N \langle \langle \hat{S}_\nu^\mu(x) \hat{S}_\mu^\nu(y) \rangle \rangle_C = (N^2 - 1) \langle \langle \hat{c}_x^\dagger \hat{c}_y \rangle \rangle_C \langle \langle \hat{c}_x \hat{c}_y^\dagger \rangle \rangle_C. \quad (199)$$

In the above x denotes a super index defining site and orbital. Aside from the normalization, this formulation gives the same result.

8.4.2 Equal-time spin correlations

A measurement of the equal-time spin correlations can be obtained by:

Call `Predefined_Obs_eq_SpinMz_measure(Latt, Latt_unit, List, GR, GRC, N_SUN, ZS, ZP, ObsZ, ObsXY, ObsXYZ)`

If $N_{\text{FL}}=2$ and $N_{\text{SUN}}=1$, then the routine returns:

$$\begin{aligned} \text{ObsZ}(\mathbf{i} - \mathbf{j}, n_i, n_j) &= 4 \langle \langle \hat{c}_{i,n_i}^\dagger S^z \hat{c}_{i,n_i} \hat{c}_{j,n_j}^\dagger S^z \hat{c}_{j,n_j} \rangle \rangle_C, \\ \text{ObsXY}(\mathbf{i} - \mathbf{j}, n_i, n_j) &= 2 \left(\langle \langle \hat{c}_{i,n_i}^\dagger S^x \hat{c}_{i,n_i} \hat{c}_{j,n_j}^\dagger S^x \hat{c}_{j,n_j} \rangle \rangle_C + \langle \langle \hat{c}_{i,n_i}^\dagger S^y \hat{c}_{i,n_i} \hat{c}_{j,n_j}^\dagger S^y \hat{c}_{j,n_j} \rangle \rangle_C \right), \\ \text{ObsXYZ} &= \frac{2 \cdot \text{ObsXY} + \text{ObsZ}}{3}. \end{aligned} \quad (200)$$

Here $\hat{c}_{i,n_i}^\dagger = (\hat{c}_{i,n_i,\uparrow}^\dagger, \hat{c}_{i,n_i,\downarrow}^\dagger)$ is a two component spinor and $S = \frac{1}{2}\sigma$, with

$$\sigma = \left(\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \right), \quad (201)$$

the Pauli spin matrices. Note that for each observable the background is stored in the `Obs_latt0` field, see Table 8.

8.4.3 Equal-time Green function

A measurement of the equal-time Green function can be obtained by:

Call `Predefined_Obs_eq_Green_measure(Latt, Latt_unit, List, GR, GRC, N_SUN, ZS, ZP, Obs)`

Which returns:

$$\text{Obs}(\mathbf{i} - \mathbf{j}, n_i, n_j) = \sum_{\sigma=1}^{N_{\text{col}}} \sum_{s=1}^{N_{\text{fl}}} \langle \langle \hat{c}_{i,n_i,\sigma,s}^\dagger \hat{c}_{j,n_j,\sigma,s} \rangle \rangle. \quad (202)$$

8.4.4 Equal-time density-density correlations

A measurement of equal-time density-density correlations can be obtained by:

Call `Predefined_Obs_eq_Den_measure(Latt, Latt_unit, List, GR, GRC, N_SUN, ZS, ZP, Obs)`

Which returns:

$$\text{Obs}(\mathbf{i} - \mathbf{j}, n_i, n_j) = \langle \langle \hat{N}_{i,n_i} \hat{N}_{j,n_j} \rangle \rangle_C, \quad (203)$$

where

$$\hat{N}_{i,n_i} = \sum_{\sigma=1}^{N_{\text{col}}} \sum_{s=1}^{N_{\text{fl}}} \hat{c}_{i,n_i,\sigma,s}^\dagger \hat{c}_{i,n_i,\sigma,s}. \quad (204)$$

The background, $\langle \langle \hat{N}_{i,n_i} \rangle \rangle_C$ is stored in `Obs0`.

8.4.5 Time-displaced Green function

A measurement of the time-displaced Green function can be obtained by:

```
Call Predefined_Obs_tau_Green_measure(Latt, Latt_unit, List, NT, GT0, GOT, G00, GTT,
                                       N_SUN, ZS, ZP, Obs)
```

Which returns:

$$\text{Obs}(\mathbf{i} - \mathbf{j}, \tau, n_i, n_j) = \sum_{\sigma=1}^{N_{\text{col}}} \sum_{s=1}^{N_{\text{fl}}} \langle \langle \hat{c}_{i,n_i,\sigma,s}(\tau) \hat{c}_{j,n_j,\sigma,s}^\dagger \rangle \rangle_C \quad (205)$$

8.4.6 Time-displaced SU(N) spin-spin correlations

A measurement of time-displaced spin-spin correlations for SU(N) models ($N_{\text{fl}} = 1$) can be obtained by:

```
Call Predefined_Obs_tau_SpinSUN_measure(Latt, Latt_unit, List, NT, GT0, GOT, G00, GTT,
                                       N_SUN, ZS, ZP, Obs)
```

$$\text{Obs}(\mathbf{i} - \mathbf{j}, \tau, n_i, n_j) = \frac{2N}{N^2 - 1} \sum_{a=1}^{N^2-1} \langle \langle \hat{c}_{i,n_i}^\dagger(\tau) T^a \hat{c}_{i,n_i}(\tau) \hat{c}_{j,n_j}^\dagger T^a \hat{c}_{j,n_j} \rangle \rangle_C \quad (206)$$

where T^a are the generators of SU(N) (see Sec. 8.4.1 for more details).

8.4.7 Time-displaced spin correlations

A measurement of time-displaced spin-spin correlations for Mz models ($N_{\text{fl}} = 2, N_{\text{col}} = 1$) is returned by:

```
Call Predefined_Obs_tau_SpinMz_measure(Latt, Latt_unit, List, NT, GT0, GOT, G00, GTT,
                                       N_SUN, ZS, ZP, ObsZ, ObsXY, ObsXYZ)
```

Which calculates the following observables:

$$\begin{aligned} \text{ObsZ}(\mathbf{i} - \mathbf{j}, \tau, n_i, n_j) &= 4 \langle \langle \hat{c}_{i,n_i}^\dagger(\tau) S^z \hat{c}_{i,n_i}(\tau) \hat{c}_{j,n_j}^\dagger S^z \hat{c}_{j,n_j} \rangle \rangle_C \\ \text{ObsXY}(\mathbf{i} - \mathbf{j}, \tau, n_i, n_j) &= 2 \left(\langle \langle \hat{c}_{i,n_i}^\dagger(\tau) S^x \hat{c}_{i,n_i}(\tau) \hat{c}_{j,n_j}^\dagger S^x \hat{c}_{j,n_j} \rangle \rangle_C \right. \\ &\quad \left. + \langle \langle \hat{c}_{i,n_i}^\dagger(\tau) S^y \hat{c}_{i,n_i}(\tau) \hat{c}_{j,n_j}^\dagger S^y \hat{c}_{j,n_j} \rangle \rangle_C \right) \\ \text{ObsXYZ} &= \frac{2 \cdot \text{ObsXY} + \text{ObsZ}}{3}. \end{aligned} \quad (207)$$

8.4.8 Time-displaced density-density correlations

A measurement of time-displaced density-density correlations for general SU(N) models is given by:

```
Call Predefined_Obs_tau_Den_measure(Latt, Latt_unit, List, NT, GT0, GOT, G00, GTT,
                                       N_SUN, ZS, ZP, Obs)
```

Which returns:

$$\text{Obs}(\mathbf{i} - \mathbf{j}, \tau, n_i, n_j) = \langle \langle \hat{N}_{i,n_i}(\tau) \hat{N}_{j,n_j} \rangle \rangle_C. \quad (208)$$

The density operator is defined in Eq. (204).

8.4.9 Dimer-Dimer correlations

Let

$$\hat{S}_\nu^\mu(x) = \hat{c}_{x,\mu}^\dagger \hat{c}_{x,\nu} - \delta_{\mu,\nu} \frac{1}{N} \sum_{\alpha=1}^N \hat{c}_{x,\alpha}^\dagger \hat{c}_{x,\alpha} \quad (209)$$

be the generators of SU(N). Dimer-Dimer correlations are defined as:

$$\langle \langle \hat{S}_\nu^\mu(x, \tau) \hat{S}_\mu^\nu(y, \tau) \hat{S}_\delta^\gamma(w) \hat{S}_\gamma^\delta(z) \rangle \rangle_C, \quad (210)$$

where the sum over repeated indices from $1 \cdots N$ is implied. The calculation is carried out for the self-adjoint antisymmetric representation of SU(N) for which $\sum_{\alpha=1}^N \hat{c}_{x,\alpha}^\dagger \hat{c}_{x,\alpha} = N/2$, such that the generators can be replaced by:

$$\hat{S}_\nu^\mu(x) = \hat{c}_{x,\mu}^\dagger \hat{c}_{x,\nu} - \delta_{\mu,\nu} \frac{1}{2}. \quad (211)$$

The function

```
Complex (Kind=Kind(0.d0)) function Predefined_Obs_dimer_tau(x, y, w, z, GT0, GOT, G00,
GTT, N_SUN, N_FL)
```

returns the value of the time-displaced dimer-dimer correlation function. The function

```
Complex (Kind=Kind(0.d0)) function Predefined_Obs_dimer_eq(x, y, w, z, GR, GRC, N_SUN,
N_FL)
```

returns the value of the equal time dimer-dimer correlation function:

$$\langle \langle \hat{S}_\nu^\mu(x, \tau) \hat{S}_\mu^\nu(y, \tau) \hat{S}_\delta^\gamma(w, \tau) \hat{S}_\gamma^\delta(z, \tau) \rangle \rangle_C. \quad (212)$$

Here, both GR and GRC are on time slice τ .

To compute the background terms, the function

```
Complex (Kind=Kind(0.d0)) function Predefined_Obs_dimer0_eq(x, y, GR, N_SUN, N_FL)
```

returns

$$\langle \langle \hat{S}_\nu^\mu(x, \tau) \hat{S}_\mu^\nu(y, \tau) \rangle \rangle_C. \quad (213)$$

All routines are programmed for $N_{\text{SUN}} = 2, 4, 6, 8$ at $N_{\text{FL}}=1$. The routines also handle the case of broken SU(2) spin symmetry corresponding to $N_{\text{FL}}=2$ and $N_{\text{SUN}}=1$. To carry out the Wick decomposition and sums over spin indices, we use the Mathematica notebooks `DimerDimer_SU2_NFL_2.nb` and `DimerDimer_SUN_NFL_1.nb`.

8.4.10 Bond-Bond correlations

The function

```
Complex (Kind=Kind(0.d0)) function Predefined_Obs_dimer_kin_eq(I,M, J, N, GR, GRC,
N_SUN, N_FL)
```

returns:

$$\langle \langle \hat{K}_{i,m} \hat{K}_{j,n} \rangle \rangle_C \text{ with } \hat{K}_{i,m} = \sum_{s=1}^{N_{\text{Fl}}} \sum_{\sigma=1}^{N_{\text{col}}} \hat{c}_{i,s,\sigma}^\dagger \hat{c}_{m,s,\sigma} + \text{H.c.} \quad (214)$$

The accompanying function

```
Complex (Kind=Kind(0.d0)) function Predefined_Obs_dimer_kin0_eq(I,M, GR, N_SUN, N_FL)
```

computes the background:

$$\langle \langle \hat{K}_{i,m} \rangle \rangle_C. \quad (215)$$

8.4.11 Expectation value of perfect squares

The function

```
Complex (Kind=Kind(0.d0)) function Predefined_Obs_V_Int(OP_Vint, GR, GRC, N_SUN )
```

returns, for a `OP_Vint` type=2 vertex, the expectation value:

$$\left\langle \left\langle \left[\sum_{s=1}^{N_{fl}} \sum_{\sigma=1}^{N_{col}} \left(\left(\sum_{x,y} \hat{c}_{x,s,\sigma}^\dagger V_{x,y}^s \hat{c}_{y,s,\sigma} \right) + \alpha_s \right) \right]^2 \right\rangle \right\rangle_C \quad (216)$$

The function is particularly useful in computing the total energy of the model at hand.

8.4.12 Cotunneling for Kondo models

The Kondo lattice model (KLM), $\hat{H}_{KLM}$ is obtained by carrying out a canonical Schrieffer-Wolf [148] transformation of the periodic Anderson model (PAM), $\hat{H}_{PAM}$. Hence, $e^{\hat{S}} \hat{H}_{PAM} e^{-\hat{S}} = \hat{H}_{KLM}$ with $\hat{S}^\dagger = -\hat{S}$. Let $\hat{f}_{x,\sigma}$ create an electron on the correlation f-orbital of the PAM. Then,

$$e^{\hat{S}} \hat{f}_{x,\sigma}^\dagger e^{-\hat{S}} \simeq \frac{2V}{U} \left(\hat{c}_{x,-\sigma'}^\dagger \hat{S}_x^{\sigma'} + \sigma' \hat{c}_{x,\sigma'}^\dagger \hat{S}_x^z \right) \equiv \frac{2V}{U} \tilde{f}_{x,\sigma'}^\dagger. \quad (217)$$

In the above, it is understood that σ' takes the value 1 (−1) for up (down) spin degrees of freedom, that $\hat{S}_x^{\sigma'} = \hat{f}_{x,\sigma'}^\dagger \hat{f}_{x,-\sigma'}$ and that $\hat{S}_x^z = \frac{1}{2} \sum_{\sigma'} \sigma' \hat{f}_{x,\sigma'}^\dagger \hat{f}_{x,\sigma'}$. Finally, $\hat{c}_{x,\sigma'}^\dagger$ corresponds to the conduction electron that hybridizes with $\hat{f}_{x,\sigma'}^\dagger$. This form matches that derived in Ref. [149] and a calculation of the former equation can be found in Ref. [150]. An identical, but more transparent formulation is given in Ref. [151] and reads:

$$\tilde{f}_{x,\sigma}^\dagger = \sum_{\sigma'} \hat{c}_{x,\sigma'}^\dagger \boldsymbol{\sigma}_{\sigma',\sigma} \cdot \hat{\mathbf{S}}_x, \quad (218)$$

where $\boldsymbol{\sigma}$ denotes the vector of Pauli spin matrices. With the above, one will readily show that the $\tilde{f}_{x,\sigma}^\dagger$ transforms as $\hat{f}_{x,\sigma}^\dagger$ under an SU(2) spin rotation. The function

```
Complex (Kind=Kind(0.d0)) function Predefined_Obs_Cotunneling(x_c, x, y_c, y, GT0, GOT,
                                                                GO0, GTT, N_SUN, N_FL)
```

returns the value of the time displaced correlation function:

$$\sum_{\sigma} \langle \langle \tilde{f}_{x,\sigma}^\dagger(\tau) \tilde{f}_{y,\sigma}(0) \rangle \rangle_C. \quad (219)$$

Here, x_c and y_c correspond to the conduction orbitals that hybridize with the x and y f-orbitals. The routine works for SU(N) symmetric codes corresponding to $N_{FL}=1$ and $N_{SUN} = 2, 4, 6, 8$. For the larger N-values, we have replaced the generators of SU(2) with that of SU(N). The routine also handles the case where spin-symmetry is broken by e.g. a Zeeman field. This corresponds to the case $N_{FL}=2$ and $N_{SUN}=1$. Note that the function only carries out the Wick decomposition and the handling of the observable type corresponding to this quantity has to be done by the user. To carry out the Wick decomposition and sums over spin indices, we use the Mathematica notebooks `Cotunneling_SU2_NFL_2.nb` and `Cotunneling_SUN_NFL_1.nb`.

8.4.13 Rényi Entropy

The module `entanglement_mod.F90` allows one to compute the 2nd Rényi entropy, S_2 , for a subsystem. Using Eq. (25), S_2 can be expressed as a stochastic average of an observable constructed from two independent simulations of the model [60]:

$$e^{-S_2} = \sum_{C_1, C_2} P(C_2)P(C_1) \det[G_A(\tau_0; C_1)G_A(\tau_0; C_2) - (\mathbb{1} - G_A(\tau_0; C_1))(\mathbb{1} - G_A(\tau_0; C_2))], \quad (220)$$

where $G_A(\tau_0; C_i)$, $i = 1, 2$ is the Green function matrix restricted to the desired subsystem A at a given time-slice τ_0 , and for the configuration C_i of the replica i . The degrees of freedom defining the subsystem A are lattice site, flavor index, and color index.

Notice that, due to its formulation, sampling S_2 requires an MPI simulation with at least 2 processes. Also, only real-space partitions are currently supported.

A measurement of the 2nd Rényi entropy can be obtained by:

Call `Predefined_Obs_scal_Renyi_Ent(GRC, List, Nsites, N_SUN, ZS, ZP, Obs)`

which returns the observable `Obs`, for which $\langle \text{Obs} \rangle = e^{-S_2}$. The subsystem A can be defined in a number of different ways, which are handled by what we call *specializations* of the subroutine, described as follows.

In the most general case, `List(:, N_FL, N_SUN)` is a three-dimensional array that contains the list of lattice sites in A for every flavor and color index; `Nsites(N_FL, N_SUN)` is then a bidimensional array that provides the number of lattice sites in the subsystem for every flavor and color index; and the argument `N_SUN` must be omitted in the call.

For a subsystem whose degrees of freedom, for a given flavor index, have a common value of color indexes, `Predefined_Obs_scal_Renyi_Ent` can be called by providing `List(:, N_FL)` as a bidimensional array that contains the list of lattice sites for every flavor index. In this case, `Nsites(N_FL)` provides the number of sites in the subsystem for any given flavor index, while `N_SUN(N_FL)` contains the number of color indexes for a given flavor index.

Finally, a specialization exists for the simple case of a subsystem whose lattice degrees of freedom are flavor- and color-independent. In this case, `List(:)` is a one-dimensional array containing the lattice sites of the subsystem. `Nsites` is the number of sites, and `N_SUN` is the number of color indexes belonging to the subsystem. Accordingly, for every element `I` of `List`, the subsystem contains all degrees of freedom with site index `I`, any flavor index, and `1 ... N_SUN` color index.

Mutual Information

The mutual information between two subsystems A and B is given by

$$I_2 = -\ln\langle \text{Renyi_A} \rangle - \ln\langle \text{Renyi_B} \rangle + \ln\langle \text{Renyi_AB} \rangle, \quad (221)$$

where `Renyi_A`, `Renyi_B`, and `Renyi_AB` are the second Rényi entropies of A , B , and $A \cup B$, respectively.

The measurements necessary for computing I_2 are obtained by:

Call `Predefined_Obs_scal_Mutual_Inf(GRC, List_A, Nsites_A, List_B, Nsites_B, N_SUN, ZS, ZP, Obs)`

which returns the 2nd Rényi entropies mentioned above, stored in the variable `Obs`. Here, `List_A` and `Nsites_A` are input parameters describing the subsystem A – with the same conventions and specializations described above – and `List_B` and `Nsites_B` are the corresponding input parameters for the subsystem B , while `N_SUN` is assumed to be identical for A and B .

8.5 Predefined trial wave functions

When using the projective algorithm (see Sec. 3), trial wave functions must be specified. These are stored in variables of the WaveFunction type (Sec. 5.5). The ALF package provides a set of predefined trial wave functions $|\Psi_{T,L/R}\rangle = \text{WF_L/R}$, returned by the call:

```
Call Predefined_TrialWaveFunction(Lattice_type, Ndim, List, Invlist, Latt, Latt_unit,
                                   N_part, N_FL, WF_L, WF_R)
```

Twisted boundary conditions ($\text{Phi_X_vec} = 0.01$) are implemented for some lattices in order to generate non-degenerate trial wave functions. Here the marker “_vec” indicates the variable may assume different values depending on the flavor (e.g., spin up and down). Currently predefined trial wave functions are flavor independent.

The predefined trial wave functions correspond to the solution of the non-interacting tight binding Hamiltonian on each of the predefined lattices. These solutions are the ground states of the predefined hopping matrices (Sec. 8.2) with default parameters, for each lattice, as follows.

8.5.1 Square

Parameter values for the predefined trial wave function on the square lattice:

```
Checkerboard = .false.
Symm         = .false.
Bulk         = .false.
N_Phi_vec    = 0
Phi_X_vec    = 0.01d0
Phi_Y_vec    = 0.d0
Ham_T_vec    = 1.d0
Ham_Chem_vec = 0.d0
Dtau         = 1.d0
```

8.5.2 Honeycomb

The twisted boundary condition for the square lattice lifts the degeneracy present at half-band filling, but breaks time reversal symmetry as well as the C_4 lattice symmetry. If time reversal symmetry is required to avoid the negative sign problem (that would be the case for the attractive Hubbard model at finite doping), then this choice of the trial wave function will introduce a negative sign. One should then use the trial wave function presented in Sec. 7.6. For the Honeycomb case, the trial wave function we choose is the ground state of the tight binding model with small next-next-nearest hopping matrix element t' [135]. This breaks the C_3 symmetry and shifts the Dirac cone away from the zone boundary. Time reversal symmetry is however not broken. Alternatively, one could include a small Kekule mass term. As shown in Sec. 3.3 both choices of trial wave functions produce good results.

8.5.3 N-leg ladder

Parameter values for the predefined trial wave function on the N-leg ladder lattice:

```
Checkerboard = .false.
Symm         = .false.
Bulk         = .false.
N_Phi_vec    = 0
Phi_X_vec    = 0.01d0
Phi_Y_vec    = 0.d0
Ham_T_vec    = 1.d0
Ham_Tperp_vec = 1.d0
```

```
Ham_Chem_vec = 0.d0
Dtau         = 1.d0
```

8.5.4 Bilayer square

Parameter values for the predefined trial wave function on the bilayer square lattice:

```
Checkerboard = .false.
Symm         = .false.
Bulk         = .false.
N_Phi_vec    = 0
Phi_X_vec    = 0.d0
Phi_Y_vec    = 0.d0
Ham_T_vec    = 1.d0
Ham_T2_vec   = 0.d0
Ham_Tperp_vec = 1.d0
Ham_Chem_vec = 0.d0
Dtau         = 1.d0
```

8.5.5 Bilayer honeycomb

Parameter values for the predefined trial wave function on the bilayer honeycomb lattice:

```
Checkerboard = .false.
Symm         = .false.
Bulk         = .false.
N_Phi_vec    = 0
Phi_X_vec    = 0.d0
Phi_Y_vec    = 0.d0
Ham_T_vec    = 1.d0
Ham_T2_vec   = 0.d0
Ham_Tperp_vec = 1.d0
Ham_Chem_vec = 0.d0
Dtau         = 1.d0
```

9 Model Classes

The ALF library comes with five model classes: (i) $SU(N)$ Hubbard models, (ii) $O(2N)$ t-V models, (iii) Kondo models, (iv) long-range Coulomb models, and (v) generic Z_2 lattice gauge theories coupled to Z_2 matter and fermions. Below we detail the functioning of these classes.

9.1 $SU(N)$ Hubbard models `Hamiltonian_Hubbard_smod.F90`

The parameter space for this model class reads:

```
&VAR_Hubbard      !! Variables for the Hubbard class
Mz                = .T.      ! Whether to use the M_z-Hubbard model: Nf=2; N_SUN must be
                             ! even. HS field couples to the z-component of magnetization

ham_T             = 1.d0     ! Hopping parameter
ham_chem          = 0.d0     ! Chemical potential
ham_U             = 4.d0     ! Hubbard interaction
ham_T2            = 1.d0     ! For bilayer systems
ham_U2            = 4.d0     ! For bilayer systems
ham_Tperp         = 1.d0     ! For bilayer systems
Continuous        = .F.     ! For continuous HS decomposition
ham_h0            = 0.d0     ! Pinning. Local magnetic field on site i=0, orbital=1
/
```

In the above listing, `ham_T` and `ham_T2` correspond to the hopping in the first and second layers respectively and `ham_Tperp` is to the interlayer hopping. The Hubbard U term has an orbital index, `ham_U` for the first and `ham_U2` for the second layers. Finally, `ham_chem` corresponds to the chemical potential. If the flag `Mz` is set to `.False.`, then the code simulates the following SU(N) symmetric Hubbard model:

$$\begin{aligned}\hat{H} = & \sum_{(i,\delta),(j,\delta')} \sum_{\sigma=1}^N T_{(i,\delta),(j,\delta')} \hat{c}_{(i,\delta),\sigma}^\dagger e^{\frac{2\pi i}{\Phi_0} \int_{i+\delta}^{j+\delta'} A(l) dl} \hat{c}_{(j,\delta'),\sigma} \\ & + \sum_i \sum_{\delta} \frac{U_{\delta}}{N} \left(\sum_{\sigma=1}^N [\hat{c}_{(i,\delta),\sigma}^\dagger \hat{c}_{(i,\delta),\sigma} - 1/2] \right)^2 - \mu \sum_{(i,\delta)} \sum_{\sigma=1}^N \hat{c}_{(i,\delta),\sigma}^\dagger \hat{c}_{(i,\delta),\sigma} \\ & + h_0 \left(\sum_{\sigma=1}^{N/2} \hat{c}_{(0,1),\sigma,1}^\dagger \hat{c}_{(0,1),\sigma,1} - \sum_{\sigma=N/2}^N \hat{c}_{(0,1),\sigma,2}^\dagger \hat{c}_{(0,1),\sigma,2} \right).\end{aligned}\quad (222)$$

The generic hopping is taken from Eq. (168) with appropriate boundary conditions given by Eq. (169). The index i runs over the unit cells, δ over the orbitals in each unit cell and σ from 1 to N and encodes the SU(N) symmetry. Note that N corresponds to `N_SUN` in the code. The chemical potential μ is relevant only for the finite temperature code. Generically the flavor index is set to unity. However, if the pinning field, h_0 , is included, then the symmetry of the Hamiltonian is reduced from U(N) to U($N/2$) $\times$ U($N/2$). In this case, one should set `N_fl` = 2. The term `ham_h0` accounts for a local magnetic field at site $i = 0$ and orbital $\delta = 1$. This is useful to probe for e.g. long-ranged magnetic order as discussed in Ref. [13]. Note that N is required to be even if the pinning field is included.

If the variable `Mz` is set to `.True.`, then the code equally requires `N_SUN` to be even and simulates the following Hamiltonian:

$$\begin{aligned}\hat{H} = & \sum_{(i,\delta),(j,\delta')} \sum_{\sigma=1}^{N/2} \sum_{s=1,2} T_{(i,\delta),(j,\delta')} \hat{c}_{(i,\delta),\sigma,s}^\dagger e^{\frac{2\pi i}{\Phi_0} \int_{i+\delta}^{j+\delta'} A(l) dl} \hat{c}_{(j,\delta'),\sigma,s} \\ & - \sum_i \sum_{\delta} \frac{U_{\delta}}{N} \left(\sum_{\sigma=1}^{N/2} [\hat{c}_{(i,\delta),\sigma,2}^\dagger \hat{c}_{(i,\delta),\sigma,2} - \hat{c}_{(i,\delta),\sigma,1}^\dagger \hat{c}_{(i,\delta),\sigma,1}] \right)^2 \\ & - \mu \sum_{(i,\delta)} \sum_{\sigma=1}^{N/2} \sum_{s=1,2} \hat{c}_{(i,\delta),\sigma,s}^\dagger \hat{c}_{(i,\delta),\sigma,s} \\ & + h_0 \left(\sum_{\sigma=1}^{N/2} \hat{c}_{(0,1),\sigma,1}^\dagger \hat{c}_{(0,1),\sigma,1} - \sum_{\sigma=N/2}^N \hat{c}_{(0,1),\sigma,2}^\dagger \hat{c}_{(0,1),\sigma,2} \right).\end{aligned}\quad (223)$$

Note that if `Mz=.T.` then the code automatically sets `N_fl`=2. Clearly at $N = 2$, both modes correspond to the Hubbard model. For N even and $N > 2$ the models differ. In particular in the latter Hamiltonian the U(N) symmetry is broken down to U($N/2$) $\times$ U($N/2$). When $h_0 \neq 0$ a local pinning field is applied at site $i = 0$ and orbital $\delta = 1$ to break the U(N) symmetry explicitly. This is useful to probe for e.g. long-ranged magnetic order as discussed in Ref. [13].

If the variable `Continuous=.T.` then the code will use the generic HS transformation:

$$e^{\alpha \hat{A}^2} = \frac{1}{\sqrt{2\pi}} \int d\phi e^{-\phi^2/2 + \sqrt{2\alpha} \hat{A}} \quad (224)$$

as opposed to the discrete version of Eq. 12. If the Langevin flag is set to false, the code will use the single spin-flip update:

$$\phi \rightarrow \phi + \text{Amplitude}(\xi - 1/2) \quad (225)$$

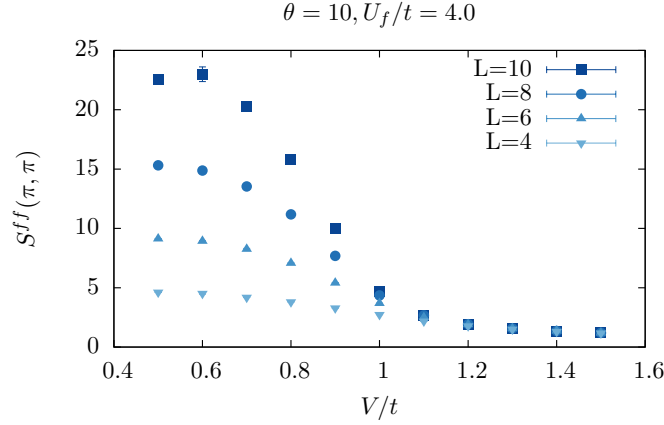


Figure 11: The periodic Anderson model. Here we plot the equal-time spin structure factor of the f-electrons at $\mathbf{q} = (\pi, \pi)$. This quantity is found in the file `SpinZ_eqJK`. The pyALF based python script `Hubbard_PAM.py` produces the data shown for the $L = 8$ lattice. One sees that for the chosen value of U_f/t the competition between the RKKY interaction and Kondo screening drives the system through a magnetic order-disorder transition at $V_c/t \simeq 1$ [152].

where ξ is a random number $\in [0,1]$ and Amplitude is defined in the `Fields_mod.F90` module. Since this model class works for all predefined lattices (see Fig. 9) it includes the $SU(N)$ periodic Anderson model on the square and Honeycomb lattices. Finally, we note that the executable for this class is given by `Hubbard.out`.

As an example, we can consider the periodic Anderson model. Here we choose the `Bilayer_square` lattice $\text{Ham}_U = \text{Ham}_{T2} = 0$, $\text{Ham}_{U2} = U_f$, $\text{Ham}_{t\text{perp}} = V$ and $\text{Ham}_T = 1$. The pyALF based python script `Hubbard_PAM.py` produces the data shown in Fig. 11 for the $L=8$ lattice.

9.2 $SU(N)$ t-V models `Hamiltonian_tV_smod.F90`

The parameter space for this model class reads:

```
&VAR_tV                                !! Variables for the t-V class
ham_T      = 1.d0                      ! Hopping parameter
ham_chem    = 0.d0                      ! Chemical potential
ham_V       = 0.5d0                    ! interaction strength
ham_T2      = 1.d0                      ! For bilayer systems
ham_V2      = 0.5d0                    ! For bilayer systems
ham_Tperp   = 1.d0                      ! For bilayer systems
ham_Vperp   = 0.5d0                    ! For bilayer systems
/
```

In the above `ham_T` and `ham_T2` and `ham_Tperp` correspond to the hopping in the first and second layers respectively and `ham_Tperp` is to the interlayer hopping. The interaction term has an orbital index, `ham_V` for the first and `ham_V2` for the second layers, and `ham_Vperp` for interlayer coupling. Note that we use the same sign conventions here for both the hopping parameters and the interaction strength. This implies a relative minus sign between here and the U_δ interaction strength of the Hubbard model (see Sec. 9.1). Finally `ham_chem` corresponds to the chemical potential. Let us introduce the operator

$$\hat{b}_{\langle(i,\delta),(j,\delta')\rangle} = \sum_{\sigma=1}^N \hat{c}_{(i,\delta),\sigma}^\dagger e^{\frac{2\pi i}{\Phi_0} \int_{i+\delta}^{j+\delta'} A(l) dl} \hat{c}_{(j,\delta'),\sigma} + \text{H.c.} \quad (226)$$

The model is then defined as follows:

$$\begin{aligned}\hat{H} = & \sum_{\langle(i,\delta),(j,\delta')\rangle} T_{(i,\delta),(j,\delta')} \hat{b}_{\langle(i,\delta),(j,\delta')\rangle} + \sum_{\langle(i,\delta),(j,\delta')\rangle} \frac{V_{(i,\delta),(j,\delta')}}{N} (\hat{b}_{\langle(i,\delta),(j,\delta')\rangle})^2 \\ & - \mu \sum_{(i,\delta)} \sum_{\sigma=1}^N \hat{c}_{(i,\delta),\sigma}^\dagger \hat{c}_{(i,\delta),\sigma}.\end{aligned}\quad (227)$$

The generic hopping is taken from Eq. (168) with appropriate boundary conditions given by Eq. (169). The index i runs over the unit cells, δ over the orbitals in each unit cell and σ from 1 to N , encoding the SU(N) symmetry. Note that N corresponds to `N_SUN` in the code. The flavor index is set to unity such that it does not appear in the Hamiltonian. The chemical potential μ is relevant only for the finite temperature code. An example showing how to run this model class can be found in the pyALF based Jupyter notebook [tV_model.ipynb](#).

As a concrete example, we can consider the Hamiltonian of the t-V model of SU(N) fermions on the square lattice,

$$\hat{H} = -t \sum_{\langle i,j \rangle} \hat{b}_{\langle i,j \rangle} - \frac{V}{N} \sum_{\langle i,j \rangle} (\hat{b}_{\langle i,j \rangle})^2 - \mu \sum_i \sum_{\sigma=1}^N \hat{c}_{i,\sigma}^\dagger \hat{c}_{i,\sigma}, \quad (228)$$

which can be simulated by setting `ham_T = t`, `ham_V = V`, and `ham_chem =  $\mu$` . At half-band filling $\mu = 0$, the sign problem is absent for $V > 0$ and for all values of N [75, 153]. For even values of N no sign problem occurs for $V > 0$ and arbitrary chemical potentials [74].

Note that in the absence of orbital magnetic fields, the model has an $O(2N)$ symmetry. This can be seen by writing the model in a Majorana basis (see e.g. Ref. [21]).

9.3 SU(N) Kondo lattice models `Hamiltonian_Kondo_smod.F90`

The Kondo lattice model we consider is an SU(N) generalization of the SU(2) Kondo-model discussed in [31, 32]. Here we follow the work of Ref. [50]. Let T^a be the $N^2 - 1$ generators of SU(N) that satisfy the normalization condition:

$$\text{Tr}[T^a T^b] = \frac{1}{2} \delta_{a,b}. \quad (229)$$

For the SU(2) case, T^a corresponds to the $T = \frac{1}{2} \boldsymbol{\sigma}$ with $\boldsymbol{\sigma}$ a vector of the three Pauli spin matrices, Eq. (201). The Hamiltonian is defined on bilayer square or honeycomb lattices, with hopping restricted to the first layer (i.e conduction orbitals $c_i^\dagger$) and spins, f-orbitals, on the second layer.

$$\begin{aligned}\hat{H} = & -t \sum_{\langle i,j \rangle} \sum_{\sigma=1}^N \left(\hat{c}_{i,\sigma}^\dagger e^{\frac{2\pi i}{\Phi_0} \int_i^j A \cdot dl} \hat{c}_{j,\sigma} + \text{H.c.} \right) - \mu \sum_{i,\sigma} \hat{c}_{i,\sigma}^\dagger \hat{c}_{i,\sigma} \\ & + \frac{U_c}{N} \sum_i \left(\hat{n}_i^c - \frac{N}{2} \right)^2 + \frac{2J}{N} \sum_{i,a=1}^{N^2-1} \hat{T}_i^{a,c} \hat{T}_i^{a,f}.\end{aligned}\quad (230)$$

In the above, i is a super-index accounting for the unit cell and orbital,

$$\hat{T}_i^{a,c} = \sum_{\sigma,\sigma'=1}^N \hat{c}_{i,\sigma}^\dagger T_{\sigma,\sigma'}^a \hat{c}_{i,\sigma'}, \quad \hat{T}_i^{a,f} = \sum_{\sigma,\sigma'=1}^N \hat{f}_{i,\sigma}^\dagger T_{\sigma,\sigma'}^a \hat{f}_{i,\sigma'}, \quad \text{and} \quad \hat{n}_i^c = \sum_{\sigma=1}^N \hat{c}_{i,\sigma}^\dagger \hat{c}_{i,\sigma}. \quad (231)$$

Finally, the constraint

$$\sum_{\sigma=1}^N \hat{f}_{i,\sigma}^\dagger \hat{f}_{i,\sigma} \equiv \hat{n}_i^f = \frac{N}{2} \quad (232)$$

holds. Some rewriting has to be carried out so as to implement the model. First, we use the relation:

$$\sum_a T_{\alpha,\beta}^a T_{\alpha',\beta'}^a = \frac{1}{2} \left(\delta_{\alpha,\beta'} \delta_{\alpha',\beta} - \frac{1}{N} \delta_{\alpha,\beta} \delta_{\alpha',\beta'} \right),$$

to show that in the unconstrained Hilbert space,

$$\frac{2J}{N} \sum_{a=1}^{N^2-1} \hat{T}_i^{a,c} \hat{T}_i^{a,f} = -\frac{J}{2N} \sum_i (\hat{D}_i^\dagger \hat{D}_i + \hat{D}_i \hat{D}_i^\dagger) + \frac{J}{N} \left(\frac{\hat{n}_i^c}{2} + \frac{\hat{n}_i^f}{2} - \frac{\hat{n}_i^c \hat{n}_i^f}{N} \right)$$

with

$$\hat{D}_i^\dagger = \sum_{\sigma=1}^N \hat{c}_{i,\sigma}^\dagger \hat{f}_{i,\sigma}.$$

In the constrained Hilbert space, $\hat{n}_i^f = N/2$, the above gives:

$$\frac{2J}{N} \sum_{a=1}^{N^2-1} \hat{T}_i^{a,c} \hat{T}_i^{a,f} = -\frac{J}{4N} \left[(\hat{D}_i^\dagger + \hat{D}_i)^2 + (i\hat{D}_i^\dagger - i\hat{D}_i)^2 \right] + \frac{J}{4}. \quad (233)$$

The perfect square form complies with the requirements of ALF. We still have to impose the constraint. To do so, we work in the unconstrained Hilbert space and add a Hubbard U -term on the f-orbitals. With this addition, the Hamiltonian we simulate reads:

$$\begin{aligned} \hat{H}_{\text{QMC}} = & -t \sum_{\langle i,j \rangle} \sum_{\sigma=1}^N \left(\hat{c}_{i,\sigma}^\dagger e^{\frac{2\pi i}{\Phi_0} \int_i^j \mathbf{A} \cdot d\mathbf{l}} \hat{c}_{j,\sigma} + \text{H.c.} \right) - \mu \sum_{i,\sigma} \hat{c}_{i,\sigma}^\dagger \hat{c}_{i,\sigma} + \frac{U_c}{N} \sum_i \left(\hat{n}_i^c - \frac{N}{2} \right)^2 \\ & - \frac{J}{4N} \left[(\hat{D}_i^\dagger + \hat{D}_i)^2 + (i\hat{D}_i^\dagger - i\hat{D}_i)^2 \right] + \frac{U_f}{N} \sum_i \left(\hat{n}_i^f - \frac{N}{2} \right)^2. \end{aligned} \quad (234)$$

The key point for the efficiency of the code, is to see that

$$\left[\hat{H}_{\text{QMC}}, \left(\hat{n}_i^f - \frac{N}{2} \right)^2 \right] = 0 \quad (235)$$

such that the constraint is implemented efficiently. In fact, for the finite temperature code at inverse temperature β , the unphysical Hilbert space is suppressed by a factor $e^{-\beta U_f / N}$.

The SU(2) case

The SU(2) case is special and allows for a more efficient implementation than the one described above. The key point is that for the SU(2) case, the Hubbard term is related to the fermion parity,

$$\left(\hat{n}_i^f - 1 \right)^2 = \frac{(-1)^{\hat{n}_i^f} + 1}{2} \quad (236)$$

such that we can omit the *current-term* $(i\hat{D}_i^\dagger - i\hat{D}_i)^2$ without violating Eq. (235). As in Refs. [31, 32, 154], the Hamiltonian that one will simulate reads:

$$\hat{\mathcal{H}} = \underbrace{-t \sum_{\langle i,j \rangle, \sigma} \left(\hat{c}_{i,\sigma}^\dagger e^{\frac{2\pi i}{\Phi_0} \int_i^j A \cdot dl} \hat{c}_{j,\sigma} + \text{H.c.} \right)}_{\equiv \hat{\mathcal{H}}_{tU_c}} + \frac{U_c}{2} \sum_i (\hat{n}_i^c - 1)^2 - \frac{J}{4} \sum_i \left(\sum_\sigma \hat{c}_{i,\sigma}^\dagger \hat{f}_{i,\sigma} + \hat{f}_{i,\sigma}^\dagger \hat{c}_{i,\sigma} \right)^2 + \underbrace{\frac{U_f}{2} \sum_i (\hat{n}_i^f - 1)^2}_{\equiv \hat{\mathcal{H}}_{U_f}}. \quad (237)$$

The relation to the Kondo lattice model follows from expanding the square of the hybridization to obtain:

$$\hat{\mathcal{H}} = \hat{\mathcal{H}}_{tU_c} + J \sum_i \left(\hat{S}_i^c \cdot \hat{S}_i^f + \hat{\eta}_i^{z,c} \cdot \hat{\eta}_i^{z,f} - \hat{\eta}_i^{x,c} \cdot \hat{\eta}_i^{x,f} - \hat{\eta}_i^{y,c} \cdot \hat{\eta}_i^{y,f} \right) + \hat{\mathcal{H}}_{U_f}, \quad (238)$$

where the η -operators relate to the spin-operators via a particle-hole transformation in one spin sector:

$$\hat{\eta}_i^\alpha = \hat{P}^{-1} \hat{S}_i^\alpha \hat{P} \quad \text{with} \quad \hat{P}^{-1} \hat{c}_{i,\uparrow} \hat{P} = (-1)^{i_x+i_y} \hat{c}_{i,\uparrow}^\dagger \quad \text{and} \quad \hat{P}^{-1} \hat{c}_{i,\downarrow} \hat{P} = \hat{c}_{i,\downarrow}. \quad (239)$$

Since the $\hat{\eta}^f$ and $\hat{S}^f$ operators do not alter the parity $[(-1)^{\hat{n}_i^f}]$ of the f -sites,

$$[\hat{\mathcal{H}}, \hat{\mathcal{H}}_{U_f}] = 0. \quad (240)$$

Thereby, and for positive values of U , doubly occupied or empty f -sites – corresponding to even parity sites – are suppressed by a Boltzmann factor $e^{-\beta U_f/2}$ in comparison to odd parity sites. Thus, essentially, choosing βU_f adequately allows one to restrict the Hilbert space to odd parity f -sites. In this Hilbert space, $\hat{\eta}^{x,f} = \hat{\eta}^{y,f} = \hat{\eta}^{z,f} = 0$ such that the Hamiltonian (237) reduces to the Kondo lattice model.

QMC implementation

The name space for this model class reads:

```
&VAR_Kondo          !! Variables for the Kondo class
ham_T                = 1.d0          ! Hopping parameter
ham_chem              = 0.d0          ! Chemical potential
ham_Uc                = 0.d0          ! Hubbard interaction on c-orbitals Uc
ham_Uf                = 2.d0          ! Hubbard interaction on f-orbitals Uf
ham_JK                = 2.d0          ! Kondo Coupling J
/
```

Aside from the usual observables we have included the scalar observable `Constraint_scal` that measures

$$\left\langle \sum_i \left(\hat{n}_i^f - \frac{N}{2} \right)^2 \right\rangle. \quad (241)$$

U_f has to be chosen large enough such that the above quantity vanishes within statistical uncertainty. For the square lattice, Fig. 12 plots the aforementioned quantity as a function of U_f for the SU(2) model. As apparent $\left\langle \sum_i \left(\hat{n}_i^f - N/2 \right)^2 \right\rangle \propto e^{-\beta U_f/2}$.

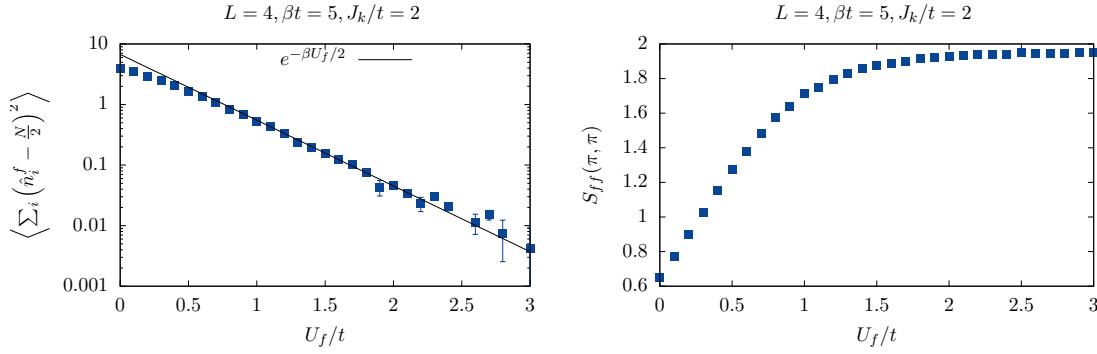


Figure 12: Left: Suppression of charge fluctuations of the f-orbitals as a function of U_f . Right: When charge fluctuations on the f-orbitals vanish, quantities such as the Fourier transform of the f spin-spin correlations at $\mathbf{q} = (\pi, \pi)$ converge to their KLM value. Typically, for the SU(2) case, $\beta U_f > 10$ suffices to reach convergent results. The pyALF script used to produce the data of the plot can be found in [Kondo.py](#)

9.4 Models with long range Coulomb interactions `Hamiltonian_LRC_smod.F90`

The model we consider here is defined for $N_{\text{FL}}=1$, arbitrary values of N_{SUN} and all the pre-defined lattices. It reads:

$$\hat{H} = \sum_{i,j} \sum_{\sigma=1}^N T_{i,j} \hat{c}_{i,\sigma}^\dagger e^{\frac{2\pi i}{\Phi_0} \int_i^j A(l) dl} \hat{c}_{j,\sigma} + \frac{1}{N} \sum_{i,j} \left(\hat{n}_i - \frac{N}{2} \right) V_{i,j} \left(\hat{n}_j - \frac{N}{2} \right) - \mu \sum_i \hat{n}_i. \quad (242)$$

In the above, $i = (\mathbf{i}, \delta_i)$ and $j = (\mathbf{j}, \delta_j)$ are super-indices encoding the unit-cell and orbital and $\hat{n}_i = \sum_{\sigma=1}^N \hat{c}_{i,\sigma}^\dagger \hat{c}_{i,\sigma}$. For simplicity, the interaction is specified by two parameters, U and α that monitor the strength of the onsite interaction and the magnitude of the Coulomb tail respectively:

$$V_{i,j} \equiv V(\mathbf{i} + \delta_i, \mathbf{j} + \delta_j) = U \begin{cases} 1 & \text{if } i = j \\ \frac{\alpha d_{\min}}{\|\mathbf{i} - \mathbf{j} + \delta_i - \delta_j\|} & \text{otherwise} \end{cases}. \quad (243)$$

Here $d_{\min}$ is the minimal distance between two orbitals. On a torus, some care has to be taken in defining the distance. Namely, with the lattice size given by the vectors $\mathbf{L}_1$ and $\mathbf{L}_2$ (see Sec. 8.1),

$$\|\mathbf{i}\| = \min_{n_1, n_2 \in \mathbb{Z}} |\mathbf{i} - n_1 \mathbf{L}_1 - n_2 \mathbf{L}_2|. \quad (244)$$

The implementation of the model follows Ref. [51], but supports various lattice geometries. We use the following HS decomposition:

$$e^{-\Delta\tau \hat{H}_V} \propto \int \prod_i d\phi_i e^{-\frac{N\Delta\tau}{4} \sum_{i,j} \phi_i V_{i,j}^{-1} \phi_j - \sum_i i\Delta\tau \phi_i (\hat{n}_i - \frac{N}{2})}, \quad (245)$$

where ϕ_i is a real variable, V is symmetric and, importantly, has to be positive definite for the Gaussian integration to be defined. The partition function reads:

$$Z \propto \int \prod_i d\phi_{i,\tau} e^{-\frac{N\Delta\tau}{4} \sum_{i,j} \phi_{i,\tau} V_{i,j}^{-1} \phi_{j,\tau}} \underbrace{\text{Tr} \left[\prod_{\tau} e^{-\Delta\tau \hat{H}_T} e^{-\sum_i i\Delta\tau \phi_{i,\tau} (\hat{n}_i - \frac{N}{2})} \right]}_{W_F(\phi)}, \quad (246)$$

such that the weight splits into bosonic and fermionic parts.

For the update, it is convenient to work in a basis where V is diagonal:

$$\text{Diag}(\lambda_1, \dots, \lambda_{\text{Ndim}}) = O^T V O \quad (247)$$

with $O^T O = 1$ and define:

$$\eta_{i,\tau} = \sum_j O_{i,j}^T \phi_{j,\tau}. \quad (248)$$

On a given time slice τ_u we propose a new field configuration with the probability:

$$T^0(\eta \rightarrow \eta') = \begin{cases} \prod_i [P P_B(\eta'_{i,\tau_u}) + (1-P)\delta(\eta_{i,\tau_u} - \eta'_{i,\tau_u})] & \text{for } \tau = \tau_u \\ \delta(\eta_{i,\tau} - \eta'_{i,\tau}) & \text{for } \tau \neq \tau_u \end{cases} \quad (249)$$

where

$$P_B(\eta_{i,\tau}) \propto e^{-\frac{N\Delta\tau}{4\lambda_i} \eta_{i,\tau}^2}, \quad (250)$$

$P \in [0, 1]$ and δ denotes the Dirac δ -function. That is, we carry out simple sampling of the field with probability P and leave the field unchanged with probability $(1 - P)$. P is a free parameter that does not change the final result but that allows one to adjust the acceptance. We then use the Metropolis-Hasting acceptance-rejection scheme and accept the move with probability

$$\min\left(\frac{T^0(\eta' \rightarrow \eta)W_B(\eta')W_F(\eta')}{T^0(\eta \rightarrow \eta')W_B(\eta)W_F(\eta)}, 1\right) = \min\left(\frac{W_F(\eta')}{W_F(\eta)}, 1\right), \quad (251)$$

where

$$W_B(\eta) = e^{-\frac{N\Delta\tau}{4} \sum_{i,\tau} \eta_{i,\tau}^2 / \lambda_i} \quad \text{and} \quad W_F(\eta) = \text{Tr} \left[\prod_{\tau} e^{-\Delta\tau \hat{H}_T} e^{-\sum_{i,j} i\Delta\tau O_{i,j} \eta_{j,\tau} (\hat{n}_i - \frac{N}{2})} \right]. \quad (252)$$

Since a local change on a single time slice in the η basis corresponds to a non-local space update in the ϕ basis, we use the routine for global updates in space to carry out the update (see Sec. 2.2.3.1).

QMC implementation

The name space for this model class reads:

```
&VAR_LRC                                !! Variables for the Long Range Coulomb class
ham_T      = 1.0                        ! Specifies the hopping and chemical potential
ham_T2     = 1.0                        ! For bilayer systems
ham_Tperp  = 1.0                        ! For bilayer systems
ham_chem   = 1.0                        ! Chemical potential
ham_U      = 4.0                        ! On-site interaction
ham_alpha  = 0.1                        ! Coulomb tail magnitude
Percent_change = 0.1                    ! Parameter P
/
```

By setting α to zero we can test this code against the Hubbard code. For a 4×4 square lattice at $\beta t = 5$, $U/t = 4$, and half-band filling, `Hamiltonian_Hubbard_smod.F90` gives $E = -13.1889 \pm 0.0017$ and `Hamiltonian_LRC_smod.F90`, $E = -13.199 \pm 0.040$. Note that for the Hubbard code we have used the default `Mz = .True..` This option breaks $SU(2)$ spin symmetry for a given HS configuration, but produces very precise values of the energy. On the other hand, the LRC code is an $SU(2)$ invariant code (as would be choosing `Mz = .False..`) and produces more fluctuations in the double occupancy. This partly explains the difference in error bars between the two codes. To produce this data, one can run the `pyALF` python script [LRC.py](#).

9.5 $\mathbf{Z}_2$ lattice gauge theories coupled to fermion and $\mathbf{Z}_2$ matter Hamiltonian_ $\mathbf{Z}_2_smod.F90$

The Hamiltonian we will consider here reads

$$\begin{aligned} \hat{H} = & -t_{\mathbf{Z}_2} \sum_{\langle i,j \rangle, \sigma} \hat{\sigma}_{\langle i,j \rangle}^z \left(\hat{\Psi}_{i,\sigma}^\dagger \hat{\Psi}_{j,\sigma} + \text{H.c.} \right) - \mu \sum_{i,\sigma} \hat{\Psi}_{i,\sigma}^\dagger \hat{\Psi}_{i,\sigma} - g \sum_{\langle i,j \rangle} \hat{\sigma}_{\langle i,j \rangle}^x \\ & + K \sum_{\square} \prod_{\langle i,j \rangle \in \partial \square} \hat{\sigma}_{\langle i,j \rangle}^z + J \sum_{\langle i,j \rangle} \hat{\tau}_i^z \hat{\sigma}_{\langle i,j \rangle}^z \hat{\tau}_j^z - h \sum_i \hat{\tau}_i^x \\ & - t \sum_{\langle i,j \rangle, \sigma} \hat{\tau}_i^z \hat{\tau}_j^z \left(\hat{\Psi}_{i,\sigma}^\dagger \hat{\Psi}_{j,\sigma} + \text{H.c.} \right) + \frac{U}{N} \sum_i \left[\sum_{\sigma} \left(\hat{\Psi}_{i,\sigma}^\dagger \hat{\Psi}_{i,\sigma} - 1/2 \right) \right]^2. \end{aligned} \quad (253)$$

The model is defined on a square lattice, and describes fermions,

$$\left\{ \hat{\Psi}_{i,\sigma}^\dagger, \hat{\Psi}_{j,\sigma'} \right\} = \delta_{i,j} \delta_{\sigma,\sigma'}, \quad \left\{ \hat{\Psi}_{i,\sigma}, \hat{\Psi}_{j,\sigma'} \right\} = 0, \quad (254)$$

coupled to bond gauge fields,

$$\hat{\sigma}_{\langle i,j \rangle}^z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad \hat{\sigma}_{\langle i,j \rangle}^x = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad \left\{ \hat{\sigma}_{\langle i,j \rangle}^z, \hat{\sigma}_{\langle i',j' \rangle}^x \right\} = 2(1 - \delta_{\langle i,j \rangle, \langle i',j' \rangle}) \hat{\sigma}_{\langle i,j \rangle}^z \hat{\sigma}_{\langle i',j' \rangle}^x \quad (255)$$

and $\mathbf{Z}_2$ matter fields:

$$\hat{\tau}_i^z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \quad \hat{\tau}_i^x = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \quad \left\{ \hat{\tau}_i^z, \hat{\tau}_{i'}^x \right\} = 2(1 - \delta_{i,i'}) \hat{\tau}_i^z \hat{\tau}_{i'}^x. \quad (256)$$

Fermions, gauge fields and $\mathbf{Z}_2$ matter fields commute with each other.

Importantly, the model has a local $\mathbf{Z}_2$ symmetry. Consider:

$$\hat{Q}_i = (-1)^{\sum_{\sigma} \hat{\Psi}_{i,\sigma}^\dagger \hat{\Psi}_{i,\sigma}} \hat{\tau}_i^x \hat{\sigma}_{i,i+a_x}^x \hat{\sigma}_{i,i-a_x}^x \hat{\sigma}_{i,i+a_y}^x \hat{\sigma}_i^x. \quad (257)$$

One can then show that $\hat{Q}_i^2 = 1$ and that

$$[\hat{Q}_i, \hat{H}] = 0. \quad (258)$$

The above allows us to assign $\mathbf{Z}_2$ charges to the operators. Since $\{\hat{Q}_i, \hat{\Psi}_{i,\sigma}^\dagger\} = 0$ we can assign a $\mathbf{Z}_2$ charge to the fermions. Equivalently $\hat{\tau}_i^z$ has a $\mathbf{Z}_2$ charge and $\hat{\sigma}_{i,j}^z$ carries $\mathbf{Z}_2$ charges at its ends. Since the total fermion number is conserved, we can assign an electric charge to the fermions. Finally, the model has an $\text{SU}(N)$ color symmetry. In fact, at zero chemical potential and $U = 0$, the symmetry is enhanced to $O(2N)$ [21]. Aspects of this Hamiltonian were investigated in Refs. [21, 25, 26, 28–30] and we refer the interested user to these papers for a discussion of the phases and phase transitions supported by the model.

QMC implementation

The name space for this model class reads:

```
&VAR_Z2_Matter      !! Variables for the Z_2 class
ham_T                = 1.0      ! Hopping for fermions
ham_TZ2              = 1.0      ! Hopping for orthogonal fermions
ham_chem              = 0.0      ! Chemical potential for fermions
ham_U                = 0.0      ! Hubbard for fermions
Ham_J                = 1.0      ! Hopping Z2 matter fields
Ham_K                = 1.0      ! Plaquette term for gauge fields
```

```

Ham_h      = 1.0      ! sigma^x-term for matter
Ham_g      = 1.0      ! tau^x-term for gauge
Dtau       = 0.1d0    ! Thereby Ltrot=Beta/dtau
Beta       = 10.d0    ! Inverse temperature
Projector  = .False.  ! To enable projective code
Theta      = 10.0     ! Projection parameter
/

```

We note that the implementation is such that if $\text{Ham_T}=0$ ($\text{Ham_TZ2}=0$) then all the terms involving the matter field ($\mathbf{Z}_2$ gauge field) are automatically set to zero. We warn the user that autocorrelation and warmup times can be large for this model class. At this point, the model is only implemented for the square lattice and does not support a symmetric Trotter decomposition.

The key point to implement the model is to define a new bond variable:

$$\hat{\mu}_{\langle i,j \rangle}^z = \hat{\tau}_i^z \hat{\tau}_j^z. \quad (259)$$

By construction, the $\hat{\mu}_{\langle i,j \rangle}^z$ bond variables have a zero flux constraint:

$$\hat{\mu}_{\langle i,i+a_x \rangle}^z \hat{\mu}_{\langle i+a_x,i+a_x+a_y \rangle}^z \hat{\mu}_{\langle i+a_x+a_y,i+a_y \rangle}^z \hat{\mu}_{\langle i+a_y,i \rangle}^z = 1. \quad (260)$$

Consider a basis where $\hat{\mu}_{\langle i,j \rangle}^z$ and $\hat{\tau}_i^z$ are diagonal with eigenvalues $\mu_{\langle i,j \rangle}$ and τ_i respectively. The map from $\{\tau_i\}$ to $\{\mu_{\langle i,j \rangle}\}$ is unique. The reverse however is valid only up to a global sign. To pin down this sign (and thereby the relative signs between different time slices) we store the fields $\mu_{\langle i,j \rangle}$ at every time slice as well as the value of the Ising field at a reference site $\tau_{i=0}$. Within the ALF, this can be done by adding a dummy operator in the Op_V list to carry this degree of freedom. With this extra degree of freedom we can switch between the two representations without loosing any information. To compute the Ising part of the action it is certainly more transparent to work with the $\{\tau_i\}$ variables. For the fermion determinant, the $\{\mu_{\langle i,j \rangle}\}$ are more convenient.

Since flipping $\hat{\tau}_i^z$ amounts to changing the sign of the four bond variables emanating from site i , the identity:

$$\hat{\tau}_i^x = \hat{\mu}_{i,i+a_x}^x \hat{\mu}_{i+a_x,i+a_x+a_y}^x \hat{\mu}_{i+a_x+a_y,i+a_y}^x \quad (261)$$

holds. Note that $\{\hat{\mu}_{\langle i,j \rangle}^z, \hat{\mu}_{\langle i',j' \rangle}^x\} = 2(1 - \delta_{\langle i,j \rangle, \langle i',j' \rangle}) \hat{\mu}_{\langle i,j \rangle}^z \hat{\mu}_{\langle i',j' \rangle}^x$, such that applying $\hat{\mu}_{\langle i,j \rangle}^x$ on an eigenstate of $\hat{\mu}_{\langle i,j \rangle}^z$ flips the field.

The model can then be written as:

$$\begin{aligned}
\hat{H} = & -t_{z_2} \sum_{\langle i,j \rangle, \sigma} \hat{\sigma}_{\langle i,j \rangle}^z (\hat{\Psi}_{i,\sigma}^\dagger \hat{\Psi}_{j,\sigma} + \text{H.c.}) - \mu \sum_{i,\sigma} \hat{\Psi}_{i,\sigma}^\dagger \hat{\Psi}_{i,\sigma} - g \sum_{\langle i,j \rangle} \hat{\sigma}_{\langle i,j \rangle}^x + K \sum_{\square} \prod_{\langle i,j \rangle \in \partial \square} \hat{\sigma}_{\langle i,j \rangle}^z \\
& + J \sum_{\langle i,j \rangle} \hat{\mu}_{\langle i,j \rangle}^z \hat{\sigma}_{\langle i,j \rangle}^z - h \sum_i \hat{\mu}_{i,i+a_x}^x \hat{\mu}_{i+a_x,i+a_x+a_y}^x \hat{\mu}_{i+a_x+a_y,i+a_y}^x \hat{\mu}_{i+a_y,i}^x \\
& - t \sum_{\langle i,j \rangle, \sigma} \hat{\mu}_{i,j}^z (\hat{\Psi}_{i,\sigma}^\dagger \hat{\Psi}_{j,\sigma} + \text{H.c.}) + \frac{U}{N} \sum_i \left[\sum_{\sigma} (\hat{\Psi}_{i,\sigma}^\dagger \hat{\Psi}_{i,\sigma} - 1/2) \right]^2
\end{aligned} \quad (262)$$

subject to the constraint of Eq. (260).

To formulate the Monte Carlo, we work in a basis in which $\hat{\mu}_{\langle i,j \rangle}^z$, $\hat{\tau}_0^z$ and $\hat{\sigma}_{\langle i,j \rangle}^z$ are diagonal:

$$\hat{\mu}_{\langle i,j \rangle}^z |\underline{s}\rangle = \mu_{\langle i,j \rangle} |\underline{s}\rangle, \quad \hat{\sigma}_{\langle i,j \rangle}^z |\underline{s}\rangle = \sigma_{\langle i,j \rangle} |\underline{s}\rangle, \quad \hat{\tau}_0^z |\underline{s}\rangle = \tau_0 |\underline{s}\rangle \quad (263)$$

with $\underline{s} = (\{\mu_{\langle i,j \rangle}\}, \{\sigma_{\langle i,j \rangle}\}, \tau_0)$. In this basis,

$$Z = \sum_{\underline{s}_1, \dots, \underline{s}_{L_\tau}} e^{-S_0(\{\underline{s}_\tau\})} \text{Tr}_F \left[\prod_{\tau=1}^{L_\tau} e^{-\Delta \tau \hat{H}_F(\underline{s}_\tau)} \right], \quad (264)$$

where

$$S_0(\{\underline{s}_\tau\}) = -\ln \left[\prod_{\tau=1}^{L_\tau} \langle \underline{s}_{\tau+1} | e^{-\Delta\tau \hat{H}_I} | \underline{s}_\tau \rangle \right],$$

$$\begin{aligned} \hat{H}_I = & -g \sum_{\langle i,j \rangle} \hat{\sigma}_{\langle i,j \rangle}^x + K \sum_{\square} \prod_{\langle i,j \rangle \in \partial \square} \hat{\sigma}_{\langle i,j \rangle}^z + J \sum_{\langle i,j \rangle} \hat{\mu}_{\langle i,j \rangle}^z \hat{\sigma}_{\langle i,j \rangle}^z \\ & - h \sum_i \hat{\mu}_{i,i+a_x}^x \hat{\mu}_{i+a_x,i+a_x+a_y}^x \hat{\mu}_{i+a_x+a_y,i+a_y}^x \end{aligned}$$

and

$$\begin{aligned} \hat{H}_F(\underline{s}) = & -t_{Z_2} \sum_{\langle i,j \rangle, \sigma} \sigma_{\langle i,j \rangle} \left(\hat{\Psi}_{i,\sigma}^\dagger \hat{\Psi}_{j,\sigma} + \text{H.c.} \right) - \mu \sum_{i,\sigma} \hat{\Psi}_{i,\sigma}^\dagger \hat{\Psi}_{i,\sigma} \\ & - t \sum_{\langle i,j \rangle, \sigma} \mu_{i,j} \left(\hat{\Psi}_{i,\sigma}^\dagger \hat{\Psi}_{j,\sigma} + \text{H.c.} \right) + \frac{U}{N} \sum_i \left[\sum_{\sigma} (\hat{\Psi}_{i,\sigma}^\dagger \hat{\Psi}_{i,\sigma} - 1/2) \right]^2. \end{aligned}$$

In the above, $|\underline{s}_{L_\tau+1}\rangle = |\underline{s}_1\rangle$. With a further HS transformation of the Hubbard term (see Sec. 8.3.1) the model is readily implemented in the ALF. Including this HS field, l , [see Eq. (12)] yields the configuration space:

$$C = (\{\mu_{\langle i,j \rangle, \tau}\}, \{\sigma_{\langle i,j \rangle, \tau}\}, \{\tau_{0,\tau}\}, \{l_{i,\tau}\}) \quad (265)$$

where the variables μ , τ and σ take the values ± 1 and l the values $\pm 1, \pm 2$.

The initial configuration as well as the moves have to respect the zero flux constraint of Eq. (260). Therefore, single spin flips of the μ fields are prohibited and the minimal move one can carry out on a given time slice is the following. We randomly choose a site i and propose a move where: $\mu_{i,i+a_x} \rightarrow -\mu_{i,i+a_x}$, $\mu_{i,i-a_x} \rightarrow -\mu_{i,i-a_x}$, $\mu_{i,i+a_y} \rightarrow -\mu_{i,i+a_y}$ and $\mu_{i,i-a_y} \rightarrow -\mu_{i,i-a_y}$. One can carry out such moves by using the global move in real space option presented in Sec. 2.2.3.1 and 5.7.1.

9.5.1 Projective approach

The program also supports a zero temperature implementation. Our choice of the trial wave function does not break any symmetries of the model and reads:

$$|\Psi_T\rangle = |\Psi_T^F\rangle \otimes_{\langle i,j \rangle} |+\rangle_{\langle i,j \rangle} \otimes_i |+\rangle_i. \quad (266)$$

For the fermion part we use a Fermi sea with small dimerization to avoid the negative sign problem at half-filling (see Sec. 7.6). For the Ising part the trial wave function is diagonal in the $\hat{\sigma}_{\langle i,j \rangle}^x$ and $\hat{\tau}_i^x$ operators:

$$\hat{\sigma}_{\langle i,j \rangle}^x |+\rangle_{\langle i,j \rangle} = |+\rangle_{\langle i,j \rangle} \quad \text{and} \quad \hat{\tau}_i^x |+\rangle_i = |+\rangle_i. \quad (267)$$

An alternative choice would be to choose a charge density wave fermionic trial wave function. This violates the partial particle-hole symmetry of the model at $U = \mu = 0$ and effectively imposes the constraint $\hat{Q}_i = 1$.

9.5.2 Observables

Apart from the standard observables discussed in Sec. 8.4 the code computes additionally

$$\langle \hat{\sigma}_{\langle i,j \rangle}^x \rangle \quad \text{and} \quad \langle \hat{\tau}_j^x \rangle,$$

which are written to file X_scal;

$$\langle \hat{\sigma}_{\langle i,i+a_x \rangle}^z \hat{\sigma}_{\langle i+a_x,i+a_x+a_y \rangle}^z \hat{\sigma}_{\langle i+a_x+a_y,i+a_y \rangle}^z \hat{\sigma}_{\langle i+a_y,i \rangle}^z \rangle$$

and

$$\langle \hat{\mu}_{\langle i,i+a_x \rangle}^z \hat{\mu}_{\langle i+a_x,i+a_x+a_y \rangle}^z \hat{\mu}_{\langle i+a_x+a_y,i+a_y \rangle}^z \hat{\mu}_{\langle i+a_y,i \rangle}^z \rangle,$$

written to file Flux_scal; and also $\langle \hat{Q}_i \rangle$ (file Q_scal). Note that the flux over a plaquette of the $\hat{\mu}_{\langle i,j \rangle}^z$ is equal to unity by construction so that this observable provides a sanity check. The file Q_eq contains the two-point correlation $\langle \hat{Q}_i \hat{Q}_j \rangle - \langle \hat{Q}_i \rangle \langle \hat{Q}_j \rangle$ and Greenf_eq the equal-time fermion Green function $\langle \hat{\tau}_i^z \hat{\Psi}_{i,\sigma}^\dagger \hat{\tau}_j^z \hat{\Psi}_{j,\sigma} \rangle$.

9.5.3 A test case: Z_2 slave spin formulation of the SU(2) Hubbard model

In this subsection, we demonstrate that the code can be used to simulate the attractive Hubbard model in the Z_2 -slave spin formulation [155]:

$$\hat{H} = -t \sum_{\langle i,j \rangle, \sigma} \hat{c}_{i,\sigma}^\dagger \hat{c}_{j,\sigma} - U \sum_i (\hat{n}_{i,\uparrow} - 1/2)(\hat{n}_{i,\downarrow} - 1/2). \quad (268)$$

In the Z_2 slave spin representation, the physical fermion, $\hat{c}_{i,\sigma}$, is fractionalized into an Ising spin carrying Z_2 charge and a fermion, $\hat{\Psi}_{i,\sigma}$, carrying Z_2 and global $U(1)$ charge:

$$\hat{c}_{i,\sigma}^\dagger = \hat{\tau}_i^z \hat{\Psi}_{i,\sigma}^\dagger. \quad (269)$$

To ensure that we remain in the correct Hilbert space, the constraint:

$$\hat{\tau}_i^x - (-1)^{\sum_\sigma \hat{\Psi}_{i,\sigma}^\dagger \hat{\Psi}_{i,\sigma}} = 0 \quad (270)$$

has to be imposed locally. Since $(\tau_i^x)^2 = 1$, the latter is equivalent to

$$\hat{Q}_i = \tau_i^x (-1)^{\sum_\sigma \hat{\Psi}_{i,\sigma}^\dagger \hat{\Psi}_{i,\sigma}} = 1. \quad (271)$$

Using

$$(-1)^{\sum_\sigma \hat{\Psi}_{i,\sigma}^\dagger \hat{\Psi}_{i,\sigma}} = \prod_\sigma (1 - 2\hat{\Psi}_{i,\sigma}^\dagger \hat{\Psi}_{i,\sigma}) = 4 \prod_\sigma (\hat{c}_{i,\sigma}^\dagger \hat{c}_{i,\sigma} - 1/2), \quad (272)$$

the Z_2 slave spin representation of the Hubbard model now reads:

$$\hat{H}_{Z_2} = -t \sum_{\langle i,j \rangle, \sigma} \hat{\tau}_i^z \hat{\tau}_j^z \hat{\Psi}_{i,\sigma}^\dagger \hat{\Psi}_{j,\sigma} - \frac{U}{4} \sum_i \hat{\tau}_i^x. \quad (273)$$

Importantly, the constraint commutes with Hamiltonian:

$$[\hat{H}_{Z_2}, \hat{Q}_i] = 0. \quad (274)$$

Hence one can foresee that the constraint will be dynamically imposed (we expect a finite-temperature Ising phase transition below which $\hat{Q}_i$ orders) and that at $T = 0$ on a finite lattice both models should give the same results.

A test run for the 8×8 lattice at $U/t = 4$ and $\beta t = 40$ gives:

$\mathbf{k}$	$\langle n_k \rangle_H$	$\langle n_k \rangle_{H_{Z_2}}$
(0,0)	$1.93348548 \pm 0.00011322$	$1.93333895 \pm 0.00010405$
$(\pi/4, \pi/4)$	$1.90120688 \pm 0.00014854$	$1.90203726 \pm 0.00017943$
$(\pi/2, \pi/2)$	$0.99942957 \pm 0.00091377$	$1.00000000 \pm 0.00000000$
$(3\pi/4, 3\pi/4)$	$0.09905425 \pm 0.00015940$	$0.09796274 \pm 0.00017943$
(π, π)	$0.06651452 \pm 0.00011321$	$0.06666105 \pm 0.00010405$

Here a Trotter time step of $\Delta\tau t = 0.05$ was used in order to minimize the systematic error which should be different between the two codes. The Hamiltonian is invariant under a partial particle-hole transformation (see Ref. [21]). Since $\hat{Q}_i$ is odd under this transformation, $\langle \hat{Q}_i \rangle = 0$. To asses whether the constraint is well imposed, the code, for this special case, computes the correlation function:

$$S_Q(\mathbf{q}) = \sum_i \langle \hat{Q}_i \hat{Q}_0 \rangle. \quad (275)$$

For the above run we obtain $S_Q(\mathbf{q} = \mathbf{0}) = 63.4 \pm 1.7$ which, for this 8×8 lattice, complies with a ferromagnetic ordering of the Ising $\hat{Q}_i$ variables. The pyALF python script that produces this data can be found in [Z2_Matter.py](#). This code was used in Refs. [28, 29].

9.6 Spin Peierls Hamiltonian_Spin_Peierls_smod.F90

The Hamiltonian that we will consider here is given by:

$$\hat{H}_{SP} = \sum_{b=\langle i,j \rangle} J_b (1 + \hat{Q}_b) \hat{\mathbf{S}}_i \cdot \hat{\mathbf{S}}_j + \frac{\hat{P}_b^2}{2M} + \frac{k}{2} \hat{Q}_b^2 \quad (276)$$

with

$$[\hat{S}_i^\alpha, \hat{S}_j^\beta] = \delta_{i,j} i \sum_{\gamma=1}^3 \epsilon^{\alpha\beta\gamma} \hat{S}_\gamma, \quad (\hat{\mathbf{S}}_i)^2 = \frac{3}{4}, \quad \text{and} \quad [\hat{P}_b, \hat{Q}_{b'}] = \frac{\delta_{b,b'}}{i}. \quad (277)$$

The model is defined on a square lattice, and the sum runs over nearest neighbor bonds $b = \langle i, j \rangle$. Including a coupling constant g in front the Einstein phonon $\hat{Q}_{\langle i,j \rangle}$ is superfluous since it can be set to unity by carrying out a canonical transformation of $\hat{Q}_{\langle i,j \rangle}$.

To implement the spin-Peierls Hamiltonian in the ALF library, we fermionize the spin degrees of freedom and use the relation

$$-\frac{1}{4} \left(\sum_{\sigma} \hat{f}_{i,\sigma}^\dagger \hat{f}_{j,\sigma} + \hat{f}_{j,\sigma}^\dagger \hat{f}_{i,\sigma} \right)^2 + \frac{1}{4} = \hat{\mathbf{S}}_i \cdot \hat{\mathbf{S}}_j \quad (278)$$

that holds in the odd parity, $(-1)^{\sum_{\sigma} \hat{f}_{i,\sigma}^\dagger \hat{f}_{i,\sigma}} = -1$, sector.

In the Monte Carlo simulation, we enforce the odd parity constraint by adding a Hubbard U term. As a consequence, the fermion Hamiltonian we consider reads:

$$\hat{H}_{QMC} = \sum_{b=\langle i,j \rangle} J_b (1 + \hat{Q}_b) \left(\frac{1}{4} - \frac{1}{4} \left(\sum_{\sigma} \hat{f}_{i,\sigma}^\dagger \hat{f}_{j,\sigma} + \hat{f}_{j,\sigma}^\dagger \hat{f}_{i,\sigma} \right)^2 \right) + \frac{\hat{P}_b^2}{2M} + \frac{k}{2} \hat{Q}_b^2 + \frac{U}{2} \sum_i (\hat{n}_i - 1)^2 \quad (279)$$

where $\hat{n}_i = \sum_{\sigma} \hat{f}_{i,\sigma}^\dagger \hat{f}_{i,\sigma}$. As for the Kondo lattice model, $[(-1)^{\sum_{\sigma} \hat{f}_{i,\sigma}^\dagger \hat{f}_{i,\sigma}}, \hat{H}_{QMC}] = 0$ such that the projection onto the physical odd parity Hilbert space turns out to be very efficient. In this Hilbert space:

$$\hat{H}_{QMC} \big|_{(-1)^{\sum_{\sigma} \hat{f}_{i,\sigma}^\dagger \hat{f}_{i,\sigma}} = -1} = \hat{H}_{SP}. \quad (280)$$

For the Monte-Carlo formulation, we consider a basis where $\hat{Q}_b|\phi\rangle = \phi_b|\phi\rangle$ such that after Trotterization, the partition function reads:

$$Z \propto \sum_{l_{b,\tau} l_{i,\tau}} \int \prod_{b,\tau} d\phi_{b,\tau} \prod_{b,\tau} \gamma(l_{b,\tau}) \prod_{i,\tau} \gamma(l_{i,\tau}) e^{-S_\phi} \quad (281)$$

$$\text{Tr} \left[\prod_{\tau} \prod_b e^{\sqrt{1+\phi_{b,\tau}} \sqrt{\Delta\tau J/4} \eta(l_{b,\tau}) \hat{K}_b} \prod_i e^{\eta(l_{i,\tau}) \sqrt{-U\Delta\tau/2} (\hat{n}_i - 1)} \right]$$

with

$$S_\phi = \sum_{b,\tau} \Delta\tau \left[\frac{M}{2} \frac{(\phi_{b,\tau+1} - \phi_{b,\tau})^2}{\Delta\tau^2} + \frac{k}{2} \left(\phi_b + \frac{J_b}{4k} \right)^2 \right]. \quad (282)$$

In the above, each site hosts a discrete HS field $l_{i,\tau}$ field and each bond discrete bond, $l_{b,\tau}$, and phonon $\phi_{b,\tau}$ fields. The phonon fields satisfy periodic boundary conditions in the temporal direction: $\phi_{b,L_{\text{Trotter}}+1} = \phi_{b,1}$.

The ALF implementation of the above Hamiltonian is straightforward, and provides an example where the type `t=4` operators introduced in 5.2 become very handy. A plain vanilla implementation of the code can be found in `Hamiltonian_Spin_Peierls_smod.F90`. It requires `N_FL=1` and `N_SUN = 2` corresponding to an $SU(2)$ spin symmetric implementation and does not include a symmetric Trotter decomposition. Aside from the standard parameters, the model specific name space reads:

```
&VAR_Spin_Peierls
ham_Jx = 1.0      ! Coupling in x-direction
ham_Jy = 1.0      ! Coupling in y-direction
ham_U = 1.0       ! Hubbard U for constraint. Typically beta U = 10
ham_Lambda = 0.1  ! Electron-phonon interaction
ham_omega0 = 0.25 ! Phonon frequency
/
```

Here, the electron phonon interaction as well as the phonon frequency are defined as

$$\lambda = \frac{1}{2k} \text{ and } \omega_0 = \sqrt{\frac{k}{M}} \quad (283)$$

respectively.

We have tested the code for an $L = 16$ chain at $\beta J = 8$, $\omega_0 = 0.25$ and various values of the electron-phonon interaction. As can be seen in Fig. 13 the results compare remarkably well to independent calculations based on the so called SSE algorithm with retarded interactions [156]. The Hamiltonian `Hamiltonian_Spin_Peierls_smod.F90` includes spin-spin correlations as well as phonon correlations, both at equal and imaginary time displaced. Scalar observables `Phi_sca1` contain the $\langle \hat{Q}_b \rangle$ for bond in the x- and y-directions.

10 Maximum Entropy

If we want to compare the data we obtain from Monte Carlo simulations with experiments, we must extract spectral information from the imaginary-time output. This can be achieved through the maximum entropy method (MaxEnt), which generically computes the image $A(\omega)$ for a given data set $g(\tau)$ and kernel $K(\tau, \omega)$:

$$g(\tau) = \int_{\omega_{\text{start}}}^{\omega_{\text{end}}} d\omega K(\tau, \omega) A(\omega). \quad (284)$$

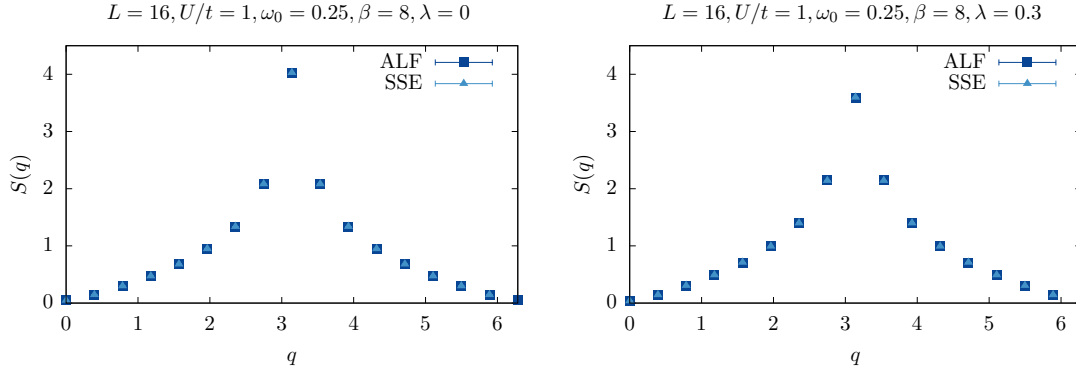


Figure 13: Simulations of the one-dimensional spin-peierls chain. For the ALF simulations, we have used $\Delta\tau J = 0.1$. The reference data was produced with the SSE algorithm with retarded interaction introduced in Ref. [157] and adapted to spin-Peierls problems in [156]. As apparent the agreement is excellent, both for the Heisenberg chain at $\lambda = 0.0$ and at finite electron phonon coupling, $\lambda = 0.3$

The ALF package includes a standard implementation of the stochastic MaxEnt, as formulated in the article of K. Beach [98], in the module `Libraries/Modules/maxent_stoch_mod.F90`. It also supports an implementation of the Bayesian based classic maximum entropy method as formulated in Refs. [99, 100], in the module `Libraries/Modules/maxent_mod.F90`. Its wrapper is found in `Analysis/Max_SAC.F90` and the Green function is read from the file `g_dat`, produced by the analysis program `Analysis/ana.out` analysis program. The logical variable `Stochastic` specified in the parameters file toggles between the two analytical continuation approaches.

In the next section we provide a quick guide on how this facility can be used, followed by sections with more detailed information.

10.1 Quick Start

- Before running the simulation, set in the file `parameters` the variable `Ltau=1`, so that the necessary time-displaced Green functions are calculated; also set a large enough number of bins
- Also in the `parameters` file, set `N_Cov=0` (for shorter runs; `N_Cov=1` might give more reliable error estimates)
- Run the Monte Carlo simulation and the analysis:

```
$ALF_DIR/Prog/ALF.out
$ALF_DIR/Analysis/ana.out *
```
- Then enter the desired results directory, e.g., `Green_0.00_0.00` (they're named in the pattern `Variable_name_kx_ky`) and copy the parameter file to it:

```
cd Green_0.00_0.00/ && cp ../parameters .
```
- Run MaxEnt:

```
$ALF_DIR/Analysis/Max_SAC.out
```

For many purposes it is practical to script some of the steps above, and an example of such a script can be found in `$ALF_DIR/Scripts_and_Parameters_files/Spectral.sh`.

10.2 General setup

The stochastic approach

The stochastic MaxEnt is essentially a parallel-tempering Monte Carlo simulation. For a discrete set of τ_i points, $i \in 1 \cdots n$, the goodness-of-fit functional, which we take as the energy reads

$$\chi^2(A) = \sum_{i,j=1}^n \left[g(\tau_i) - \overline{g(\tau_i)} \right] C^{-1}(\tau_i, \tau_j) \left[g(\tau_j) - \overline{g(\tau_j)} \right], \quad (285)$$

with $\overline{g(\tau_i)} = \int d\omega K(\tau_i, \omega) A(\omega)$ and C the covariance matrix. The set of N_α inverse temperatures considered in the parallel tempering is given by $\alpha_m = \alpha_{st} R^m$, for $m = 1 \cdots N_\alpha$ and a constant R . The default model is, $D(\omega)$, is required to have the same sum-rule as the spectral function,

$$\int d\omega D(\omega) = \int d\omega A(\omega) = M_0. \quad (286)$$

Since the default model is a positive function, the function

$$x(\omega) = \frac{1}{M_0} \int_{-\infty}^{\omega} d\omega' D(\omega') \quad (287)$$

is invertible (provided that the default model does not vanish in the considered frequency range) and takes values $x \in [0, 1]$. Consider

$$n(x) = \frac{A(\omega^{-1}(x))}{D(\omega^{-1}(x))}, \quad (288)$$

then

$$\int_0^1 dx n(x) = 1 \quad \text{and} \quad \overline{g(\tau)} = M_0 \int_0^1 dx K(\tau, \omega^{-1}(x)) n(x). \quad (289)$$

The phase space corresponds to all possible spectral functions $n(x)$ satisfying a given sum rule and the required positivity. Finally, the partition function reads $Z = \int \mathcal{D}n e^{-\alpha \chi^2(n)}$ [98], such that for a given “inverse temperature” α , the image is given by:

$$\langle n(x) \rangle = \frac{\int \mathcal{D}n e^{-\alpha \chi^2(n)} n(x)}{\int \mathcal{D}n e^{-\alpha \chi^2(n)}}. \quad (290)$$

In the code, $n(x)$ is parametrized by a set of N_γ Dirac δ functions:

$$n(x) = \sum_{i=1}^{N_\gamma} a_i \delta(x - x_i). \quad (291)$$

The code samples $n(x)$ and at the end, builds the spectral function:

$$A(\omega) = n(x(\omega)) D(\omega). \quad (292)$$

To produce a histogram of $A(\omega)$ we divide the frequency range in N_{dis} intervals.

Besides the parameters included in the namelist `VAR_Max_Stoch` set in the file `parameters` (see Sec. 5.7), also the variable `N_cov`, from the namelist `VAR_errors`, is required to run the maxent code. Recalling: `N_cov = 1` (`N_cov = 0`) sets that the covariance will (will not) be taken into account.

The Bayesian approach

It is beyond the scope of this documentation to provide a full account of the classic Maximum Entropy method, and the interested reader is referred to Refs. [99, 100] for detailed accounts. Bayes theorem states that the combined probability of events X, Y, Z ($P(X, Y, Z)$) is given by the probability of X given Y, Z ($P(X|Y, Z)$) times the prior probability of Y, Z ($P(Y, Z)$). Let us apply this to investigate the combined probability of A, α given the combined Default D and Monte-Carlo data G . The result reads:

$$P(A, \alpha|G, D) = P(G|A, D, \alpha)P(A|D, \alpha)P(\alpha|D)/P(G|D) \quad (293)$$

We will assume that D and α are not correlated, such that $P(\alpha, D) = P(\alpha)P(D)$. As a consequence, $P(\alpha|D) = P(\alpha)$. Similarly if we assume that G and D are not correlated, then $P(G|D) = P(G)$. In the Bayesian approach, the likelihood function $P(G|A, m, \alpha)$ reads:

$$P(G|A, D, \alpha) = \frac{1}{(2\pi)^{n/2} \sqrt{\det C}} e^{-\chi^2(A)/2}. \quad (294)$$

As apparent it is independent on the default and value of α . The entropic prior reads:

$$P(A|D, \alpha) = e^{\frac{1}{\alpha} S(A, D)} \text{ with } S(A, D) = \int d\omega \left(A(\omega) - D(\omega) - A(\omega) \log \frac{A(\omega)}{D(\omega)} \right) \quad (295)$$

Normalization is insured by the measure $\mathcal{D}A = \prod_i \sqrt{\frac{1}{\alpha 2\pi}} \frac{dA_i}{\sqrt{A_i}}$ with $A_i = A(\omega_i) \Delta\omega$.

Putting everything together, we obtain:

$$P(A, \alpha|G, D) = \frac{1}{Z(\alpha)} e^{Q(A, \alpha)} \text{ with } Q(A, \alpha) = \frac{1}{\alpha} S(A, D) - \chi^2(A)/2 \quad (296)$$

and

$$\frac{1}{Z(\alpha)} = \frac{P(\alpha)}{P(G)} \frac{1}{(2\pi)^{n/2} \sqrt{\det C}}. \quad (297)$$

The last piece of information we need, is the prior probability of α . We could for instance choose a constant, reflecting the fact that we have no prior information. Alternatively we can use Jeffrey's prior $P(\alpha) = \alpha$. This choice does not lead to important differences since the integration measure already includes a high power of α .

For a given value of α , one can maximize $P(A, \alpha|G, D)$ with respect to A , so as to obtain the most probable image, $A^*(\alpha)$. However this leaves us with the free parameter α . The probability of α given G and D reads

$$P(\alpha|G, D) = \int \mathcal{D}A P(A, \alpha|G, D). \quad (298)$$

In the classic Maximum Entropy method, α is chosen so as to maximize $P(\alpha|G, D)$. Denoting this value of α by α^* , the final image produced by the classic Maximum Entropy method is $A^*(\alpha^*)$.

Input files

The parameters for the Maximum entropy are listed in the `parameters` file detailed in Sec. 5.7.1. In the namespace `VAR_errors` the variable `N_Cov` triggers the use of the covariance matrix, and in the namespace `VAR_Max_Stoch` the variable `Stochastic` toggles between the stochastic and classic analytical continuation schemes. The reader is referred to Sec. 5.7.1 for a complete description of the parameters.

In addition to the aforementioned parameter file, the MaxEnt program requires the output of the analysis of the time-displaced functions. The program `Anaylsis/ana.out` (see Sec. 6.3) generates, for each k -point, a directory named `Variable_name_kx_ky`. In this directory the file `g_dat` contains the required information for the MaxEnt code, which is formatted as follows:

```
<# of tau-points> <# of bins > <beta> <Norb> <Channel>
do tau = 1, # of tau-points
  tau,  sum_alpha <S_{alpha,alpha}^{(corr)}(k,tau)>,  error
enddo
do tau1 = 1, # of tau-points
  do tau2 = 1, # of tau-points
    C(tau1,tau2)
  enddo
enddo
```

The specification of the default model is optional. If the file `Default` exists then this default will be used. We note that the default model corresponds to the output of the MaxEnt code. That is $S(q, \omega)$ for particle-hole quantities, $A(k, \omega)$ in the particle channel, etc. The format of the input is identical to that of the output file `Green` containing the spectral function (see below). Hence the result of one run can be used as a default model for another run. This turns out to be very useful when carrying out temperature scans. Alternatively, the default model can be specified as

```
do om = 1, # of frequencies
  read om, D(om)
enddo
```

The code will automatically detect one of the two aforementioned formats. If no default is provided, then a flat default model with correct sum rule is used.

Output files

The code produces the following output files.

The stochastic approach

- The files `Aom_n` contains the average spectral function at inverse temperature α_n . This corresponds to $\langle A_n(\omega) \rangle = \frac{1}{Z} \int \mathcal{D}A(\omega) e^{-\alpha_n \chi^2(A)} A(\omega)$. The file contains three columns: ω , $\langle A_n(\omega) \rangle$, and $\Delta \langle A_n(\omega) \rangle$.
- The files `Aom_ps_n` contain the average image over the inverse temperatures α_n to α_{N_γ} , see Ref. [98] for more details. Its first three columns have the same meaning as for the files `Aom_n`.
- The file `Green` contains the Green function, obtained from the spectral function through

$$G(\omega) = -\frac{1}{\pi} \int_{-\infty}^{\infty} d\Omega \frac{A(\Omega)}{\omega - \Omega + i\delta}, \quad (299)$$

where $\delta = \Delta\omega = (\omega_{\text{end}} - \omega_{\text{start}})/N_{\text{dis}}$ and the image corresponds to that of the file `Aom_ps_n` with $n = N_\alpha - 10$. The first column of the `Green` file is a placeholder for post-processing. The last three columns correspond to ω , $\text{Re } G(\omega)$, $\text{Im } G(\omega)$. The above serves as a smoothing procedure of the spectral function. In particular if $\delta = 0^+$ then $\text{Im } G(\omega) = A(\omega)$.

- One of the most important output files is `energies`, which lists $\alpha_n, \langle \chi^2 \rangle, \Delta \langle \chi^2 \rangle$.
- `best_fit` gives the values of a_i and ω_i (recall that $A(\omega) = \sum_{i=1}^{N_\gamma} a_i \delta(\omega - \omega_i)$) corresponding to the last configuration of the lowest temperature run.
- The file `data_out` facilitates crosschecking. It lists $\tau, g(\tau), \Delta g(\tau)$, and $\int d\omega K(\tau, \omega) A(\omega)$, where the image corresponds to the best fit (i.e. the lowest temperature). This data should give an indication of how good the fit actually is. Note that `data_out` contains only the data points that have passed the tolerance test.
- Two dump files are also generated, `dump_conf` and `dump_Aom`. Since the MaxEnt is a Monte Carlo code, it is possible to improve the data by continuing a previous simulation. The data in the dump files allow you to do so. These files are only generated if the variable `checkpoint` is set to `.true..`
- The file `Info_MaxEnt` lists a summary used parameters and best χ^2 .

The essential question is: Which image should one use? There is no ultimate answer to this question in the context of the stochastic MaxEnt. The only rule of thumb is to consider temperatures for which the χ^2 is comparable to the number of data points.

The Bayesian approach

The output for the classic MaxEnt contains the file `Data_out_c1`, the image `Green_c1` as well as a summary of the parameter used in the file `Info_MaxEnt_c1`. The format of these files does not differ from the stochastic maxent code.

Scripts

In the directory `Scripts_and_Parameters_files` there is a shell script `Spectral.sh` that can help for the running of the code, and production of plots. The included shell script produces the plots of Fig. 14, and will have to be adapted (i.e. set of k-points) for different system sizes.

10.3 Single-particle quantities: Channel=P

For the single-particle Green function,

$$\langle \hat{c}_k(\tau) \hat{c}_k^\dagger(0) \rangle = \int d\omega K_p(\tau, \omega) A_p(k, \omega), \quad (300)$$

with

$$K_p(\tau, \omega) = \frac{1}{\pi} \frac{e^{-\tau\omega}}{1 + e^{-\beta\omega}} \quad (301)$$

and, in the Lehmann representation,

$$A_p(k, \omega) = \frac{\pi}{Z} \sum_{n,m} e^{-\beta E_m} (1 + e^{-\beta\omega}) |\langle n | c_n | m \rangle|^2 \delta(E_m - E_n - \omega). \quad (302)$$

Here $(\hat{H} - \mu \hat{N}) |n\rangle = E_n |n\rangle$. Note that $A_p(k, \omega) = -\text{Im } G^{\text{ret}}(k, \omega)$, with

$$G^{\text{ret}}(k, \omega) = -i \int dt \Theta(t) e^{i\omega t} \langle \{ \hat{c}_k(t), \hat{c}_k^\dagger(0) \} \rangle. \quad (303)$$

The above follows from the identity $\Theta(t) = \frac{i}{2\pi} \int d\omega' \frac{e^{-i\omega't}}{\omega' + i0^+}$. Finally, the sum rule reads

$$\int d\omega A_p(k, \omega) = \pi \langle \{\hat{c}_k, \hat{c}_k^\dagger\} \rangle = \pi (\langle \hat{c}_k(\tau=0) \hat{c}_k^\dagger(0) \rangle + \langle \hat{c}_k(\tau=\beta) \hat{c}_k^\dagger(0) \rangle). \quad (304)$$

Using the `Max_Sac.F90` with `Channel="P"` will load the above kernel in the `MaxEnt` library. In this case the back transformation is set to unity. Note that for each configuration of fields we have $\langle \langle \hat{c}_k(\tau=0) \hat{c}_k^\dagger(0) \rangle \rangle_C + \langle \langle \hat{c}_k(\tau=\beta) \hat{c}_k^\dagger(0) \rangle \rangle_C = \langle \langle \{\hat{c}_k, \hat{c}_k^\dagger\} \rangle \rangle_C = 1$, hence, if both the $\tau=0$ and $\tau=\beta$ data points are included, the covariance matrix will have a zero eigenvalue and the χ^2 measure is not defined. Therefore, for the particle channel the program omits the $\tau=\beta$ data point. There are special particle-hole symmetric cases where the $\tau=0$ data point shows no fluctuations – in such cases the code omits the $\tau=0$ data point as well.

10.4 Single-particle quantities with particle-hole symmetry: `Channel=P_PH`

Assume that the single particle Green function has an additional symmetry, $G(\bullet, \tau) = G(\bullet, \beta - \tau)$ where $\bullet$ is a placeholder for a momentum or real-space index. Then one can use the `Channel=P_PH` tag. In this case the analysis will average over $G(\bullet, \tau)$ and $G(\bullet, \beta - \tau)$ to provide an improved estimator, and the output files of the analysis, `g_dat`, will contain data with τ in the range $\tau \in [0, \beta]$. As a consequence of the particle hole symmetry the spectral function is an even function of the frequency. For this channel we constrain the frequency range to $\omega \in [0, \infty]$ to obtain:

$$\langle \hat{c}_\bullet(\tau) \hat{c}_\bullet^\dagger(0) \rangle = \int_0^\infty d\omega \underbrace{[K_p(\tau, \omega) + K_p(\tau, -\omega)]}_{\equiv K_{p-ph}(\tau, \omega)} A_p(\bullet, \omega). \quad (305)$$

The above defines the kernel for this channel. If this channel is specified in the data file, then the lower bound of the frequency range will be set to zero by the program.

10.5 Particle-hole quantities: `Channel=PH`

Imaginary-time formulation

For particle-hole quantities such as spin-spin or charge-charge correlations, the kernel reads

$$\langle \hat{S}(q, \tau) \hat{S}(-q, 0) \rangle = \frac{1}{\pi} \int d\omega \frac{e^{-\tau\omega}}{1 - e^{-\beta\omega}} \chi''(q, \omega). \quad (306)$$

This follows directly from the Lehmann representation

$$\chi''(q, \omega) = \frac{\pi}{Z} \sum_{n,m} e^{-\beta E_n} |\langle n | \hat{S}(q) | m \rangle|^2 \delta(\omega + E_n - E_m) (1 - e^{-\beta\omega}) \quad (307)$$

with

$$\chi(q, \omega) = i \int dt \Theta(t) e^{i\omega t} \langle [\hat{S}(q, t), \hat{S}(-q, 0)] \rangle.$$

Since the linear response to a hermitian perturbation is real, $\chi''(q, \omega) = -\chi''(-q, -\omega)$ and hence $\langle \hat{S}(q, \tau) \hat{S}(-q, 0) \rangle$ is a symmetric function around $\beta = \tau/2$ for systems with inversion symmetry – the ones we consider here. When `Channel=PH` the analysis program `ana.out` uses this symmetry to provide an improved estimator.

The stochastic MaxEnt requires a sum rule, and hence the kernel and image have to be adequately redefined. Let us consider $\coth(\beta\omega/2) \chi''(q, \omega)$. For this quantity, we have the sum rule, since

$$\int d\omega \coth(\beta\omega/2) \chi''(q, \omega) = 2\pi \langle \hat{S}(q, \tau=0) \hat{S}(-q, 0) \rangle, \quad (308)$$

which is just the first point in the data. Therefore,

$$\langle \hat{S}(q, \tau) \hat{S}(-q, 0) \rangle = \int d\omega \underbrace{\frac{1}{\pi} \frac{e^{-\tau\omega}}{1 - e^{-\beta\omega}} \tanh(\beta\omega/2)}_{K_{pp}(\tau, \omega)} \underbrace{\coth(\beta\omega/2) \chi''(q, \omega)}_{A_{ph}(\omega)} \quad (309)$$

and one computes $A(\omega)$. Note that since χ'' is an odd function of ω one restricts the integration range to positive values of ω . Hence:

$$\langle \hat{S}(q, \tau) \hat{S}(-q, 0) \rangle = \int_0^\infty d\omega \underbrace{(K_{pp}(\tau, \omega) + K_{pp}(\tau, -\omega))}_{K_{ph}(\tau, \omega)} A_{ph}(\omega). \quad (310)$$

In the code, ω_{start} is set to zero by default and the kernel K_{ph} is defined in the routine `XKER_ph`.

In general, one would like to produce the dynamical structure factor that gives the susceptibility according to

$$S(q, \omega) = \chi''(q, \omega) / (1 - e^{-\beta\omega}). \quad (311)$$

In the code, the routine `BACK_TRANS_ph` transforms the image A to the desired quantity:

$$S(q, \omega) = \frac{A_{ph}(\omega)}{1 + e^{-\beta\omega}}. \quad (312)$$

Matsubara-frequency formulation

The ALF library uses imaginary time. It is, however, possible to formulate the MaxEnt in Matsubara frequencies. Consider:

$$\chi(q, i\Omega_m) = \int_0^\beta d\tau e^{i\Omega_m \tau} \langle \hat{S}(q, \tau) \hat{S}(-q, 0) \rangle = \frac{1}{\pi} \int d\omega \frac{\chi''(q, \omega)}{\omega - i\Omega_m}. \quad (313)$$

Using the fact that $\chi''(q, \omega) = -\chi''(-q, -\omega) = -\chi''(q, -\omega)$ one obtains

$$\begin{aligned} \chi(q, i\Omega_m) &= \frac{1}{\pi} \int_0^\infty d\omega \left(\frac{1}{\omega - i\Omega_m} - \frac{1}{-\omega - i\Omega_m} \right) \chi''(q, \omega) \\ &= \frac{2}{\pi} \int_0^\infty d\omega \frac{\omega^2}{\omega^2 + \Omega_m^2} \frac{\chi''(q, \omega)}{\omega} \\ &\equiv \int_0^\infty d\omega K(\omega, i\Omega_m) A(q, \omega), \end{aligned} \quad (314)$$

with

$$K(\omega, i\Omega_m) = \frac{\omega^2}{\omega^2 + \Omega_m^2} \quad \text{and} \quad A(q, \omega) = \frac{2}{\pi} \frac{\chi''(q, \omega)}{\omega}. \quad (315)$$

The above definitions produce an image that satisfies the sum rule:

$$\int_0^\infty d\omega A(q, \omega) = \frac{1}{\pi} \int_{-\infty}^\infty d\omega \frac{\chi''(q, \omega)}{\omega} \equiv \chi(q, i\Omega_m = 0). \quad (316)$$

10.6 Optical conductivity: Channel=PH_C

A slight modification of the above is required to compute the optical conductivity. In the Lehmann representation, this quantity reads:

$$\sigma'(\omega) = \frac{\pi}{Z} \frac{1}{\omega} \sum_{n,m} e^{-\beta E_n} |\langle n | \hat{J}_p | m \rangle|^2 \delta(\omega + E_n - E_m) (1 - e^{-\beta \omega}) \quad (317)$$

where $\hat{J}_p$ is a model dependent paramagnetic current operator. One can show that the sum rule,

$$\int d\omega \omega \coth(\beta \omega / 2) \sigma'(\omega) = 2\pi \langle \hat{J}_p(\tau = 0) \hat{J}_p(0) \rangle \quad (318)$$

holds such that:

$$\langle \hat{J}(\tau) \hat{J}(0) \rangle = \int d\omega \underbrace{\frac{1}{\pi} \frac{e^{-\tau \omega}}{1 - e^{-\beta \omega}} \tanh(\beta \omega / 2)}_{K_{pp}(\tau, \omega)} \underbrace{\omega \coth(\beta \omega / 2) \sigma'(\omega)}_{A_C(\omega)} \quad (319)$$

and one computes $A_C(\omega)$. Since $\hat{J}_p$ is hermitian, $\sigma'(\omega)$ is an even function of frequency, the above can be simplified to

$$\langle \hat{J}_p(\tau) \hat{J}_p(0) \rangle = \int_0^\infty d\omega \underbrace{(K_{pp}(\tau, \omega) + K_{pp}(\tau, -\omega))}_{K_{ph}(\tau, \omega)} A_C(\omega). \quad (320)$$

As apparent, the only difference with the particle-hole channel of Sec. 10.5 is in the back transformation routine BACK_TRANS_ph_C required to obtain the optical conductivity:

$$\sigma'(\omega) = \frac{\tanh(\beta \omega / 2)}{\omega} A_C(\omega) \quad (321)$$

10.7 Particle-Particle quantities: Channel=PP

Similarly to the particle-hole channel, the particle-particle channel is also a bosonic correlation function. Here, however, we do not assume that the imaginary time data is symmetric around the $\tau = \beta/2$ point. We use the kernel K_{pp} defined in Eq. (309) and consider the whole frequency range. The back transformation yields

$$\frac{\chi''(\omega)}{\omega} = \frac{\tanh(\beta \omega / 2)}{\omega} A(\omega). \quad (322)$$

If no default is provided, then we use a flat default for the $A(\omega)$ that satisfies the sum rule: $\int d\omega A(\omega) = \pi (\langle \hat{S}(q, \beta) \hat{S}(-q, 0) \rangle + \langle \hat{S}(q, 0) \hat{S}(-q, 0) \rangle)$. Here, we use the same notation as in Eq. (309).

10.8 Zero-temperature, projective code: Channel=T0

In the zero temperature limit, the spectral function associated to an operator $\hat{O}$ reads:

$$A_o(\omega) = \pi \sum_n |\langle n | \hat{O} | 0 \rangle|^2 \delta(E_n - E_0 - \omega), \quad (323)$$

such that

$$\langle 0 | \hat{O}^\dagger(\tau) \hat{O}(0) | 0 \rangle = \int d\omega K_o(\tau, \omega) A_o(\omega), \quad (324)$$

with

$$K_0(\tau, \omega) = \frac{1}{\pi} e^{-\tau\omega}. \quad (325)$$

The zeroth moment of the spectral function reads

$$\int d\omega A_o(\omega) = \pi \langle 0 | \hat{O}^\dagger(0) \hat{O}(0) | 0 \rangle, \quad (326)$$

and hence corresponds to the first data point.

In the zero-temperature limit one does not distinguish between particle, particle-hole, or particle-particle channels. Using the `Max_Sac.F90` with `Channel="T0"` loads the above kernel in the `MaxEnt` library. In this case the back transformation is set to unity. The code will also cut-off the tail of the imaginary time correlation function if the relative error is greater than the variable `Tolerance`.

10.9 Integration over the spectral function: QFI and $\frac{\partial I}{\partial V}$

In many cases, one would like to compute the integral

$$(F, A) = \int d\omega F(\omega) A(\omega) \quad (327)$$

for a user defined function $F(\omega)$ and the spectral function $A(\omega)$ produced in the stochastic `MaxEnt`. An example of an application where this can come in handy is the calculation of the quantum Fisher information (QFI) [158] or susceptibilities. As seen above, the stochastic `MaxEnt` encodes the image $n(x)$ as a set of Dirac δ -functions (Eq. (291)); thus rendering the integration very convenient to carry out. Since we stochastically sample the image, we can easily estimate the fluctuations of the result. In fact,

$$(F, A) = M_0 \int_0^1 dx F(\Phi^{-1}(x)) n(x). \quad (328)$$

The function $F(\omega)$ is provided by the user in the call to the stochastic `MaxEnt` routine. The program then computes (F, A) at the given sets of temperatures α (see Eq. 290) provided by the user.

One can use the above to efficiently compute susceptibilities— typically of the form:

$$\chi = \int_0^\beta d\tau g(\tau) = \int d\omega \underbrace{\int_0^\beta d\tau K(\tau, \omega)}_{\equiv F(\omega)} A(\omega). \quad (329)$$

Here, the analytic continuation is used as an optimal fit function to the imaginary time data, allowing the minimization of finite discretization time errors.

Another example is the quantum Fisher information [158] (QFI). In the particle-hole channel, this quantity reads:

$$F_Q(T) = \int_0^\infty d\omega \frac{4}{\pi} \tanh\left(\frac{\beta\omega}{2}\right) \chi''(\omega) = \int_0^\infty d\omega \underbrace{\frac{4}{\pi} \tanh^2\left(\frac{\beta\omega}{2}\right)}_{\equiv F(\omega)} A(\omega). \quad (330)$$

The last step follows for Eq. 309. In the default implementation the QFI is computed for the particle-hole channel, and the results are written in the file `QFI.dat`. The first column of this file corresponds to the inverse temperature, $1/\alpha$, the second column to the QFI, and the third

column to the error bar. The function F is specified in the module `maxent_wrapper_mod.F90` in the `Analysis` directory. This function is passed to the stochastic MaxEnt routine in the `called` from the wrapper `Max_SAC.F90`. The user can modify the function F to compute other quantities and specify the output of the file name.

For the particle channel the default corresponds to the calculation of $\frac{\partial I}{\partial V}\big|_{V=0}$, given by the Meier-Wingreen formula [159]:

$$\frac{\partial I}{\partial V}\bigg|_{V=0} = - \int d\omega A(\omega) \frac{\partial f(\beta\omega)}{\partial \omega} \quad (331)$$

with $f(x) = 1/(1 + e^x)$ the Fermi function. The evaluation of this quantity is based on the work of Ref. [160] where the Fermi function is approximated by:

$$f(x) - \frac{1}{2} = - \sum_{\alpha=1}^{N/2} \left(\frac{R_{\alpha}}{x - i/b_{\alpha}} + \frac{R_{\alpha}}{x + i/b_{\alpha}} \right). \quad (332)$$

For this parametrization, it turns out that truncating the sum to ten terms provides a perfect approximation for $|x| < 40$. In the MaxEnt wrapper `Max_SAC.F90` the call to the routine `Set_Ra_ba(N)` sets the parameters R_{α} and b_{α} for the approximation of the Fermi function (see Ref. [160]). The default value of N is set to ten resulting in the aforementioned truncation, but the user can change it to improve the approximation for larger values of $|x|$. With the above, one can estimate:

$$\frac{\partial I}{\partial V}\bigg|_{V=0} = \int d\omega A(\omega) \left[-\frac{2}{\beta} \sum_{\alpha>0} R_{\alpha} \operatorname{Re} \frac{1}{(i/(\beta b_{\alpha}) - \omega)^2} \right] \quad (333)$$

The results are written in the file `dIdV.dat` with the first column corresponding to the temperature, the second column to $\frac{\partial I}{\partial V}\big|_{V=0}$, and the third column to the error bar.

10.10 Dynamics of the one-dimensional half-filled Hubbard model

To conclude this section, we show the example of the one-dimensional Hubbard model, which is known to show spin-charge separation (see Ref. [161] and references therein). The data of Fig. 14 was produced with the `pyALF` python script `Hubbard_1D.py`, and the spectral function plots with the bash script `Spectral.sh`.

11 Conclusions and Future Directions

In its present form, the auxiliary-field QMC code of the ALF project allows us to simulate a large class of non-trivial models, both efficiently and at minimal programming cost. The package contains many advanced functionalities, including a projective formulation, various updating schemes, better control of Trotter errors, predefined structures that facilitate reuse, a large class of models, continuous fields and, finally, stochastic and Maximum Entropy codes for analytical continuation. Also, the usability of the code has been continuously improved. In particular the `pyALF` project provides a Python interface to the ALF which substantially facilitates running the code for established models. This ease of use renders ALF a powerful tool to for benchmarking new algorithms.

There are further capabilities that we would like to see in future versions of ALF. It would be desirable to include additional lattice fermion algorithms such as the CT-INT [89, 162]. In this process we also aim at improving the modularity of the code, such that common parts for different algorithms can be reused. There are many methods aimed a circumventing the

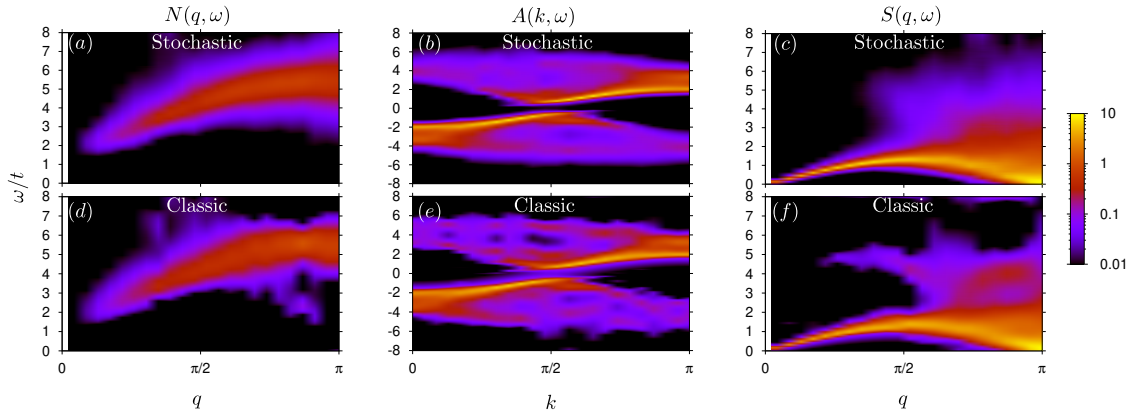


Figure 14: Dynamics of the one-dimensional half-filled Hubbard model on a 46-site chain, with $U/t=4$ and $\beta t = 10$. The top (bottom) panels correspond to stochastic (Classic) Maxent runs. (a,d) Dynamical charge structure factor, (b,e) single particle spectral function and (c,f) dynamical spin structure factor. Data obtained using the pyALF python script [Hubbard_1D.py](#), considering 400 bins of 200 sweeps each and taking into account the covariance matrix for the MaxEnt. The parameters for the MaxEnt that differ from the default values are also listed in the python script.

negative sign problem. This includes variational implementations of the projective code. It would certainly be desirable to implement such schemes in the ALF. One further imminent challenge is to make the code GPU ready since most computer centers are shifting towards GPU-based supercomputers. At first glance it is not obvious how to achieve this efficiently since the size of our matrices is small.

Acknowledgments

We are very grateful to B. Danu, S. Beyl, M. Hohenadler, M. Raczkowski, T. Sato, M. Ulybyshev, Z. Wang, and M. Weber for their constant support during the development of this project. We equally thank G. Hager, M. Wittmann, and G. Wellein for useful discussions and overall support. And we extend our special thanks to the user community for its valuable feedback. FFA would also like to thank T. Lang and Z. Y. Meng for developments of the auxiliary field code as well as to T. Grover. MB, FFA and FG thank the Bavarian Competence Network for Technical and Scientific High Performance Computing (KONWIHR) for financial support. FG, JH, and JS thank the SFB-1170 for financial support under projects Z03 and C01. FPT is funded by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) – project number 414456783. JSEP thanks the DFG for financial support under the project AS120/14-1, dedicated to the further development of the ALF library. Part of the optimization of the code was carried out during the Porting and Tuning Workshop 2016 offered by the Forschungszentrum Jülich. Calculations performed to extensively test this package were carried out both on SuperMUC-NG at the Leibniz Supercomputing Centre and on JURECA [163] at the Jülich Supercomputing Centre. We thank both institutions for the generous allocation of computing time.

A Practical implementation of Wick decomposition of $2n$ -point correlation functions of two imaginary times

In this Appendix we briefly outline how to compute $2n$ point correlation functions of the form:

$$\lim_{\epsilon \rightarrow 0} \sum_{\sigma_1, \sigma'_1, \dots, \sigma_n, \sigma'_n, s_1, s'_1 \dots s_n, s'_n} f(\sigma_1, \sigma'_1, \dots, \sigma_n, \sigma'_n, s_1, s'_1 \dots s_n, s'_n) \langle \langle \mathcal{T} \left(\hat{c}_{x_1, \sigma_1, s_1}^\dagger(\tau_{1, \epsilon}) \hat{c}_{x'_1, \sigma'_1, s'_1}(\tau'_{1, \epsilon}) - a_1 \right) \dots \left(\hat{c}_{x_n, \sigma_n, s_n}^\dagger(\tau_{n, \epsilon}) \hat{c}_{x'_n, \sigma'_n, s'_n}(\tau'_{n, \epsilon}) - a_n \right) \rangle \rangle_C. \quad (334)$$

Here, σ is a color index and s a flavor index such that

$$\langle \langle \mathcal{T} \hat{c}_{x, \sigma, s}^\dagger(\tau) \hat{c}_{x', \sigma', s'}(\tau') \rangle \rangle_C = \langle \langle \mathcal{T} \hat{c}_{x, s}^\dagger(\tau) \hat{c}_{x', s}(\tau') \rangle \rangle_C \delta_{s, s'} \delta_{\sigma, \sigma'}. \quad (335)$$

That is, the single-particle Green function is diagonal in the flavor index and color independent. To define the time ordering we will assume that all times differ but that $\lim_{\epsilon \rightarrow 0} \tau_{n, \epsilon}$ as well as $\lim_{\epsilon \rightarrow 0} \tau'_{n, \epsilon}$ take the values 0 or τ . Let

$$G_s(I, J) = \lim_{\epsilon \rightarrow 0} \langle \langle \mathcal{T} c_{x_I, s}^\dagger(\tau_{I, \epsilon}) c_{x_J, s}(\tau'_{J, \epsilon}) \rangle \rangle_C. \quad (336)$$

The $G_s(I, J)$ are uniquely defined by the time-displaced correlation functions that enter the ObserT routine in the Hamiltonian files. They are defined in Eq. (162) and read:

$$\begin{aligned} \text{GT0}(\mathbf{x}, \mathbf{y}, s) &= \langle \langle \hat{c}_{x, s}(\tau) \hat{c}_{y, s}^\dagger(0) \rangle \rangle_C = \langle \langle \mathcal{T} \hat{c}_{x, s}(\tau) \hat{c}_{y, s}^\dagger(0) \rangle \rangle_C \\ \text{GOT}(\mathbf{x}, \mathbf{y}, s) &= -\langle \langle \hat{c}_{y, s}^\dagger(\tau) \hat{c}_{x, s}(0) \rangle \rangle_C = \langle \langle \mathcal{T} \hat{c}_{x, s}(0) \hat{c}_{y, s}^\dagger(\tau) \rangle \rangle_C \\ \text{G00}(\mathbf{x}, \mathbf{y}, s) &= \langle \langle \hat{c}_{x, s}(0) \hat{c}_{y, s}^\dagger(0) \rangle \rangle_C \\ \text{GTT}(\mathbf{x}, \mathbf{y}, s) &= \langle \langle \hat{c}_{x, s}(\tau) \hat{c}_{y, s}^\dagger(\tau) \rangle \rangle_C. \end{aligned} \quad (337)$$

For instance, let $\tau_{I, \epsilon} > \tau'_{J, \epsilon}$ and $\lim_{\epsilon \rightarrow 0} \tau_{I, \epsilon} = \lim_{\epsilon \rightarrow 0} \tau'_{J, \epsilon} = \tau$. Then

$$G_s(I, J) = \langle \langle c_{x_I, s}^\dagger(\tau) c_{x_J, s}(\tau) \rangle \rangle_C = \delta_{x_I, x_J} - \text{GTT}(x'_J, x_I, s). \quad (338)$$

Using the formulation of Wick's theorem of Eq. (24), Eq. (334) reads:

$$\sum_{\sigma_1, \sigma'_1, \dots, \sigma_n, \sigma'_n, s_1, s'_1 \dots s_n, s'_n} f(\sigma_1, \sigma'_1, \dots, \sigma_n, \sigma'_n, s_1, s'_1 \dots s_n, s'_n) \det \begin{bmatrix} G_{s_1}(1, 1) \delta_{s_1, s'_1} \delta_{\sigma_1, \sigma'_1} - \alpha_1 & G_{s_1}(1, 2) \delta_{s_1, s'_2} \delta_{\sigma_1, \sigma'_2} & \dots & G_{s_1}(1, n) \delta_{s_1, s'_n} \delta_{\sigma_1, \sigma'_n} \\ G_{s_2}(2, 1) \delta_{s_2, s'_1} \delta_{\sigma_2, \sigma'_1} & G_{s_2}(2, 2) \delta_{s_2, s'_2} \delta_{\sigma_2, \sigma'_2} - \alpha_2 & \dots & G_{s_2}(2, n) \delta_{s_2, s'_n} \delta_{\sigma_2, \sigma'_n} \\ \vdots & \vdots & \ddots & \vdots \\ G_{s_n}(n, 1) \delta_{s_n, s'_1} \delta_{\sigma_n, \sigma'_1} & G_{s_n}(n, 2) \delta_{s_n, s'_2} \delta_{\sigma_n, \sigma'_2} & \dots & G_{s_n}(n, n) \delta_{s_n, s'_n} \delta_{\sigma_n, \sigma'_n} - \alpha_n \end{bmatrix}. \quad (339)$$

The symbolic evaluation of the determinant as well as the sum over the color and flavor indices can be carried out with Mathematica. This produces a long expression in terms of the functions $G(I, J, s)$ that can then be included in the code. The Mathematica notebooks that we use can be found in the directory Mathematica of the ALF directory. As an open source alternative to Mathematica, the user can consider the Sympy Python library.

B Performance, memory requirements and parallelization

As mentioned in the introduction, the auxiliary field QMC algorithm scales linearly in inverse temperature β and as a cube in the volume N_{dim} . Using fast updates, a single spin flip requires $(N_{\text{dim}})^2$ operations to update the Green function upon acceptance. As there are $L_{\text{Trotter}} \times N_{\text{dim}}$ spins to be visited, the total computational cost for one sweep is of the order of $\beta(N_{\text{dim}})^3$. This operation alongside QR-decompositions required for stabilization dominates the performance, see Fig. 15. A profiling analysis of our code shows that 80-90% of the CPU time is spend in ZGEMM calls of the BLAS library provided in the MKL package by Intel. Consequently, the single-core performance is next to optimal.

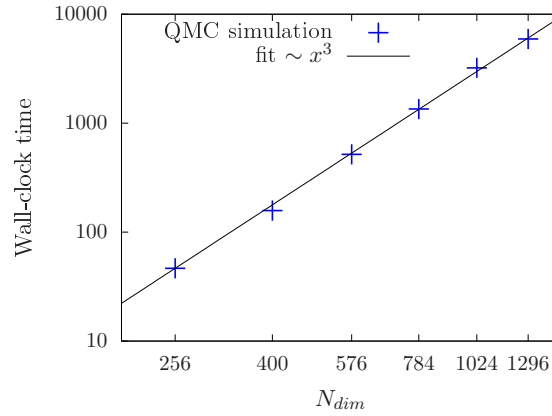


Figure 15: Volume scaling behavior of the auxiliary field QMC code of the ALF project on SuperMUC (phase 2/Haswell nodes) at the LRZ in Munich. The number of sites N_{dim} corresponds to the system volume. The plot confirms that the leading scaling order is due to matrix multiplications such that the runtime is dominated by calls to ZGEMM.

For the implementation which scales linearly in β , one has to store $2 \times N_{\text{fl}} \times L_{\text{Trotter}}/\text{NWrap}$ intermediate propagation matrices of dimension $N_{\text{dim}} \times N_{\text{dim}}$. Hence the memory cost scales as βN_{dim}^2 and for large lattices and/or low temperatures this dominates the total memory requirements that can exceed 2 GB memory for a sequential version.

The above estimates of βN_{dim}^3 for CPU time and βN_{dim}^2 for memory implicitly assume Hamiltonians where the interaction is a sum of local terms. Recently Landau level projection schemes for the regularization of continuum field theories have been introduced in the realm of the auxiliary field QMC algorithm [70, 71]. In this case the interaction is not local, such that the matrices stored in the `Op_V` array of `Observable` type are of dimension of N_{dim} . Since the dimension of the `Op_V` array scales as N_{dim} , the memory requirement scales as N_{dim}^3 . In these algorithms, a single field couples to a $N_{\text{dim}} \times N_{\text{dim}}$ matrix, such that updating it scales as N_{dim}^3 . Furthermore, and as mentioned in Sec. 2.4, for non-local Hamiltonians the Trotter time step has to be scaled as $1/N_{\text{dim}}$ so as to maintain a constant systematic error. Taking all of this into account, yields a CPU time that scales as βN_{dim}^5 . Hence this approach is expensive both in memory and CPU time.

At the heart of Monte Carlo schemes lies a random walk through the given configuration space. This is easily parallelized via MPI by associating one random walker to each MPI task. For each task, we start from a random configuration and have to invest the autocorrelation time T_{auto} to produce an equilibrated configuration. Additionally we can also profit from an OpenMP parallelized version of the BLAS/LAPACK library for an additional speedup, which

also effects equilibration overhead $N_{\text{MPI}} \times T_{\text{auto}}/N_{\text{OMP}}$, where N_{MPI} is the number of cores and N_{OMP} the number of OpenMP threads. For a given number of independent measurements N_{meas} , we therefore need a wall-clock time given by

$$T = \frac{T_{\text{auto}}}{N_{\text{OMP}}} \left(1 + \frac{N_{\text{meas}}}{N_{\text{MPI}}} \right). \quad (340)$$

As we typically have $N_{\text{meas}}/N_{\text{MPI}} \gg 1$, the speedup is expected to be almost perfect, in accordance with the performance test results for the auxiliary field QMC code on SuperMUC (see Fig. 16 (left)).

For many problem sizes, 2 GB memory per MPI task (random walker) suffices such that we typically start as many MPI tasks as there are physical cores per node. Due to the large amount of CPU time spent in MKL routines, we do not profit from the hyper-threading option. For large systems, the memory requirement increases and this is tackled by increasing the amount of OpenMP threads to decrease the stress on the memory system and to simultaneously reduce the equilibration overhead (see Fig. 16 (right)). For the displayed speedup, it was crucial to pin the MPI tasks as well as the OpenMP threads in a pattern which keeps the threads as compact as possible to profit from a shared cache. This also explains the drop in efficiency from 14 to 28 threads where the OpenMP threads are spread over both sockets.

We store the field configurations of the random walker as checkpoints, such that a long simulation can be easily split into several short simulations. This procedure allows us to take advantage of chained jobs using the dependency chains provided by the batch system.

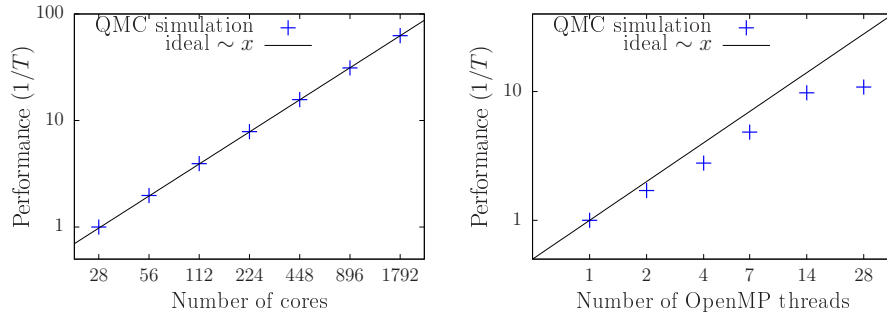


Figure 16: MPI (left) and OpenMP (right) scaling behavior of the auxiliary field QMC code of the ALF project on SuperMUC (phase 2/Haswell nodes) at the LRZ in Munich. The MPI performance data was normalized to 28 cores and was obtained using a problem size of $N_{\text{dim}} = 400$. This is a medium to small system size that is the least favorable in terms of MPI synchronization effects. The OpenMP performance data was obtained using a problem size of $N_{\text{dim}} = 1296$. Employing 2 and 4 OpenMP threads introduces some synchronization/management overhead such that the per-core performance is slightly reduced, compared to the single thread efficiency. Further increasing the amount of threads to 7 and 14 keeps the efficiency constant. The drop in performance of the 28 thread configuration is due to the architecture as the threads are now spread over both sockets of the node. To obtain the above results, it was crucial to pin the processes in a fashion that keeps the OpenMP threads as compact as possible.

C Licenses and Copyrights

The ALF code is provided as an open source software such that it is available to all and we hope that it will be useful. If you benefit from this code we ask that you acknowledge the ALF collaboration as mentioned on our website <https://alf.physik.uni-wuerzburg.de>. The git repository at <https://github.com/ALF-QMC/ALF> gives us the tools to create a small but vibrant community around the code and provides a suitable entry point for future contributors and future developments. The website is also the place where the original source files can be found. Its public release make it necessary to add copyright headers to our source code, which is licensed under a GPL license to keep the source as well as any future work in the community. And the Creative Commons licenses are a good way to share our documentation and it is also well accepted by publishers. Therefore this document is licensed to you under a CC-BY-SA license. This means you can share it and redistribute it as long as you cite the original source and license your changes under the same license. The details are in the file `license.CCBYSA`, which you should have received with this documentation. To express our desire for a proper attribution we decided to make this a visible part of the license. To that end we have exercised the rights of section 7 of GPL version 3 and have amended the license terms with an additional paragraph that expresses our wish that if an author has benefited from this code that he/she should consider giving back a citation as specified on <https://alf.physik.uni-wuerzburg.de>. This is not something that is meant to restrict your freedom of use, but something that we strongly expect to be good scientific conduct. The original GPL license can be found in the file `license.GPL` and the additional terms can be found in `license.additional`. In favour to our users, the ALF code contains part of the Lapack implementation version 3.6.1 from <http://www.netlib.org/lapack>. Lapack is licensed under the modified BSD license whose full text can be found in `license.BSD`. With that being said, we hope that the ALF code will prove to you to be a suitable and high-performance tool that enables you to perform quantum Monte Carlo studies of solid state models of unprecedented complexity.

The ALF project's contributors.

COPYRIGHT

Copyright © 2016-2022, The ALF Project.

The ALF Project Documentation is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License. You are free to share and benefit from this documentation as long as this license is preserved and proper attribution to the authors is given. For details see the ALF project website alf.physik.uni-wuerzburg.de and the file `license.CCBYSA`.

References

- [1] R. Blankenbecler, D. J. Scalapino and R. L. Sugar, *Monte carlo calculations of coupled boson-fermion systems.*, Phys. Rev. D **24**, 2278 (1981), doi:[10.1103/PhysRevD.24.2278](https://doi.org/10.1103/PhysRevD.24.2278).
- [2] S. White, D. Scalapino, R. Sugar, E. Loh, J. Gubernatis and R. Scalettar, *Numerical study of the two-dimensional hubbard model*, Phys. Rev. B **40**, 506 (1989), doi:[10.1103/PhysRevB.40.506](https://doi.org/10.1103/PhysRevB.40.506).

- [3] G. Sugiyama and S. Koonin, *Auxiliary field monte-carlo for quantum many-body ground states*, Annals of Physics **168**(1), 1 (1986), doi:[10.1016/0003-4916\(86\)90107-7](https://doi.org/10.1016/0003-4916(86)90107-7).
- [4] S. Sorella, S. Baroni, R. Car and M. Parrinello, *A novel technique for the simulation of interacting fermion systems*, Europhysics Letters (EPL) **8**(7), 663 (1989), doi:[10.1209/0295-5075/8/7/014](https://doi.org/10.1209/0295-5075/8/7/014).
- [5] S. Duane, A. D. Kennedy, B. J. Pendleton and D. Roweth, *Hybrid Monte Carlo*, Phys. Lett. **B195**, 216 (1987), doi:[10.1016/0370-2693\(87\)91197-X](https://doi.org/10.1016/0370-2693(87)91197-X).
- [6] F. Assaad and H. Evertz, *World-line and determinantal quantum monte carlo methods for spins, phonons and electrons*, In H. Fehske, R. Schneider and A. Weiße, eds., *Computational Many-Particle Physics*, vol. 739 of *Lecture Notes in Physics*, pp. 277–356. Springer, Berlin Heidelberg, ISBN 978-3-540-74685-0, doi:[10.1007/978-3-540-74686-7_10](https://doi.org/10.1007/978-3-540-74686-7_10) (2008).
- [7] D. J. Scalapino, *Numerical Studies of the 2D Hubbard Model*, pp. 495–526, Springer New York, New York, NY, ISBN 978-0-387-68734-6, doi:[10.1007/978-0-387-68734-6_13](https://doi.org/10.1007/978-0-387-68734-6_13) (2007).
- [8] J. P. F. LeBlanc, A. E. Antipov, F. Becca, I. W. Bulik, G. K.-L. Chan, C.-M. Chung, Y. Deng, M. Ferrero, T. M. Henderson, C. A. Jiménez-Hoyos, E. Kozik, X.-W. Liu *et al.*, *Solutions of the two-dimensional hubbard model: Benchmarks and results from a wide range of numerical algorithms*, Phys. Rev. X **5**, 041041 (2015), doi:[10.1103/PhysRevX.5.041041](https://doi.org/10.1103/PhysRevX.5.041041).
- [9] M. Hohenadler, T. C. Lang and F. F. Assaad, *Correlation effects in quantum spin-hall insulators: A quantum monte carlo study*, Phys. Rev. Lett. **106**, 100403 (2011), doi:[10.1103/PhysRevLett.106.100403](https://doi.org/10.1103/PhysRevLett.106.100403).
- [10] D. Zheng, G.-M. Zhang and C. Wu, *Particle-hole symmetry and interaction effects in the kane-mele-hubbard model*, Phys. Rev. B **84**, 205121 (2011), doi:[10.1103/PhysRevB.84.205121](https://doi.org/10.1103/PhysRevB.84.205121).
- [11] F. F. Assaad, M. Bercx and M. Hohenadler, *Topological invariant and quantum spin models from magnetic π fluxes in correlated topological insulators*, Phys. Rev. X **3**, 011015 (2013), doi:[10.1103/PhysRevX.3.011015](https://doi.org/10.1103/PhysRevX.3.011015).
- [12] J. S. Hofmann, F. F. Assaad, R. Queiroz and E. Khalaf, *Search for correlation-induced adiabatic paths between distinct topological insulators*, Phys. Rev. Research **2**, 023390 (2020), doi:[10.1103/PhysRevResearch.2.023390](https://doi.org/10.1103/PhysRevResearch.2.023390).
- [13] F. F. Assaad and I. F. Herbut, *Pinning the order: The nature of quantum criticality in the hubbard model on honeycomb lattice*, Phys. Rev. X **3**, 031010 (2013), doi:[10.1103/PhysRevX.3.031010](https://doi.org/10.1103/PhysRevX.3.031010).
- [14] F. Parisen Toldin, M. Hohenadler, F. F. Assaad and I. F. Herbut, *Fermionic quantum criticality in honeycomb and π -flux hubbard models: Finite-size scaling of renormalization-group-invariant observables from quantum monte carlo*, Phys. Rev. B **91**, 165108 (2015), doi:[10.1103/PhysRevB.91.165108](https://doi.org/10.1103/PhysRevB.91.165108).
- [15] Y. Otsuka, S. Yunoki and S. Sorella, *Universal quantum criticality in the metal-insulator transition of two-dimensional interacting dirac electrons*, Phys. Rev. X **6**, 011029 (2016), doi:[10.1103/PhysRevX.6.011029](https://doi.org/10.1103/PhysRevX.6.011029).

- [16] S. Chandrasekharan and A. Li, *Quantum critical behavior in three dimensional lattice gross-neveu models*, Phys. Rev. D **88**, 021701 (2013), doi:[10.1103/PhysRevD.88.021701](https://doi.org/10.1103/PhysRevD.88.021701).
- [17] V. Ayyar and S. Chandrasekharan, *Massive fermions without fermion bilinear condensates*, Phys. Rev. D **91**, 065035 (2015), doi:[10.1103/PhysRevD.91.065035](https://doi.org/10.1103/PhysRevD.91.065035).
- [18] Y. Liu, Z. Wang, T. Sato, M. Hohenadler, C. Wang, W. Guo and F. F. Assaad, *Superconductivity from the condensation of topological defects in a quantum spin-hall insulator*, Nature Communications **10**(1), 2658 (2019), doi:[10.1038/s41467-019-10372-0](https://doi.org/10.1038/s41467-019-10372-0).
- [19] Z.-X. Li, Y.-F. Jiang, S.-K. Jian and H. Yao, *Fermion-induced quantum critical points*, Nature Communications **8**(1), 314 (2017), doi:[10.1038/s41467-017-00167-6](https://doi.org/10.1038/s41467-017-00167-6).
- [20] M. Raczkowski, R. Peters, T. T. Phùg, N. Takemori, F. F. Assaad, A. Honecker and J. Vahedi, *Hubbard model on the honeycomb lattice: From static and dynamical mean-field theories to lattice quantum monte carlo simulations*, Phys. Rev. B **101**, 125103 (2020), doi:[10.1103/PhysRevB.101.125103](https://doi.org/10.1103/PhysRevB.101.125103).
- [21] F. F. Assaad and T. Grover, *Simple fermionic model of deconfined phases and phase transitions*, Phys. Rev. X **6**, 041049 (2016), doi:[10.1103/PhysRevX.6.041049](https://doi.org/10.1103/PhysRevX.6.041049).
- [22] T. Sato, M. Hohenadler and F. F. Assaad, *Dirac fermions with competing orders: Non-landau transition with emergent symmetry*, Phys. Rev. Lett. **119**, 197203 (2017), doi:[10.1103/PhysRevLett.119.197203](https://doi.org/10.1103/PhysRevLett.119.197203).
- [23] T. Sato, M. Hohenadler, T. Grover, J. McGreevy and F. F. Assaad, *Topological terms on topological defects: a quantum Monte Carlo study*, arXiv e-prints arXiv:2005.08996 (2020), <https://arxiv.org/abs/2005.08996>.
- [24] Z. Wang, Y. Liu, T. Sato, M. Hohenadler, C. Wang, W. Guo and F. F. Assaad, *Doping-induced quantum spin Hall insulator to superconductor transition*, arXiv e-prints arXiv:2006.13239 (2020), <https://arxiv.org/abs/2006.13239>.
- [25] S. Gazit, M. Randeria and A. Vishwanath, *Emergent dirac fermions and broken symmetries in confined and deconfined phases of z_2 gauge theories*, Nat. Phys. **13**(5), 484 (2017), doi:[10.1038/nphys4028](https://doi.org/10.1038/nphys4028).
- [26] S. Gazit, F. F. Assaad, S. Sachdev, A. Vishwanath and C. Wang, *Confinement transition of z_2 gauge theories coupled to massless fermions: Emergent quantum chromodynamics and $so(5)$ symmetry*, Proceedings of the National Academy of Sciences (2018), doi:[10.1073/pnas.1806338115](https://doi.org/10.1073/pnas.1806338115), <http://www.pnas.org/content/early/2018/07/06/1806338115.full.pdf>.
- [27] X. Y. Xu, Y. Qi, L. Zhang, F. F. Assaad, C. Xu and Z. Y. Meng, *Monte carlo study of lattice compact quantum electrodynamics with fermionic matter: The parent state of quantum phases*, Phys. Rev. X **9**, 021022 (2019), doi:[10.1103/PhysRevX.9.021022](https://doi.org/10.1103/PhysRevX.9.021022).
- [28] M. Hohenadler and F. F. Assaad, *Fractionalized metal in a falicov-kimball model*, Phys. Rev. Lett. **121**, 086601 (2018), doi:[10.1103/PhysRevLett.121.086601](https://doi.org/10.1103/PhysRevLett.121.086601).
- [29] M. Hohenadler and F. F. Assaad, *Orthogonal metal in the hubbard model with liberated slave spins*, Phys. Rev. B **100**, 125133 (2019), doi:[10.1103/PhysRevB.100.125133](https://doi.org/10.1103/PhysRevB.100.125133).

- [30] S. Gazit, F. F. Assaad and S. Sachdev, *Fermi-surface reconstruction without symmetry breaking*, arXiv:1906.11250 arXiv:1906.11250 (2019), <https://arxiv.org/abs/1906.11250>.
- [31] F. F. Assaad, *Quantum monte carlo simulations of the half-filled two-dimensional kondo lattice model*, Phys. Rev. Lett. **83**, 796 (1999), doi:[10.1103/PhysRevLett.83.796](https://doi.org/10.1103/PhysRevLett.83.796).
- [32] S. Capponi and F. F. Assaad, *Spin and charge dynamics of the ferromagnetic and anti-ferromagnetic two-dimensional half-filled kondo lattice model*, Phys. Rev. B **63**, 155114 (2001), doi:[10.1103/PhysRevB.63.155114](https://doi.org/10.1103/PhysRevB.63.155114).
- [33] T. Sato, F. F. Assaad and T. Grover, *Quantum monte carlo simulation of frustrated kondo lattice models*, Phys. Rev. Lett. **120**, 107201 (2018), doi:[10.1103/PhysRevLett.120.107201](https://doi.org/10.1103/PhysRevLett.120.107201).
- [34] J. S. Hofmann, F. F. Assaad and T. Grover, *Fractionalized fermi liquid in a frustrated kondo lattice model*, Phys. Rev. B **100**, 035118 (2019), doi:[10.1103/PhysRevB.100.035118](https://doi.org/10.1103/PhysRevB.100.035118).
- [35] B. Danu, F. F. Assaad and F. Mila, *Exploring the kondo effect of an extended impurity with chains of co adatoms in a magnetic field*, Phys. Rev. Lett. **123**, 176601 (2019), doi:[10.1103/PhysRevLett.123.176601](https://doi.org/10.1103/PhysRevLett.123.176601).
- [36] B. Danu, M. Vojta, F. F. Assaad and T. Grover, *Kondo breakdown in a spin-1/2 chain of adatoms on a dirac semimetal*, Phys. Rev. Lett. **125**, 206602 (2020), doi:[10.1103/PhysRevLett.125.206602](https://doi.org/10.1103/PhysRevLett.125.206602).
- [37] Y. Schattner, S. Lederer, S. A. Kivelson and E. Berg, *Ising nematic quantum critical point in a metal: A monte carlo study*, Phys. Rev. X **6**, 031028 (2016), doi:[10.1103/PhysRevX.6.031028](https://doi.org/10.1103/PhysRevX.6.031028).
- [38] O. Grossman, J. S. Hofmann, T. Holder and E. Berg, *Specific Heat of a Quantum Critical Metal*, arXiv e-prints arXiv:2009.11280 (2020), <https://arxiv.org/abs/2009.11280>.
- [39] X. Y. Xu, K. Sun, Y. Schattner, E. Berg and Z. Y. Meng, *Non-fermi liquid at (2 + 1)D ferromagnetic quantum critical point*, Phys. Rev. X **7**, 031058 (2017), doi:[10.1103/PhysRevX.7.031058](https://doi.org/10.1103/PhysRevX.7.031058).
- [40] Z. H. Liu, G. Pan, X. Y. Xu, K. Sun and Z. Y. Meng, *Itinerant quantum critical point with fermion pockets and hotspots*, Proceedings of the National Academy of Sciences **116**(34), 16760 (2019), doi:[10.1073/pnas.1901751116](https://doi.org/10.1073/pnas.1901751116), <https://www.pnas.org/content/116/34/16760.full.pdf>.
- [41] E. Berg, M. A. Metlitski and S. Sachdev, *Sign-problem-free quantum monte carlo of the onset of antiferromagnetism in metals*, Science **338**(6114), 1606 (2012), doi:[10.1126/science.1227769](https://doi.org/10.1126/science.1227769), <http://www.sciencemag.org/content/338/6114/1606.full.pdf>.
- [42] H.-K. Tang, X. Yang, J. Sun and H.-Q. Lin, *Berezinskii-kosterlitz-thoules phase transition of spin-orbit coupled fermi gas in optical lattice*, EPL (Europhysics Letters) **107**(4), 40003 (2014), doi:[10.1209/0295-5075/107/40003](https://doi.org/10.1209/0295-5075/107/40003).
- [43] J. S. Hofmann, E. Berg and D. Chowdhury, *Superconductivity, pseudogap, and phase separation in topological flat bands*, Phys. Rev. B **102**, 201112 (2020), doi:[10.1103/PhysRevB.102.201112](https://doi.org/10.1103/PhysRevB.102.201112).

- [44] V. Peri, Z. Song, B. A. Bernevig and S. D. Huber, *Fragile topology and flat-band superconductivity in the strong-coupling regime*, arXiv:2008.02288 (2020), <https://arxiv.org/abs/2008.02288>.
- [45] F. F. Assaad, *Phase diagram of the half-filled two-dimensional $SU(n)$ hubbard-heisenberg model: A quantum monte carlo study*, Phys. Rev. B **71**, 075103 (2005), doi:[10.1103/PhysRevB.71.075103](https://doi.org/10.1103/PhysRevB.71.075103).
- [46] T. C. Lang, Z. Y. Meng, A. Muramatsu, S. Wessel and F. F. Assaad, *Dimerized solids and resonating plaquette order in $SU(n)$ -dirac fermions*, Phys. Rev. Lett. **111**, 066401 (2013), doi:[10.1103/PhysRevLett.111.066401](https://doi.org/10.1103/PhysRevLett.111.066401).
- [47] F. H. Kim, K. Penc, P. Nataf and F. Mila, *Linear flavor-wave theory for fully antisymmetric $su(n)$ irreducible representations*, Phys. Rev. B **96**, 205142 (2017), doi:[10.1103/PhysRevB.96.205142](https://doi.org/10.1103/PhysRevB.96.205142).
- [48] D. Wang, Y. Li, Z. Cai, Z. Zhou, Y. Wang and C. Wu, *Competing orders in the 2d half-filled $SU(2n)$ hubbard model through the pinning-field quantum monte carlo simulations*, Phys. Rev. Lett. **112**, 156403 (2014), doi:[10.1103/PhysRevLett.112.156403](https://doi.org/10.1103/PhysRevLett.112.156403).
- [49] F. H. Kim, F. F. Assaad, K. Penc and F. Mila, *Dimensional crossover in the $su(4)$ heisenberg model in the six-dimensional antisymmetric self-conjugate representation revealed by quantum monte carlo and linear flavor-wave theory*, Phys. Rev. B **100**, 085103 (2019), doi:[10.1103/PhysRevB.100.085103](https://doi.org/10.1103/PhysRevB.100.085103).
- [50] M. Raczkowski and F. F. Assaad, *Phase diagram and dynamics of the $SU(n)$ symmetric kondo lattice model*, Phys. Rev. Research **2**, 013276 (2020), doi:[10.1103/PhysRevResearch.2.013276](https://doi.org/10.1103/PhysRevResearch.2.013276).
- [51] M. Hohenadler, F. Parisen Toldin, I. F. Herbut and F. F. Assaad, *Phase diagram of the kane-mele-coulomb model*, Phys. Rev. B **90**, 085146 (2014), doi:[10.1103/PhysRevB.90.085146](https://doi.org/10.1103/PhysRevB.90.085146).
- [52] H.-K. Tang, E. Laksono, J. N. B. Rodrigues, P. Sengupta, F. F. Assaad and S. Adam, *Interaction-driven metal-insulator transition in strained graphene*, Phys. Rev. Lett. **115**, 186602 (2015), doi:[10.1103/PhysRevLett.115.186602](https://doi.org/10.1103/PhysRevLett.115.186602).
- [53] H.-K. Tang, J. N. Leaw, J. N. B. Rodrigues, I. F. Herbut, P. Sengupta, F. F. Assaad and S. Adam, *The role of electron-electron interactions in two-dimensional dirac fermions*, Science **361**(6402), 570 (2018), doi:[10.1126/science.aao2934](https://doi.org/10.1126/science.aao2934), <http://science.sciencemag.org/content/361/6402/570.full.pdf>.
- [54] M. Raczkowski and F. F. Assaad, *Interplay between the edge-state magnetism and long-range coulomb interaction in zigzag graphene nanoribbons: Quantum monte carlo study*, Phys. Rev. B **96**, 115155 (2017), doi:[10.1103/PhysRevB.96.115155](https://doi.org/10.1103/PhysRevB.96.115155).
- [55] J. N. Leaw, H.-K. Tang, P. Sengupta, F. F. Assaad, I. F. Herbut and S. Adam, *Electronic ground state in bilayer graphene with realistic Coulomb interactions*, Phys. Rev. B **100**(12), 125116 (2019), doi:[10.1103/PhysRevB.100.125116](https://doi.org/10.1103/PhysRevB.100.125116), [1903.06177](https://doi.org/10.1103/PhysRevB.100.125116).
- [56] M. Rigol, A. Muramatsu, G. G. Batrouni and R. T. Scalettar, *Local quantum criticality in confined fermions on optical lattices*, Phys. Rev. Lett. **91**, 130403 (2003), doi:[10.1103/PhysRevLett.91.130403](https://doi.org/10.1103/PhysRevLett.91.130403).
- [57] D. Lee, *Lattice simulations for few- and many-body systems*, Progress in Particle and Nuclear Physics **63**(1), 117 (2009), doi:[http://dx.doi.org/10.1016/j.pnpnp.2008.12.001](https://doi.org/http://dx.doi.org/10.1016/j.pnpnp.2008.12.001).

- [58] Z. Wang, F. F. Assaad and F. Parisen Toldin, *Finite-size effects in canonical and grand-canonical quantum monte carlo simulations for fermions*, Phys. Rev. E **96**, 042131 (2017), doi:[10.1103/PhysRevE.96.042131](https://doi.org/10.1103/PhysRevE.96.042131).
- [59] T. Shen, Y. Liu, Y. Yu and B. Rubenstein, *Finite temperature auxiliary field quantum monte carlo in the canonical ensemble*, arXiv:2010.09813 (2020), <https://arxiv.org/abs/2010.09813>.
- [60] T. Grover, *Entanglement of interacting fermions in quantum monte carlo calculations*, Phys. Rev. Lett. **111**, 130402 (2013), doi:[10.1103/PhysRevLett.111.130402](https://doi.org/10.1103/PhysRevLett.111.130402).
- [61] P. Broecker and S. Trebst, *Rényi entropies of interacting fermions from determinantal quantum monte carlo simulations*, Journal of Statistical Mechanics: Theory and Experiment **2014**(8), P08015 (2014), doi:[10.1088/1742-5468/2014/08/p08015](https://doi.org/10.1088/1742-5468/2014/08/p08015).
- [62] F. F. Assaad, T. C. Lang and F. Parisen Toldin, *Entanglement spectra of interacting fermions in quantum monte carlo simulations*, Phys. Rev. B **89**, 125121 (2014), doi:[10.1103/PhysRevB.89.125121](https://doi.org/10.1103/PhysRevB.89.125121).
- [63] F. F. Assaad, *Stable quantum monte carlo simulations for entanglement spectra of interacting fermions*, Phys. Rev. B **91**, 125146 (2015), doi:[10.1103/PhysRevB.91.125146](https://doi.org/10.1103/PhysRevB.91.125146).
- [64] F. Parisen Toldin and F. F. Assaad, *Entanglement hamiltonian of interacting fermionic models*, Phys. Rev. Lett. **121**, 200602 (2018), doi:[10.1103/PhysRevLett.121.200602](https://doi.org/10.1103/PhysRevLett.121.200602).
- [65] F. Parisen Toldin, T. Sato and F. F. Assaad, *Mutual information in heavy-fermion systems*, Phys. Rev. B **99**, 155158 (2019), doi:[10.1103/PhysRevB.99.155158](https://doi.org/10.1103/PhysRevB.99.155158).
- [66] F. Parisen Toldin and F. F. Assaad, *Entanglement studies of interacting fermionic models*, J. Phys.: Conf. Ser. **1163**, 012056 (2019), doi:[10.1088/1742-6596/1163/1/012056](https://doi.org/10.1088/1742-6596/1163/1/012056), [1810.06595](https://arxiv.org/abs/1810.06595).
- [67] C. Chen, X. Y. Xu, J. Liu, G. Batrouni, R. Scalettar and Z. Y. Meng, *Symmetry-enforced self-learning monte carlo method applied to the holstein model*, Phys. Rev. B **98**, 041102 (2018), doi:[10.1103/PhysRevB.98.041102](https://doi.org/10.1103/PhysRevB.98.041102).
- [68] C. Chen, X. Y. Xu, Z. Y. Meng and M. Hohenadler, *Charge-density-wave transitions of dirac fermions coupled to phonons*, Phys. Rev. Lett. **122**, 077601 (2019), doi:[10.1103/PhysRevLett.122.077601](https://doi.org/10.1103/PhysRevLett.122.077601).
- [69] O. Bradley, G. G. Batrouni and R. T. Scalettar, *Superconductivity and charge density wave order in the 2d holstein model*, arXiv:2011.11703 (2020), <https://arxiv.org/abs/2011.11703>.
- [70] M. Ippoliti, R. S. K. Mong, F. F. Assaad and M. P. Zaletel, *Half-filled landau levels: A continuum and sign-free regularization for three-dimensional quantum critical points*, Phys. Rev. B **98**, 235108 (2018), doi:[10.1103/PhysRevB.98.235108](https://doi.org/10.1103/PhysRevB.98.235108).
- [71] Z. Wang, R. S. K. Mong and F. F. Zaletel, Michael P. and. Assaad, *Phases of the (2+1) dimensional so(5) non-linear sigma model with topological term*, arXiv:2003.08368 <https://arxiv.org/abs/2003.08368>.
- [72] G. Pan, W. Wang, A. Davis, Y. Wang and Z. Y. Meng, *Yukawa-syk model and self-tuned quantum criticality*, arXiv:2001.06586 (2020), <https://arxiv.org/abs/2001.06586>.

- [73] H. Zhang, Z. Zhao, D. Gautreau, M. Raczkowski, A. Saha, V. O. Garlea, H. Cao, T. Hong, H. O. Jeschke, S. D. Mahanti, T. Birol, F. F. Assaad *et al.*, *Coexistence and interaction of spinons and magnons in an antiferromagnet with alternating antiferromagnetic and ferromagnetic quantum spin chains*, Phys. Rev. Lett. **125**, 037204 (2020), doi:[10.1103/PhysRevLett.125.037204](https://doi.org/10.1103/PhysRevLett.125.037204).
- [74] C. Wu and S.-C. Zhang, *Sufficient condition for absence of the sign problem in the fermionic quantum monte carlo algorithm*, Phys. Rev. B **71**, 155115 (2005), doi:[10.1103/PhysRevB.71.155115](https://doi.org/10.1103/PhysRevB.71.155115).
- [75] E. F. Huffman and S. Chandrasekharan, *Solution to sign problems in half-filled spin-polarized electronic systems*, Phys. Rev. B **89**, 111101 (2014), doi:[10.1103/PhysRevB.89.111101](https://doi.org/10.1103/PhysRevB.89.111101).
- [76] Z.-X. Li, Y.-F. Jiang and H. Yao, *Solving the fermion sign problem in quantum monte carlo simulations by majorana representation*, Phys. Rev. B **91**, 241117 (2015), doi:[10.1103/PhysRevB.91.241117](https://doi.org/10.1103/PhysRevB.91.241117).
- [77] Z. C. Wei, C. Wu, Y. Li, S. Zhang and T. Xiang, *Majorana positivity and the fermion sign problem of quantum monte carlo simulations*, Phys. Rev. Lett. **116**, 250601 (2016), doi:[10.1103/PhysRevLett.116.250601](https://doi.org/10.1103/PhysRevLett.116.250601).
- [78] J. Hubbard, *Calculation of partition functions*, Phys. Rev. Lett. **3**, 77 (1959), doi:[10.1103/PhysRevLett.3.77](https://doi.org/10.1103/PhysRevLett.3.77).
- [79] M. Troyer and U.-J. Wiese, *Computational complexity and fundamental limitations to fermionic quantum monte carlo simulations*, Phys. Rev. Lett. **94**, 170201 (2005), doi:[10.1103/PhysRevLett.94.170201](https://doi.org/10.1103/PhysRevLett.94.170201).
- [80] S. Duane and J. B. Kogut, *Hybrid stochastic differential equations applied to quantum chromodynamics*, Phys. Rev. Lett. **55**, 2774 (1985), doi:[10.1103/PhysRevLett.55.2774](https://doi.org/10.1103/PhysRevLett.55.2774).
- [81] J. Hirsch, *Discrete hubbard-stratonovich transformation for fermion lattice models*, Phys. Rev. B **28**, 4059 (1983), doi:[10.1103/PhysRevB.28.4059](https://doi.org/10.1103/PhysRevB.28.4059).
- [82] A. D. Sokal, *Monte Carlo methods in statistical mechanics: Foundations and new algorithms*, doi:[10.1007/978-1-4899-0319-8_6](https://doi.org/10.1007/978-1-4899-0319-8_6), Lecture notes from Cours de Troisième Cycle de la Physique en Suisse Romande. Updated in 1996 for the Cargèse Summer School on “Functional Integration: Basics and Applications” (1989).
- [83] H. G. Evertz, G. Lana and M. Marcu, *Cluster algorithm for vertex models*, Phys. Rev. Lett. **70**, 875 (1993), doi:[10.1103/PhysRevLett.70.875](https://doi.org/10.1103/PhysRevLett.70.875).
- [84] A. W. Sandvik, *Stochastic series expansion method with operator-loop update*, Phys. Rev. B **59**, R14157 (1999), doi:[10.1103/PhysRevB.59.R14157](https://doi.org/10.1103/PhysRevB.59.R14157).
- [85] O. Syljuåsen and A. Sandvik, *Quantum monte carlo with directed loops*, Phys. Rev. E **66**, 046701 (2002), doi:[10.1103/PhysRevE.66.046701](https://doi.org/10.1103/PhysRevE.66.046701).
- [86] J. E. Hirsch and R. M. Fye, *Monte carlo method for magnetic impurities in metals*, Phys. Rev. Lett. **56**, 2521 (1986), doi:[10.1103/PhysRevLett.56.2521](https://doi.org/10.1103/PhysRevLett.56.2521).
- [87] E. Gull, A. J. Millis, A. I. Lichtenstein, A. N. Rubtsov, M. Troyer and P. Werner, *Continuous-time monte carlo methods for quantum impurity models*, Rev. Mod. Phys. **83**, 349 (2011), doi:[10.1103/RevModPhys.83.349](https://doi.org/10.1103/RevModPhys.83.349).

- [88] F. F. Assaad, *DMFT at 25: Infinite Dimensions: Lecture Notes of the Autumn School on Correlated Electrons*, vol. 4, chap. 7. Continuous-time QMC Solvers for Electronic Systems in Fermionic and Bosonic Baths, Verlag des Forschungszentrum Jülich, Jülich, ISBN 978-3-89336-953-9 (2014).
- [89] F. F. Assaad and T. C. Lang, *Diagrammatic determinantal quantum monte carlo methods: Projective schemes and applications to the hubbard-holstein model*, Phys. Rev. B **76**, 035116 (2007), doi:[10.1103/PhysRevB.76.035116](https://doi.org/10.1103/PhysRevB.76.035116).
- [90] R. T. Scalettar, D. J. Scalapino and R. L. Sugar, *New algorithm for the numerical simulation of fermions*, Phys. Rev. B **34**, 7911 (1986), doi:[10.1103/PhysRevB.34.7911](https://doi.org/10.1103/PhysRevB.34.7911).
- [91] S. Beyl, F. Goth and F. F. Assaad, *Revisiting the hybrid quantum monte carlo method for hubbard and electron-phonon models*, Phys. Rev. B **97**, 085144 (2018), doi:[10.1103/PhysRevB.97.085144](https://doi.org/10.1103/PhysRevB.97.085144).
- [92] S. Dürr, Z. Fodor, J. Frison, C. Hoelbling, R. Hoffmann, S. D. Katz, S. Krieg, T. Kurth, L. Lellouch, T. Lippert, K. K. Szabo and G. Vulvert, *Ab initio determination of light hadron masses*, Science **322**(5905), 1224 (2008), doi:[10.1126/science.1163233](https://doi.org/10.1126/science.1163233), <http://science.sciencemag.org/content/322/5905/1224.full.pdf>.
- [93] S. Sorella, *The phase diagram of the hubbard model by variational auxiliary field quantum monte carlo*, arXiv:2101.07045 (2021), doi:[10.48550/ARXIV.2101.07045](https://doi.org/10.48550/ARXIV.2101.07045).
- [94] F. F. Assaad, *Quantum monte carlo methods on lattices: The determinantal method*, In J. Grotendorst, D. Marx and A. Muramatsu., eds., *Lecture notes of the Winter School on Quantum Simulations of Complex Many-Body Systems: From Theory to Algorithms.*, vol. 10, pp. 99–155. Publication Series of the John von Neumann Institute for Computing, Jülich (2002).
- [95] Y. Motome and M. Imada, *A quantum monte carlo method and its applications to multi-orbital hubbard models*, Journal of the Physical Society of Japan **66**(7), 1872 (1997), doi:[10.1143/JPSJ.66.1872](https://doi.org/10.1143/JPSJ.66.1872), <http://dx.doi.org/10.1143/JPSJ.66.1872>.
- [96] F. F. Assaad, M. Imada and D. J. Scalapino, *Charge and spin structures of a $d_{x^2-y^2}$ superconductor in the proximity of an antiferromagnetic mott insulator*, Phys. Rev. B **56**, 15001 (1997), doi:[10.1103/PhysRevB.56.15001](https://doi.org/10.1103/PhysRevB.56.15001).
- [97] A. Sandvik, *Stochastic method for analytic continuation of quantum monte carlo data*, Phys. Rev. B **57**, 10287 (1998), doi:[10.1103/PhysRevB.57.10287](https://doi.org/10.1103/PhysRevB.57.10287).
- [98] K. S. D. Beach, *Identifying the maximum entropy method as a special limit of stochastic analytic continuation*, eprint arXiv:cond-mat/0403055 (2004), <https://arxiv.org/abs/cond-mat/0403055>.
- [99] M. Jarrell and J. Gubernatis, *Bayesian inference and the analytic continuation of imaginary-time quantum monte carlo data*, Physics Reports **269**, 133 (1996).
- [100] W. von der Linden, Applied Physics A **60**, 155 (1995).
- [101] R. M. Fye, *New results on trotter-like approximations*, Phys. Rev. B **33**, 6271 (1986), doi:[10.1103/PhysRevB.33.6271](https://doi.org/10.1103/PhysRevB.33.6271).
- [102] M. Iazzi and M. Troyer, *Efficient continuous-time quantum monte carlo algorithm for fermionic lattice models*, Phys. Rev. B **91**, 241118 (2015), doi:[10.1103/PhysRevB.91.241118](https://doi.org/10.1103/PhysRevB.91.241118).

- [103] S. M. A. Rombouts, K. Heyde and N. Jachowicz, *Quantum monte carlo method for fermions, free of discretization errors*, Phys. Rev. Lett. **82**, 4155 (1999), doi:[10.1103/PhysRevLett.82.4155](https://doi.org/10.1103/PhysRevLett.82.4155).
- [104] E. Gull, P. Werner, O. Parcollet and M. Troyer, *Continuous-time auxiliary-field monte carlo for quantum impurity models*, EPL (Europhysics Letters) **82**(5), 57003 (2008), doi:[10.1209/0295-5075/82/57003](https://doi.org/10.1209/0295-5075/82/57003).
- [105] S. Rombouts, K. Heyde and N. Jachowicz, *A discrete hubbard-stratonovich decomposition for general, fermionic two-body interactions*, Physics Letters A **242**, 271 (1998), doi:[10.1016/S0375-9601\(98\)00197-2](https://doi.org/10.1016/S0375-9601(98)00197-2).
- [106] D. Rost, E. V. Gorelik, F. Assaad and N. Blümer, *Momentum-dependent pseudogaps in the half-filled two-dimensional hubbard model*, Phys. Rev. B **86**, 155109 (2012), doi:[10.1103/PhysRevB.86.155109](https://doi.org/10.1103/PhysRevB.86.155109).
- [107] D. Rost, F. Assaad and N. Blümer, *Quasi-continuous-time impurity solver for the dynamical mean-field theory with linear scaling in the inverse temperature*, Phys. Rev. E **87**, 053305 (2013), doi:[10.1103/PhysRevE.87.053305](https://doi.org/10.1103/PhysRevE.87.053305).
- [108] N. Blümer, *Multigrid Hirsch-Fye quantum Monte Carlo method for dynamical mean-field theory*, arXiv:0801.1222 (2008), <https://arxiv.org/abs/0801.1222>.
- [109] L. Wang, Y.-H. Liu and M. Troyer, *Stochastic series expansion simulation of the $t - v$ model*, Phys. Rev. B **93**, 155117 (2016), doi:[10.1103/PhysRevB.93.155117](https://doi.org/10.1103/PhysRevB.93.155117).
- [110] E. Huffman and S. Chandrasekharan, *Fermion bag approach to hamiltonian lattice field theories in continuous time*, Phys. Rev. D **96**, 114502 (2017), doi:[10.1103/PhysRevD.96.114502](https://doi.org/10.1103/PhysRevD.96.114502).
- [111] E. Huffman and S. Chandrasekharan, *Fermion-bag inspired hamiltonian lattice field theory for fermionic quantum criticality* (2019), <https://arxiv.org/abs/1912.12823>.
- [112] F. Goth, *Higher order auxiliary field quantum monte carlo methods*, arXiv:2009.04491 (2020), <https://arxiv.org/abs/2009.04491>.
- [113] I. Peschel, *Calculation of reduced density matrices from correlation functions*, Journal of Physics A: Mathematical and General **36**(14), L205 (2003), doi:[10.1088/0305-4470/36/14/101](https://doi.org/10.1088/0305-4470/36/14/101).
- [114] Z.-Q. Wan, S.-X. Zhang and H. Yao, *Mitigating sign problem by automatic differentiation*, arXiv:2010.01141 (2020), <https://arxiv.org/abs/2010.01141>.
- [115] D. Hangleiter, I. Roth, D. Nagaj and J. Eisert, *Easing the monte carlo sign problem*, Science Advances **6**(33) (2020), doi:[10.1126/sciadv.abb8341](https://doi.org/10.1126/sciadv.abb8341), <https://advances.sciencemag.org/content/6/33/eabb8341.full.pdf>.
- [116] J. Liu, Y. Qi, Z. Y. Meng and L. Fu, *Self-learning monte carlo method*, Phys. Rev. B **95**, 041101 (2017), doi:[10.1103/PhysRevB.95.041101](https://doi.org/10.1103/PhysRevB.95.041101).
- [117] X. Y. Xu, Y. Qi, J. Liu, L. Fu and Z. Y. Meng, *Self-learning quantum monte carlo method in interacting fermion systems*, Phys. Rev. B **96**, 041119 (2017), doi:[10.1103/PhysRevB.96.041119](https://doi.org/10.1103/PhysRevB.96.041119).
- [118] K. Hukushima and K. Nemoto, *Exchange monte carlo method and application to spin glass simulations*, Journal of the Physical Society of Japan **65**(6), 1604 (1996), doi:[10.1143/JPSJ.65.1604](https://doi.org/10.1143/JPSJ.65.1604), <http://dx.doi.org/10.1143/JPSJ.65.1604>.

- [119] C. J. Geyer, *Markov chain monte carlo maximum likelihood*, In *Computing Science and Statistics: Proceedings of the 23rd Symposium on the Interface*, pp. 156–163. American Statistical Association, New York (1991).
- [120] C. W. Gardiner, *Handbook of Stochastic Methods*, Springer (1985).
- [121] G. G. Batrouni, G. R. Katz, A. S. Kronfeld, G. P. Lepage, B. Svetitsky and K. G. Wilson, *Langevin simulations of lattice field theories*, Phys. Rev. D **32**, 2736 (1985), doi:[10.1103/PhysRevD.32.2736](https://doi.org/10.1103/PhysRevD.32.2736).
- [122] G. G. Batrouni and R. T. Scalettar, *Langevin simulations of a long-range electron-phonon model*, Phys. Rev. B **99**, 035114 (2019), doi:[10.1103/PhysRevB.99.035114](https://doi.org/10.1103/PhysRevB.99.035114).
- [123] C. Davies, G. Batrouni, G. Katz, A. Kronfeld, P. Lepage, P. Rossi, B. Svetitsky and K. Wilson, *Langevin simulations of lattice field theories using fourier acceleration*, Journal of Statistical Physics **43**(5), 1073 (1986), doi:[10.1007/BF02628331](https://doi.org/10.1007/BF02628331).
- [124] U. Wolff, *Collective monte carlo updating for spin systems*, Phys. Rev. Lett. **62**, 361 (1989), doi:[10.1103/PhysRevLett.62.361](https://doi.org/10.1103/PhysRevLett.62.361).
- [125] M. Imada, *A fermion simulation algorithm by molecular dynamics technique combined with monte carlo method*, Journal of the Physical Society of Japan **57**(8), 2689 (1988), doi:[10.1143/JPSJ.57.2689](https://doi.org/10.1143/JPSJ.57.2689), <https://doi.org/10.1143/JPSJ.57.2689>.
- [126] S. Beyl, *Hybrid Quantum Monte Carlo for Condensed Matter Models*, Doctoral thesis, Universität Würzburg, doi:[10.25972/OPUS-19122](https://doi.org/10.25972/OPUS-19122) (2020).
- [127] E. Loh, J. Gubernatis, R. Scalettar, R. Sugar and S. White, *Stable matrix-multiplication algorithms for low-temperature numerical simulations of fermions*, In *Interacting Electrons in Reduced Dimensions*, pp. 55–60. Springer, ISBN 978-1-4613-0565-1, doi:[10.1007/978-1-4613-0565-1](https://doi.org/10.1007/978-1-4613-0565-1) (1989).
- [128] E. Y. LOH, J. E. GUBERNATIS, R. T. SCALETTAR, S. R. WHITE, D. J. SCALAPINO and R. L. SUGAR, *Numerical stability and the sign problem in the determinant quantum monte carlo method*, International Journal of Modern Physics C **16**(08), 1319 (2005), doi:[10.1142/S0129183105007911](https://doi.org/10.1142/S0129183105007911), <https://doi.org/10.1142/S0129183105007911>.
- [129] Z. Bai, C. Lee, R.-C. Li and S. Xu, *Stable solutions of linear systems involving long chain of matrix multiplications*, Linear Algebra and its Applications **435**(3), 659 (2011), doi:[10.1016/j.laa.2010.06.023](https://doi.org/10.1016/j.laa.2010.06.023).
- [130] C. Bauer, *Fast and stable determinant quantum Monte Carlo*, SciPost Phys. Core **2**, 11 (2020), doi:[10.21468/SciPostPhysCore.2.2.011](https://doi.org/10.21468/SciPostPhysCore.2.2.011).
- [131] J. Demmel and K. Veselić, *Jacobi’s method is more accurate than qr*, SIAM Journal on Matrix Analysis and Applications **13**(4), 1204 (1992), doi:[10.1137/0613074](https://doi.org/10.1137/0613074), <https://doi.org/10.1137/0613074>.
- [132] J. Dongarra, M. Gates, A. Haidar, J. Kurzak, P. Luszczek, S. Tomov and I. Yamazaki, *The singular value decomposition: Anatomy of optimizing an algorithm for extreme scale*, SIAM Review **60**(4), 808 (2018), doi:[10.1137/17M1117732](https://doi.org/10.1137/17M1117732), <https://doi.org/10.1137/17M1117732>.
- [133] A. van der Sluis, *Condition numbers and equilibration of matrices*, Numerische Mathematik **14**(1), 14 (1969), doi:[10.1007/BF02165096](https://doi.org/10.1007/BF02165096).

- [134] M. Feldbacher and F. F. Assaad, *Efficient calculation of imaginary-time-displaced correlation functions in the projector auxiliary-field quantum monte carlo algorithm*, Phys. Rev. B **63**, 073105 (2001), doi:[10.1103/PhysRevB.63.073105](https://doi.org/10.1103/PhysRevB.63.073105).
- [135] D. Ixert, F. F. Assaad and K. P. Schmidt, *Mott physics in the half-filled hubbard model on a family of vortex-full square lattices*, Phys. Rev. B **90**, 195133 (2014), doi:[10.1103/PhysRevB.90.195133](https://doi.org/10.1103/PhysRevB.90.195133).
- [136] J. W. Negele and H. Orland, *Quantum Many body systems*, Frontiers in physics. Addison-Wesley, Redwood City, Calif. u.a., ISBN 9780201125931 (1988).
- [137] W. Krauth, *Statistical Mechanics: Algorithms and Computations*, Oxford University Press (2006).
- [138] C. J. Geyer, *Practical Markov chain Monte Carlo*, Statistical Science **7**(4), 473 (1992), doi:[10.1214/ss/1177011137](https://doi.org/10.1214/ss/1177011137).
- [139] R. M. Neal, *Probabilistic inference using Markov chain Monte Carlo methods*, Department of Computer Science, University of Toronto Toronto, Ontario, Canada (1993).
- [140] M. Bercx, J. S. Hofmann, F. F. Assaad and T. C. Lang, *Spontaneous particle-hole symmetry breaking of correlated fermions on the lieb lattice*, Phys. Rev. B **95**, 035108 (2017), doi:[10.1103/PhysRevB.95.035108](https://doi.org/10.1103/PhysRevB.95.035108).
- [141] B. Efron and C. Stein, *The jackknife estimate of variance*, Ann. Statist. **9**(3), 586 (1981), doi:[10.1214/aos/1176345462](https://doi.org/10.1214/aos/1176345462).
- [142] S. Chakravarty, B. I. Halperin and D. R. Nelson, *Low-temperature behavior of two-dimensional quantum antiferromagnets*, Phys. Rev. Lett. **60**, 1057 (1988), doi:[10.1103/PhysRevLett.60.1057](https://doi.org/10.1103/PhysRevLett.60.1057).
- [143] M. B. Thompson, *A Comparison of Methods for Computing Autocorrelation Time*, arXiv:1011.0175 (2010), <https://arxiv.org/abs/1011.0175>.
- [144] A. Parola, S. Sorella, M. Parrinello and E. Tosatti, *d-wave, dimer, and chiral states in the two-dimensional hubbard model*, Phys. Rev. B **43**, 6190 (1991), doi:[10.1103/PhysRevB.43.6190](https://doi.org/10.1103/PhysRevB.43.6190).
- [145] I. Milat, F. Assaad and M. Sigrist, *Field induced magnetic ordering transition in kondo insulators*, The European Physical Journal B - Condensed Matter and Complex Systems **38**(4), 571 (2004), doi:[10.1140/epjb/e2004-00154-5](https://doi.org/10.1140/epjb/e2004-00154-5).
- [146] M. Bercx, T. C. Lang and F. F. Assaad, *Magnetic field induced semimetal-to-canted-antiferromagnet transition on the honeycomb lattice*, Phys. Rev. B **80**, 045412 (2009), doi:[10.1103/PhysRevB.80.045412](https://doi.org/10.1103/PhysRevB.80.045412).
- [147] M.-S. Vaezi, A.-R. Negari, A. Moharramipour and A. Vaezi, *Amelioration for the sign problem: An adiabatic quantum monte carlo algorithm*, Phys. Rev. Lett. **127**, 217003 (2021), doi:[10.1103/PhysRevLett.127.217003](https://doi.org/10.1103/PhysRevLett.127.217003).
- [148] J. R. Schrieffer and P. A. Wolff, *Relation between the anderson and kondo hamiltonians*, Phys. Rev. **149**, 491 (1966), doi:[10.1103/PhysRev.149.491](https://doi.org/10.1103/PhysRev.149.491).
- [149] T. A. Costi, *Kondo effect in a magnetic field and the magnetoresistivity of kondo alloys*, Phys. Rev. Lett. **85**, 1504 (2000), doi:[10.1103/PhysRevLett.85.1504](https://doi.org/10.1103/PhysRevLett.85.1504).

- [150] M. Raczkowski and F. F. Assaad, *Emergent coherent lattice behavior in kondo nanosystems*, Phys. Rev. Lett. **122**, 097203 (2019), doi:[10.1103/PhysRevLett.122.097203](https://doi.org/10.1103/PhysRevLett.122.097203).
- [151] M. Maltseva, M. Dzero and P. Coleman, *Electron cotunneling into a kondo lattice*, Phys. Rev. Lett. **103**, 206402 (2009), doi:[10.1103/PhysRevLett.103.206402](https://doi.org/10.1103/PhysRevLett.103.206402).
- [152] M. Vekić, J. W. Cannon, D. J. Scalapino, R. T. Scalettar and R. L. Sugar, *Competition between antiferromagnetic order and spin-liquid behavior in the two-dimensional periodic anderson model at half filling*, Phys. Rev. Lett. **74**, 2367 (1995), doi:[10.1103/PhysRevLett.74.2367](https://doi.org/10.1103/PhysRevLett.74.2367).
- [153] Z.-X. Li, Y.-F. Jiang and H. Yao, *Majorana-time-reversal symmetries: A fundamental principle for sign-problem-free quantum monte carlo simulations*, Phys. Rev. Lett. **117**, 267002 (2016), doi:[10.1103/PhysRevLett.117.267002](https://doi.org/10.1103/PhysRevLett.117.267002).
- [154] K. S. D. Beach, P. A. Lee and P. Monthoux, *Field-induced antiferromagnetism in the kondo insulator*, Phys. Rev. Lett. **92**, 026401 (2004), doi:[10.1103/PhysRevLett.92.026401](https://doi.org/10.1103/PhysRevLett.92.026401).
- [155] A. Rüegg, S. D. Huber and M. Sigrist, *z_2 -slave-spin theory for strongly correlated fermions*, Phys. Rev. B **81**, 155118 (2010), doi:[10.1103/PhysRevB.81.155118](https://doi.org/10.1103/PhysRevB.81.155118).
- [156] M. Weber, *Valence bond order in a honeycomb antiferromagnet coupled to quantum phonons*, Phys. Rev. B **103**, L041105 (2021), doi:[10.1103/PhysRevB.103.L041105](https://doi.org/10.1103/PhysRevB.103.L041105).
- [157] M. Weber, F. F. Assaad and M. Hohenadler, *Directed-loop quantum monte carlo method for retarded interactions*, Phys. Rev. Lett. **119**, 097401 (2017), doi:[10.1103/PhysRevLett.119.097401](https://doi.org/10.1103/PhysRevLett.119.097401).
- [158] P. Hauke, M. Heyl, L. Tagliacozzo and P. Zoller, *Measuring multipartite entanglement through dynamic susceptibilities*, Nature Physics **12**(8), 778 (2016), doi:[10.1038/nphys3700](https://doi.org/10.1038/nphys3700).
- [159] Y. Meir and N. S. Wingreen, *Landauer formula for the current through an interacting electron region*, Phys. Rev. Lett. **68**, 2512 (1992), doi:[10.1103/PhysRevLett.68.2512](https://doi.org/10.1103/PhysRevLett.68.2512).
- [160] C. Karrasch, V. Meden and K. Schönhammer, *Finite-temperature linear conductance from the matsubara green's function without analytic continuation to the real axis*, Phys. Rev. B **82**, 125114 (2010), doi:[10.1103/PhysRevB.82.125114](https://doi.org/10.1103/PhysRevB.82.125114).
- [161] A. Abendschein and F. F. Assaad, *Temperature dependence of spectral functions for the one-dimensional hubbard model: Comparison with experiments*, Phys. Rev. B **73**, 165119 (2006), doi:[10.1103/PhysRevB.73.165119](https://doi.org/10.1103/PhysRevB.73.165119).
- [162] A. N. Rubtsov, V. V. Savkin and A. I. Lichtenstein, *Continuous-time quantum monte carlo method for fermions*, Phys. Rev. B **72**, 035122 (2005), doi:[10.1103/PhysRevB.72.035122](https://doi.org/10.1103/PhysRevB.72.035122).
- [163] Jülich Supercomputing Centre, *JURECA: General-purpose supercomputer at Jülich Supercomputing Centre*, Journal of large-scale research facilities **2**, A62 (2016), doi:[10.17815/jlsrf-2-121](https://doi.org/10.17815/jlsrf-2-121).

Index

Bilayer_square, 85	Kagome, 86
Honeycomb, 85	Precision Green , 34
NWrap, 34	Precision Phase, 34
N_leg_ladder, 85	Square , 84
Bilayer_honeycomb, 85	Symm, 30–32
Checkerboard, 29	Triangular, 86